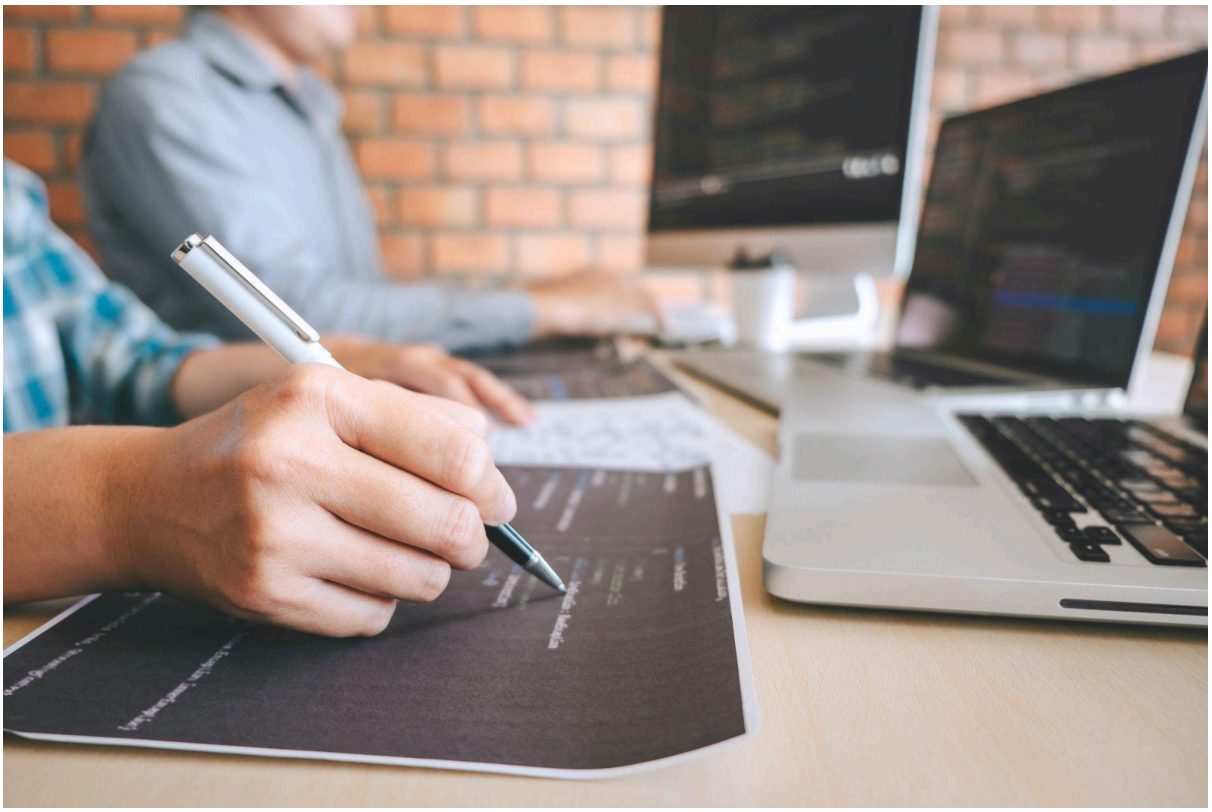
Laboratorio de Programación I

CARLOS ARNALDO GONZALEZ



Asistente virtual de estudio.

¡Escaneá y consultá!

COLEGIO UNIVERSITARIO IES

Editorial

Agradecimientos

Agradecemos a todos aquellos que han aportado su investigación, su experiencia y su tiempo para la elaboración de este TID.

Agradecemos, finalmente, a Fernando FRÍAS, Director de la Carrera de ANALISTA DE SISTEMAS del Colegio Universitario IES.

Autor



Carlos González

- Analista de Sistemas de Computación, egresado del *Colegio Universitario IES*.
- Desarrollador .Net - GIS - MOTOROLA, Córdoba.
- Desarrollador C y .Net - VATES SA, Córdoba.
- Desarrollador C/C++ - CDSI Argentina S.A, Córdoba Docente de las materias Laboratorio de Programación 1, 2 y 3, Visual Studio .NET (lenguaje Visual Basic .NET) en el *Colegio Universitario IES*.
- Docente de la materias Estructura de Datos, prácticas con lenguaje C#, Visual Studio .NET 2010 en el *Colegio Universitario IES*.

Equipo de producción

Producción y dirección general

- Directora general: María Braganza
- Vicedirector general: Alberto Rabbat
- Director Académico: Hugo Fernando Frias
- Vicedirectora Académica: Erica Bongiovanni

Planificación y coordinación general

- Coordinador de estudios a distancia: Erica Bongiovanni
- Dirección de carrera: Hugo Fernando Frias
- Coordinación de proceso editorial: Nicolás Irusta

Producción Editorial

- Sebastian Benito
- Diego Oliva

Producción Académica

- Ana Giró
- Telmo Torres

Coordinación de sistemas

- Marcela Giraudo

Uso de marcas

Cláusulas de uso de marcas y derechos de autor de terceros.

- Uso atípico de marca ajena: Exclusión de los usos no comerciales del Derecho Marcario. PERSPECTIVAS S.A. en su carácter de titular de los derechos intelectuales sobre la presente obra declara por esta vía que el uso que realiza de marcas comerciales de terceros lo es sólo a los fines informativos y didácticos, para mejor comprensión de los lectores y alumnos del contenido de la obra, siendo el mismo de carácter atípico (uso atípico de marca ajena) y lícito. Este uso, a tenor de la jurisprudencia vigente queda fuera del ius prohibendi, que detenta el titular de cada marca registrada, atento no ser el mismo de carácter comercial en relación al producto que distinguen las referidas marcas, y por ende de índole marcario.-
- Uso de derechos de autor en videos, diskettes, imágenes y audio: Libre utilización -Uso privado- de obras protegidas. PERSPECTIVAS S.A. en su carácter de titular de los derechos intelectuales sobre la presente obra declara por esta vía que el uso que realiza de determinadas grabaciones (audio y video), e imágenes (fotografías) de terceros, lo es a los fines informativos y didácticos, para mejor comprensión de los alumnos del contenido de la obra, siendo el mismo de carácter privado y no comercial, y desde ya respetando el derecho de cita, esto es declarando en toda ocasión la cita o fuente (obra y autor) de la cual se toman los fragmentos de obras de terceros para incorporarlos a la presente (Convenio de Berna, Acta de París, 1971 – Art. 10, § 2 y § 3).-
- Modificación de obras literarias por el titular de los derechos patrimoniales. PERSPECTIVAS S.A. en su carácter de titular de los derechos intelectuales (patrimoniales) sobre la presente obra, aclara que ha autorizado a Carlos GONZÁLES a la modificación respecto de la obra original publicada con el título de Laboratorio de programación I (08/2011) con N° de ISBN 978-987-600-178-6, cuyos autores son Juan Casale - Miguel Carrizo, constituyéndose en autores morales de la obra referida y modificada. Se declara a todo efecto que, los derechos intelectuales se ceden y mantienen a favor de su titular, PERSPECTIVAS S.A.

Copyright

González, Carlos

Laboratorio de programación I / Carlos González ; coordinación general de María Teresa de las Casas ; dirigido por José Alberto Rabbat. - 1a edición para el alumno - Córdoba : IES Siglo 21, 2022.

Libro digital, Otros

Archivo Digital: descarga y online

ISBN 978-987-600-402-2

1. Aplicaciones Informáticas. I. Casas, María Teresa de las, coord. II. Rabbat, José Alberto, dir. III.

Título.

CDD 005.102

2da Edición

© 2022 - Editorial IES Siglo 21

Buenos Aires 563

TE: 54-351-4211717

5000 - Córdoba

Queda hecho el depósito que establece la Ley 11723

Libro de edición argentina

No se permite la reproducción parcial o total, el almacenamiento, el alquiler, la transmisión o la transformación de este libro, en cualquier forma o por cualquier medio, sea electrónico o mecánico, mediante fotocopias, digitalización u otros métodos, sin el permiso previo y escrito del editor. Su infracción está penada por las leyes 11723 y 25446.

Se terminó de replicar durante el mes de Abril de 2022 en el departamento de Logística en Editorial IES Siglo 21.

Índice

Índice 5
Cómo está organizado este texto 7
Introducción 8
Esquema 9
Situación profesional 1: Lo que ves es lo que obtienes 10
SP1/H1: Interfaz 12
SP1/Autoevaluación 1 47
SP1/H2: IDE (Integrated Development Environment). Entorno de desarrollo integrado 49
SP1/Autoevaluación 2 58
SP1/H3: Visual Studio 59
SP1/Autoevaluación 3 67
SP1/Ejercicio resuelto 68
SP1/Ejercicio por resolver 85
SP1/Evaluación de paso 87
Situación profesional 2: La lógica en la programación 89
SP2/H1: Diseño de formularios y controles estándares 91
SP2/Autoevaluación 1 99
SP2/H2: Controles 100
SP2/Autoevaluación 2 159
SP2/H3: Identificadores, constantes, variables, tipos de datos 160
y operadores 160
SP2/Autoevaluación 3 168
SP2/H4: Estructuras de control 170
SP2/Autoevaluación 4 197
SP2/Ejercicio resuelto 198
SP2/Ejercicio por resolver 206
SP2/Evaluación de paso 207
Situación profesional 3: Auto Test 208
SP3/H1: Tipos de datos definidos por el usuario, tipos 'struct' 210
SP3/Autoevaluación 1 216
SP3/H2: Arreglos 217
SP3/Autoevaluación 2 221
SP3/H3: Arreglos de una dimensión 222
SP3/Autoevaluación 3 233
SP3/H4: Elementos del lenguaje para conversiones y validaciones de datos 235
SP3/Autoevaluación 4 245

SP3/Ejercicio resuelto 246
SP3/Ejercicio por resolver 262
SP3/Evaluación de paso 263
Situación profesional 4: Arreglos multidimensionales - Triatlón 265
SP4/H1: Arreglos multidimensionales 267
SP4/Autoevaluación 1 276
SP4/H2: Control DataGridView 278
SP4/Autoevaluación 2 287
SP4/H3: Control PictureBox 288
SP4/Autoevaluación 3 298
SP4/Ejercicio resuelto 299
SP4/Ejercicio por resolver 317
SP4/Evaluación de paso 318
Cierre 320
Bibliografía 321

Cómo está organizado este texto

Usted está en presencia de este texto que los autores proponen para la comprensión y estudio de la asignatura. Ha sido preparado y diseñado para facilitar el acceso al conocimiento, a partir de una secuencia cuyo punto de partida es la práctica profesional cotidiana y no la teoría alejada de la realidad.

Está organizado de la siguiente manera:

- **Introducción**. Indica qué papel desempeña la asignatura dentro de la carrera y los conceptos básicos que usted conocerá.
- **Esquema**. Muestra los enlaces que unen los conceptos centrales de la asignatura entre sí.
- **Situación profesional.** Lo ubica frente a un problema de la práctica profesional cotidiana que puede ser resuelto, ya que existe al menos una solución para ello, a través de conocimientos específicos que en cada caso se aportan.
- **Herramientas**. Son los conocimientos necesarios para resolver la Situación profesional planteada.
- **Autoevaluación**. Para que usted compruebe si ha comprendido correctamente lo que se explicó en una herramienta, los autores proponen la resolución de actividades y le ofrecen las respuestas.
- **Ejercicio resuelto.** Bajo este título encontrará una manera de resolver los problemas de práctica profesional planteados, con la selección de las herramientas pertinentes.
- **Ejercicio por resolver.** Ahora le toca a usted. Es el momento de aplicar las herramientas a una situación profesional nueva o similar a la ya expuesta. Todas las dudas que le aparezcan podrán ser planteadas a su docente.
- **Evaluación de paso.** Para que usted compruebe si ha comprendido correctamente lo que se explicó en las distintas herramientas que hasta el momento se han presentado, los autores proponen la resolución de actividades y le ofrecen las respuestas.
- **Bibliografía.** Se indican los textos, revistas y links de consulta a los que podrá recurrir para complementar o ampliar algunos temas.

Introducción



Link del video: https://www.youtube.com/watch?v=qILRl4sfSzA

De la mano y en conjunto con la materia **Programación Lógica 1**, usted iniciará la diagramación de interfaces gráficas de usuarios y el desarrollo de aplicaciones.

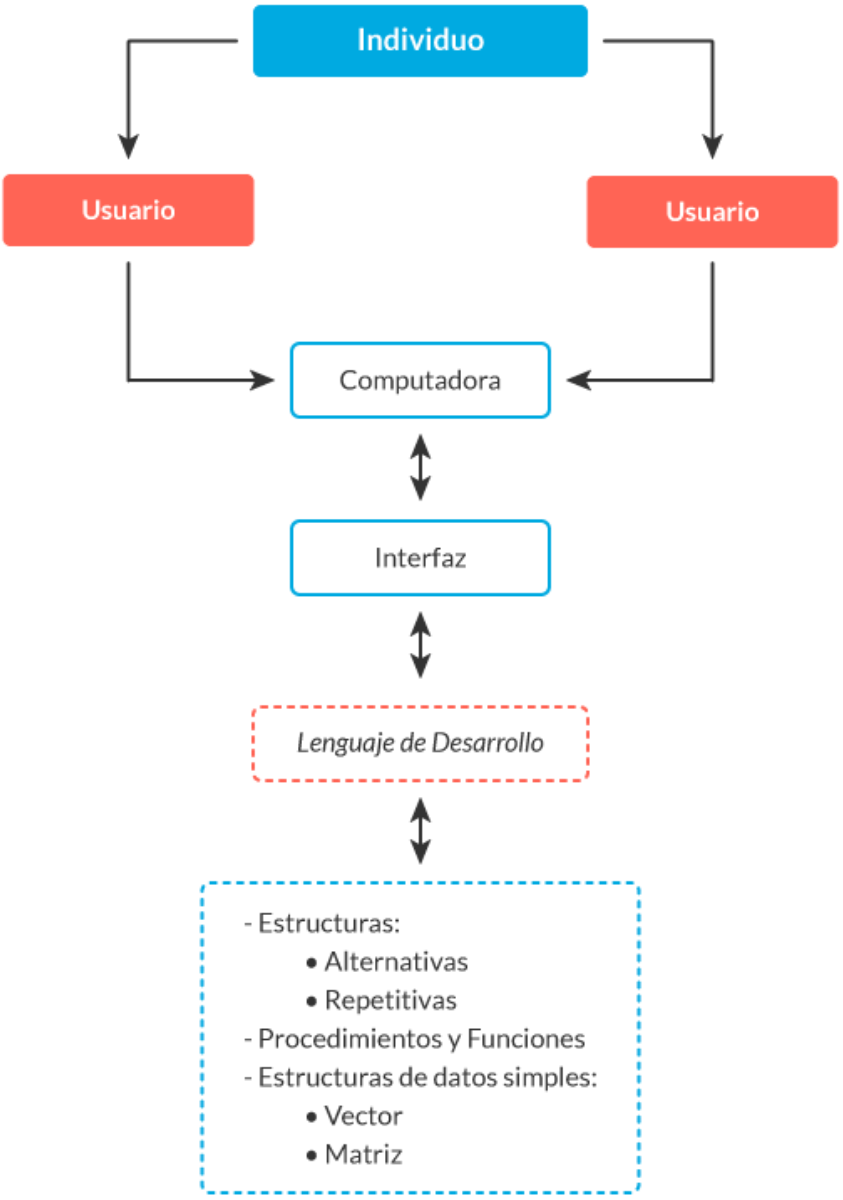
En este libro recorreremos las bases del desarrollo y programación de aplicaciones informáticas, comenzando por el diseño de interfaces gráficas de usuarios, conociendo la utilización de lenguajes de programación, y finalizando con el código fuente de la programación.

Encontraremos en las diferentes situaciones la necesidad de integrar visiones, de interpretar modelos de negocios, de conocer a nuestro mayor cliente, el usuario.

Tenemos las bases para iniciar el poderoso conocimiento que nos conducirá al desarrollo de aplicaciones. El ámbito es amplio, las herramientas y la ayuda es inmensa. Inicie con este texto el camino a su futuro como programador/desarrollador.

El autor

Esquema


Individuo
Usuario
Usuario
Computadora
Interfaz
Lenguaje de Desarrollo
- Estructuras:
• Alternativas
• Repetitivas
- Procedimientos y Funciones
- Estructuras de datos simples:
• Vector
• Matriz

Situación profesional 1: Lo que ves es lo que obtienes



Link del video: https://www.youtube.com/watch?v=HhR-5rjoUyU

Texto del video:

Una empresa dedicada a investigación de mercado, ONE S.A., contrata a la organización donde usted trabaja como desarrollador, para que confeccione un módulo de software, el cual debe permitir el ingreso de datos rápido, debe ser fácil de ser entendido y debe poder recibir información fácilmente.

El módulo puntualmente es sobre las investigaciones en Interés del Público (general y específico por barrio) en productos electrónicos (puntualmente SmartPhones, PC, Notebook, Netbook) caracterizándolos como MUY INTERESADO, INTERESADO, POCO INTERESADO, DESINTERESADO.

La forma de recolección de datos es a través de encuestas por barrio. Luego se procesa la información en la empresa.

Para esto, usted debe interpretar y analizar el modelo de negocio que le provee ONE S.A., luego evaluar cómo deberán ser las interfaces gráficas y programarlo.

En conjunto le solicitan el prototipo de una aplicación de escritorio para identificar a los usuarios que accederán al sistema de la organización.

Debe confeccionar el formulario de acceso (usuario y contraseña) y la ventana de bienvenida.

Le indican que al sistema pueden ingresar dos usuarios denominados "Administrador" y "Operador" cuyas contraseñas son "adm135$" y "ope246$" respectivamente.

Como requerimientos para esta aplicación le solicitan que el formulario de identificación ingrese los datos del usuario y contraseña en cajas de texto, no se pueden ingresar más de 20 caracteres en las cajas de texto y los caracteres de la contraseña no se visualizan cuando se digitan, la interfaz también posee un botón de comando denominado "Aceptar" para verificar los datos ingresados.

La aplicación se bloquea ante tres intentos fallidos de identificación del usuario.

Si el usuario y la contraseña son correctos se abre un formulario principal con una etiqueta de bienvenida y se oculta el formulario de identificación.

El formulario principal tiene un botón de comando denominado "Salir" que al ser pulsado cierra el formulario.

SP1/H1: Interfaz

¿Cómo interactuamos con un dispositivo electrónico? ¿Cómo interpretan lo que necesitamos? Son preguntas que a pocas personas les pueden interesar, sin embargo, en el rubro de la informática, debemos saber que para comunicarnos con estos dispositivos y obtener lo que deseamos, debemos conocer un lenguaje que nos permita "**conversar**" en el idioma de un equipo. Este idioma, a bajo nivel, es el **binario** (*los conocidos 0 y 1*).

En el transcurso de la materia, veremos una introducción sobre cómo obtenemos lo que deseamos de estos dispositivos; luego, pasaremos a conocer un **lenguaje** para decirle a un dispositivo que realice lo que deseamos.

En nuestra vida cotidiana, interactuamos con distintos tipos de aparatos electrónicos que, en algunos casos, nos facilitan las tareas diarias. El método por el cual interactuamos con dichos dispositivos, generalmente es por medio de una **interfaz**. Existen varios tipos de interfaces, en los próximos párrafos veremos algunos ejemplos.

Según la **RAE**, el término "**interfaz**" significa "conexión física y funcional entre dos aparatos o sistemas independientes", como puede suceder entre una persona y un dispositivo electrónico (computadora, celular, **smartphone**[1], etc.).

[1] Smartphone: Un smartphone es un teléfono móvil que incorpora características de una computadora personal. Los smartphones suelen permitir al usuario instalar nuevas aplicaciones, aumentando así sus funcionalidades. Esto es posible porque ejecutan un sistema operativo potente de fondo. Incluyen un mini teclado, una pantalla táctil, un lápiz óptico, etc. Los smartphones incluyen asimismo acceso a internet, servicios de e-mail, cámara integrada, navegador web, procesador de textos, etcétera.



Link del video: https://www.youtube.com/watch?v=mzorto5p0_E

Texto del video:

Interfaz como instrumento: desde esta perspectiva la interfaz es una "prótesis" o "extensión" de nuestro cuerpo. El mouse es un instrumento que extiende las funciones de nuestra mano y las lleva a la pantalla bajo forma de cursor.

Así por ejemplo, la pantalla de una computadora es interfaz entre el usuario y el disco duro de la misma.

Interfaz como superficie



Link del video: https://www.youtube.com/watch?v=_LvoZRKDSgU

Texto del video:

Interfaz como superficie: algunos consideran que la interfaz nos transmite instrucciones que nos informan sobre su uso. La superficie de un objeto (real o virtual) nos habla por medio de sus formas, texturas, colores, etcétera.

Interfaz como espacio

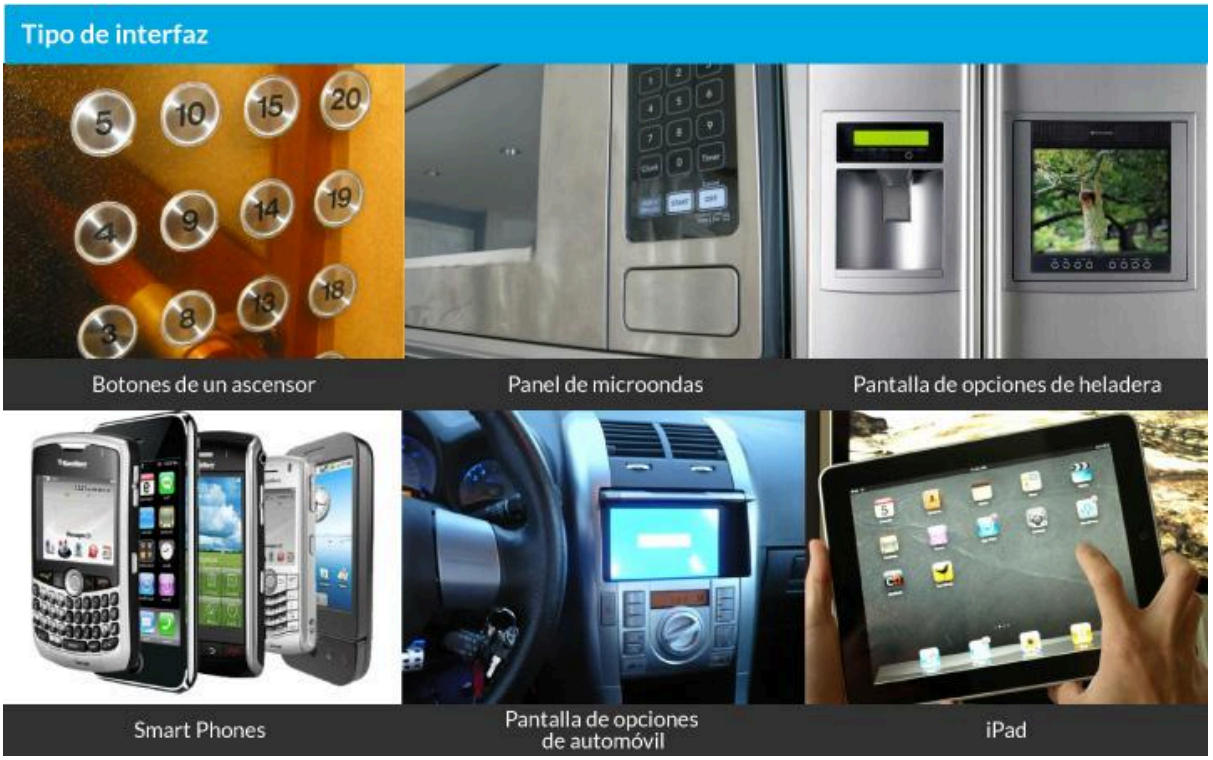


Link del video: https://www.youtube.com/watch?v=qFS8p8sGe94

Texto del video:

Interfaz como espacio: desde esta perspectiva la interfaz es el lugar de la interacción, el espacio donde se desarrollan los intercambios y sus manualidades. Se puede considerar el espacio donde se va a llevar a cabo la interacción. Por ejemplo, en la siguiente imagen, tiene como espacio la ubicación de un modelo, que interactúa con un software 3D de una computadora.

En las siguientes imágenes, podemos ver otros tipos de interfaz que comunican a las personas con un dispositivo electrónico:



Y muchos otros dispositivos... En nuestra situación profesional encontramos que el modelo de negocio que se presenta en el trabajo de **ONE S.A.**, debe llevarse a cabo mediante encuestas en la calle, además, en este modelo debemos gestionar la información recopilada. Para las encuestas en las calles, podemos utilizar interfaces como las **colectoras de datos específicos**, dispositivos como los **smartphones** o **tablas digitales (tablets)**. Para la gestión de información, podemos utilizar interfaces provistos por una computadora de escritorio o portátil.

Hasta aquí, hemos visto algunos de los ejemplos de interfaz, para ampliar la perspectiva de este concepto, debemos conocer la definición de **interfaz de usuario**.

Interfaz de usuario

Las interfaces de usuario son aquellas que incluyen menús, ventanas, teclado, mouse, algunos sonidos que la computadora realiza, en general, todos aquellos canales por los cuales se permite la comunicación entre el ser humano y la computadora. La mejor interacción humano-máquina es a través de una adecuada interfaz (**interfaz de usuario**), que le brinde comodidad y eficiencia en las tareas que se deban realizar. A su

vez, podemos encontrar distintos tipos de interfaces de usuarios (similares al concepto de interfaz desarrollado previamente). Estos tipos son:

- **Interfaz de hardware**, a nivel de los dispositivos utilizados para ingresar, procesar y entregar los datos: teclado, mouse y pantalla.
- **Interfaz de software**, destinada a entregar información acerca de los procesos y herramientas de control, a través de lo que el usuario observa habitualmente en la pantalla.
- **Interfaz de software-hardware**, que establece un puente entre la máquina y las personas, permite a la máquina captar la instrucción y al humano entender el código binario traducido a información legible.

Cómo interactuamos con una interfaz



Tipos de interfaces de usuario

- *Interfaces de hardware:* se trata de un conjunto de controles o dispositivos que permiten la interacción hombre-máquina, de modo que permiten introducir o leer datos del equipo, mediante pulsadores, reguladores e instrumentos. Por ejemplo: botones del ascensor.

- *Interfaces de software:* son programas o parte de ellos, que permiten expresar nuestros deseos a la computadora o visualizar su respuesta. Por ejemplo: programas que permiten manejar un smartphone, autogestión de alumnos, página de internet para compras de artículos, entre otros.

En conclusión, en una **interfaz de usuario**[2], la idea fundamental es la mediación entre hombre y máquina. La interfaz es lo que "media", lo que facilita la comunicación, la interacción, entre dos sistemas de diferente naturaleza, típicamente el ser humano y una máquina como las computadoras. Esto implica, además, que se trata de un sistema de traducción, ya que los dos "hablan" lenguajes diferentes: **verbo-icónico** en el caso del hombre y **binario** en el caso del procesador electrónico.

Sentidos involucrados

Para interactuar con las distintas interfaces, se pueden involucrar los principales sentidos de un ser humano, como:

[2] Interfaz de usuario: Se define como conjunto de componentes empleados por los usuarios para comunicarse con las computadoras. El usuario dirige el funcionamiento de la máquina mediante instrucciones, denominadas genéricamente "entradas". Las entradas se introducen mediante diversos dispositivos, por ejemplo un teclado, y se convierten en señales electrónicas que pueden ser procesadas por la computadora. Estas señales se transmiten a través de circuitos conocidos como bus, y son coordinadas y controladas por la unidad de proceso central y por un soporte lógico conocido como sistema operativo. Una vez que la UPC ha ejecutado las instrucciones indicadas por el usuario, puede comunicar los resultados mediante señales electrónicas, o salidas, que se transmiten por el bus a uno o más dispositivos de salida, por ejemplo una impresora o un monitor.



Link del video: https://www.youtube.com/watch?v=WIOW_dsHims

Texto del video:

Vista: principal sentido utilizado para interactuar con los diferentes dispositivos, especialmente para la salida de información. Existen dispositivos especiales para ciertas discapacidades, que pueden utilizarse para interactuar en las pantallas como puntero de mouse.

Tacto: la utilización de los dedos para interactuar con dispositivos en la entrada de datos es muy utilizada. Aquellas personas con problemas visuales pueden encontrar dispositivos como este.

Olfato: se siguen desarrollando métodos para digitalizar aromas que puedan ser percibidos por los usuarios de computadoras.

Movilidad psicomotriz: las consolas de videojuegos son aquellas que iniciaron la mayor interacción del cuerpo humano con un dispositivo electrónico. El que generó mayor sensación es la Nintendo Wii, luego sus sucesores Playstation Move, Xbox Kinect y lo que depare el futuro. Las últimas consolas toman como dispositivo o mando de control, todo el cuerpo.

Evolución de las interfaces

No todas las interfaces que comunican a un usuario con una computadora han sido tan sofisticadas como las que encontramos actualmente. Para ello veremos una breve reseña de la evolución en las interfaces computacionales.

Command-Line Interface (CLI) - Interfaces de línea de comandos

Encontraremos el sistema operativo **DOS**[3], el **sistema operativo**[4] de las primeras computadoras, que es el estilo más antiguo de interacción hombre-máquina. El usuario escribe órdenes utilizando un lenguaje formal con un vocabulario y una sintaxis propia (los mandatos en el caso del DOS). Se usa un teclado y las órdenes están encaminadas a realizar una acción.



Link del video: https://www.youtube.com/watch?v=8ZfhTasvfD8

Texto del video:

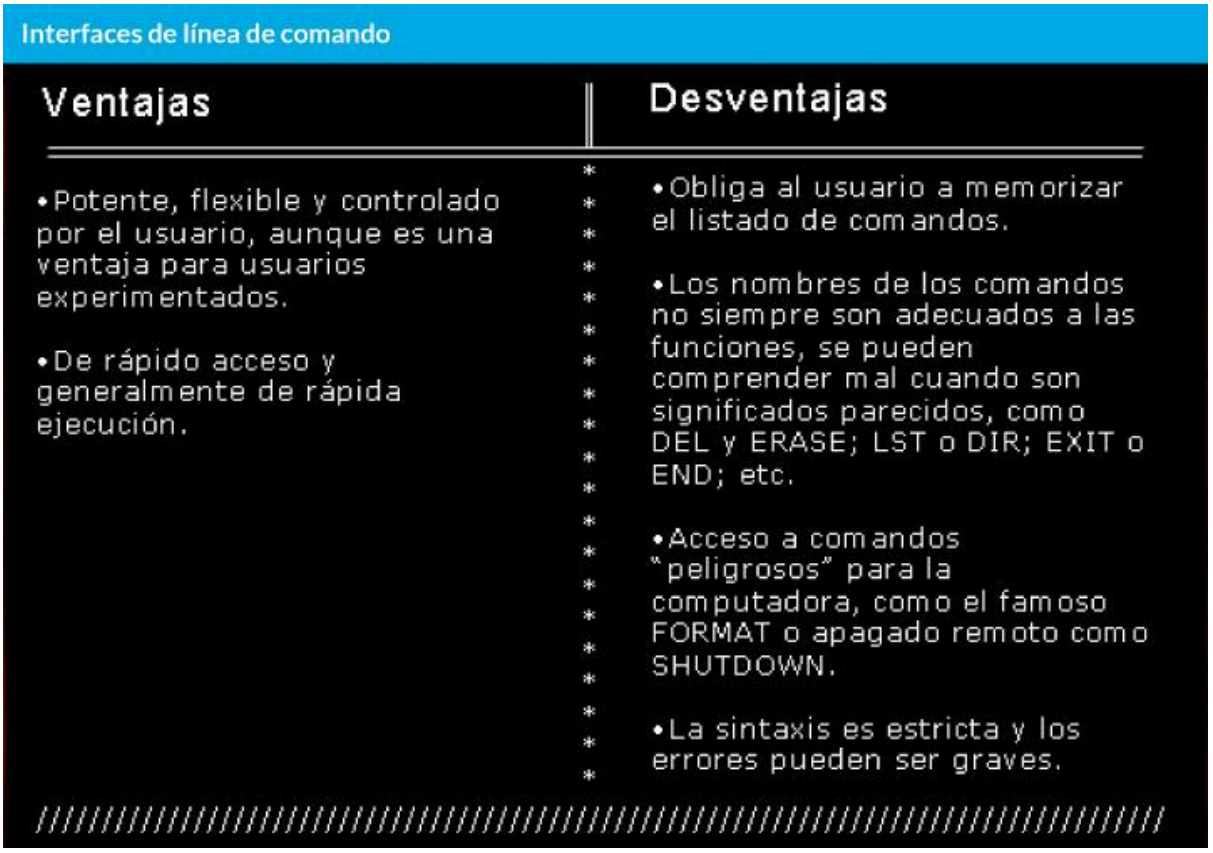
Por ejemplo, si tiene un sistema operativo Microsoft puede ir al botón INICIO, Ejecutar: Escribir cmd y aceptar. Escribir la palabra dir luego Enter. Nos listará el directorio. Escribir la palabra ipconfig, luego Enter.Nos listará el directorio, para finalizar escribimos exit y luego Enter.

El usuario no recibe mucha información del sistema, y debe conocer cómo funciona la computadora y dónde están los directorios (*carpetas*) de los programas, es decir debe conocer el amplio listado de comandos para poder acceder a las facilidades de la computadora. El modelo de la interfaz es del programador, no de un usuario común,

[3] DOS: Las siglas significan Disk Operating System, Sistema Operativo de Disco. Fue creado para computadoras de la familia IBM PC, que utilizaban los procesadores Intel 8086 y 8088, de 16 bits, siendo el primer sistema operativo popular para esta plataforma.

[4] SO – Sistema Operativo: Software base que permite que interactúe el hardware con el usuario.

debido a la complejidad de comprensión necesaria. Cabe destacar que todos los comandos ejecutados son cargados en memoria **RAM**[5] para su procesamiento.



Graphical User Interfaces (GUI) - Interfaces gráficas de usuarios

Las interfaces gráficas de usuarios se abren paso para facilitar el uso de una computadora personal o computadora al público en general, y no necesariamente conocer comandos o interactuar tan "planamente" con nuestro equipo.

El diseño gráfico de cada pantalla en un sistema operativo, nos permite acceder a distintas utilidades y facilidades de los mismos. De esta forma, el usuario conoce que un pequeño gráfico con un texto (*ícono*), llamado "Documentos" o "Imágenes", puede contener archivos de uso diario y nuestras imágenes. Los medios gráficos facilitan la

[5] RAM: La memoria de acceso aleatorio (Random Access Memory, RAM) se utiliza como memoria de trabajo de computadoras y otros dispositivos para el sistema operativo, los programas y la mayor parte del software. En la RAM se cargan todas las instrucciones que ejecuta la unidad central de procesamiento (CPU) y otras unidades del computador, además de contener los datos que manipulan los distintos programas. Se denominan «de acceso aleatorio» porque se puede leer o escribir en una posición de memoria con un tiempo de espera igual para cualquier posición, no siendo necesario seguir un orden para acceder (acceso secuencial) a la información de la manera más rápida posible.

comprensión del usuario promedio para acceder a las distintas funciones de la computadora.

Reseña histórica

Los tres estilos más comunes de interfaces gráficas hombre-computadora son:

- Lo que tú ves es lo que puedes conseguir (WYSIWYG -What you see is what you get-)[6].
- Manipulación directa.
- Interfaces de usuario basadas en íconos.

Una **GUI** es una representación gráfica en la pantalla de la computadora basada en los programas, datos y objetos, así como de la interacción con ellos. Una GUI proporciona al usuario las herramientas para realizar sus operaciones.

Características de una GUI:

- Posee un dispositivo indicador / apuntador / control.
- Promueve la consistencia de la interfaz entre programas.
- Los usuarios pueden ver en la pantalla los gráficos y textos tal como se verán impresos.
- Sigue el **paradigma**[7] de la **interacción**[8] objeto-acción.
- Permite la transferencia de información entre programas.
- Se pueden manipular en la pantalla directamente los objetos y la información.

[6] What You See Is What You Get (en español, "lo que ves es lo que obtienes"), es una frase aplicada a los procesadores de texto y otros editores de texto con formato que permiten escribir un documento mostrando directamente el resultado final, frecuentemente el resultado impreso.

[7] Interfaz de usuario: Se define como conjunto de componentes empleados por los usuarios para comunicarse con las computadoras. El usuario dirige el funcionamiento de la máquina mediante instrucciones, denominadas genéricamente entradas. Las entradas se introducen mediante diversos dispositivos, por ejemplo un teclado, y se convierten en señales electrónicas que pueden ser procesadas por la computadora. Estas señales se transmiten a través de circuitos conocidos como bus, y son coordinadas y controladas por la unidad de proceso central y por un soporte lógico conocido como sistema operativo. Una vez que la UPC ha ejecutado las instrucciones indicadas por el usuario, puede comunicar los resultados mediante señales electrónicas, o salidas, que se transmiten por el bus a uno o más dispositivos de salida, por ejemplo una impresora o un monitor.

[8] Interacción: Se produce de acuerdo con ciertos paradigmas interactivos, los actuales son: - La computadora de mesa. - La realidad virtual. - La computación ubicua. - La realidad aumentada.

- Provee elementos de interfaz estándar como menús y diálogos.
- Existe una muestra visual de la información y los objetos (íconos y ventanas).
- Proporciona respuesta visual a las acciones del usuario.
- Existe información visual de las acciones y modos del usuario/sistema (*menús*, *paletas*).
- Existen controles gráficos (*widgets*) para la selección e introducción de la información.
- Permite a los usuarios personalizar la interfaz y las interacciones.
- Proporciona flexibilidad en el uso de dispositivos de entrada (*teclado/mouse*).
- Permite manipular los objetos e información de la pantalla, no solo presentarla.

Para usar un GUI, los usuarios deben conocer (o aprender) una serie de conceptos: organización del sistema (archivos, carpetas), diferentes tipos de íconos y efectos de las acciones sobre ellos, elementos básicos de una ventana, uso de los controles del GUI, uso del mouse, etc.

Los GUI usan el estilo **objeto-acción**, en contraposición a la **acción-objeto** de los CUI o las **interfaces de menú**[9]. El usuario selecciona un objeto, y después la acción a realizar sobre dicho objeto. Los objetos son el principal foco de atención del usuario, lo cual resulta más natural y próximo a su modelo mental.

El módulo de programación que tenemos designado, debe ser programado en interfaces gráficas, ya que esto cumplirá con los requisitos que la organización dispuso en este trabajo. Los medios gráficos nos van a permitir mayor interpretación y manejo sencillo del modelo de negocio que debemos desarrollar.

Hasta aquí, conocemos las diferentes interfaces de usuario que se pueden presentar en un sistema informático. Actualmente las interfaces gráficas son avanzadas donde podemos tener acceso, comprensión a mucha información y actividades de negocio.

[9] Interfaces de menú: Un menú es una lista de opciones que se muestran en la pantalla o en una ventana de la pantalla para que los usuarios elijan la opción que deseen. Los menús permiten dos cosas: navegar dentro de un sistema, presentando rutas que llevan de un sitio a otro, y seleccionar elementos de una lista, que representan propiedades o acciones que los usuarios desean realizar sobre algún objeto. Las interfaces de menús aparecen cuando la computadora se vuelve una herramienta de usuario y no sólo de programadores. Las actuales interfaces gráficas u orientadas a objetos siguen utilizando este tipo de interfaces (los distintos estilos de interfaces no son mutuamente excluyentes).

Características humanas del diseño de interfaz

Factores humanos

Al diseñar interfaces de usuario debemos tener en cuenta las **habilidades cognitivas**[10] y de percepción de las personas, y adaptar el programa a ellas. Una de las cosas más importantes que una interfaz puede hacer es reducir la dependencia de las personas de su propia memoria, librándolos de recordar cosas innecesariamente, por ejemplo: información que apareció en una pantalla anterior, o a repetir operaciones ya realizadas, por ejemplo: introducir un mismo dato repetidas veces.

Consideraciones que debemos tener en cuenta:

Velocidad de aprendizaje(link): se pretende que la persona aprenda a usar el sistema lo más pronto posible.

Velocidad de respuesta: el tiempo necesario para realizar una operación en el sistema.

Tasa de errores: porcentaje de errores que comete el usuario.

Retención: cuánto recuerda el usuario sobre el uso del sistema en un período de tiempo.

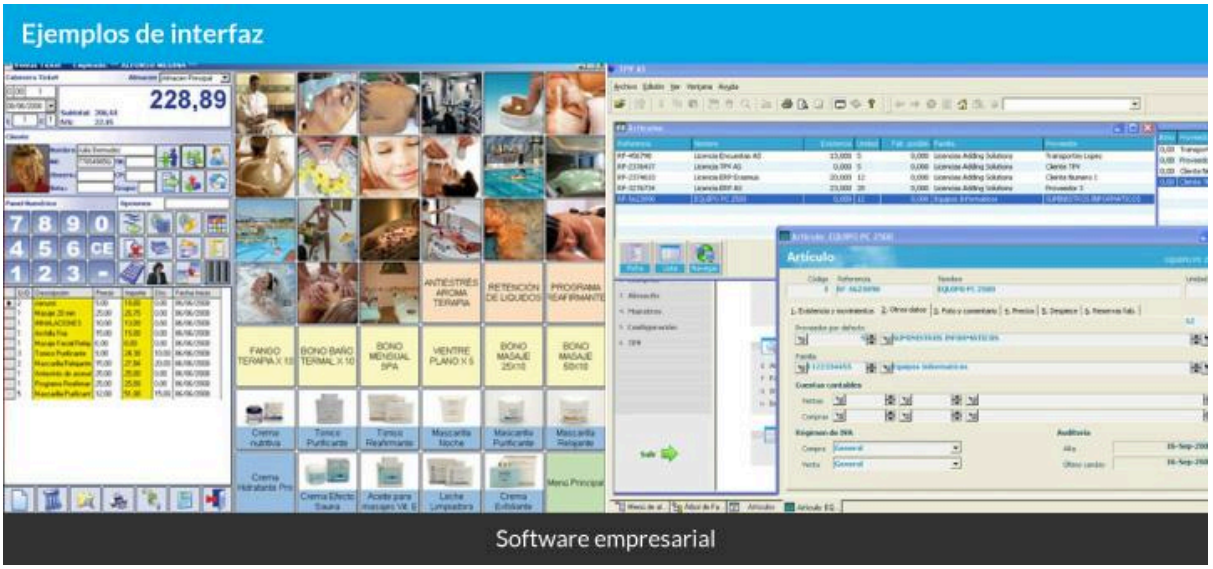
Satisfacción: se refiere a que el usuario esté a gusto con el sistema.

Observe los siguientes ejemplos, en un ámbito educativo y en un ámbito comercial identifique las características que puede detallar de cada una; puede contemplar mejor los ejemplos en las páginas web de la fuente:

[10] Habilidades cognitivas: El concepto de cognición (del latín: cognoscere, "conocer") hace referencia a la facultad de los seres de procesar información a partir de la percepción, el conocimiento adquirido (experiencia) y características subjetivas que permiten valorar la información.



Fuente: http://querecursos.com - http://www.clubpenguin.com



Fuente: http://www.amsystem.es - http://www.addingsolutions.com

Adecuación

- **Características físicas:** cada persona tiene diferentes características y disposiciones físicas. A algunas personas que no les gustan utilizar tanto el teclado mientras que a otras sí, es por eso que hay teclados ergonómicos. Lo mismo sucede con el mouse.



Fuente: Microsoft.com

- **Ambiente:** el lugar donde va a ser usado el sistema. Cada interfaz tiene que adecuarse al lugar.



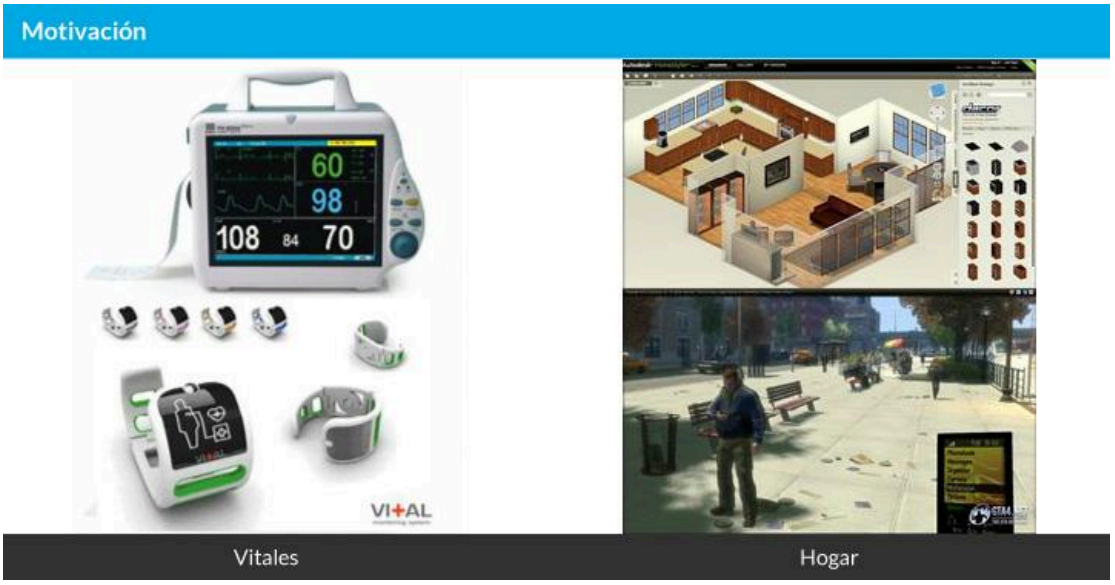
Fuente: Seedmech.com - Taobao.com

- **Visibilidad:** Tomar en cuenta la cantidad de iluminación del lugar. Por ejemplo: ¿se refleja el brillo en la pantalla?
- **Personalidad:** de acuerdo a la edad, nivel socio-económico, etcétera (ver ejemplos de software empresarial y educativo expuesto anteriormente).
- **Cultura:** en Egipto no tienen las mismas pantallas, ventanas, etcétera. Este factor es importante si la intención es vender un sistema a nivel internacional (ver ejemplos de software empresarial y educativo expuesto anteriormente).

Según la función

Motivación:

- **Sistemas vitales:** son de vida o muerte; muchas personas dependen de ellos. Ejemplo: un sistema para reactores nucleares. Este sistema trabaja en tiempo real, y es de suma importancia la seguridad y efectividad del mismo.
- **Sistemas comerciales e industriales:** sirven para aumentar la productividad y vender más.
- **Sistemas de oficina, hogar y juegos:** un factor importante es el mercado a quién está dirigido; tienen que ser muy amigables y satisfacer al cliente.
- **Sistemas de investigación:** realizan tareas muy específicas y tratan de imitar el medio en el que se desenvuelve el usuario.



Podemos analizar las siguientes funciones del módulo de **ONE S.A.:**

Ambiente:

- Recolección de datos: calle.
- Gestión de información: oficina.

Visibilidad:

- Recolección de datos: luz ambiente (sol).
- Gestión de información: luz tubos fluorescentes.

Cultura y personalidad:

- Recolección de datos: encuestador, educación media, 20 a 30 años.

- Gestión de información: administrador, educación universitaria, 30 a 50 años.

Facilidad de uso:

- Recolección de datos: recolectora de datos.
- Gestión de información: computadora.

Aprendizaje:

- Recolección de datos: debe ser una interfaz de fácil uso, y que solo sea utilizada para encuesta.
- Gestión de información: debe ser una interfaz no muy compleja, con datos específicos y funciones complejas al nivel del administrador.

Hasta aquí, vimos las características humanas que debemos tener en cuenta para la confección o desarrollo de interfaces de usuario. Tenemos que tener muy en claro a qué grupo de personas estará dirigido nuestro desarrollo de interfaces. Lo invitamos a investigar sobre las diferentes aplicaciones que facilitan la vida de personas que no cuentan con todos sus sentidos, ya que son interfaces ingeniosas y llenas de funcionalidad.

Pasos para el diseño de interfaz

En el proceso de diseño de una interfaz de usuario se pueden encontrar varias técnicas, a saber:

Cuatro fases o pasos fundamentales	
Reunir y analizar la información del usuario	Ya hemos reunido la información del usuario. Continuaremos con los siguientes puntos.
Diseñar la interfaz de usuario	Es importante dedicar tiempo y recursos a esta fase, antes de entrar en la codificación. En esta fase se definen los objetivos de usabilidad del programa, las tareas del usuario, los objetos y acciones de la interfaz, los iconos, vistas y representaciones visuales de los objetos, los menús de los objetos y ventanas. Todos los elementos visuales se pueden diseñar dibujándolas y luego ver la aprobación del usuario sobre las mismas.
Construir la interfaz de usuario	Es interesante realizar un prototipo previo, una primera versión del programa que se realice rápidamente y permita visualizar el producto para poderlo probar antes de codificarlo definitivamente.
Validar la interfaz de usuario	Se deben realizar pruebas de usabilidad del producto, de ser posible con los propios usuarios finales del mismo.

Veamos los nueve pasos en el proceso de diseño "centrado en las tareas":

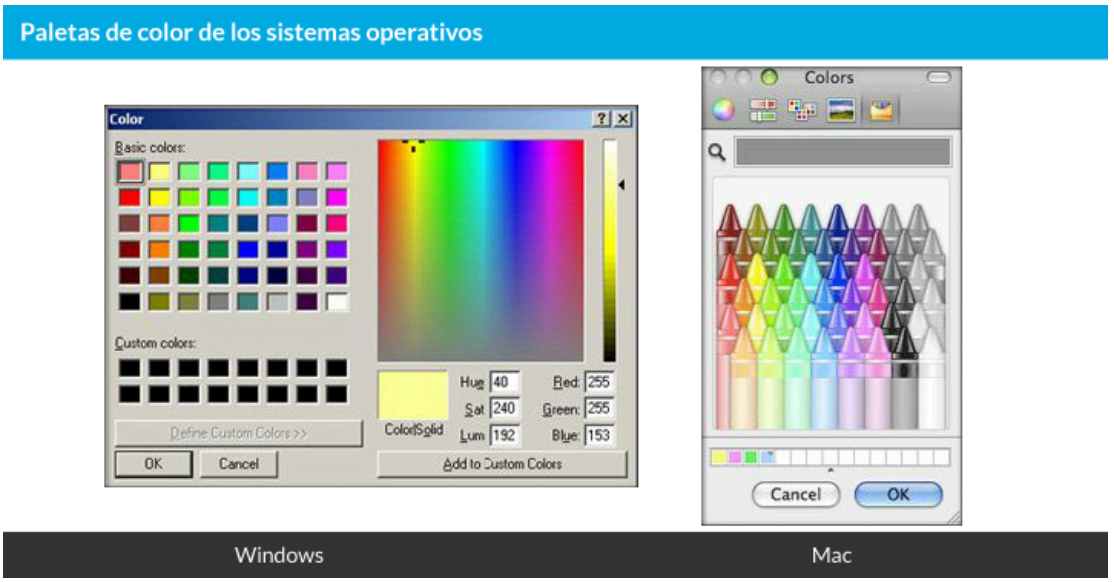
1. Entender quién usará el sistema para hacer qué.
2. Elegir tareas representativas para el diseño.
3. Bosquejar un diseño.
4. Pensar acerca del diseño.
5. Crear un prototipo.
6. Evaluarlo con los usuarios.
7. Construirlo.
8. Testearlo.
9. Cambiarlo / modificarlo.

Otras técnicas

- **Presentación de la información:** no se deben colocar demasiados objetos en la pantalla, y los que existan deben estar bien distribuidos. Cada elemento visual influye en el usuario no sólo por sí mismo, sino también por su combinación con el resto de elementos presentes en la pantalla.

Elementos de diseño de pantalla y su percepción visual

- **Análisis Color:** Es probablemente el elemento de la interfaz que con más frecuencia es mal utilizado. El color comunica información, no es sólo decorativo (ejemplo: reforzar mensajes de error). Deben utilizarse combinaciones adecuadas (por ejemplo, las paletas proporcionadas por los sistemas operativos). El color debe atraer la atención, pero no cansar después de un tiempo de trabajo. Es especialmente importante seguir las líneas de diseño existentes.



- **Análisis Audio:** primero es preciso ver cuándo es más apropiado que la información visual. Segundo, determinar el sonido adecuado. Tercero, permitir la personalización (volumen y desactivación). Como en el caso de los colores existen guías de uso. En lugares de trabajo abiertos, puede ser poco efectivo; además, puede ser embarazoso para algunas personas. El sonido debe usarse para informar, no cuando no añade nada nuevo (por ejemplo, un mensaje de aviso de correo o de bienvenida, respectivamente, al iniciar una sesión de trabajo).

- **Análisis Animación:** se define como un cambio en el tiempo de la apariencia visual de un elemento gráfico. Ejemplos de su uso: progreso de acciones (copia de ficheros en Windows, instalación de programas), estado de procesos (íconos

de impresora), acciones posibles (cambios en el cursor al desplazar el mouse). La animación puede ayudar a subrayar íconos importantes, mostrar el estado de un objeto particular o explicar su comportamiento.



Link del video: https://www.youtube.com/watch?v=8MUkciu5roA

- **Diseño internacional:** debe hacerse un uso adecuado de la terminología. Hay mucho trabajo en este campo. Debe tenerse cuidado con las diferencias culturales (gestos, terminología, dibujos, formatos de teléfonos o calendarios, etc.).

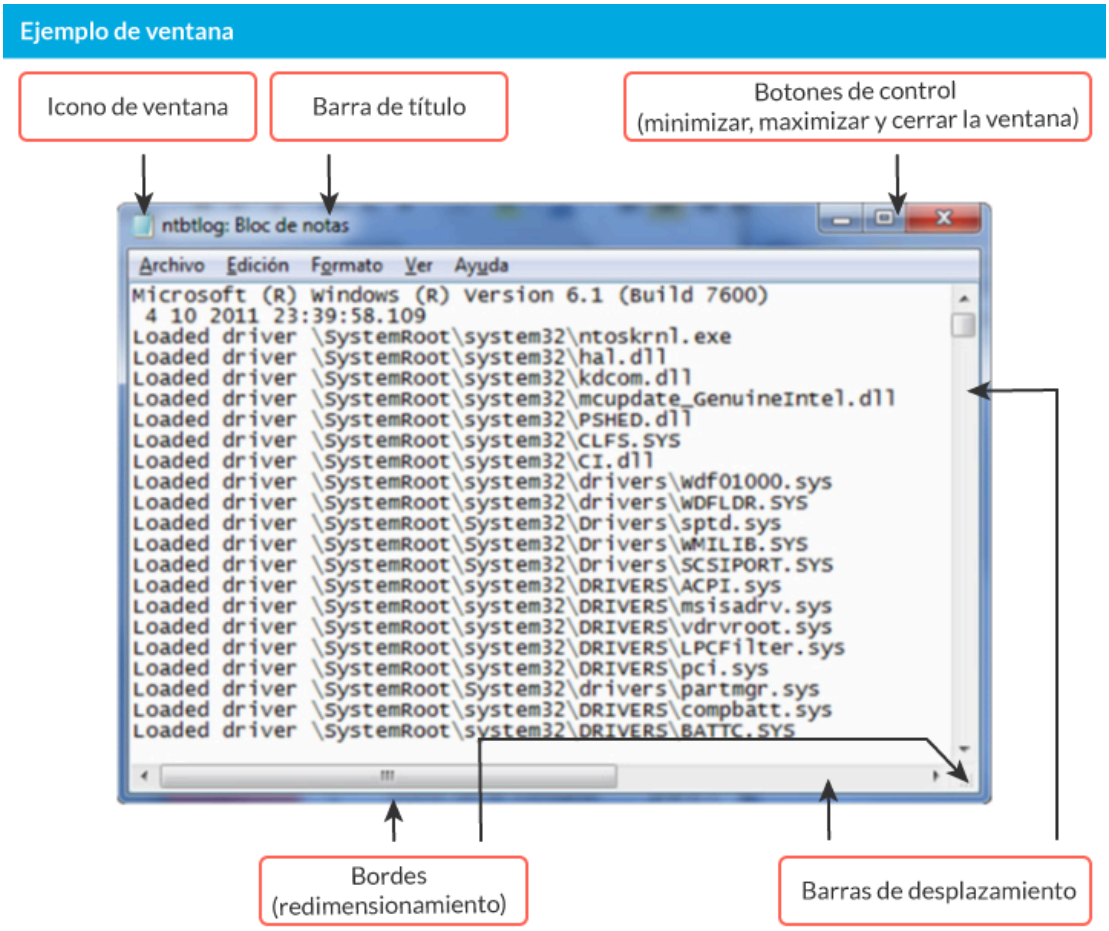


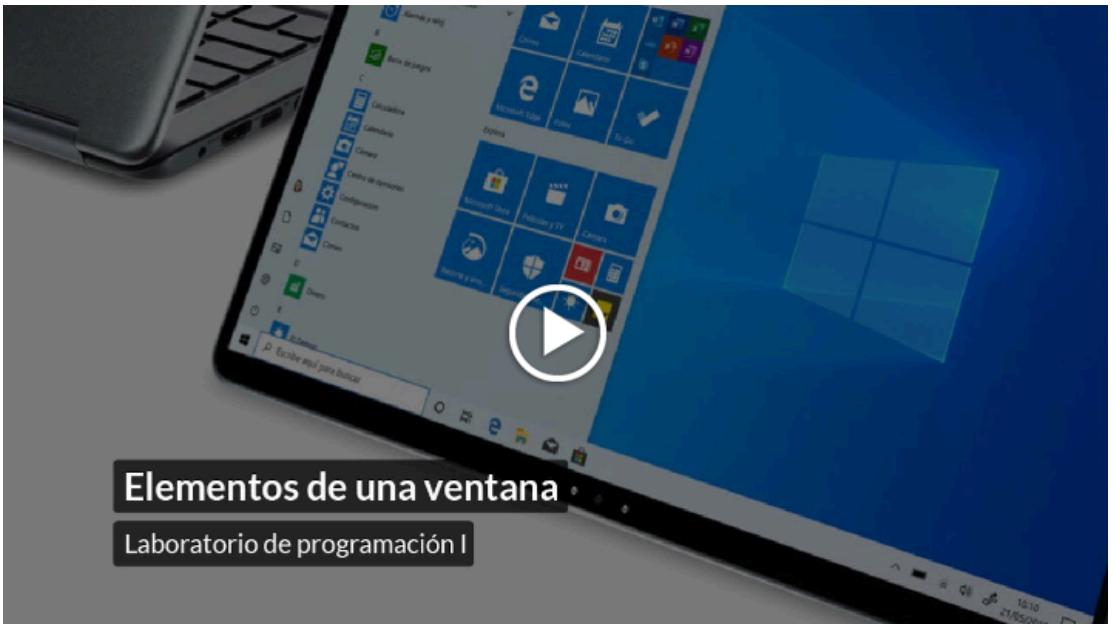
Componentes básicos de un sistema de Interfaz GUI

Para resolver nuestra situación profesional deberemos conocer los distintos componentes del GUI para usarlos de manera adecuada, a fin de poder ofrecer al usuario una interfaz con las características ya señaladas hasta ahora. Todo entorno GUI se basa en el uso de Ventanas, Íconos, un área de trabajo o Escritorio, y por supuesto, un dispositivo señalador (el mouse).

- **Escritorio:** es el "fondo" del ambiente GUI. Se denomina así porque es una abstracción de lo que representa un escritorio en el mundo real. Sobre él se despliegan los objetos (herramientas, recursos, etcétera) que utilizamos frecuentemente. Sirve de contenedor para todos los objetos y componentes del entorno GUI. La mayoría de los ambientes ofrecen la posibilidad de colocar sobre el escritorio alguna imagen o "papel tapiz", dándole mayor atractivo gráfico.

- **Ventana**: es el área que ocupa un programa para desplegar todos sus componentes de interfaz con el usuario. Normalmente suele ser rectangular y suele disponer de ciertos controles implícitos para controlar su apariencia, tamaño y ubicación.





Link del video: https://www.youtube.com/watch?v=wM3GixBepdg

Texto del video:

No todas las ventanas disponen de los elementos señalados. Como siempre, todo depende de la aplicación particular de la ventana. Por ejemplo, veamos la ventana de la Calculadora de Windows.

Evidentemente, no tendría sentido redimensionarla ni permitir sobre ella funciones como la de "maximizar". Si se hace más pequeña, quedarán botones sin verse, y si se agranda, no hay ningún otro control por mostrar más que los ya presentes en su tamaño inicial. Incluso hay ventanas, como esta otra, que ni siquiera disponen de barra de título. Suelen ser ventanas con las que el usuario interactúa bastante poco. Su uso está reservado para mensajes de sistema.

Muchos programas actuales están apostando a estilos de interfaces de usuario no convencionales, con bordes curvilíneos, controles GUI de diseño propio, etcétera. Pero, ante todo, un desarrollo sumamente prolijo e intuitivo. Si no, ¡no sirve!

Estas dos imágenes corresponden a un programa de audio/video. Es un reproductor de archivos de audio en varios formatos. Dispone, entre otras características, de la posibilidad de que el usuario elija qué interfaz desea que el programa presente. Los diseños son de los más variados y se caracteriza por disponer de controles GUI de desarrollo propio.

Controles GUI de desarrollo propio



Modelos estándar de interfaces y componentes

Existen diferentes modelos generales de ventanas, básicamente tres:

- Modelo SDI (Single Document Interface)
- Modelo MDI (Múltiple Document Interface)

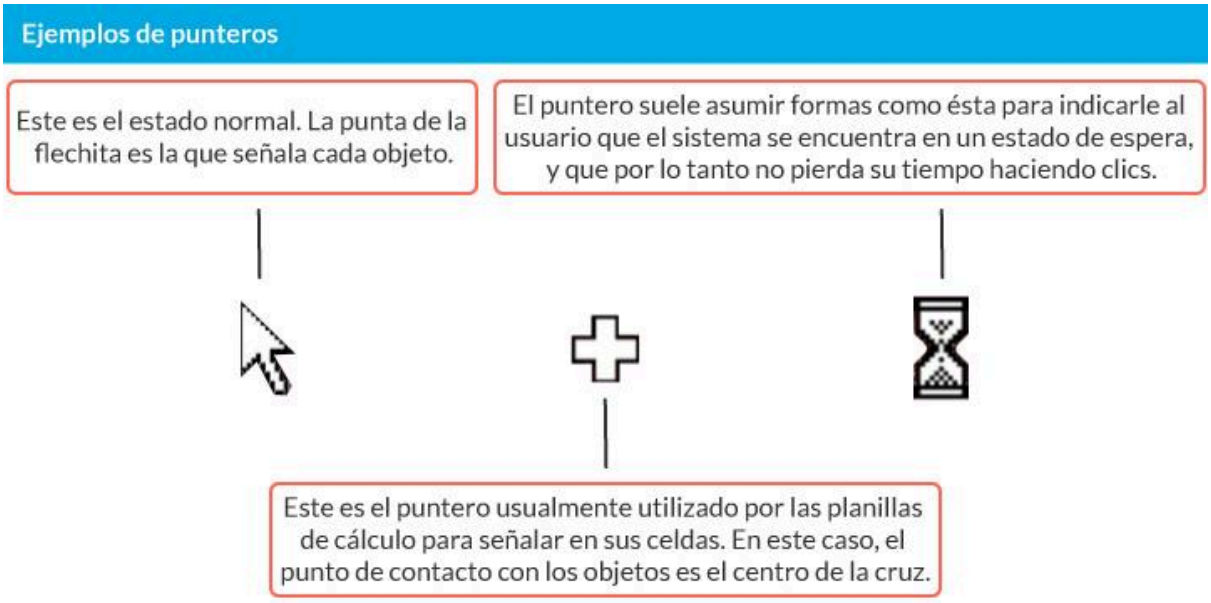
• Modelo de Ventana de Explorador

- **Ícono:** es una representación gráfica de reducidas dimensiones, normalmente con un área cuadrada de no más de un par de centímetros de lado, que caracteriza gráficamente un objeto o recurso en los ambientes GUI. Existen infinidad de tamaños y estilos, y su creación es tarea específica de los diseñadores gráficos. Los hay planos y con efecto de relieve, en blanco y negro, escala de grises y a colores, pequeños, medianos y grandes, etcétera. Éstos son algunos ejemplos:

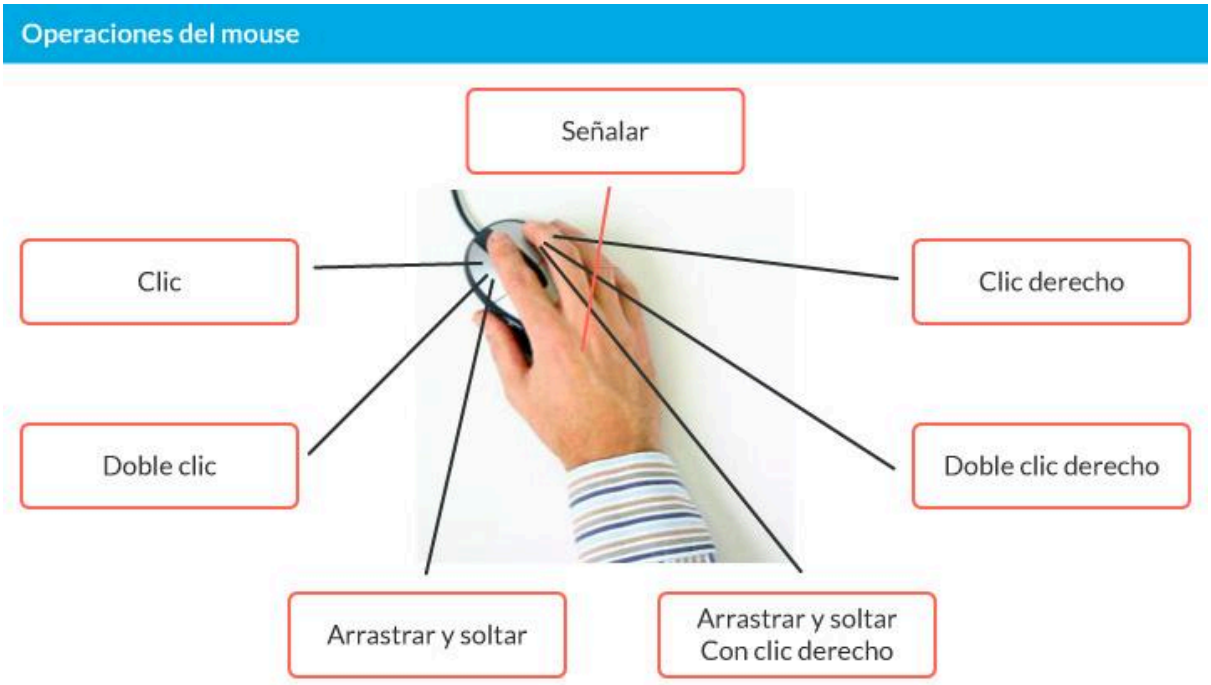


- **Puntero del mouse**: como mencionamos anteriormente, el "mouse" (o cualquier otro dispositivo señalador como Trackball, Lápiz Óptico, Touch Pad, etc.) es nuestra "mano virtual" dentro del escritorio GUI. Siempre tiene uno y sólo un punto de contacto, que es la parte del puntero que virtualmente "toca" a los objetos cuando el usuario hace click. Por ello, el rol del puntero que manejamos con él es importantísimo.

Las diferentes formas del puntero nos dan información relativa sobre qué es lo que nuestra mano virtual está en capacidad de realizar a cada momento. También nos brinda información respecto del estado del sistema. Veamos algunos ejemplos:



Algunas de las operaciones fundamentales que se pueden realizar con el mouse son las siguientes:



- **Señalar:**
  Mover el mouse hasta que el punto de contacto del puntero se ubique encima de un objeto.

- **Clic:**
  Consiste en, una vez señalado un objeto, pulsar y soltar rápidamente el botón principal (izquierdo).

- **Doble clic:**
  Son dos clics seguidos, bien rápidos y sin mover el mouse entre uno y otro. Si el mouse se mueve o si la velocidad con que se realiza no es la suficiente, el sistema interpretará la acción como "dos eventos Clic" y no un evento "Doble Clic".

- **Clic derecho:**
  Su nombre lo indica, es un clic sobre un objeto, pero con el botón secundario o alternativo del mouse (normalmente el derecho, excepto en los mouse para zurdos).

- **Doble clic derecho:**
  Es muy extraño encontrar sistemas que tengan programadas tareas para un doble clic derecho, esta operación prácticamente no se utiliza nunca, aunque es sin duda posible.

- **Arrastrar y soltar:**
  Esta es una acción sumamente común con el mouse. Consiste en apuntar un objeto, pulsar sobre él el botón principal y sin soltarlo desplazar el mouse, con lo que se produce un efecto gráfico como si estuviera llevándose consigo el objeto en cuestión (se va pegado al puntero del mouse). Finalmente, una vez que el objeto arrastrado se encuentra encima de su destino, suelta el botón, con lo que se produce el efecto gráfico como si lo "soltara". Probablemente esta es la más concreta de las analogías entre el mouse y las manos de los seres humanos, con las que habitualmente movemos y acomodamos los objetos que tenemos en nuestro entorno real (o virtual). Esta acción es, por ejemplo, la que llevamos a cabo cuando queremos deshacernos de un documento y lo "tiramos" a la papelera de reciclaje, disponible en casi todos los ambientes GUI.

- **Arrastrar y soltar con el botón derecho:**
  Es exactamente idéntica a la operación anterior, sólo que su efecto es diferente. Normalmente esta acción despliega un menú de opciones sobre el objeto arrastrado, dependiendo de en qué lugar se lo haya soltado, preguntando al usuario qué es exactamente lo que quiere hacer.
  Por ejemplo, si arrastramos y soltamos con el botón derecho del mouse un archivo de una carpeta a otra en Windows, al soltarlo aparecerá un menú emergente preguntándonos qué queremos hacer con ese archivo: Moverlo de la carpeta origen a la de destino del movimiento, o si queremos copiarlo, o crear un acceso directo, etc.

Algunos entornos, como el de Apple Macintosh, disponen de un solo botón en el mouse, por lo que la operación de click derecho no es posible.

En cambio estos sistemas realizan el equivalente a tal tarea combinando la presión de alguna tecla especial en el teclado, al mismo tiempo que se hace click con el único botón del mouse.

¿Ventanas o Cuadros de diálogos?

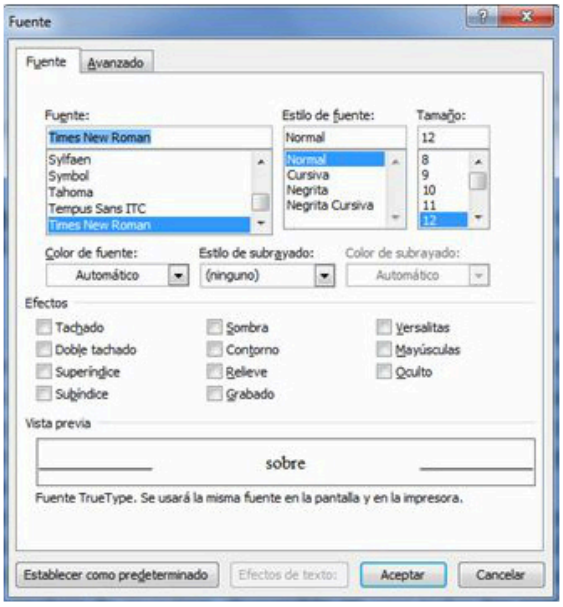
Normalmente una ventana le permite al usuario trabajar en algo concreto. Por ejemplo la ventana de un editor de texto le permite desarrollar su texto, configurar su aspecto, alineación, tipos y estilos de letra, etc. La ventana de la calculadora le permite operarla para realizar los cálculos que quiera.

No obstante, en muchos programas de aplicación se requiere de la interacción concreta del usuario para responder una pregunta al programa, o para configurar algún componente o aspecto particular. Es entonces que los programas despliegan una ventana, normalmente más pequeña que la ventana del programa propiamente dicha. Esta ventana, subordinada a la anterior, se denomina genéricamente "Cuadro de diálogo".

Un cuadro de diálogo tiene ciertas características particulares: normalmente no son redimensionales (su tamaño es fijo); por ende, suelen no disponer de barras de desplazamiento. Contienen, en su mayoría, controles GUI estándar. Aparecen encima de la ventana de la aplicación que los dispara y no le permiten al usuario volver a la ventana "madre" sin responder o cerrar debidamente el Cuadro de diálogo (esta característica se denomina "modalidad", por eso se dice que son "ventanas modales").

Veamos un ejemplo sobre el editor de textos Microsoft Word. Si usted está escribiendo una carta y toma la opción del menú Formato → Párrafo, emergerá encima del documento que está editando el Cuadro de diálogo de "Párrafo" (tecla rápida Ctrl+M en cualquier versión de Office).

Cuadro de diálogo



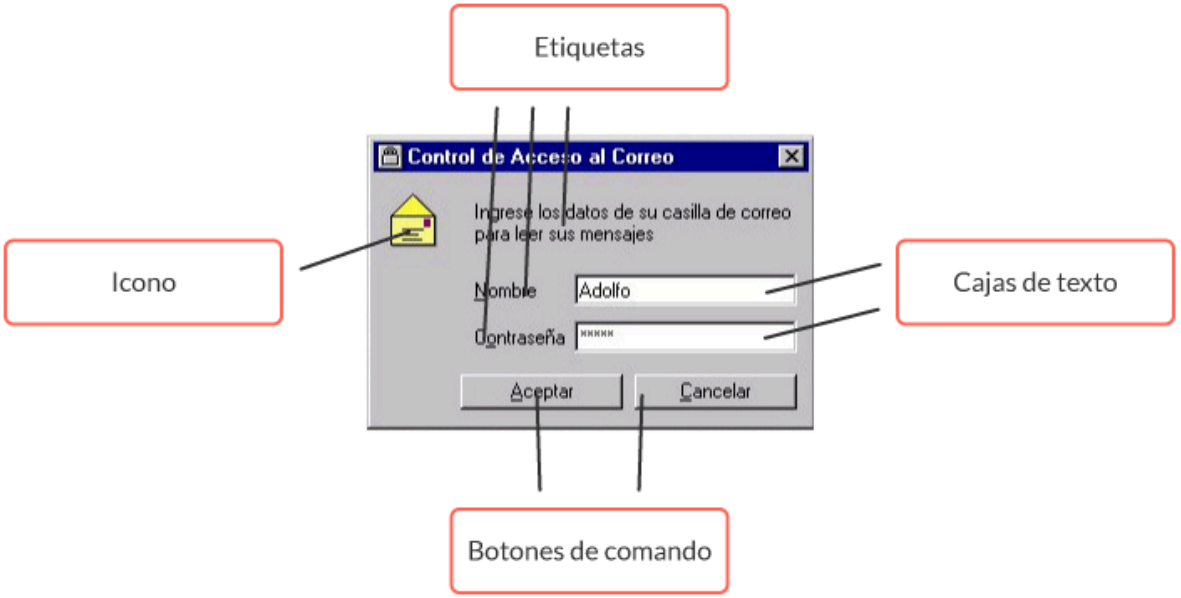
Hasta que no cierre este cuadro de diálogo, no podrá continuar la edición de su documento.

Para trabajar en el diseño de Interfaces de Usuario GUI, particularmente la de los cuadros de diálogo, necesitaremos conocer en detalle todos y cada uno de los controles GUI de uso estándar, sin dejar de considerar todas las reglas de diseño y técnicas GUI hasta aquí analizadas.

Algunos controles GUI estándar

Considérense inicialmente los siguientes controles GUI estándar:

Controles GUI estándar



- **Ícono:**
  Su utilización como "cara" de los botones de comando y barras de herramientas ya ha sido referida. Sirven además para utilizarlos como elementos estético-semánticos en los cuadros de diálogos, para que el Usuario, al verlo, sepa el carácter de la ventana emergente o el tipo de situación general que la misma representa.

- **Etiquetas:**
  Sirven para indicar rótulos con el nombre para los datos solicitados al usuario (ver etiquetas "Nombre" y "Contraseña"). Sirven también para indicar al usuario una situación en particular, brindar instrucciones de los pasos a seguir, establecer una pregunta, etc. Es como la "voz" de la ventana o cuadro de diálogo que le habla al usuario.

- **Cajas de texto:**
  Se utilizan para que el usuario pueda ingresar allí datos a través del teclado. Estos datos pueden ser del tipo textual (un nombre), numérico (un importe), de fecha, etc. Algunas características particulares de las cajas de texto:

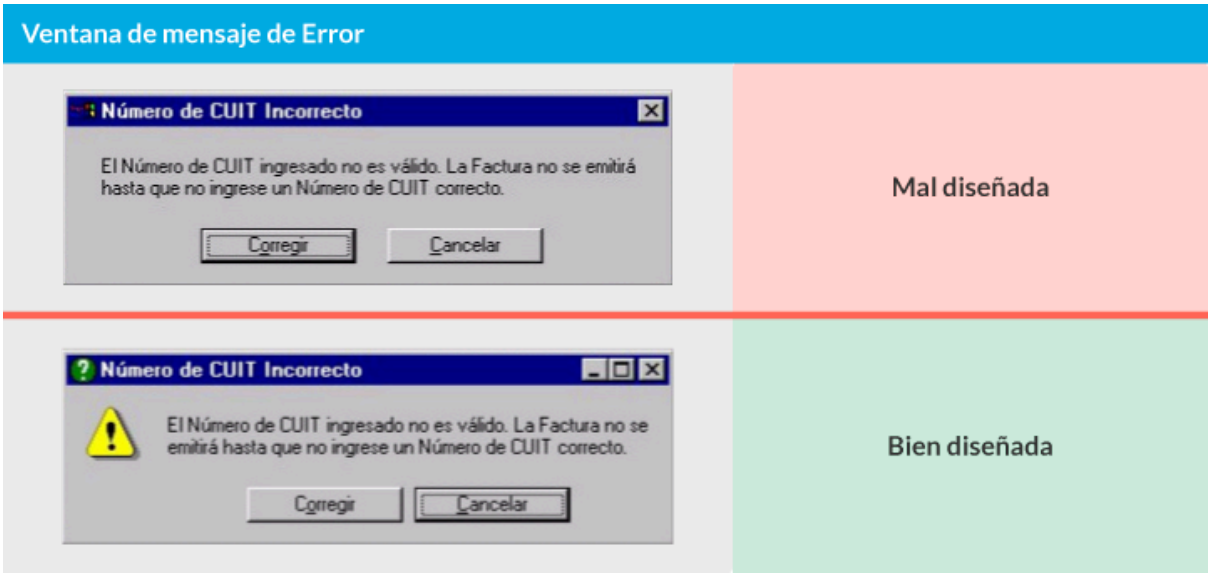
    - Se puede establecer una longitud máxima para el texto que en ella se introduzca.

    - Se puede controlar que sólo acepte ciertos caracteres (por ejemplo, sólo números si estoy solicitando el ingreso de una edad).

- Se puede usar para ingresar contraseñas (la caja de texto no mostrará los caracteres ingresados, sino que en su reemplazo presentará asteriscos para que si la pantalla está siendo vista por alguien en ese momento no vea la contraseña).

- **Botones de comandos:**
  Sirven para que el usuario los presione (clic) cuando quiera dar una orden particular al sistema. Esa orden debe estar claramente indicada en el botón, ya sea utilizando como identificador sobre su superficie un título de texto o un icono.

Veamos un ejemplo de Ventana de Mensaje de error:



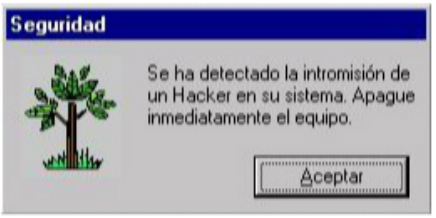
La segunda ventana no sólo es más agradable visualmente al usuario, sino que éste, al verla, inmediatamente reconoce la situación de que se ha presentado algún problema y no puede continuar hasta no resolverlo. No necesita leer el texto completo del cuadro de dialogo para saber eso.

De todas formas, lo leerá (aunque no siempre) para enterarse sobre los detalles en cuanto a cuál ha sido el problema.

El rol semántico (el significado que los usuarios le otorgan) de los íconos es fundamental en el diseño de interfaces GUI. No es cuestión de poner íconos por todos lados porque sí, porque nos gustan. Lejos de beneficiar al usuario, lo perjudicaríamos. Veamos el siguiente ejemplo:

Rol semántico, ejemplos





Ante la aparición de cualquiera de estos dos cuadros de diálogo, el usuario se vería obligado a leer su texto completo para enterarse de qué es lo que pasa. El ícono no le dice nada de nada, excepto que al programador le gustaban los árboles. Es preferible no usar íconos porque sí, sino reflexionar detenidamente sobre su rol semántico.

Algoritmia y Código de programa: el "software" que da vida a las Interfaces de Usuario

Lo tratado hasta aquí nos da un claro panorama sobre el rol de las interfaces con el usuario en los programas de computadoras, y de las reglas y principios que deben regir el diseño de las mismas. Para resolver el problema planteado en nuestra situación profesional deberemos apelar a todo esto, pero deberemos hacer, además, que el programa se comporte de manera adecuada. Ahora bien: ¿cómo es que logramos que estas interfaces "tomen vida"?

De nada sirve presentar al usuario un montón de interfaces bonitas y bien diseñadas si éstas no son capaces de permitirle interactuar con ellas, brindándole las herramientas necesarias para que desarrolle su trabajo, es decir, posibilitando la ejecución de las actividades para las que el **programa**[11] ha sido creado. La respuesta a este interrogante está precisamente en el desarrollo del "software" que ejecute las tareas necesarias, respondiendo a las acciones del usuario.

Todos los componentes de hardware de la computadora están preparados, en su conjunto, para ejecutar prácticamente cualquier tarea que le programemos. Son

[11] Programa: Interfaz de usuario y conjunto de instrucciones (expresadas en el lenguaje de programación específico) que definen su comportamiento.

"capaces" de mucho, pero por sí solos no hacen absolutamente nada. Una computadora sin "software" es como una canilla sin agua o un auto apagado.

El proceso de desarrollo de software consiste precisamente en diseñar las interfaces de las aplicaciones que ejecutará el ordenador, y la **"programación"**[12] de su comportamiento.

Esta tarea de programación no es otra cosa que indicarle a la computadora, en un lenguaje que ella sea capaz de interpretar, de manera secuencial y precisa, qué acciones debe realizar.

Una de las principales características del software es que el ordenador ejecuta una a una las instrucciones que recibe, secuencialmente. Es justamente por ello que nuestra tarea como programadores será la de indicarle al ordenador esta secuencia de operaciones para que al ejecutarla logre resolver el problema o alcanzar el objetivo planteado.

Decimos que las instrucciones debemos dárselas en un lenguaje que la computadora sea capaz de interpretar, indicándole en ese lenguaje comandos que sea capaz de ejecutar. Es por esto que resulta clave conocer las capacidades del sistema.

El lenguaje de la computadora

La máquina sólo entiende instrucciones indicadas en un lenguaje electrónico, completamente compuesto de dígitos binarios (ceros y unos) que se implementan como tensión o no tensión en pistas de un complicado circuito electrónico.

Para actividades tan elementales como la de calcular un interés hacen falta cientos (a veces miles) de pequeñas instrucciones individuales, sumamente elementales, que ejecutadas en la secuencia correcta producen el resultado deseado. Claro está que los seres humanos no manejamos ese tipo de lenguaje, ni en su forma ni en su nivel de

[12] Programa: Interfaz de usuario y conjunto de instrucciones (expresadas en el lenguaje de programación específico) que definen su comportamiento.

simplicidad. Es por eso que para programar el software de un ordenador recurrimos a los denominados **"lenguajes de programación"**[13].

Un lenguaje de programación se encarga de recibir los comandos que como **programadores**[14] impartimos al sistema y traducirlos al lenguaje de la computadora. Normalmente esta tarea de traducción convierte cada instrucción que conceptualmente manejamos los programadores (llamadas instrucciones de alto nivel) en cientos o miles de instrucciones a nivel del lenguaje de la máquina (o bajo nivel) que implementan la orden que le hemos dado. De esto se deduce naturalmente que los Lenguajes de Programación son capaces de manejar un conjunto finito y sumamente preciso de instrucciones de alto nivel para poder traducirlas mecánicamente.

Dadas las características de los lenguajes de programación como traductores, resulta importantísimo respetar la sintaxis y formato de cada instrucción que los mismos reciban por parte del programa que desarrolla un programador.

En general, el formato de las instrucciones de los lenguajes de programación consiste en verbos (en imperativo o infinitivo) que son las "órdenes" que le dicen qué hacer, casi siempre seguido de algún objeto con el cual realizar la acción.

Algunos ejemplos son:

- *Ingresar al Sistema*

- *Abrir el archivo de clientes*

- *Finalizar la ejecución del programa*

- *Imprimir "Hola"*

[13] Lenguaje de Programación: Define el conjunto de operaciones o instrucciones que la computadora es capaz de realizar, y la sintaxis específica con que deben indicarse, para que el traductor pueda convertirlas en instrucciones en el lenguaje de la máquina.

[14] Programador: Es la persona que desarrolla un programa.



Link del video: https://www.youtube.com/watch?v=V8ArTYdXfmY

Texto del video:

El modo de indicar algunas órdenes al lenguaje no es muy coloquial, sino que se ajusta taxativamente a las reglas sintácticas del lenguaje de programación. Por ejemplo, la orden de "Sumar 2 más 3 y guardar el resultado en X", se indica en programación como: x = 2+3.

Modelo de funcionamiento basado en eventos

Una cuestión tan importante como "qué hacer" es "cuándo hacerlo". El modelo de programación que estudiaremos en esta materia y que será luego aplicado por usted en los lenguajes de programación que estudie más adelante es el denominado "Modelo de Programación Estructurada basada en Objetos conducida por Eventos".

Esto consiste en desarrollar el programa indicando al ordenador que acción o conjunto de acciones desarrollar, de manera estructurada, ante la ocurrencia de un fenómeno o **Evento**[15] en el sistema.

Normalmente es el Usuario quien produce los eventos en el sistema. Entonces, lo que programamos no es otra cosa que "qué hacer" ante cada acción del usuario. Para cada

[15] Evento: Es un suceso que ocurre en el sistema y que el programa es capaz de detectar, para reaccionar al mismo ejecutando las acciones que el programador haya especificado. Por ejemplo: Un "click" del mouse es un evento. Pulsar una tecla en una caja de texto es un evento. Posicionarse sobre un objeto es un evento. Cerrar una ventana es un evento.

"**acción**" a la que nos interese que el programa responda, debemos programar la "**reacción**" correspondiente.

Veamos un ejemplo: si el usuario hace click sobre un botón llamado "cmdCancelar", lo que hace en realidad es producir el evento click sobre el objeto cmdCancelar. Si nos interesa que la interfaz que está utilizando reaccione a ese click, debemos entonces programar un **Procedimiento**[16] que se ejecutará de principio a fin cuando ese evento ocurra. El que sigue es un posible ejemplo.

PSEUDO: Ejemplo de procedimiento

```
Procedimiento cmdCancelar: Click
Detener la reproducción del CD
Cerrar la ventana del programa
Fin-Procedimiento
```

Como puede observar, el nombre de ese procedimiento será conformado por el nombre del objeto seguido del signo de dos puntos (:), seguido a su vez del nombre del evento.

Decíamos anteriormente que se trataba de un lenguaje estructurado. La que acabamos de analizar es una estructura que contiene un inicio, un desarrollo y un fin.

Hasta aquí, hemos recorrido los conceptos de interfaces y los sentidos involucrados en las mismas. Destacamos el uso de interfaces tipo gráficas con interacción al usuario, de éstas, rescatamos los estándares que se utilizan para su confección y las normas generales de armado y diseño.

Finalizamos con una introducción a lo que representa "un programa".

En el caso de nuestra situación profesional, debemos destacar, que ya podemos determinar algunos ítems necesarios para el desarrollo de interfaces gráficas, por ejemplo:

[16] Procedimiento: Es un conjunto de instrucciones (también llamadas comandos) dadas en un orden específico a la computadora para que produzcan, en conjunto, un resultado o acción. Tener en cuenta que: *Los procedimientos no se ejecutan solos, sino como consecuencia ante la ocurrencia de un evento que los "dispara". *Si ocurre un evento y el programador no ha especificado para el mismo ningún procedimiento, el programa simplemente "no hará nada".

- Empresa: ONE S.A.
- Rubro: investigación de mercado
- Necesidad del usuario: realizar la recolección de encuestas; procesamiento de encuestas.
- Reunimos la información sobre el modelo de negocio: Interés del Público (general y específico por barrio) en productos electrónicos (puntualmente SmartPhones, PC, Notebook, Netbook) caracterizándolos como MUY INTERESADO, INTERESADO, POCO INTERESADO, DESINTERESADO.
  La forma de recolección de datos es a través de encuestas por barrio. Luego se procesa la información en la empresa.

Con esta información podemos iniciar la confección de interfaces gráficas según los intereses del usuario/cliente.

SP1/Autoevaluación 1

Indique las opciones correctas

1) ¿Cuáles de los siguientes sentidos debemos tener en cuenta en el diseño de una interfaz gráfica?:

- ☐ Tacto
- ☐ Olfato
- ☐ Visión
- ☐ Sensibilidad
- ☐ Sabor
- ☐ Olores
- ☐ Calor

2) Indique los 2 elementos principales que debemos tener en cuenta para la confección de una interfaz gráfica de usuario:

- ☐ Color
- ☐ Líneas
- ☐ Objetos 3D
- ☐ Imágenes
- ☐ Usabilidad
- ☐ Fácil comprensión

Indique si es verdadero o falso

3) Los componentes electrónicos no necesitan interfaces para poder realizar sus tareas:

- ○ Verdadero
- ○ Falso

4) Una interfaz es solamente un medio gráfico que nos permite comunicarnos con dispositivos electrónicos.

- ○ Verdadero
- ○ Falso

5) Las interfaces de tipo texto son más fáciles de manejar porque no necesitan el mouse.

- ○ Verdadero
- ○ Falso

Respuestas correctas[17]

[17] 1)Tacto,Visión; 2)Usabilidad,Fácil comprensión; 3)Falso; 4)Falso; 5)Falso.

SP1/H2: IDE (Integrated Development Environment). Entorno de desarrollo integrado

Para resolver el pedido de desarrollo de nuestro cliente, vamos a necesitar utilizar y conocer un **IDE** (Entorno de Desarrollo Integrado). Existen muchos tipos de **IDE**, en este caso utilizaremos la **IDE de *Microsoft Visual Studio*.**

El concepto de una **IDE** es un entorno de desarrollo informático, un programa compuesto por un conjunto de herramientas de programación.

Puede dedicarse exclusivamente a un solo lenguaje de programación o bien a varios.



Link del video: https://www.youtube.com/watch?v=IVEvgc1IhyA

Texto del video:

Un IDE es un entorno de programación que ha sido empaquetado como un programa de aplicación; consiste en un editor de código, un compilador, un depurador y un constructor de interfaz gráfica (GUI).

Los IDE proveen un entorno de trabajo amigable para la mayoría de los lenguajes de programación tales como C++, Python, Java, C#, Visual Basic, etcétera. Es posible que un mismo IDE pueda funcionar con varios lenguajes de programación. Este es el caso de **Eclipse**, al que mediante plugins se le pueden añadir soporte de lenguajes adicionales. Observemos otros IDE que en algunos casos son de uso gratuito:

- Mono
- Wingware
- Eric
- NetBeans
- Visual Studio
- Eclipse
- JetBrains

Puntualmente, dentro del IDE de *Visual Studio*, utilizaremos el **lenguaje de programación**[18] conocido como *C#* (se pronuncia C sharp). C# tiene sus orígenes en la plataforma .Net de Microsoft y está basado en la sintaxis del lenguaje C++ y Java. C# es un lenguaje de programación orientado a objetos de múltiples usos, permitiendo generar programas con distintos tipos de interfaz, por ejemplo, aplicaciones de consola con interfaz de texto, aplicaciones windows con interfaz gráfica o incluso aplicaciones sin ningún tipo de interfaz con el usuario que se usan para comunicar a programas entre sí.

Algunas de las características del lenguaje:

- Sintaxis sencilla que facilita al desarrollador la escritura de código.
- Sistema de tipo unificado, permitiendo realizar operaciones comunes y que los valores de todos los tipos se puedan almacenar, transportar y utilizar de manera coherente.
- Orientación a componentes. Hemos dicho que C# es lenguaje orientado a objetos, pero también a componentes porque permite definir propiedades sin necesidad de crear métodos o usar eventos sin tratar con punteros a funciones.
- Espacio de nombres. Se puede aislar o agrupar código.

[18] Lenguaje de programación: Idioma artificial diseñado para expresar computaciones que pueden ser llevadas a cabo por máquinas como las computadoras (lenguaje máquina). Pueden utilizarse para crear programas o aplicaciones que controlen el comportamiento físico y lógico de una máquina, para expresar algoritmos con precisión, o como modo de comunicación humana. Está formado por un conjunto de símbolos y reglas sintácticas y semánticas que definen su estructura y el significado de sus elementos y expresiones. Al proceso por el cual se escribe, se prueba, se depura, se compila y se mantiene el código fuente de un programa informático se le llama "programación".

- Bibliotecas. Todos los compiladores de C# tienen un mínimo de biblioteca de clases disponibles para usar.
- Integración con otros lenguajes.
- Multihilo. En C# puedes dividir el código en múltiples hilos de ejecución, trabajar en paralelo y sincronizarlos al final.

Se considera a **Andrés Hejlsberg** como el impulsor de la creación de un nuevo lenguaje que finalmente resultó ser C#.

Andrés Hejlsberg decidió formar un equipo de trabajo en 1999 para trabajar en un nuevo lenguaje de programación, con la premisa que fuera sencillo, con seguridad de tipos y orientado a objetos. Su sintaxis sería fácil de aprender y muy intuitiva, lo que hace que cualquier persona familiarizada con algún otro tipo de lenguaje de programación aprenda en poco tiempo a utilizarlo.

La primera versión operativa del lenguaje fue la 1.0, liberada el año 2002 junto con Microsoft .Net Framework como plataforma de desarrollo, desde entonces se han sucedido las versiones 2.0 en el año 2005, 3.0 en 2007, 4.0 en 2010, 5.0 en 2012, 6.0 en 2015 y 7.0 en 2019.

Seguramente el lenguaje seguirá evolucionando agregando nuevas características y funcionalidades.

Ahora conocemos que:

- El IDE que utilizaremos es *Visual Studio*.
- El lenguaje de programación es *C#*.

Además de saber qué utilizaremos para desarrollar aplicaciones, debemos conocer que contamos con herramientas tales como los frameworks.

Un ***framework*** sería como un escritorio donde podemos obtener distintos objetos útiles, por ejemplo, colores y estructuras específicas, formas de ventana, acceso a datos, entre otros. Si la computadora donde voy a instalar la aplicación, tiene el framework adecuado, el programa se ejecutará sin ninguna dificultad.

Un ***Framework.net*** es un componente integral de *Windows* que admite la creación y la ejecución de aplicaciones o servicios *Web*. El diseño de .Net Framework está enfocado a cumplir los siguientes objetivos:

- Proporcionar un entorno coherente de programación orientada a objetos (POO).
- Proporcionar un entorno de ejecución de código que reduzca lo máximo posible la implementación de software y los conflictos de versiones.
- Ofrecer un entorno de ejecución de código seguro, un entorno de ejecución de código que elimine los problemas de rendimiento de los entornos en los que se utilizan secuencias de comandos o intérpretes de comandos.
- Ofrecer al programador una experiencia coherente entre tipos de aplicaciones Windows y web.
- Basar toda la comunicación en estándares del sector para asegurar que el código de .Net Framework se pueda integrar con otros tipos de código.

Para profundizar un poco más sobre los conceptos de Framework y .Net recomendamos acceder a estos videos:

Qué es un Framework (video).

¿Qué es NET? ¿Y .NET Core? ¿Para qué sirve? (video).

.Net Framework contiene dos componentes principales: ***Common Language Runtime*** y la biblioteca de clases de ***.Net Framework***.

Common Language Runtime **- CLR** es la base de la tecnología .Net Framework. Este tiempo de ejecución se puede considerar un agente que administra el código en tiempo de ejecución y proporciona servicios centrales *como la administración de memoria, la administración de subprocesos y la interacción remota*. El concepto de administración de código es un principio básico del tiempo de ejecución (Runtime). El código destinado al tiempo de ejecución se denomina *código administrado*, mientras que el resto se conoce como *código no administrado*.

La **biblioteca de clases**, el segundo componente principal de *.Net* Framework, es una colección completa orientada a objetos de tipos reutilizables que se pueden emplear

para desarrollar aplicaciones que abarcan desde las tradicionales herramientas de interfaz gráfica de usuario (GUI) o de línea de comandos (CLI) hasta las aplicaciones basadas en las innovaciones más recientes proporcionadas por *ASP.NET*, como formularios *Web Forms* y los Servicios *Web* XML.

.Net Framework se puede alojar en componentes no administrados que carguen *Common Language Runtime* - CLR en sus procesos e inicien la ejecución de código administrado, con lo que se crea un entorno de software en el que se pueden utilizar funciones administradas y no administradas. *.Net Framework* no solo proporciona varios hosts en tiempo de ejecución, sino que también admite el desarrollo de estos hosts por parte de terceros. Por ejemplo, cuando compilamos un programa y lo ejecutamos en el ordenador del cliente MSIL o IL (*Microsoft Immediate Language)*, compilamos un ***Assembly***[19] (que es un fichero de .Net), es igual que el lenguaje que escojamos sea C#, VB.NET o Python, el código generado es en lenguaje MSIL (similar al ***Bytecode***[20] de Java). Este *MSIL* es un lenguaje intermedio común a todos los sistemas operativos que soporten .NET Framework.

Cuando hacemos clic sobre el archivo **.exe** generado por ese código *MSIL* es interceptado por el *.Net Framework*, en concreto por el *CLR*.

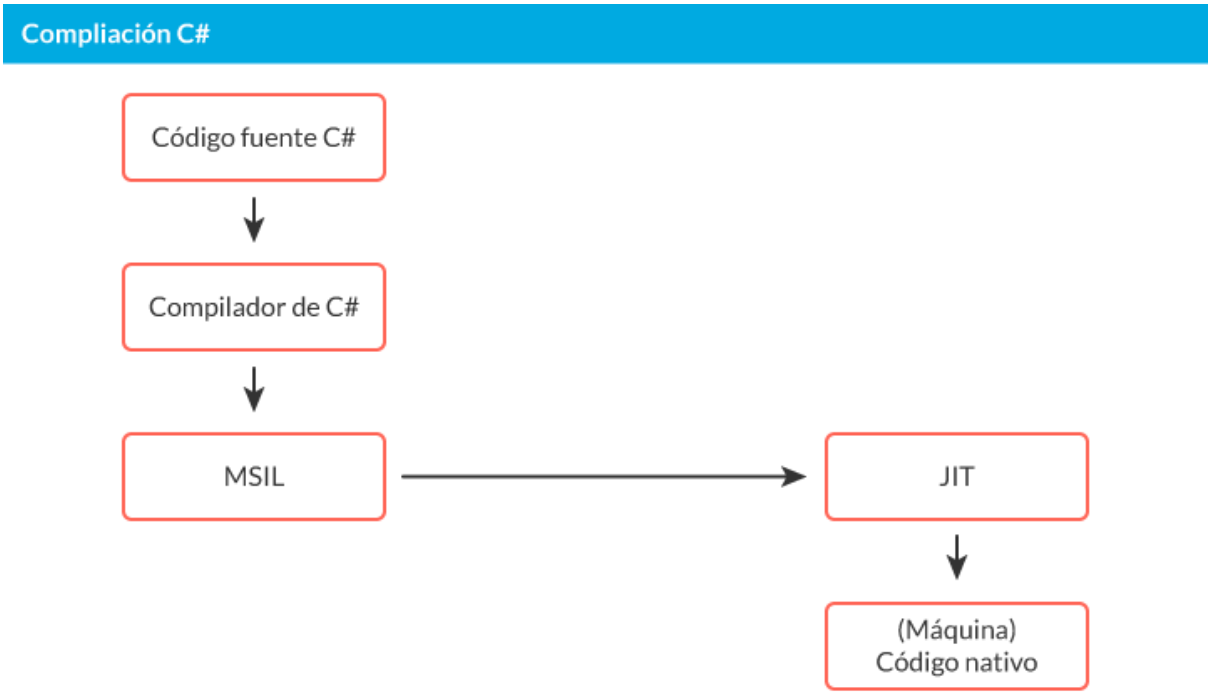
El *CLR* o (*Common Language Runtime*) es el motor de todo .NET, este CLR vuelve a compilar esta vez para generar un código nativo, es decir optimizado para el sistema operativo y el hardware actual. Esta compilación la realiza el compilador llamado JIT (Just In Time).

EL JIT o *Just In Time Compilator* es otro programa de *.Net Framework*, es el encargado de convertir el código MSIL en código de nuestro sistema operativo, este código se almacena en caché mientras nuestro .exe MSIL permanece inalterable.

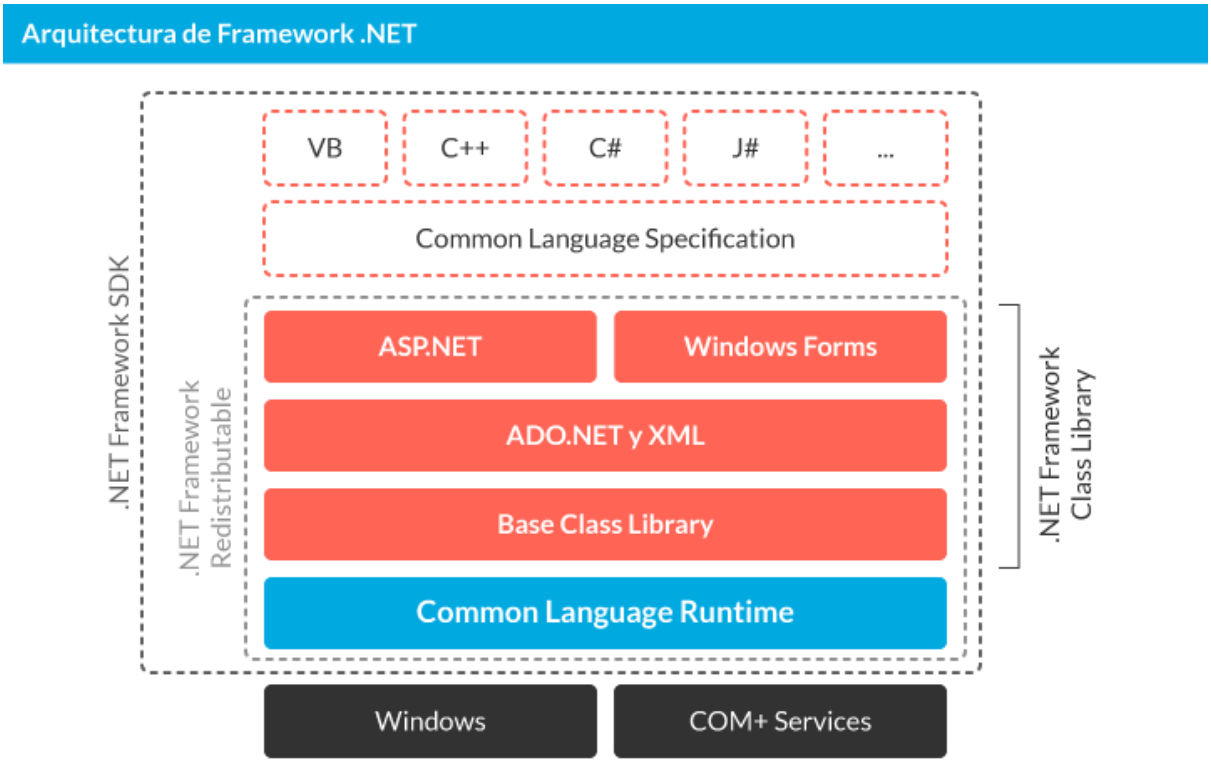
---

[19] Assembly: El lenguaje Assembly (a veces mal llamado "Ensamblador" por su traducción literal al español) es un tipo de lenguaje de bajo nivel utilizado para escribir programas informáticos, y constituye la representación más directa del código máquina específico para cada arquitectura de computadoras legible por un programador.

[20] Bytecode: El bytecode es un código intermedio más abstracto que el código máquina. Habitualmente es tratado como un archivo binario que contiene un programa ejecutable similar a un módulo objeto, que es un archivo binario producido por el compilador cuyo contenido es el código objeto o código máquina.



Arquitectura de Framework .Net y Core .Net

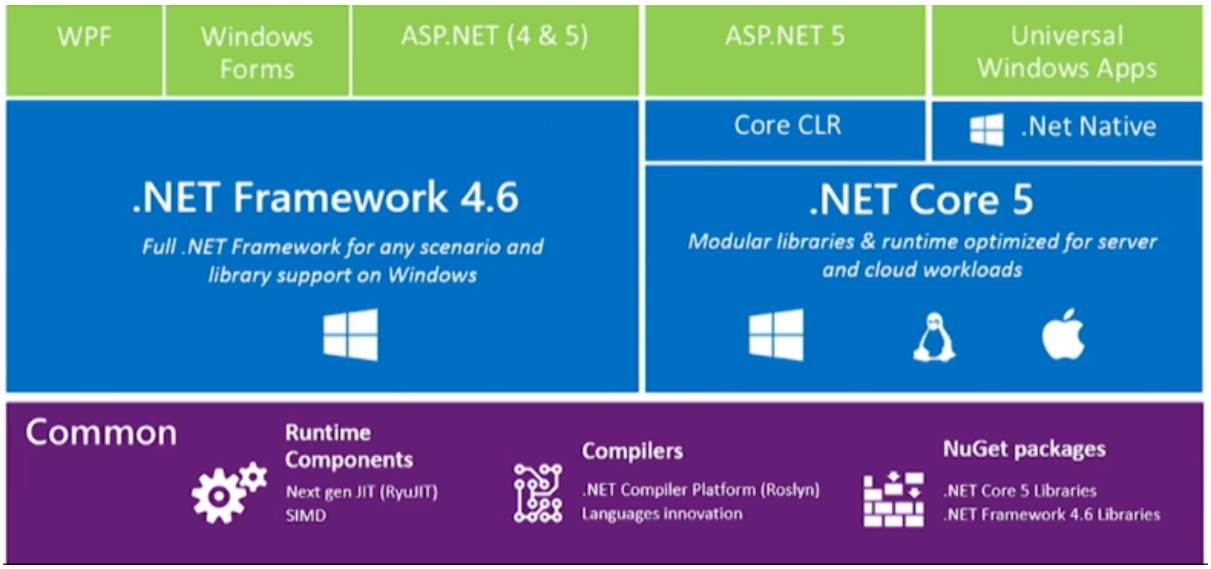


En la figura se pueden apreciar las distintas partes que componen al *.Net Framework*, incluidas el entorno de ejecución de aplicaciones (CLR, en azul), el conjunto de bibliotecas de funcionalidad reutilizable (*.Net Framework Class Library*, en verde) y los

compiladores y herramientas de desarrollo para los lenguajes .Net (en rojo). Todos estos componentes se notan por encima de la familia de sistemas operativos *Windows*.

Dentro del conjunto de la *.Net Framework Class Library* se distinguen 4 subcomponentes principales:

- La ***Base Class Library*** (BCL - Biblioteca de Clases Base), que contiene la funcionalidad más comúnmente utilizada para el desarrollo de todo tipo de aplicaciones. Algunos ejemplos de la funcionalidad provista por la BCL son el manejo de colecciones, cadenas de texto, entrada/salida, threading, operaciones matemáticas y dibujos 2D.
- ***ADO.NET***, que contiene un conjunto de clases que permiten interactuar con bases de datos relacionales y documentos XML como repositorios de información persistente.
- ***ASP.NET***, que constituye la tecnología dentro del .Net Framework para construir aplicaciones con interfaz de usuario Web (es decir, aplicaciones cuya lógica se encuentra centralizada en uno o varios servidores y que los clientes pueden acceder usando un browser o navegador mediante una serie de protocolos y estándares como HTTP y HTML).
- ***Windows Forms*** (o simplemente *WinForms)*, que constituye la tecnología dentro del *.NET Framework* que permite crear aplicaciones con interfaz de usuario basada en formularios y ventanas *Windows* de funcionalidad rica y que se ejecutan directamente en los clientes.



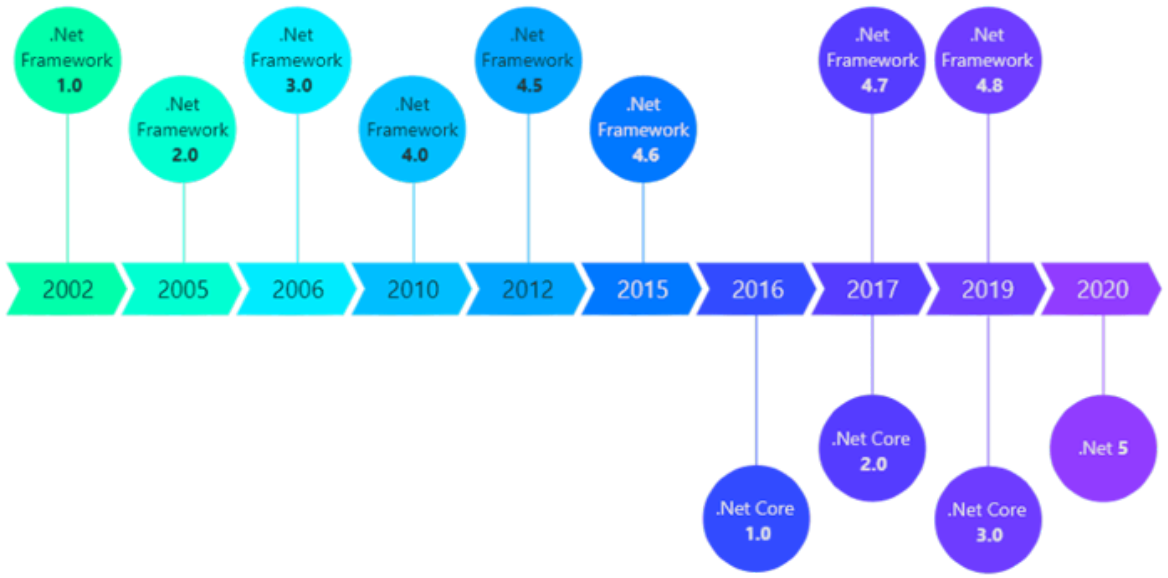
Fuente:

https://social.msdn.microsoft.com/Forums/azure/es-ES/68b3feeb-bbad-4980-86b4-95fe923037f2/net-core-vs-net-native-no-entiendo-estoy-confundido?forum=vbes

En esta figura podemos apreciar la diferencia entre:

.Net Framework 4.6: Es un framework para todo lo relacionado con .NET, pero carece de bibliotecas y tiempo de ejecución optimizado para ***implementaciones multiplataforma (Windows, Linux y Mac) y en la nube.***

.Net Core: Por otro lado, está optimizado principalmente para implementaciones multiplataforma y en la nube.



Fuente: https://dev.to/mikechatsky/net-core-for-net-frameworkers-n8m

Instalación de Visual Studio

Al ver la teoría relacionada con el origen del lenguaje de programación, qué es un IDE y qué es un Framework de .Net, procederemos a la instalación del software.

Instalación de Visual Studio (link)

Para versiones anteriores de Visual Studio puede visitar este link de descargas.

En todos los casos se recomienda descargar e instalar la versión "**Community**" ya que es gratuita y orientada a estudiantes, colaboradores y usuarios particulares, las otras

versiones disponibles de VS: "**Professional**" y "**Enterprise**", requieren el pago de una licencia y se recomiendan para organizaciones o equipos de trabajo en desarrollo.

Requerimientos para la instalación e información sobre el programa

Podremos obtener información sobre los requerimientos básicos para instalar Visual Studio en la siguiente web.

Con todos los conocimientos presentados en esta situación profesional, podremos avanzar a lo que más nos interesa, la utilización de ***Microsoft Visual Studio.***

Indique si es verdadero o falso

1) Un IDE es una framework que es utilizada como base para la programación, brindándonos clases y librerías.

○ Verdadero

○ Falso

2) C# es un IDE de Microsoft

○ Verdadero

○ Falso

3) Definido en forma sencilla, a un Framework lo podemos comparar a un escritorio de trabajo, del cual obtendremos distintas herramientas útiles, por ejemplo un libro de información podría ser comparado a las librerías de programación.

○ Verdadero

○ Falso

4) Originalmente, el lenguaje C# se creó a partir de C++ y Java.

○ Verdadero

○ Falso

Indique las opciones correctas

5) ¿Cuáles de las siguientes características pertenecen al Framework .NET?:

☐ a) CLR
☐ b) Biblioteca de clases
☐ c) MSIL
☐ d) Windows Forms
☐ e) COM+ Services
☐ f) Runtime Library
☐ g) JIT
☐ h) IIS

Respuestas correctas[21]

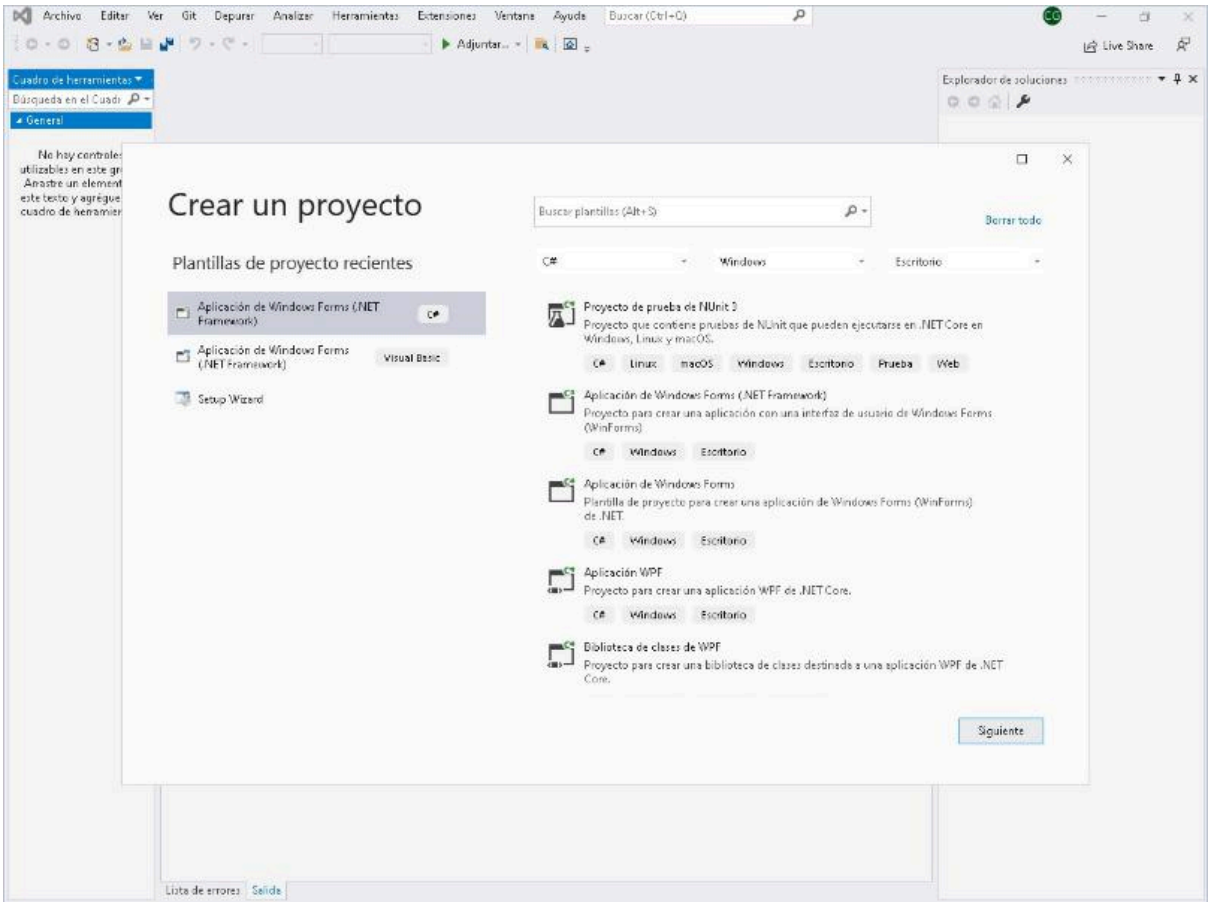
[21] 1)Verdadero; 2)Falso; 3)Verdadero; 4)Verdadero; 5)a,b,c,d,f,g.

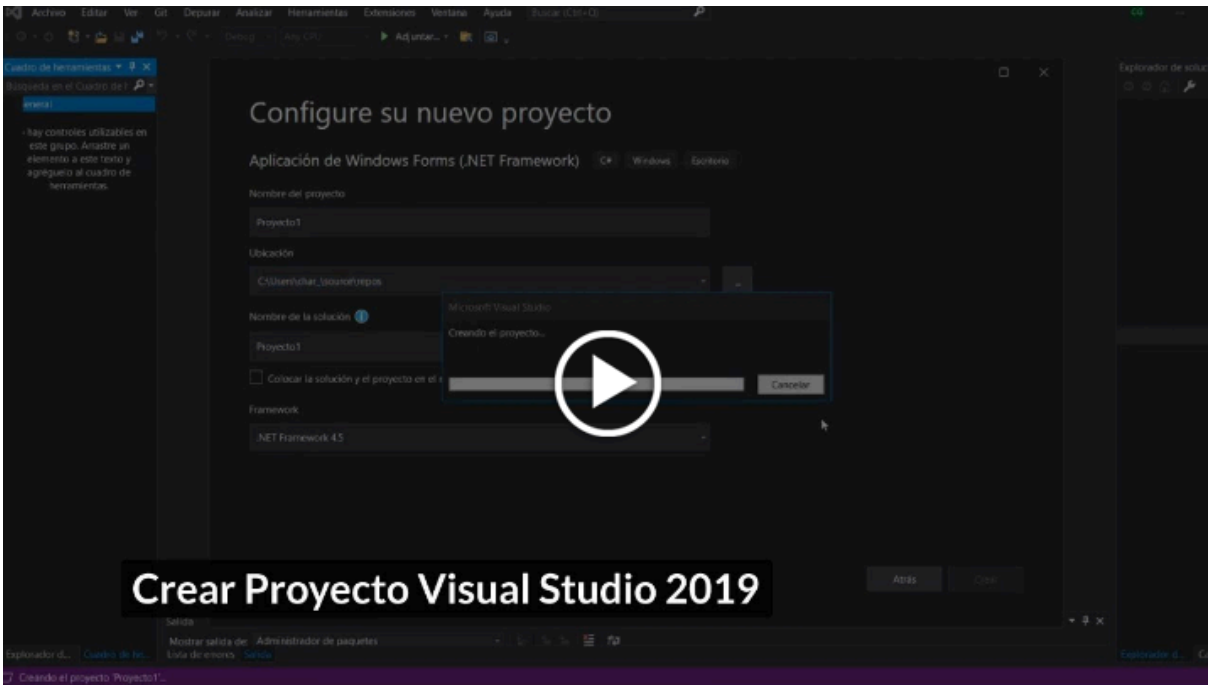
SP1/H3: Visual Studio

Para desarrollar lo que nos solicitó el usuario/cliente, vamos a utilizar ***Visual C#*** *(lenguaje de programación)*, integrado en ***Microsoft Visual Studio*** *(entorno de desarrollo)*. A fin de poder realizar esta tarea, debemos conocer cómo es el entorno luego de la instalación del mismo.

El entorno de desarrollo (**IDE**) se compone de varios elementos: la barra de herramientas **Menú**, barra de herramientas **Estándar**, varias ventanas de herramientas que se acoplan u ocultan automáticamente a la izquierda, parte inferior y a la derecha, así como en el espacio del editor. Las ventanas de herramientas, menús y barras de herramientas disponibles dependen del tipo de proyecto o archivo en el que esté trabajando.

Para empezar, ejecute Visual Studio que tiene instalado. Las versiones mantienen cierto estándar en sus interfaces, por ello, al iniciarse generalmente mostrará una interfaz como la siguiente:





Fuente: Carlos Gonzalez. (2022, enero 10). CrearProyectoVS2019 [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=xGxh0gwcjT0

Link del video: https://www.youtube.com/watch?v=uBBS7K0SOrE

Cuando creamos un nuevo proyecto o trabajamos con proyectos existentes, *Visual Studio .Net* crea un determinado conjunto de archivos que soportan nuestro desarrollo. Al crear un nuevo proyecto, también se crea una solución, aunque la solución únicamente tenga un proyecto. Se crea una carpeta para cada solución en la carpeta **\Mis documentos\Visual Studio Projects** que contiene los archivos **.sln** (la ubicación puede cambiar dependiendo de las opciones seleccionadas en el momento de crear el proyecto).

- **Archivos de solución (.sln)**

  La extensión de archivo ***NombreSolución.sln*** se utiliza para archivos de solución que enlazan uno o más proyectos; almacena información global.

Cada proyecto es una única aplicación almacenada en su propia carpeta. Dentro de la carpeta de proyecto se encuentra el archivo de configuración del proyecto y los archivos reales que constituyen el proyecto. El archivo de configuración del proyecto es un documento XML que contiene referencias a todos los elementos del proyecto,

como formularios y clases, además de referencias de proyecto y opciones de compilación.

Los archivos de proyecto C# utilizan una extensión **.csproj**. Estas extensiones permiten diferenciar entre archivos escritos en otros lenguajes compatibles con .Net y facilitan la inclusión de múltiples proyectos basados en diferentes lenguajes en la misma solución.

Los archivos que no estén basados en un lenguaje de programación tendrán sus propias extensiones. Por ejemplo, un archivo *Crystal Reports* utiliza la extensión **.rpt** y un archivo de texto utiliza **.txt**. Archivos ensamblados del proyecto (**.dll**).

AssemblyInfo.cs.

El archivo AssemblyInfo se utiliza para describir la información general del ensamblado, como la versión y los atributos del mismo.

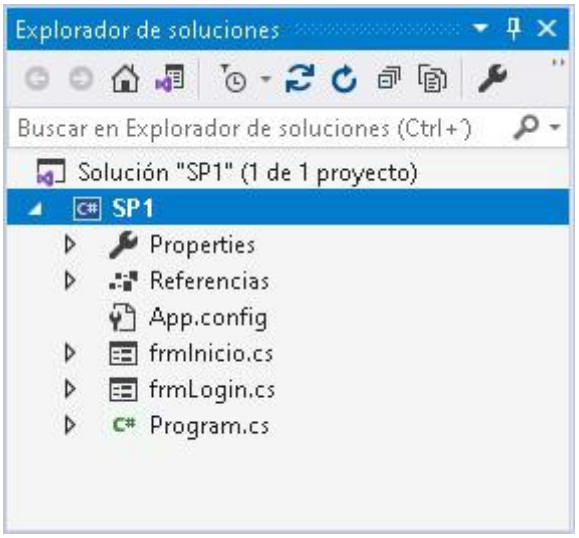
Resumen de los tipos de archivos:

Tipos de archivos	
.sln	Solución
.suo	Personalización de solución
.vbproj	Proyecto Visual Basic
.csproj	Proyecto C#
.vb	Archivo con código Visual Basic
.cs	Archivo con código C#
.dll	Librería

En general todas las aplicaciones que vayamos a hacer, en el IDE mostrarán las siguientes características:

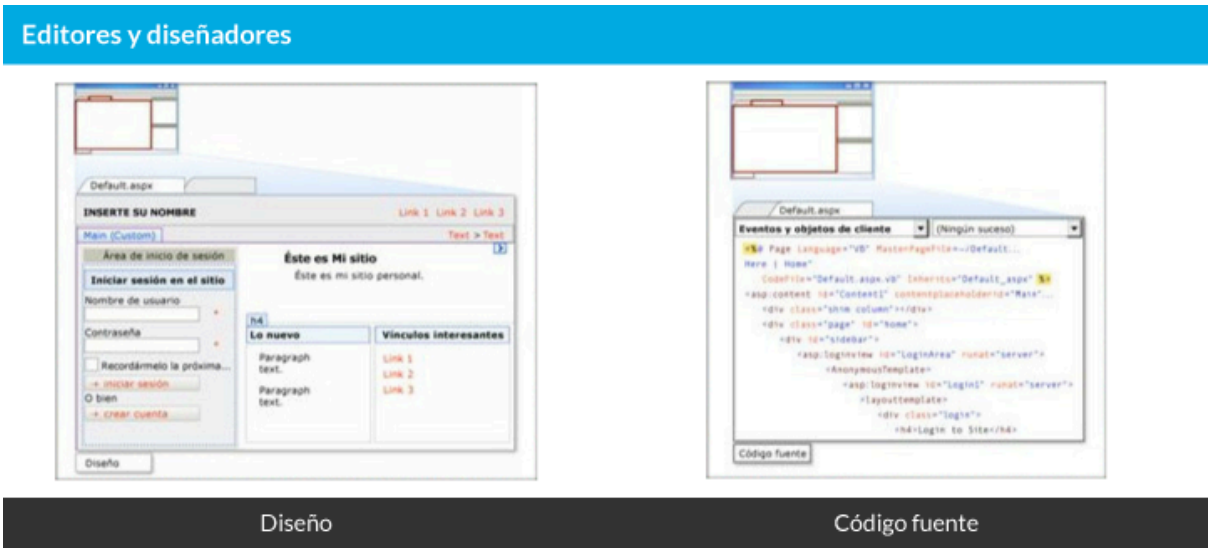
- **Explorador de soluciones:** muestra soluciones, sus proyectos y los elementos incluidos en dichos proyectos. En el Explorador de soluciones, puede abrir

archivos para editar, agregar nuevos archivos a un proyecto y ver las propiedades de las soluciones, proyectos y elementos.



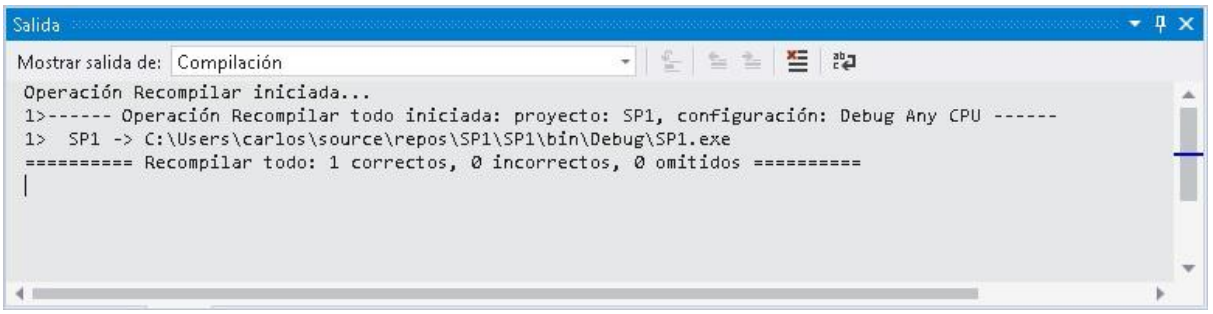
- **Los editores y diseñadores:** el editor y los diseñadores que utilice dependerán del tipo de archivo o documento que esté creando. El Editor de texto es el procesador de textos básico del IDE, mientras que el Editor de código es el editor de código fuente básico.

Otros editores, como el *Editor CSS*, el Diseñador *HTML* y el Diseñador de páginas web, comparten muchas de las funciones del Editor de código, junto con mejoras específicas en el tipo de código o de marcado admitido. Normalmente tienen dos vistas: una vista de diseño gráfico y la vista de código subyacente o vista de código fuente. La vista de diseño le permite especificar la ubicación de los controles y otros elementos en la interfaz de usuario o la página web. Puede arrastrar con facilidad un control desde el Cuadro de herramientas y colocarlo en la superficie de diseño.

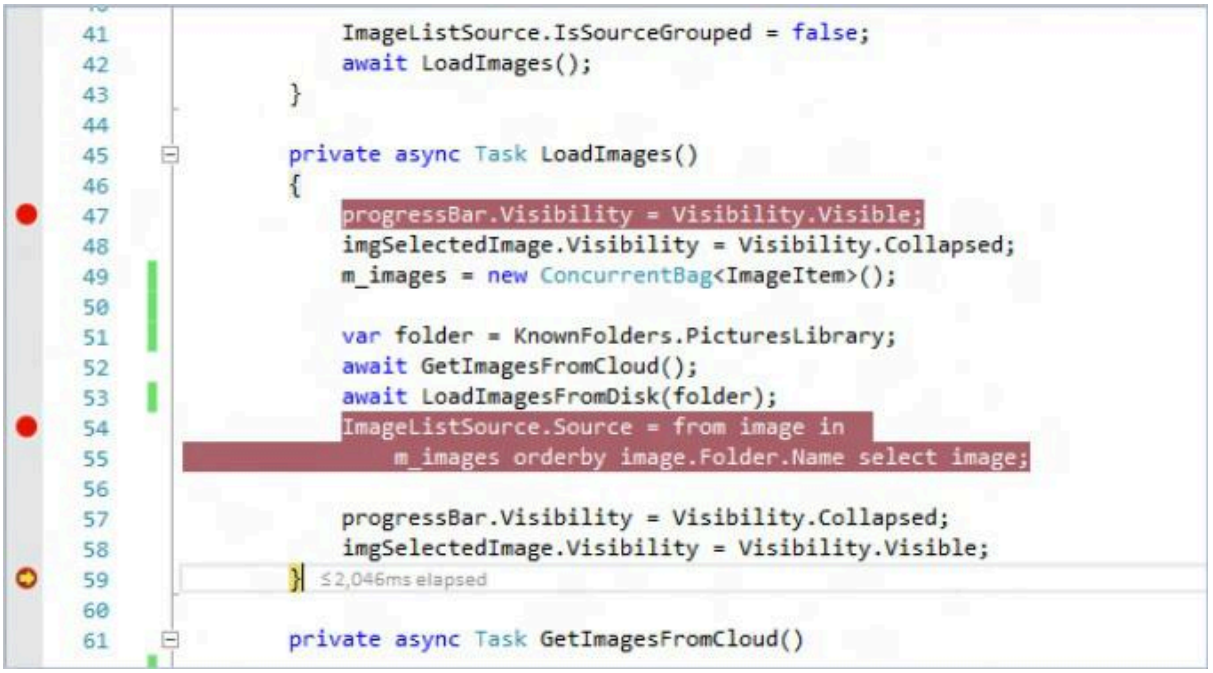


- **Herramientas de compilación y depuración:** con las configuraciones de compilación puede seleccionar los componentes que se van a generar, excluir los que no se van a generar y determinar cómo se van a generar los proyectos seleccionados y en qué plataforma.

Cuando genera, está comenzando el proceso de depuración. La compilación de la aplicación le ayuda a detectar errores de compilación. Estos errores pueden deberse a una sintaxis incorrecta, a palabras clave mal escritas o a divergencias entre los tipos. La ventana Salida (para ver la misma ir al menú Ventana y seleccionar Salida) muestra estos tipos de errores o mostrará que la generación del proyecto no tuvo errores.

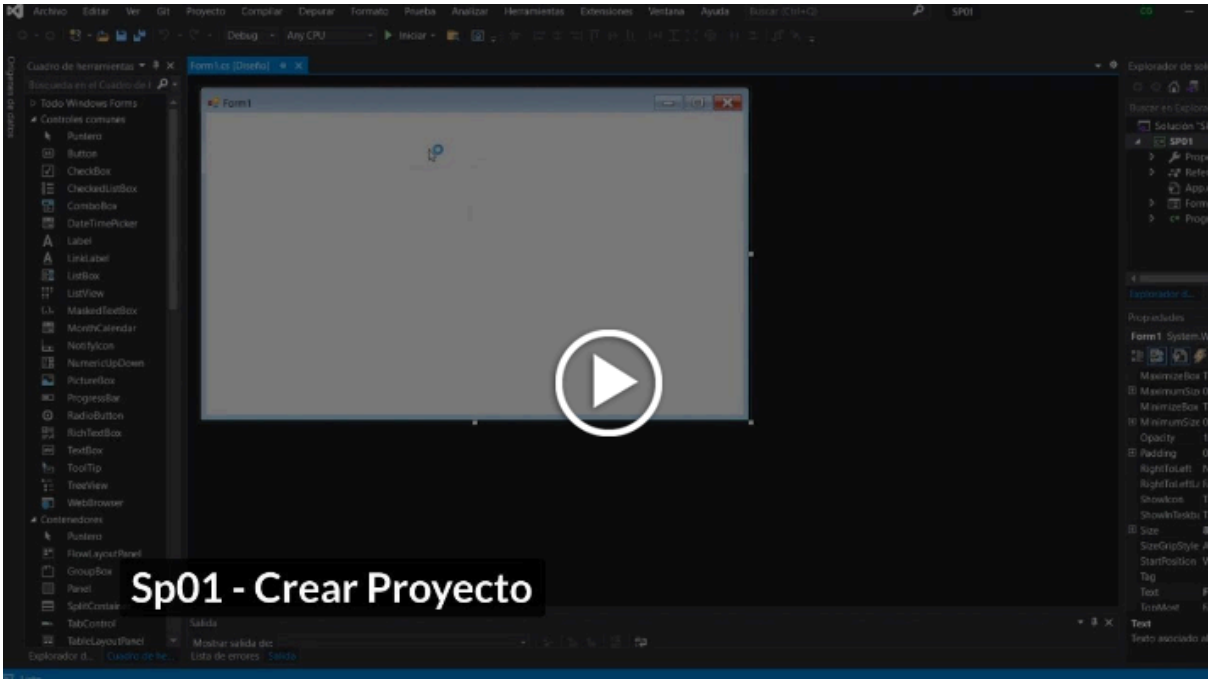


Después de generar la aplicación, puede utilizar el depurador para detectar y corregir problemas como errores lógicos y semánticos que se descubren en tiempo de ejecución. En el modo de interrupción, puede examinar las variables locales y otros datos pertinentes utilizando herramientas como Ventanas de variables y Memoria (para ver la misma, ir al menú Ventana y seleccionarlas).



Crear un proyecto

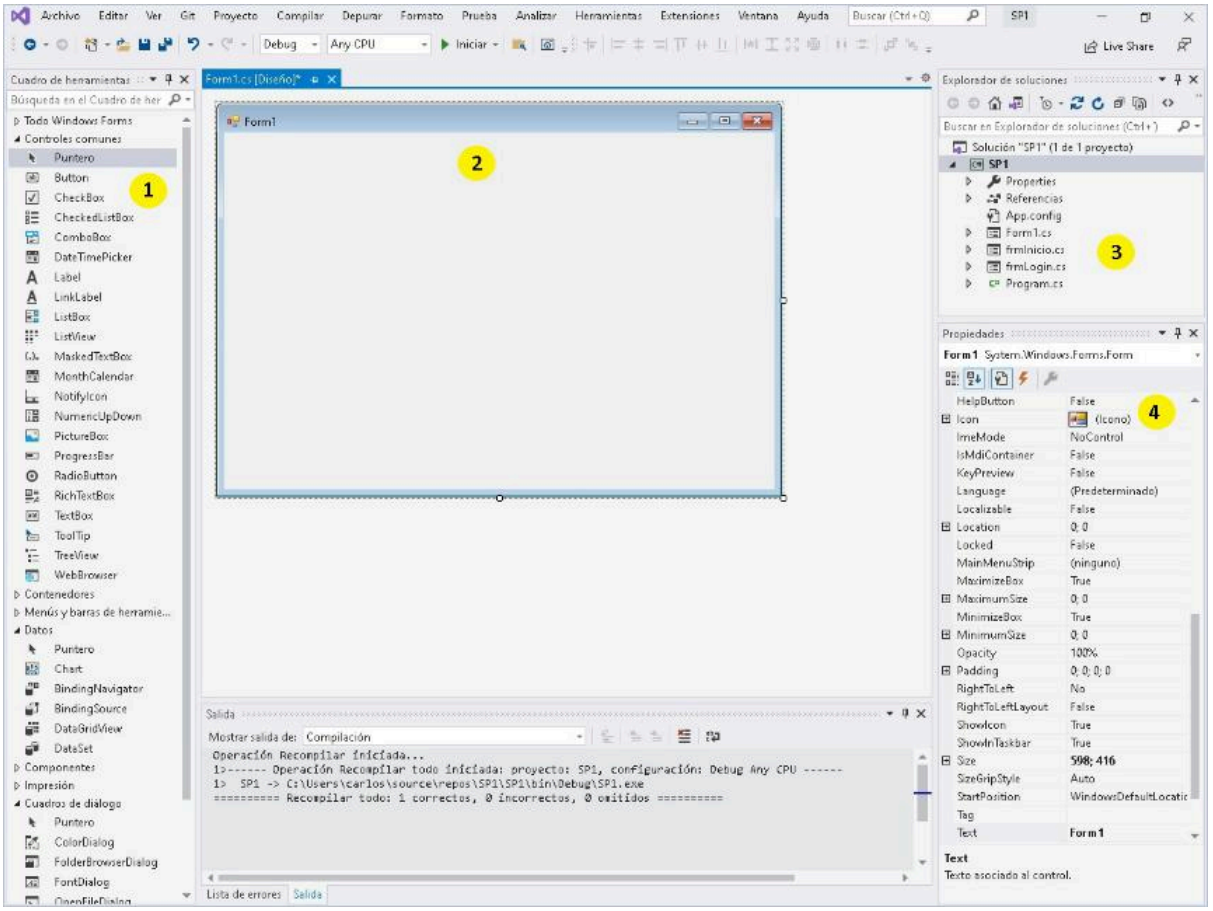
Empecemos a resolver los requerimientos que nos solicita nuestro cliente. Para ello, crearemos un proyecto desde cero.



Fuente: Carlos Gonzalez. (2022, enero 10). Sp01 CrearProyecto [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=51_mI0-yEdQ

Link del video: https://www.youtube.com/watch?v=JwnMq5FPvK4

Al realizar estos pasos para crear un proyecto, tendremos la siguiente IDE disponible:



1: Cuadro de Herramientas, contiene los componentes que podemos usar para componer el diseño de la interfaz gráfica en cada formulario.

2: Ventana de diseñador de interfaz gráfica, permite agregar los componentes y editar su ubicación, tamaño, alineación, espaciado, etc.

3: Ventana del explorador de soluciones, muestra los componentes que forman parte del proyecto.

4: Ventana de propiedades, permite revisar y modificar las propiedades específicas de cada componente ubicado en la interfaz gráfica.

Diseño de interfaces. Propiedades de formulario

Los objetos de un formulario son los elementos de desarrollo básico de una aplicación en *Visual C# .Net*; las ventanas reales con las que interactúa el usuario cuando ejecuta la aplicación. Los formularios tienen sus propios eventos, propiedades y métodos con los que se puede controlar su apariencia y comportamiento.

El primer paso para diseñar un formulario consiste en establecer sus propiedades. Puede establecer las propiedades en **tiempo de diseño**[22] en la ventana de Propiedades o en **tiempo de ejecución**[23] escribiendo el código.

En resumen, en esta situación profesional hemos visto cómo trabaja un Lenguaje de Programación Visual, en este caso con C# .Net. Conocimos sus componentes y la forma de administrar proyectos para poder desarrollar Aplicaciones, como así también cuáles son los tipos de objetos que maneja y sus características.



Link del video: https://www.youtube.com/watch?v=DW3zP30Z6-s

Texto del video:

En esta situación profesional aprendimos los conceptos básicos de interfaz, especialmente lo relacionado el uso de interfaz gráfica.

También vimos una introducción a Visual Studio .Net y la forma de crear un proyecto para construir una aplicación de escritorio usando como lenguaje a C#.

Finalmente vimos cómo conectar un evento con el código que necesitamos ejecutar como respuesta a dicho evento.

[22] Tiempo de diseño: Es el momento de desarrollo de la pantalla.

[23] Tiempo de ejecución: Es el momento de ejecución de la pantalla.

SP1/Autoevaluación 3

Indique la opción correcta

1) ¿Qué contiene un archivo de extensión .csproj?:

- ○ El código escrito en C#
- ○ La estructura del proyecto completo
- ○ La definición de la interfaz gráfica

2) En el entorno integrado de Visual Studio .NET, ¿Qué muestra la ventana Explorador de Soluciones?:

- ○ Los controles comunes
- ○ Los componentes del proyecto
- ○ Los objetos y funciones

Indique si es verdadero o falso

3) En el entorno integrado de Visual Studio .NET, el Cuadro de herramientas permite un rápido acceso a los componentes usados para la interfaz gráfica.

- ○ Verdadero
- ○ Falso

4) Un archivo .cs puede contener declaraciones, por ejemplo, de constantes y variables.

- ○ Verdadero
- ○ Falso

5) En las aplicaciones controladas por eventos, la aplicación es la que controla qué partes del código y en qué secuencia se ejecutan.

- ○ Verdadero
- ○ Falso

Respuestas correctas[24]

[24] 1)La estructura del proyecto completo; 2)Los componentes del proyecto; 3)Verdadero; 4)Verdadero; 5)Falso

SP1/Ejercicio resuelto

Para comprender el modelo de negocio de la organización, debemos separar las necesidades, requerimientos y propuestas que podemos hacer con respecto a la solicitud, y generar un prototipo antes de la programación para que la misma sea aprobada.

Situación: Investigaciones en Interés Público.
Requerimiento: Software y hardware para la obtención y procesamiento de datos.

Propuesta:

- **Hardware de recolección de datos:** recomendamos que para la digitalización de la información, utilicen dispositivos móviles, como por ejemplo Tablet PC, SmartPhones o Colectoras de datos.
- **Software de gestión:** Tendremos dos tipos de software:
    - **Colectora de Datos:** pantalla intuitiva donde se llevan a cabo los registros de las encuestas. Puede ser táctil para recolectar la información mucho más rápido.
    - **Programa de Escritorio (computadora):** software de gestión que servirá para descargar la información y administrarla.

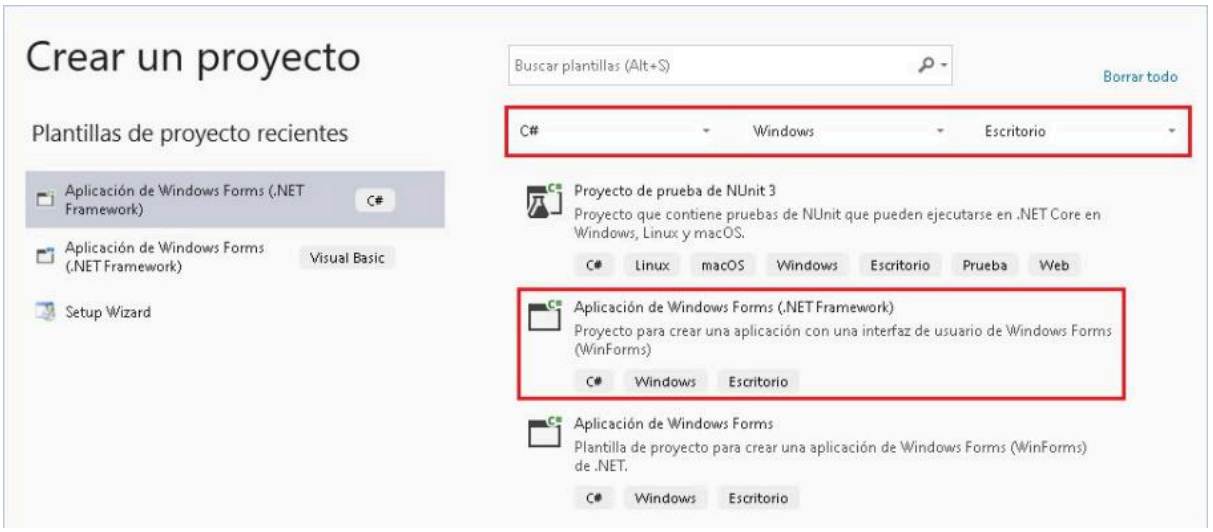
Prototipos: como vio en la materia Programación, usted debe dibujar los prototipos para proponer al cliente cómo se verían las pantallas que interactuarían con el usuario.

Para la solicitud de la segunda parte, en la cual la programación es especificada y no requieren prototipos, hará lo siguiente:

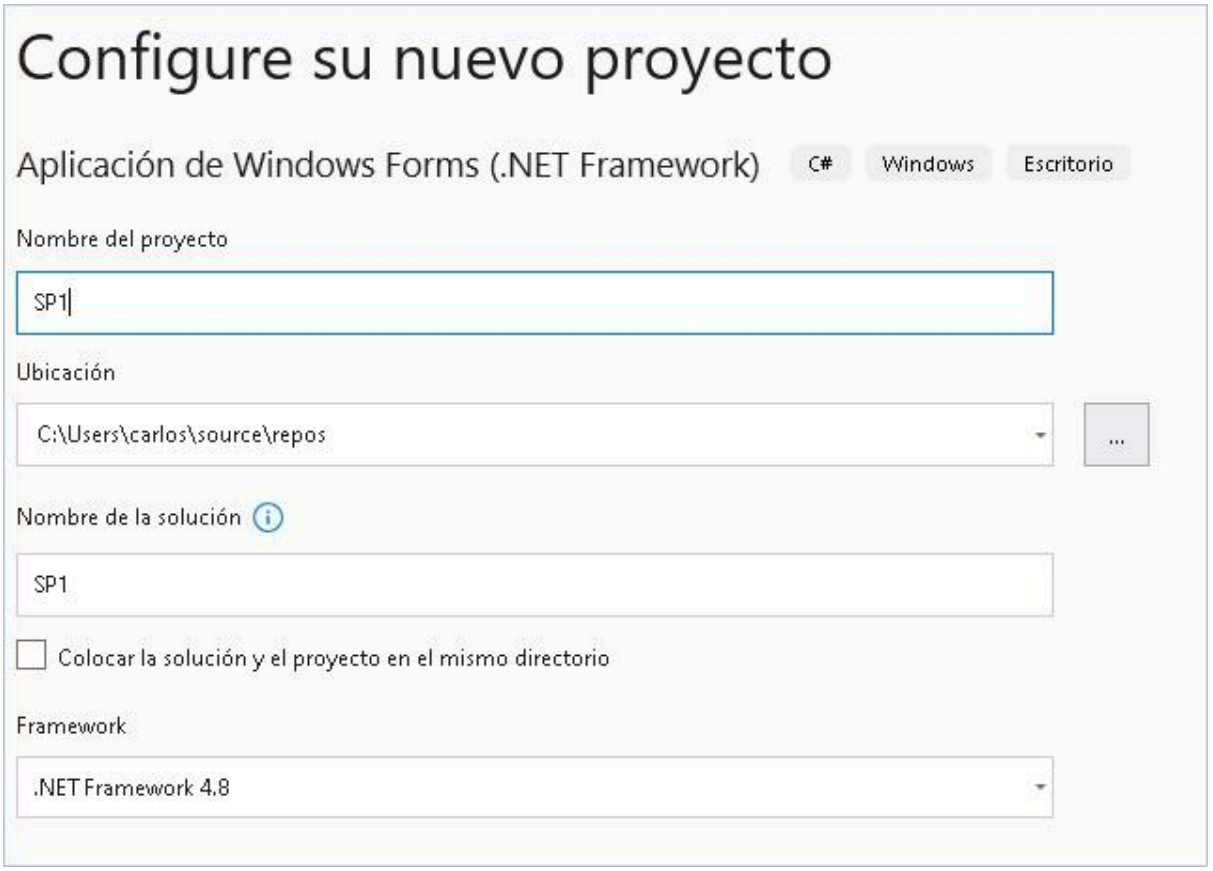
1. Crear un proyecto nuevo con Visual Studio: Plantilla de Proyecto: **Aplicación de Windows Forms (.Net Framework)**, Lenguaje: C#, Plataforma: Windows, Tipo de Proyecto: Escritorio.
2. Realizar el diseño de la interfaz para el formulario de Login.
3. Agregar al proyecto un segundo formulario y realizar su diseño como formulario de inicio.
4. Escribir el código en lenguaje C# que le otorgue la funcionalidad solicitada a cada formulario.
5. Ejecutar y probar la aplicación, corregir errores si fuera necesario.

Creación del Proyecto con Visual Studio

Desde Visual Studio elegimos la opción de crear un nuevo proyecto y elegimos la plantilla "Aplicación de Windows Forms (.Net Framework) con lenguaje C#:



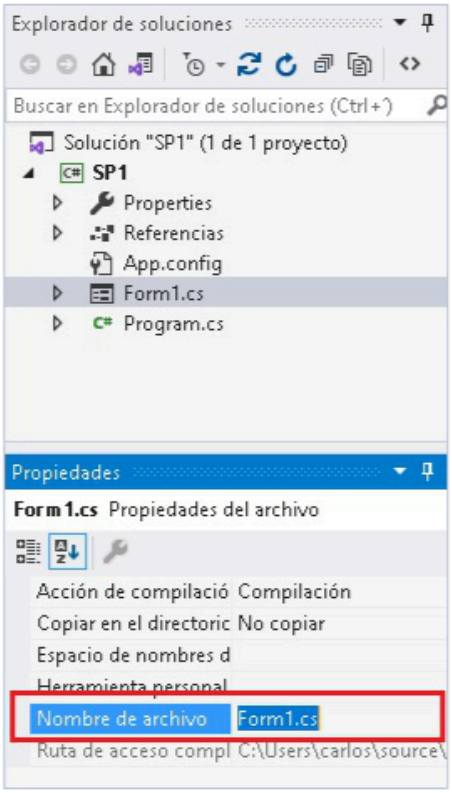
En el siguiente paso de la creación escribimos el nombre de nuestro proyecto, en este caso "SP1", elegimos la carpeta dónde se ubicará el proyecto y hacemos click en el botón "Crear" para finalizar la creación. Las demás las dejamos con los valores que se muestran por defecto.

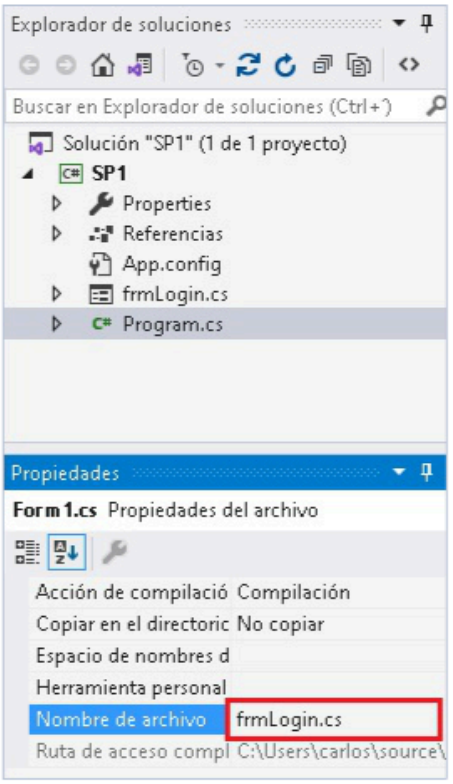


Una vez creado el proyecto ya podemos comenzar a trabajar en él.

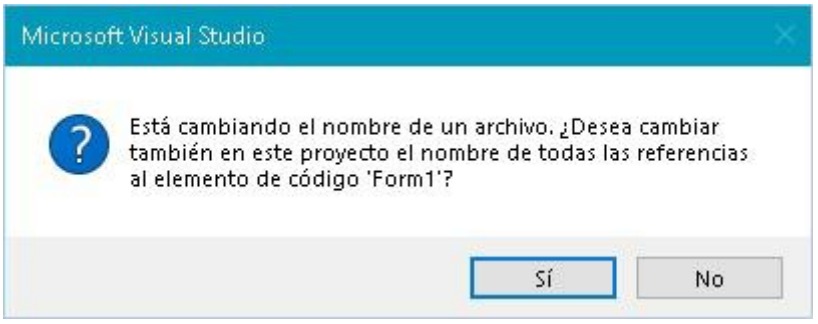
Diseño del formulario de Login

Inicialmente procedemos a cambiar el nombre del archivo de nuestro primer formulario, para ello en la ventana del explorador de soluciones, arriba a la derecha ubicamos y seleccionamos el formulario "Form1.cs", luego desde la ventana de propiedades seleccionamos la propiedad "Nombre de archivo" y cambiamos el valor de la propiedad que muestra "Form1.cs" por "frmLogin.cs"



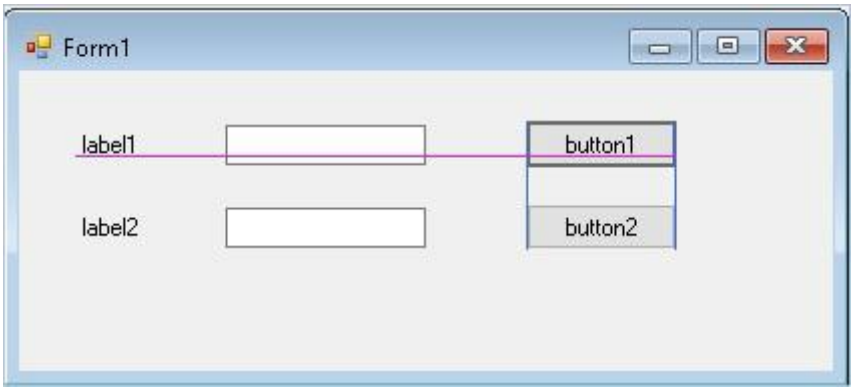


Al presionar la tecla "Enter" para confirmar el cambio de nombre se mostrará una ventana pidiendo confirmación:



Presionamos el botón "Si". De esa forma nos aseguramos que el nuevo nombre será aplicado a todo el código de nuestra aplicación.

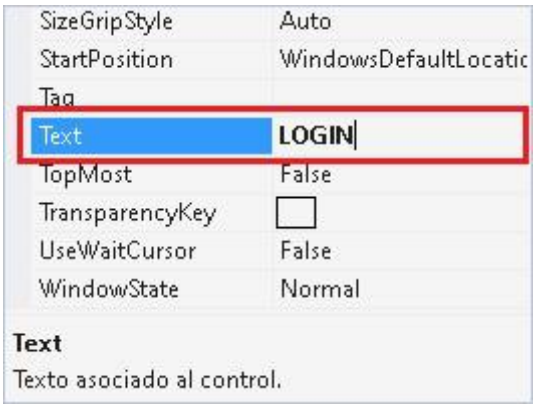
Ahora continuamos con el diseño de nuestro formulario, colocaremos en él, arrastrando desde el cuadro de herramientas 2 controles de tipo "Label", 2 controles de tipo "TextBox" y 2 controles de tipo "Button" y los ubicamos para obtener la disposición mostrada, también ajustamos el tamaño del formulario arrastrando desde sus bordes derecho e inferior.



Podemos ayudarnos con las líneas guías que se muestran sobre el formulario cuando movemos un componente para conseguir una distribución uniforme y bien alineada de todos los elementos.

La siguiente tarea consiste en asignar a cada elemento los valores adecuados en sus propiedades, comenzando por la propiedad "Name" de cada componente para poder identificarlos correctamente cuando necesitemos referenciarlos desde el código en C#. La forma de trabajo es simple, seleccionamos el componente en el editor de la interfaz (formulario, Label, TextBox, etc.) y luego en la ventana de propiedades ubicamos y modificamos los valores de las propiedades que necesitemos ajustar.

Comenzamos con el formulario, a su propiedad "Text" asignamos el valor "LOGIN"



El texto "LOGIN" se muestra como título en la ventana del formulario. Además, modificamos la propiedad "ControlBox" asignando el valor "False", eso hará que los botones de control del formulario, minimizar, maximizar y cerrar ya no se muestren.

Para el resto de los componentes del formulario aplicamos estos cambios:

Para "label1": Propiedad Valor
Text Usuario

Para "label2": Propiedad Valor
Text Contraseña

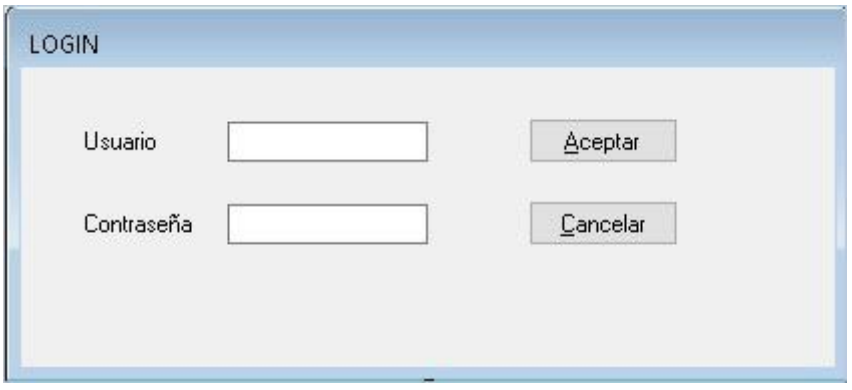
Para "textBox1": Propiedad Valor
Name txtUsuario
maxLength 20

Para "textBox2": Propiedad Valor
Name txtContrasenia
maxLength 20
paswordChar *

Para "button1": Propiedad Valor
Name btnAceptar
Text &Aceptar

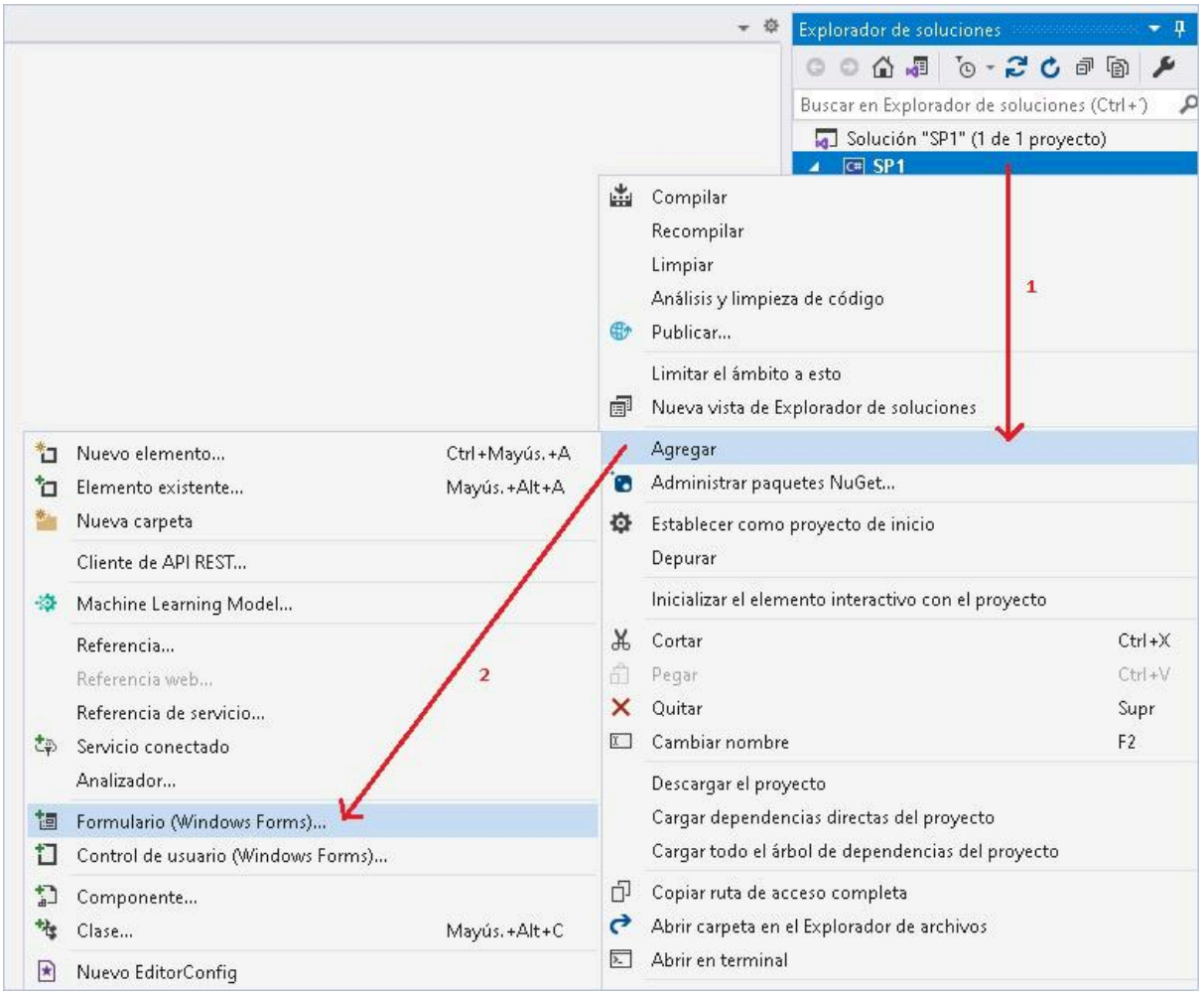
Para "button2": Propiedad Valor
Name btnCancelar
Text &Cancelar

Resulta:

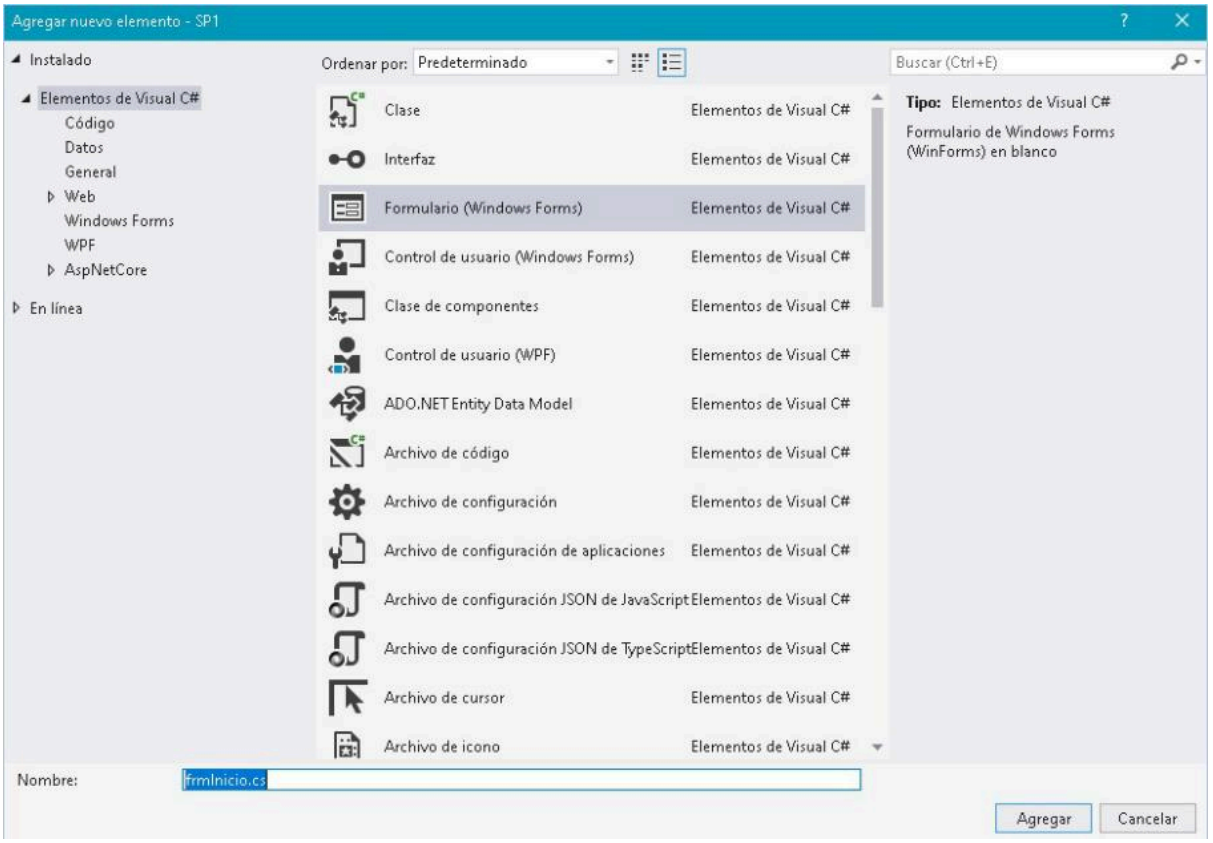


Continuamos el desarrollo ahora con el segundo formulario, debemos agregar un nuevo componente al proyecto, entonces desde la ventana del explorador de soluciones, seleccionamos el nombre del proyecto (SP1) y hacemos click con el botón

derecho del mouse para abrir el menú desplegable, elegimos la opción "Agregar" (1) y del submenú elegimos "Formulario (Windows Forms)... (2):



Nos solicitará el nombre del nuevo formulario, escribiremos en este caso "frmInicio" y presionamos el botón "Agregar":



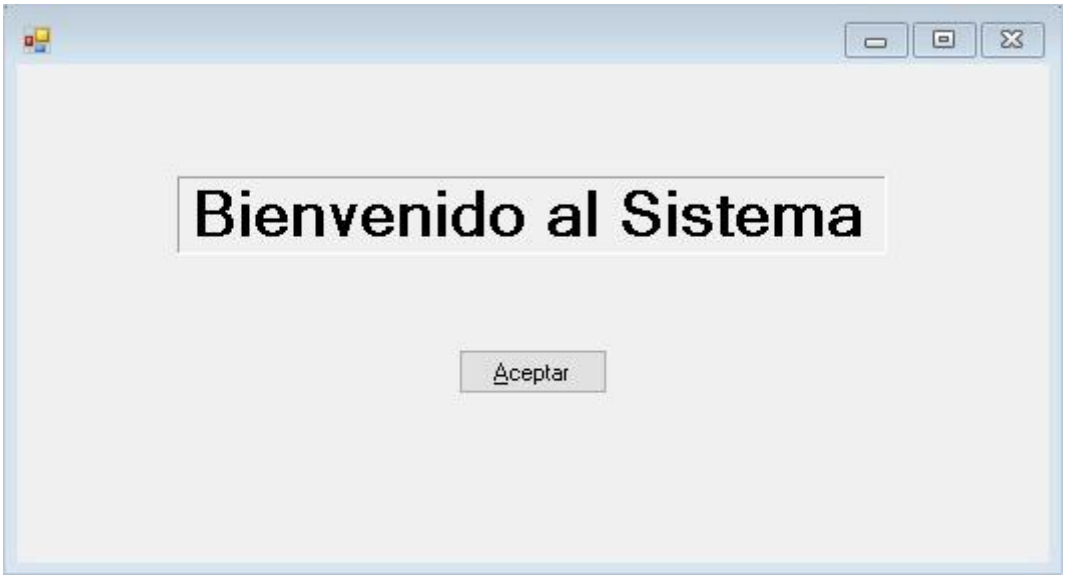
Continuamos trabajando en el diseño de este nuevo formulario colocando en él un componente "Label" y otro "Button", los ubicamos de forma tal que se vean centrados en el formulario. Desde la ventana de propiedades asignaremos estos valores:

Para "frmInicio" Propiedad Valor
Text (vacía)

Para "label1": Propiedad Valor
Text Bienvenido al Sistema
BorderStyle Fixed 3D
Font Size 24
Font Bold True

Para "button1": Propiedad Valor
Name btnAceptar
Text &Aceptar

Resulta:

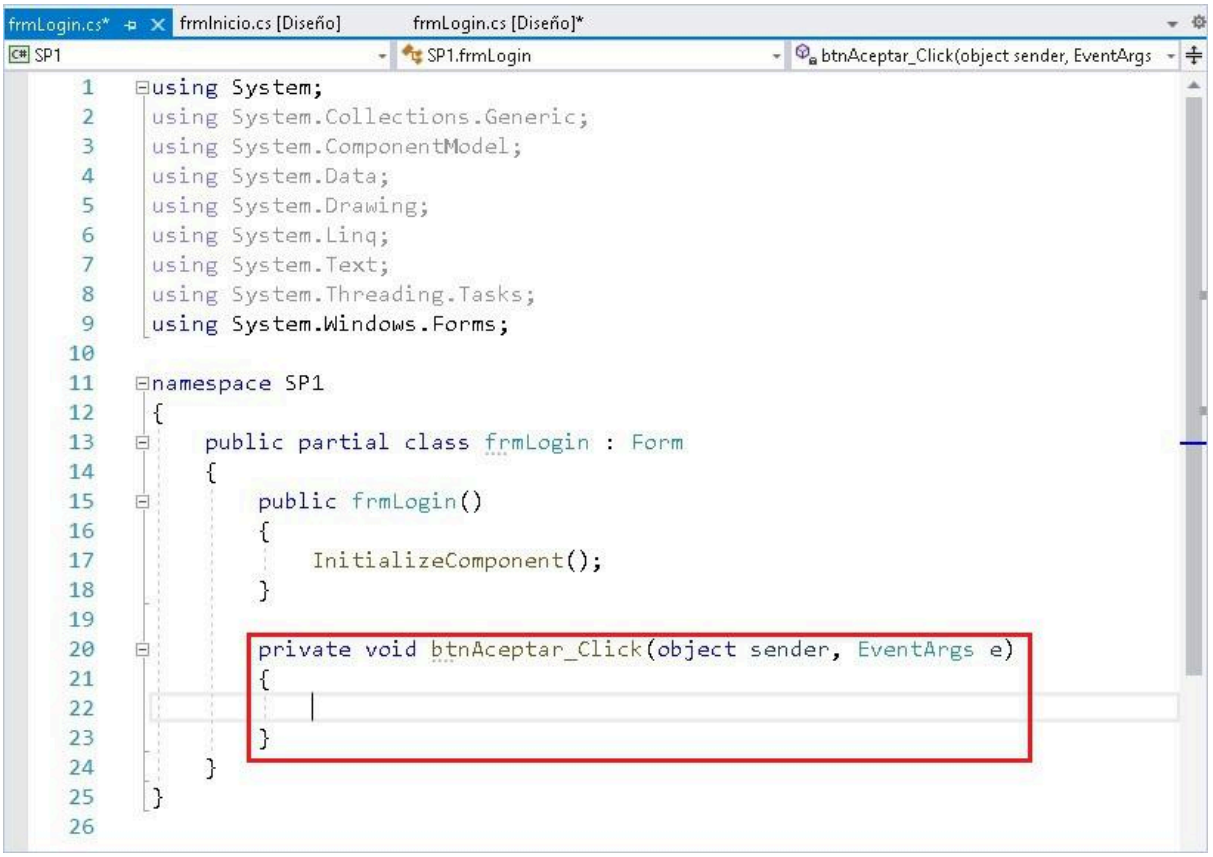


Y ya tenemos los dos formularios completos en su etapa de diseño, debemos ahora continuar con la escritura del código en C# para obtener la funcionalidad requerida en cada uno de ellos.

Programación del formulario frmLogin

En este formulario la interacción del usuario con la aplicación ocurre con los botones de comando, por lo tanto, deberemos programar los eventos que ocurren cuando el usuario hace click sobre el botón. Cada componente de la interfaz puede reaccionar a varios eventos, pero tiene un evento predefinido como el evento por defecto. En el código un evento está representado por un bloque de código que será ejecutado cuando el evento tiene lugar, generalmente en respuesta a una acción que el usuario realiza sobre la interfaz, como el caso del click sobre un botón. A estos bloques de código se los denomina, en general, como procedimientos o funciones. Veamos entonces cómo crear los bloques de código para el evento Click de los botones.

Tomamos en primer lugar el botón "btnAceptar" del formulario frmLogin, seleccionamos el botón en la interfaz y hacemos doble click sobre él, esa acción genera el código del evento Click y nos lleva directamente al editor de código en una nueva ventana del editor de Visual Studio:



Vemos el encabezado del código para el evento click del botón "btnAceptar", el nombre del procedimiento creado está formado por el nombre del botón (btnAceptar) seguido de un guión bajo y el nombre del evento (Click), los valores entre paréntesis que siguen al nombre del evento son parámetros que le indican al evento cuál fue el objeto que generó el evento y ciertos argumentos que aportan datos particulares al evento. Todo esto se genera en forma automática al hacer doble click sobre el botón. Las acciones que deseamos ejecutar cuando el evento se dispare las debemos programar entre las llaves { }.

Según lo expresado en el requerimiento inicial para este formulario, debemos controlar los valores ingresados para el usuario y su contraseña, si son correctos debemos mostrar el segundo formulario, en caso contrario mostraremos un mensaje de error. Además, si el error ocurre 3 veces cerraremos el formulario.

El código completo del evento resulta así:

C #: Evento click btnAceptar completo

```
int intentos = 0;  // contador de intentos de acceso
  // evento Click del botón "btnAceptar"
  private void btnAceptar_Click(object sender, EventArgs e)
  {
      if( (txtUsuario.Text == "Administrador" &&
              txtContrasenia.Text == "adm135$") ||
          (txtUsuario.Text == "Operador" &&
              txtContrasenia.Text == "ope246$") )
      {
          this.Hide(); // oculta este formualrio
          frmInicio f = new frmInicio(); // crea el frmInicio
          f.Text = txtUsuario.Text; // asigna el texto de título
          f.ShowDialog(); // visualiza y ejecuta el frmInicio
          this.Show(); // visualiza nuevamente este formulario
      }
      else
      {
          MessageBox.Show("Datos incorrectos. Acceso Denegado.");
          intentos++; // incrementa el contador de intentos fallidos
          if(intentos == 3) // si es 3 se cierra el formulario
          {
              this.Close();
          }
      }
  }
```

Usamos la estructura alternativa "if" - "else" para preguntar si los valores de usuario y contraseña son correctos, en ese caso se oculta el formulario frmLogin, se crea y visualiza el formulario frmInicio y se espera a que éste se cierre, cuando eso ocurre, se visualiza nuevamente el formulario frmLogin, dejando todo en el estado inicial. En caso de no haber ingresado valores correctos para el usuario y contraseña, se mostrará un mensaje de error y se incrementa el contador de intentos erróneos, implementado con la variable "intentos", luego usando nuevamente la estructura "if" preguntamos si el contador llegó al valor 3, en caso afirmativo se cierra el formulario.

En el código de C# se pueden escribir comentarios, que son anotaciones que agrega el programador para explicar las acciones que realiza el código o para describir algo que considera importante de tener en cuenta, el comentario comienza con doble barra: (//) y es válido hasta el fin de la línea que lo contiene.

La variable "intentos" debe ser declarada fuera del evento, en este caso la declaración estará antes del bloque del evento.

Más adelante veremos con detalle el significado de cada uno de los elementos que forman el código de este evento, por ahora lo usaremos tal como está para que nuestra aplicación pueda funcionar.

Repetimos el procedimiento para crear el evento click del botón "btnCancelar" y completamos su implementación escribiendo este código:

C #: Evento click boton Cancelar

```
// evento Click del botón "btnCancelar"
  private void btnCancelar_Click(object sender, EventArgs e)
  {
      this.Close(); // cierra el formulario
  }
```

Acá con una sola sentencia es suficiente para lograr cerrar el formulario y ya tenemos el código completo del formulario "frmLogin".

Continuaremos ahora con la implementación en el formulario frmInicio

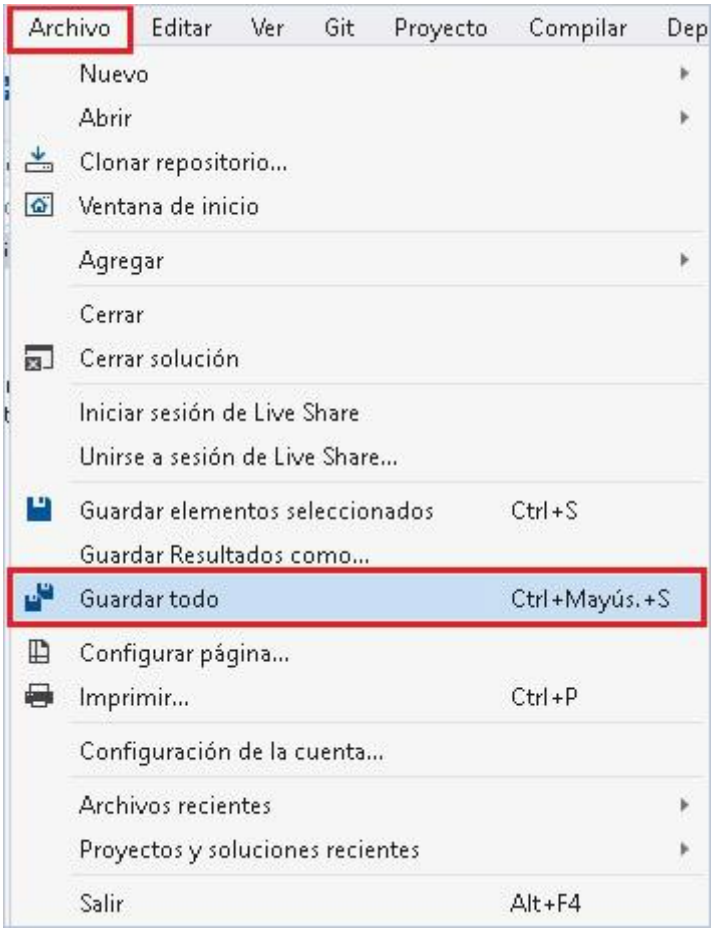
Desde la ventana de diseño del formulario "frmInicio", seleccionamos el botón "btnAceptar" y hacemos doble click en él, se generará el encabezado del evento Click y lo completaremos de esta forma:

C #: frmInicio evento click btnAceptar

```
// evento Click del botón "btnAceptar"
  private void btnAceptar_Click(object sender, EventArgs e)
  {
      this.Close(); // cierra el formulario
  }
```

De esa forma conseguimos que el formulario se cierre y podamos retornar al formulario de login tal como se solicitó inicialmente.

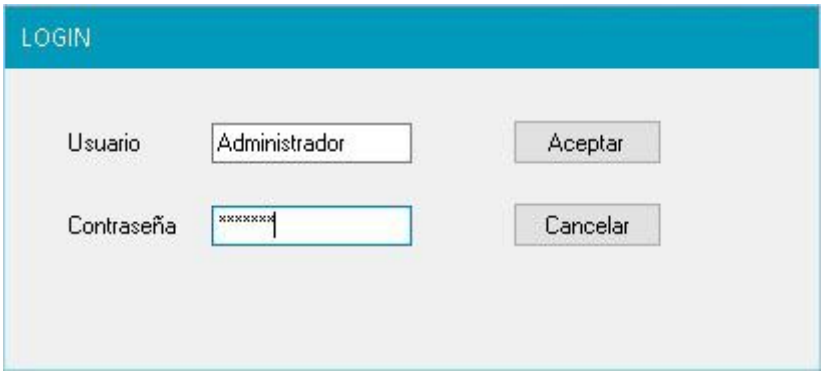
Así finalizamos el desarrollo de la aplicación, resta guardar todos los cambios y ejecutar para verificar que todo esté correcto.



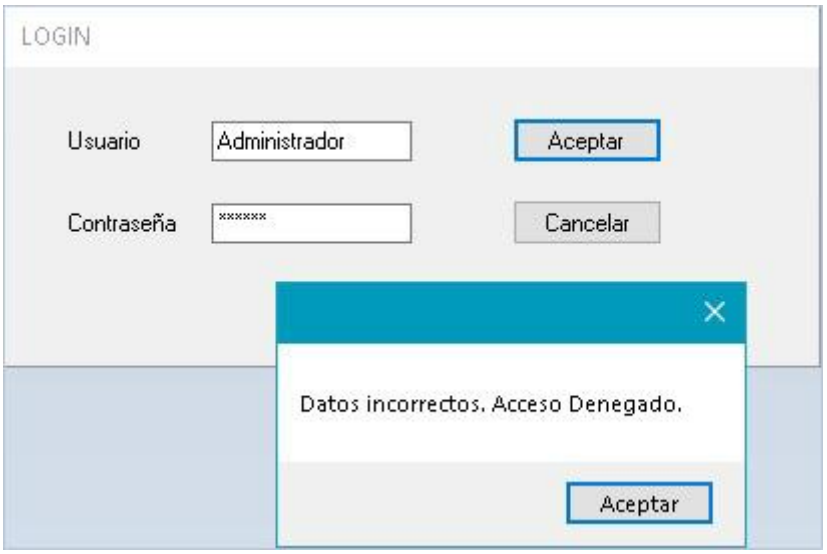
Para ejecutar:



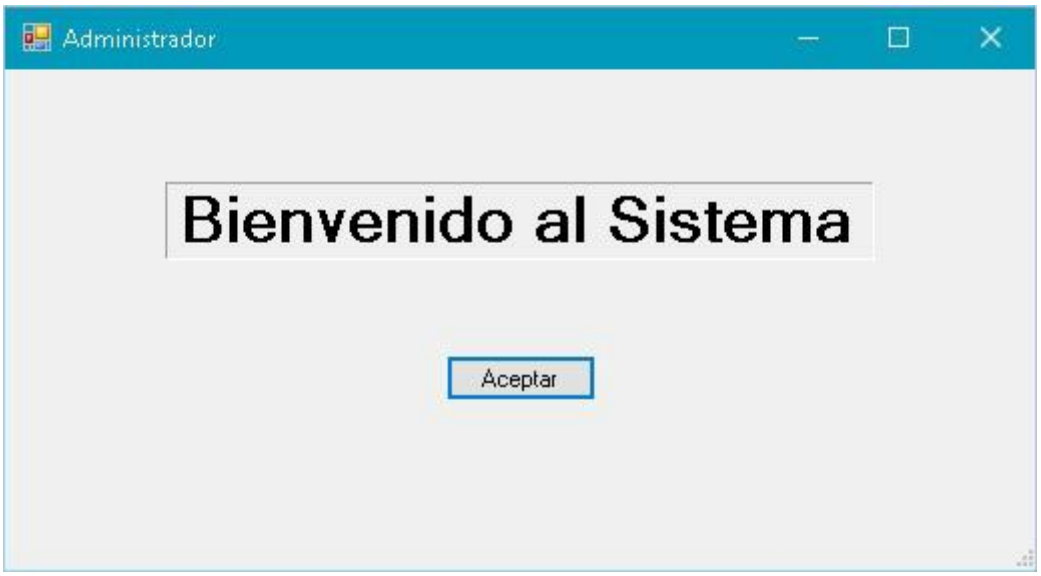
Así finalmente obtenemos la aplicación en funcionamiento:



Si los datos ingresados no son correctos obtenemos el mensaje de error:

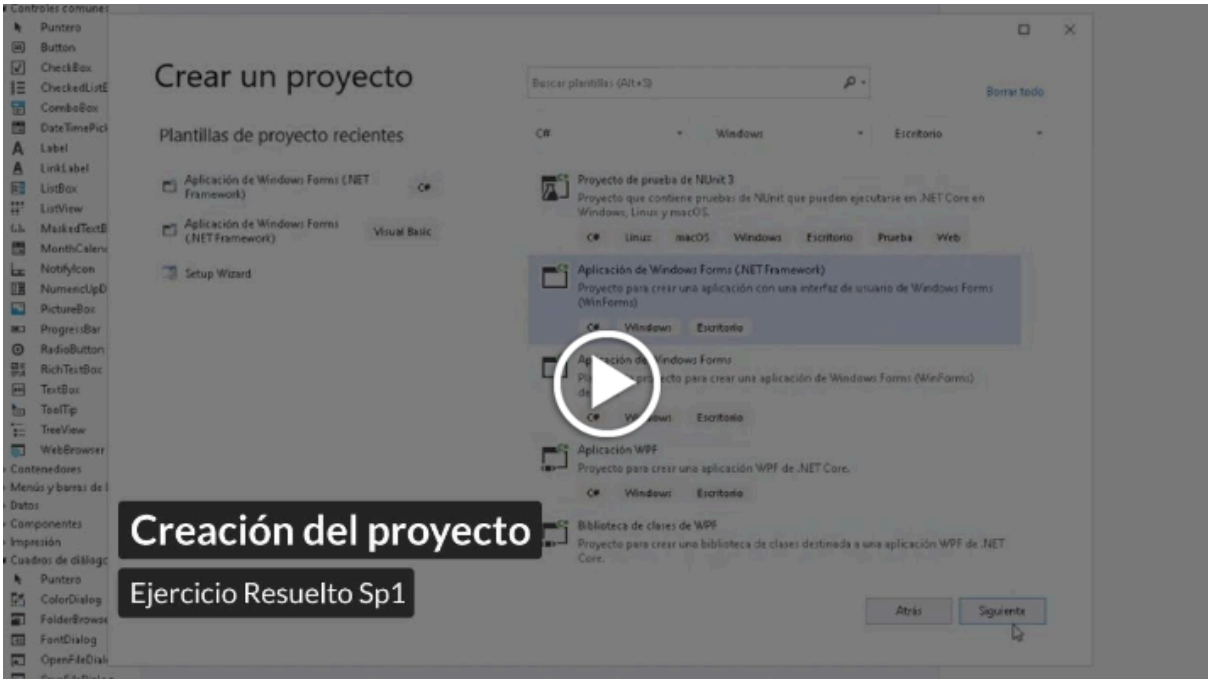


Si los datos de usuario y contraseña son correctos accedemos al sistema:



Videos de la creación, diseño, implementación y ejecución de la aplicación:

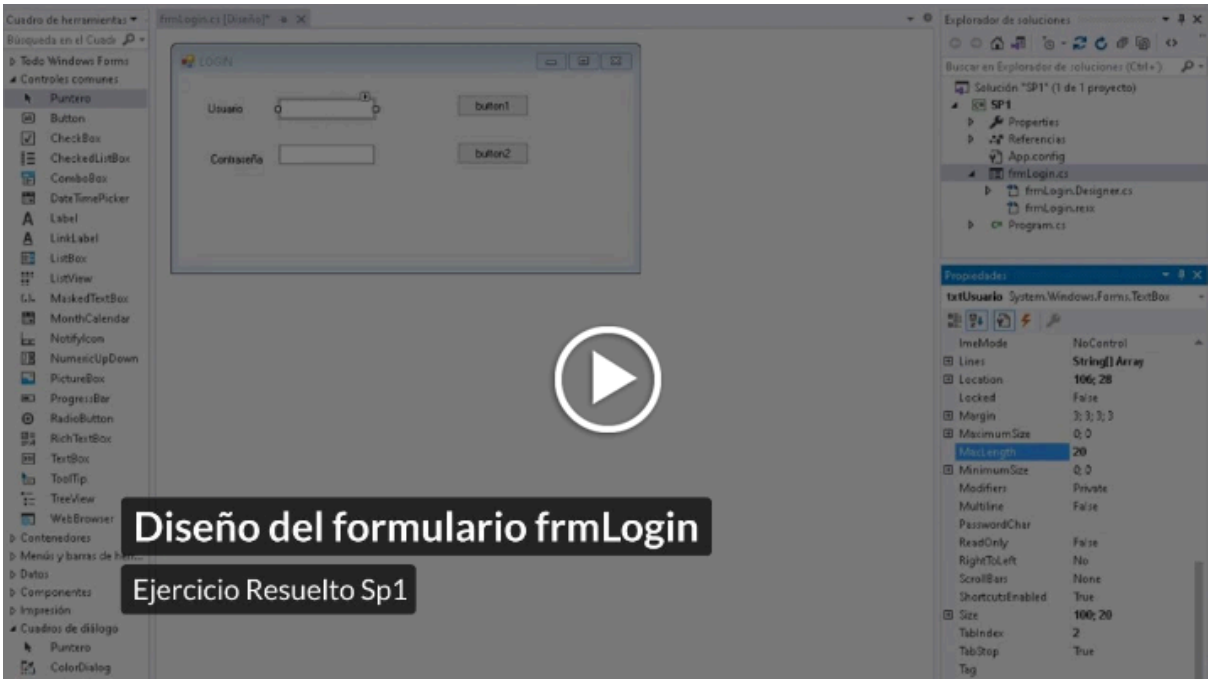
Creación del proyecto



Fuente: Carlos Gonzalez. (2021, diciembre 12). SP1 Crear Proyecto [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=9EyaIMfSygQ

Link del video: https://www.youtube.com/watch?v=6nuQiyhW9TU

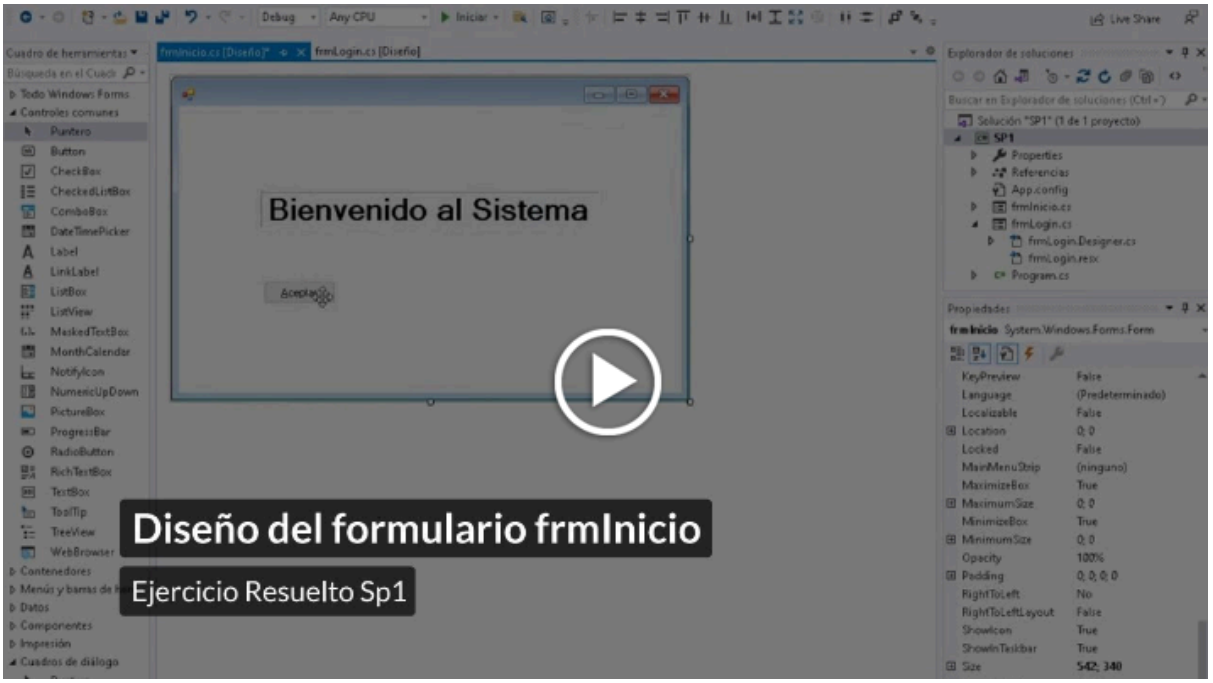
Diseño del formulario frmLogin



Fuente: Carlos Gonzalez. (2021, diciembre 12). SP1 Diseño frmLogin [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=E1weLpZW3sQ

Link del video: https://www.youtube.com/watch?v=X16YqDNYrKM

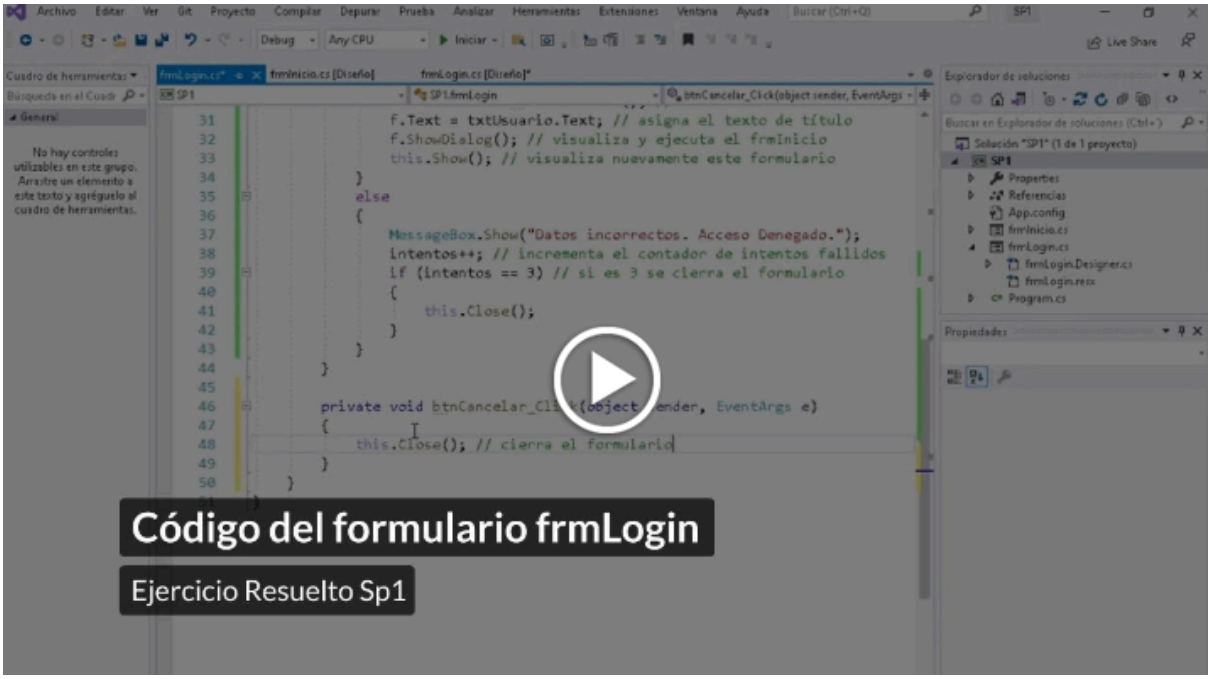
Diseño del formulario frmInicio



Fuente: Carlos Gonzalez. (2021, diciembre 12). SP1 Diseño frmInicio [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=VqWkrg-oirU

Link del video: https://www.youtube.com/watch?v=5v0nBFkH-b4

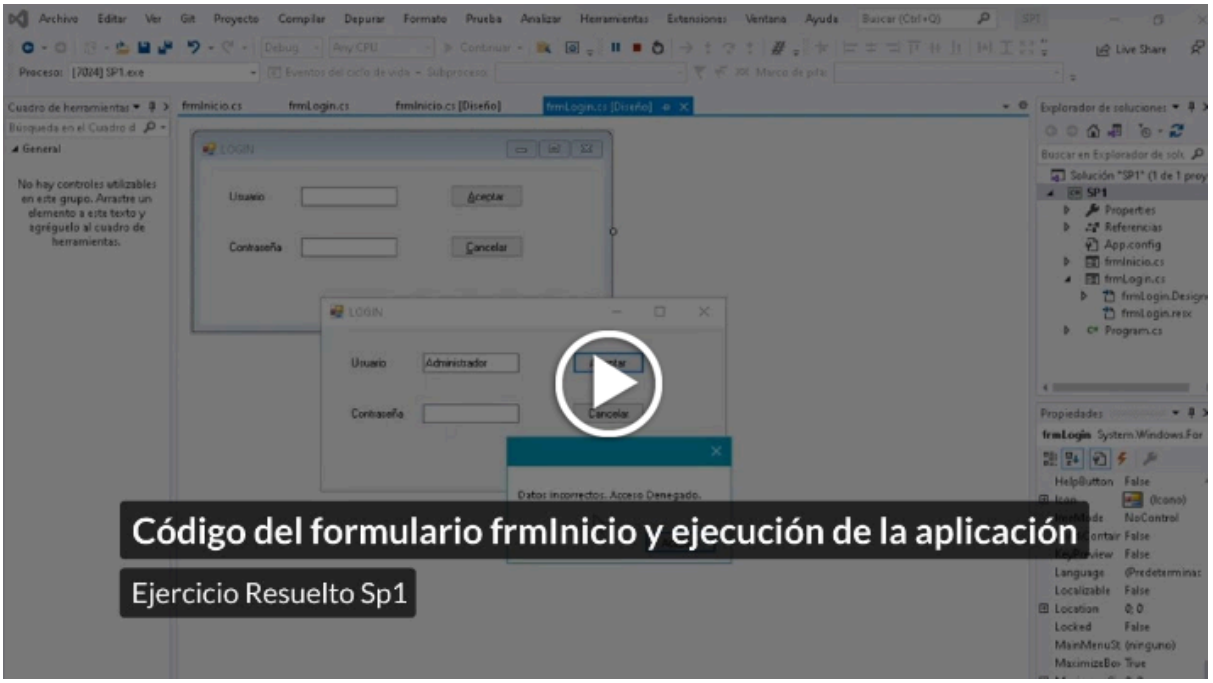
Código del formulario frmLogin



Fuente: Carlos Gonzalez. (2021, diciembre 12). SP1 Código frmLogin [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=QqaSy-HbPm0

Link del video: https://www.youtube.com/watch?v=A8x-rq9dMnE

Código del formulario frmInicio y ejecución de la aplicación



Fuente: Carlos Gonzalez. (2021, diciembre 12). SP1 Codigo frmInicio y Ejecucion [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=Nn40jx3Pysc

Link del video: https://www.youtube.com/watch?v=uMGH2iuxTvw

Descargue los **archivos del proyecto completo**

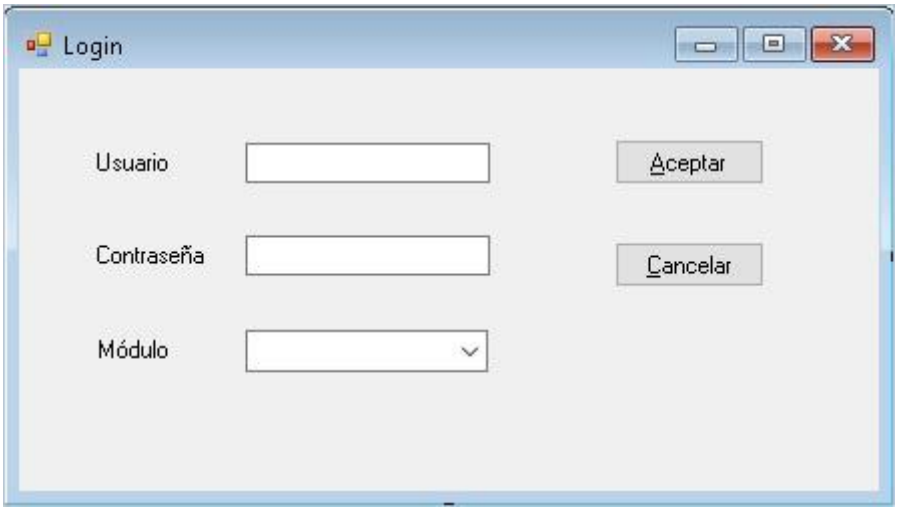
SP1/Ejercicio por resolver

Sintepart SRL, fábrica de autopartes y repuestos de automóviles *Fiat*, necesita el desarrollo de un sistema de facturación. Para ello, ha solicitado a una consultora informática el desarrollo de dicha aplicación. Usted acaba de ingresar como pasante del área de Ingeniería de software de esta última empresa, para desarrollar tareas de analista programador.

Según las indicaciones que le ha enviado el líder del desarrollo de este proyecto, deberá diseñar una primera interfaz gráfica que inicie y presente el sistema. También debe tener en cuenta que le solicitan una pantalla de Login o acceso. El mismo debe contar con una lista desplegable donde se detallan los módulos (ADM – SIST – COM – VTA). A su vez debe ingresar el usuario que accede a ese módulo.

- Primera interfaz gráfica debe ser de bienvenida al sistema con el logo de Sintepart SRL.

- Segunda interfaz gráfica como la siguiente imagen:



Debe configurar la caja de texto de usuario:

- máximo de letras 10.
- fuente de color azul.

Debe configurar la caja de texto de contraseña:

- máximo de letras 10.

- configure para que aparezcan "#" en vez de los caracteres que realmente se escriben.

Para el módulo se coloca un control de tipo comboBox que mostrará estas opciones:

ADM → Administración

SIST → Sistemas

COM → Compras

VTA → Ventas

Usuarios:

Adm → Administrador. Puede acceder al módulo ADM, COM y VTA,

Contraseña: @1a

John → puede acceder solo al módulo SIST. Contraseña: *2b

Ceci → puede acceder a ADM y VTA. Contraseña: *@3c

God → puede acceder a todos los módulos. Contraseña: *@#4d

El evento Click del botón "Aceptar" debe controlar que los datos ingresados sean correctos, si es así y el usuario tiene acceso al módulo seleccionado entonces se ocultará el formulario de Login y se mostrará el formulario de Bienvenida, en caso contrario se mostrará un cuadro de mensajes con el texto "Usuario y/o contraseña incorrectos para el módulo seleccionado". Si la falla se presenta dos veces en forma consecutiva se cerrará el formulario de Login.

El evento Click del botón "Cancelar" debe cerrar el formulario.

SP1/Evaluación de paso

Indique si es verdadero o falso

1) Un framework es utilizado como base para la programación, brindándonos clases y librerías.

- ○ Verdadero
- ○ Falso

2) Originalmente, el lenguaje Visual Basic se llamaba QBasic.

- ○ Verdadero
- ○ Falso

3) NET Framework tiene una serie de componentes que permiten procesar información de código fuente, comprensible para el programador, a lenguaje máquina:

- ○ Verdadero
- ○ Falso

4) Cuando creamos y guardamos un proyecto en Visual Studio, los archivos que debemos transportar o almacenar son solamente aquellos que tienen la extensión .vbproj.

- ○ Verdadero
- ○ Falso

Indique las opciones correctas

5) Indique las características generales de una interfaz gráfica para una aplicación de escritorio:

- ☐ Ventana
- ☐ Colores
- ☐ Icono
- ☐ Barras de desplazamiento
- ☐ Objetos 3D
- ☐ Imágenes
- ☐ Barra de título

Indique la opción

6) ¿Cuál de las siguientes características permiten al Framework .NET el acceso a datos?

- ○ CLR
- ○ Bibliotecas de clases
- ○ MSIL
- ○ ADO. NET

Respuestas correctas[25]

[25] 1)Verdadero; 2)Falso; 3)Verdadero; 4)Falso; 5)Icono,Barras de desplazamiento,Barra de título; 6)ADO. NET.

Situación profesional 2: La lógica en la programación

El administrador de un complejo de cabañas, le ha solicitado a usted, en calidad de pasante, el desarrollo de una aplicación para el registro de la reserva de cabañas.

Existen dos tipos de cabañas, la cabaña tipo "A" para 4 personas y la cabaña tipo "B" para 8 personas; sus precios base por día son US$ 20 y US$ 34 respectivamente; al precio base se le añade US$ 1 por persona que ocupa la cabaña, es decir, una cabaña clase "A" con 2 personas tiene un costo diario de US$ 22.

Las cabañas están equipadas con cocina, heladera y televisión, estos adicionales tienen un costo diario de US$ 1, US$ 1,50 y US$ 2 respectivamente.

La forma de pago puede ser en efectivo o con las siguientes tarjetas: Card Red, Card Green y Card Blue. La tarjeta Card Red tiene un recargo del 10% y las otras tarjetas un recargo del 20%.

El estado inicial de la interfaz es: el combo tipo de cabaña tiene el valor "A", el combo personas el valor 1, la caja de texto días el valor 1, las casillas de verificación de los adicionales no están marcadas, el botón de opción efectivo está seleccionado, el combo tarjetas está deshabilitado, las cajas de texto para ingresar el nombre del titular de la reserva y los teléfonos de contacto están inicializados, el botón de comando "Aceptar" está deshabilitado.

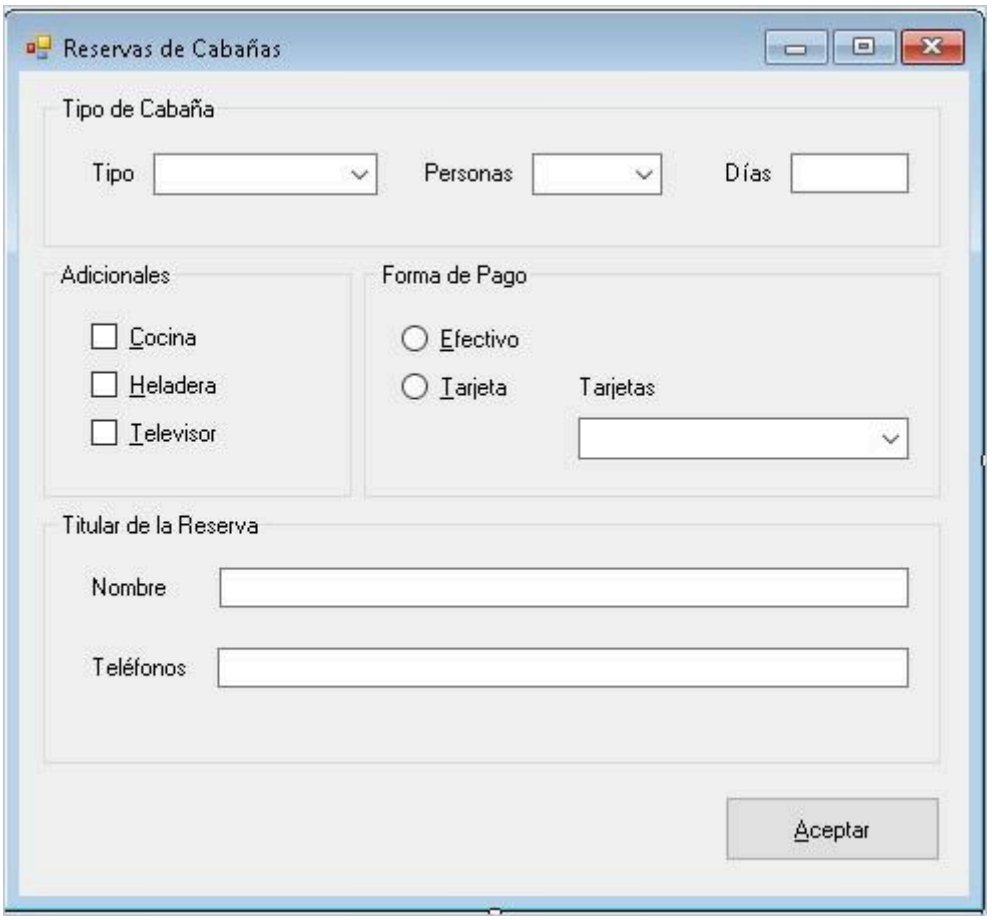
En el combo personas tiene los valores de 1 a 4 si la clase de cabaña es "A" y los valores de 1 a 8 si la clase de cabaña es "B".

El combo tarjetas se habilita y queda seleccionada la tarjeta Card Red al elegir en forma de pago el botón de opción tarjeta. Cuando se elige el botón de opción efectivo el combo tarjetas se deshabilita y no queda seleccionada ninguna tarjeta.

El botón de comando "Aceptar" se habilita si la caja de texto días contiene un valor mayor a cero, se ingresa el nombre del titular y los teléfonos de contacto en las cajas de texto. Si alguno de los datos mencionados anteriormente falta, el botón se deshabilita.

Al pulsar el botón de comando "Aceptar" se muestra en un cuadro de diálogo el costo de la reserva y a continuación se establece la interfaz a su estado inicial.

El formulario solicitado debe tener este diseño:



SP2/H1: Diseño de formularios y controles estándares

Para desarrollar lo que nos solicitó el cliente, vamos a utilizar el lenguaje **C#**, integrado en **Microsoft Visual Studio**. Para poder realizar esta tarea, debemos conocer cómo es el entorno luego de la instalación del mismo.

En esta situación profesional usaremos el formulario (**Form**), ya que es el principal elemento de interfaz gráfica utilizada para el desarrollo de aplicaciones. En el formulario mostrado en el ejercicio resuelto de la situación profesional anterior se ve el diseño de los controles y las propiedades utilizadas para configurarlo. Analizaremos cada una de las partes y controles ingresados.

Definición de soluciones y proyectos

Visual Studio dispone de dos ***contenedores*** que nos ayudan a administrar los elementos necesarios para el desarrollo de programas, como ser: referencias, conexiones de datos, carpetas y archivos. Estos contenedores se denominan **soluciones** y **proyectos**.

Las **soluciones** contienen los elementos necesarios para crear aplicaciones. Una solución incluye uno o más proyectos, archivos y metadatos que ayudan a definir la solución en conjunto. Se genera automáticamente una solución al crear un nuevo proyecto, también se almacena la definición para una solución en dos archivos: **.sln** y **.suo** (*características que vimos en herramientas anteriores*). El archivo de definición de solución (**.sln**) almacena los metadatos que definen la solución, como:

- Los proyectos asociados a la solución.
- Los elementos que no están asociados a un proyecto determinado.
- Las configuraciones de compilación que determinan las configuraciones de proyecto a aplicar a cada tipo de compilación.

Los metadatos que se almacenan en el archivo **.suo** durante la creación de una solución y la configuración de sus propiedades, se utilizan para personalizar el **IDE** cuando la solución esté activa.

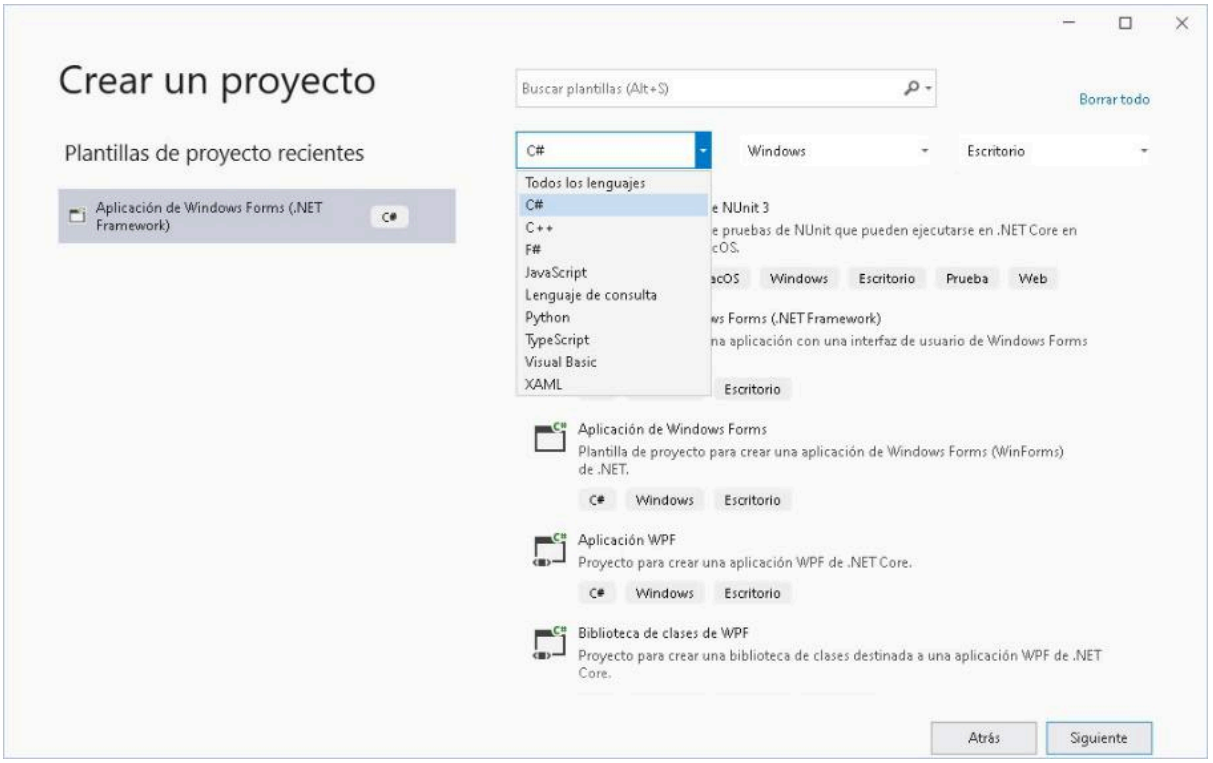
Los **proyectos** en una solución se utilizan para administrar de forma lógica, compilar, y depurar los elementos que forman parte del desarrollo. El resultado de un proyecto suele ser un programa **ejecutable (.exe)**, un archivo de biblioteca de vínculos dinámicos **(.dll)** o un módulo, entre otros.

En Visual Studio podemos encontrar varias plantillas predefinidas de proyecto. Puede utilizar estas plantillas para crear el contenedor de tipo proyecto básico y un conjunto de elementos adicionales que podría necesitar para desarrollar la aplicación, clase, control, o de biblioteca. Por ejemplo: crear una aplicación para Windows, el proyecto le facilitará un elemento de formulario **Windows Forms** para que pueda personalizarlo. Del mismo modo, si decide crear una aplicación Web, el proyecto le ofrecerá un elemento de formulario **Web Forms**.

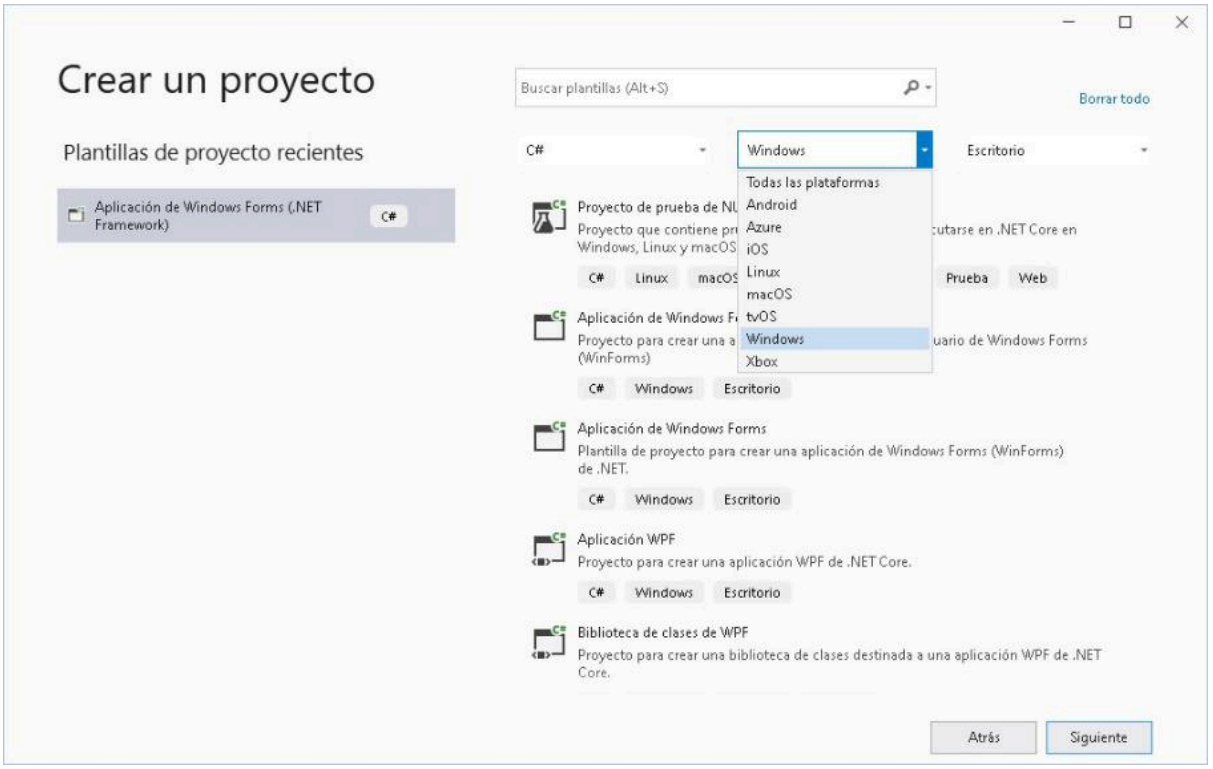
Creando un proyecto en Visual Studio

Como *Visual Studio* dispone de muchos tipos de proyectos con distintos lenguajes y plataformas de ejecución hay que prestar atención a las opciones que se seleccionen en la creación de un nuevo proyecto. Ya comentamos anteriormente que vamos a trabajar con el lenguaje C#, para la plataforma de *Windows* y con aplicaciones de escritorio, esos tres elementos los elegiremos en el momento de crear un nuevo proyecto.

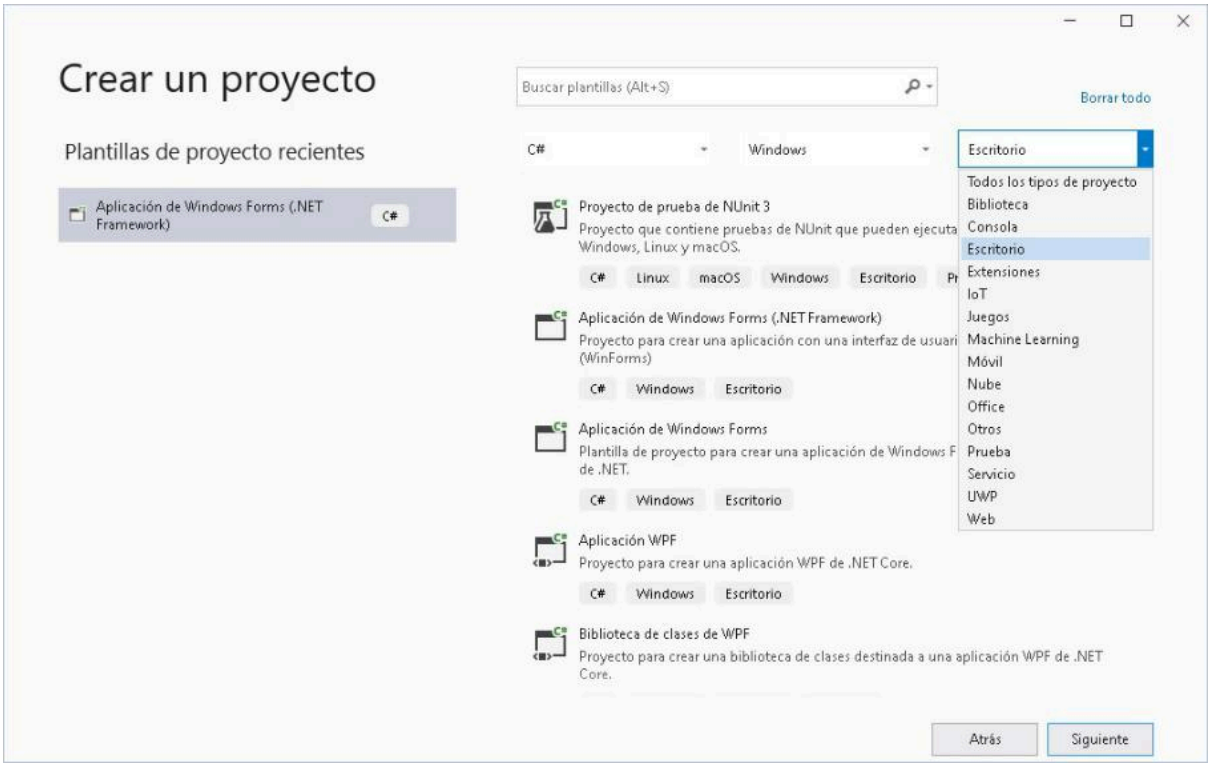
La primer pantalla que nos muestra *Visual Studio* para comenzar a crear un proyecto es esta:



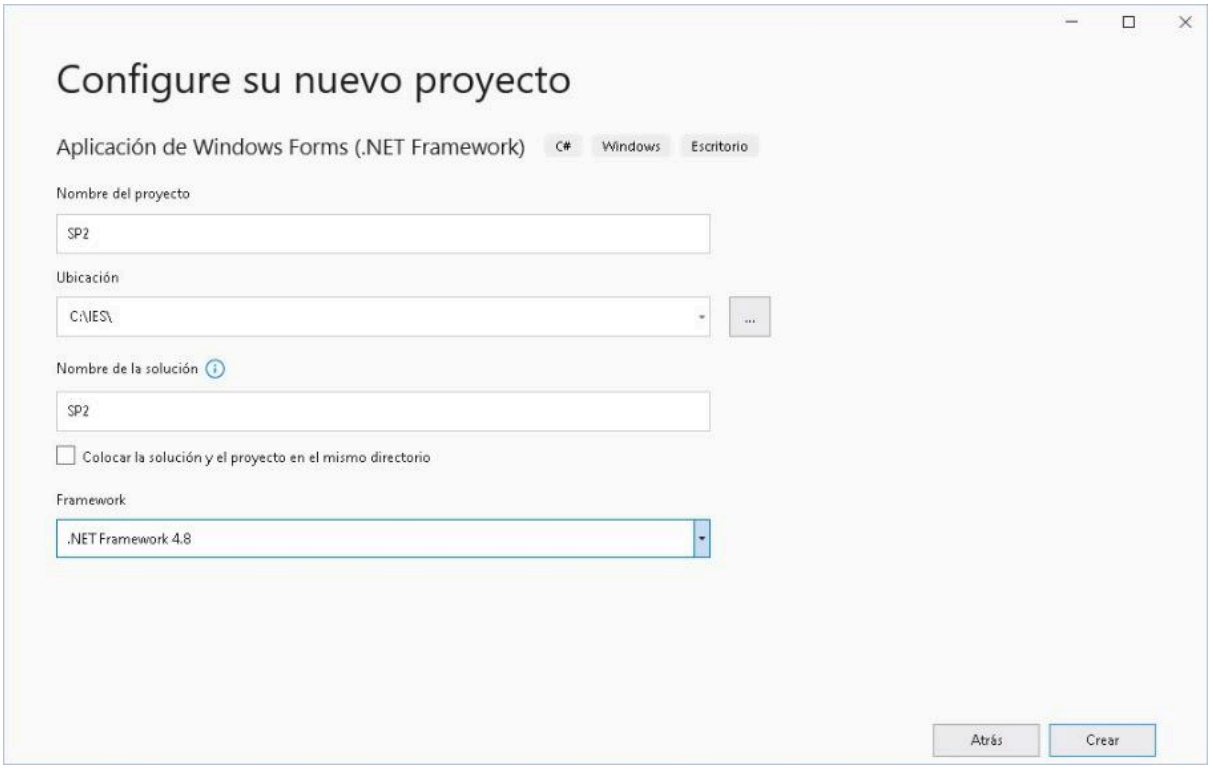
Observe la lista desplegable con los lenguajes disponibles, elegimos el lenguaje "C#", luego:



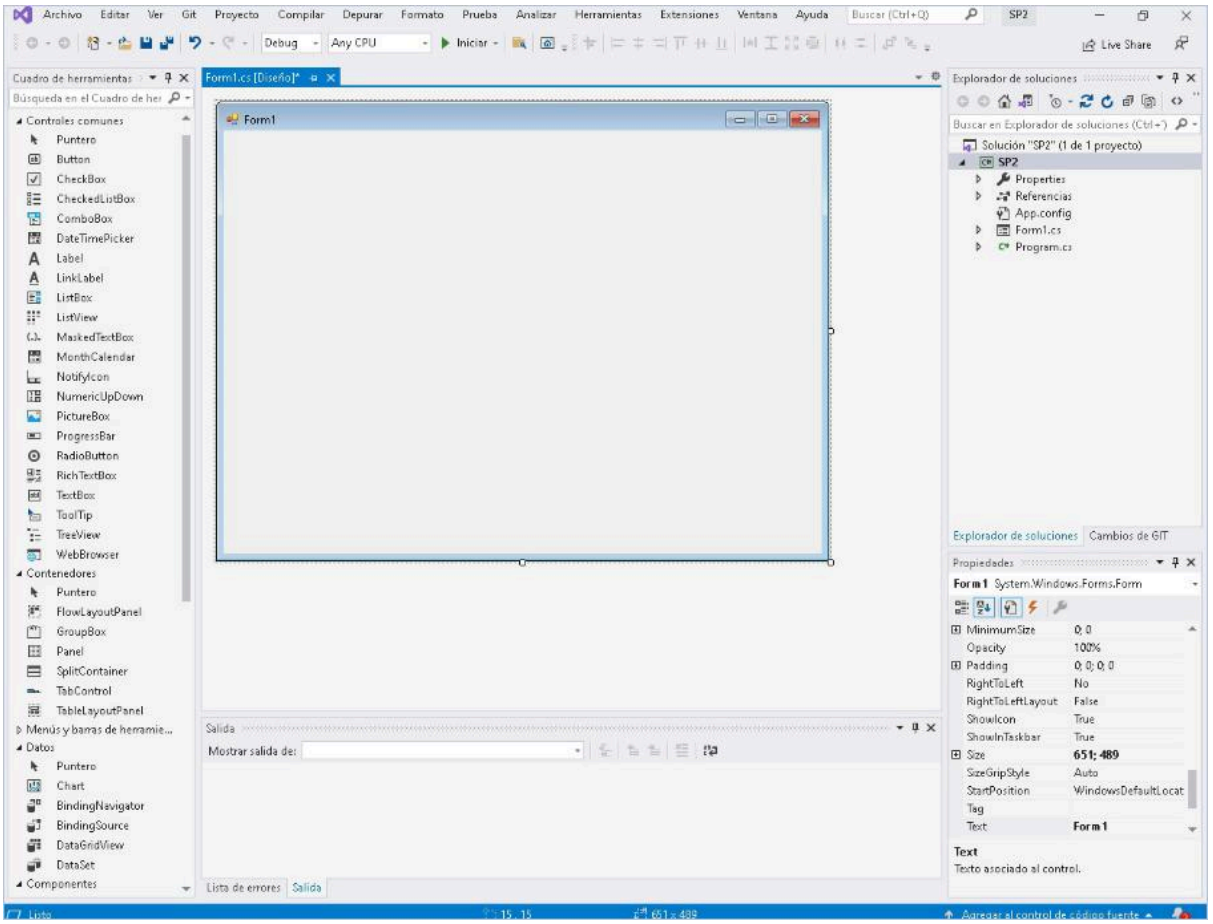
En la segunda lista desplegable seleccionamos la plataforma destino de nuestra aplicación, es decir sobre qué sistema operativo se va a ejecutar, elegimos "*Windows*".



En la tercer lista desplegable elegimos el tipo de aplicación a desarrollar, elegimos "**Escritorio**" ya que de ese tipo serán todas las aplicaciones que desarrollaremos en esta materia. A continuación, click en el botón "**Siguiente**".



En esta pantalla debemos escribir el nombre que tendrá el proyecto, por ejemplo, "SP2", elegimos la ubicación en el disco que le queremos dar y finalmente elegimos la versión del *Framework* de .NET que usaremos, por defecto nos mostrará la última versión disponible, *Framework 4.8* en el ejemplo anterior, se recomienda mantener la opción por defecto. Ahora click en el botón "**Crear**" y ya tendremos el proyecto listo para empezar a trabajar en él.



Al finalizar la creación del proyecto tendremos el entorno de desarrollo de *Visual Studio* y en el centro vemos el formulario que debemos diseñar y programar para nuestra aplicación.

El primer objeto con que nos encontramos es el **formulario**. De hecho, cada vez que iniciamos *Visual Studio* y elegimos un proyecto de ***Windows Form***, nos presenta en pantalla un nuevo formulario, que tiene por defecto el nombre de **Form1**.

El formulario es un objeto que sirve de contenedor de otros objetos. El nombre de *formulario* lo toma precisamente porque, al igual que un formulario de papel, contiene

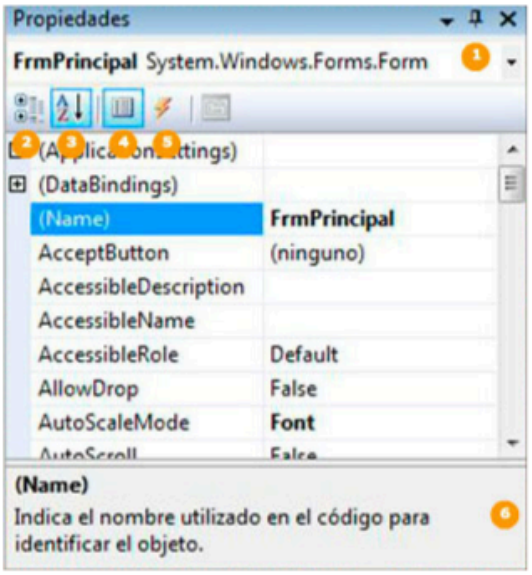
textos, lugares donde podemos escribir, figuras, cajas en las que podemos elegir una u otra opción, etcétera. En este cuadro gráfico que llamamos *formulario*, podemos introducir todo lo detallado anteriormente.

En realidad, un formulario es lo que normalmente conocemos por **Ventana**. El nombre de formulario le viene muy bien cuando ese formulario es una ventana donde vamos a introducir datos alfanuméricos. Cuando en lugar de introducir datos, lo que tenemos es una ventana, en la que se reproducen exactamente los controles de operación de una máquina, parece en principio que sería más correcto el nombre de "ventana".

Posiblemente a lo largo de este texto nos referiremos al formulario con el nombre de ventana, o incluso, pantalla. Procuraremos usar el nombre apropiado: formulario, en el futuro se encontrará también con *WebForm*, formularios web.

Como cualquier otro objeto de *Visual C# .Net*, un formulario tiene propiedades métodos y eventos; pasemos a analizar qué función cumplen los más importantes:

- **Propiedades:** para visualizar el cuadro de Propiedades, presione F4, sino también puede ubicarlo desde el menú **Ver / Ventana Propiedades**. Las propiedades están agrupadas de la siguiente manera:



1 - Objeto seleccionado

2 - Ordena las propiedades por categoría

3 - Alfabético (como muestra actualmente la imagen)

4 - Propiedades: muestra las propiedades del objeto seleccionado

5 - Eventos: muestra los eventos del objeto seleccionado

6 - Descripción de la propiedad seleccionada

El cuadro de **Propiedades** permite visualizar la definición de la propiedad al momento de seleccionarla. Por ejemplo, para la propiedad *Opacity* (que le da al formulario un efecto de transparencia, el cual mejora notablemente el aspecto de la misma) veremos la definición "**Porcentaje de la opacidad del control**".

Entre las propiedades más utilizadas, tenemos las siguientes:

- **Apariencia**
  - *BackColor:* el color de fondo del componente.
  - *Font:* fuente utilizada para mostrar texto en el control.
  - *ForeColor:* color de primer plano de este componente, utilizado para mostrar texto.
  - *Text:* texto asociado al control (Título del formulario).

- **Comportamiento**
  - *Enabled:* indica si el control está habilitado.

- **Diseño**
  - *Name:* indica el nombre utilizado en el código para identificar el objeto.
  - *AutoSize:* especifica si un control cambiará automáticamente de tamaño para ajustarse a su contenido.
  - *Size:* tamaño del control, en píxeles.
  - *StartPosition:* determina la posición del formulario cuando aparece por primera vez.
  - *WindowState:* determina el estado visual inicial del formulario.

- **Estilo de Ventana**
  - *ControlBox:* determina si el formulario tiene un cuadro de menú Control o Sistema.
  - *Icon:* indica el icono para el formulario.
  - *IsMdiContainer:* determina si el formulario es un contenedor MDI.
  - *MaximizeBox:* determina si el formulario tiene un cuadro para maximizar en la esquina superior derecha de la barra de título.
  - *MinimizeBox:* determina si el formulario tiene un cuadro para minimizar en la esquina superior derecha de la barra de título.

- *ShowInTaskBar:* determina si el formulario aparece en la barra de tareas de *Windows*.
- *AcceptButton:* Botón Aceptar del formulario. Se activa al presionar ENTER.
- *CancelButton:* Botón Cancelar del formulario. Se activa al presionar ESC.

➢ **Eventos:** un evento es una acción producida o generada por el usuario o por el programa. Un evento puede ejecutar un método que se comporte como procedimiento (no devuelve valores) o como un método (si devuelve valores). Para visualizar los eventos debemos ver el cuadro de Propiedades, presione **F4**, sino también puede ubicarlo desde el menú **Ver / Ventana Propiedades**, en el cuadro de **Propiedades** hace clic sobre el botón **Eventos**. Algunos de los eventos más frecuentes que se desencadenan en un formulario, son:

- **Click:** ocurre cuando el usuario hace click con el mouse sobre el formulario.
- **Load:** ocurre cuando se ejecuta el formulario y antes de mostrarlo en pantalla ejecutará todo el código que contenga dicho evento.
- **Resize:** ocurre cuando se cambia el tamaño del formulario.

Para ver qué eventos tiene el objeto formulario, basta con seleccionarlo, ir a la caja de **Propiedades** y presionar el botón **Eventos**.

Un proyecto puede tener varios formularios y dentro de cada formulario, los controles necesarios para cada paso o estado de la aplicación (*como vimos en el ejercicio resuelto de la situación profesional 1*). En nuestra situación profesional haremos uso de varias de las propiedades y eventos mencionados anteriormente. Por ejemplo, para cargar un **ComboBox** (*lista desplegable*), lo haremos en el evento **Load** del formulario. También utilizaremos las propiedades **Text**, **Name**, etc. Por último, podremos decir que un formulario puede servir también como soporte de textos o dibujos.

Hasta aquí, hemos visto el uso general del entorno de desarrollo *Visual Studio*. Con el mismo podremos comenzar a confeccionar el formulario principal para el programa que debemos realizar para el usuario/cliente de nuestra actual situación profesional.

SP2/Autoevaluación 1

Indique la opción correcta

1) ¿Con qué propiedad se puede establecer si el tamaño del formulario puede ser modificado por el usuario cuando la aplicación se está ejecutando?

- ○ WindowState
- ○ FormBorderStyle
- ○ AutoSizeMode

2) ¿Cuál es el evento que se ejecuta al inicializar un formulario?

- ○ WindowStart
- ○ FormLoad
- ○ FormOpen
- ○ LoadForm

3) ¿Cuál es la finalidad de establecer las propiedades del formulario, como tamaño, título, color, etc.?

- ○ Mejorar la apariencia de la aplicación para el usuario.
- ○ Una configuración estándar.

Indique si es verdadero o falso

4) El Formulario es el soporte de los objetos que componen una interfaz gráfica.

- ○ Verdadero
- ○ Falso

Indique las opciones correctas

5) ¿Cuáles son propiedades básicas de un formulario para controlar su apariencia?:

- ☐ WindowState
- ☐ Cursor
- ☐ Key Preview
- ☐ FormBorderStyle

Respuestas correctas[26]

[26] 1)FormBorderStyle; 2)FormLoad; 3)Mejorar la apariencia de la aplicación para el usuario; 4)Verdadero; 5)WindowState,FormBorderStyle.

SP2/H2: Controles

Tal como la situación profesional nos pide, tendremos que hacer un formulario para resolver lo solicitado, pero con el formulario no resolvemos la situación planteada; nos faltan los objetos fundamentales para enviar y recibir datos desde la aplicación: estos objetos son los **controles**. Entonces, una vez introducido un Formulario, se pueden colocar los objetos (controles) que forman parte de la aplicación.

Entre los controles principales que vienen instalados por defecto en (*recordemos que Visual Studio es un entorno integrado capaz de desarrollar en múltiples lenguajes de programación y en distintas tecnologías, entre ellas: aplicaciones de consola, aplicaciones para móviles, aplicaciones web, servicios web, etc.*) o también denominados **controles estándar** o **controles comunes**, como los que se muestran en imágenes en los próximos párrafos. Los controles, lo mismo que el Formulario, tienen propiedades y métodos.

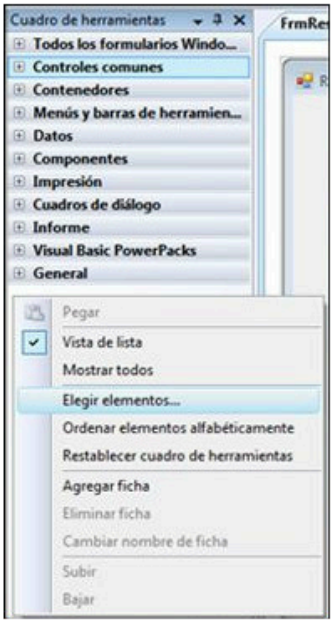
Las **propiedades** definen las características del control. Los **métodos** definen la forma de actuar (la forma de proceder) de un control frente a un evento.

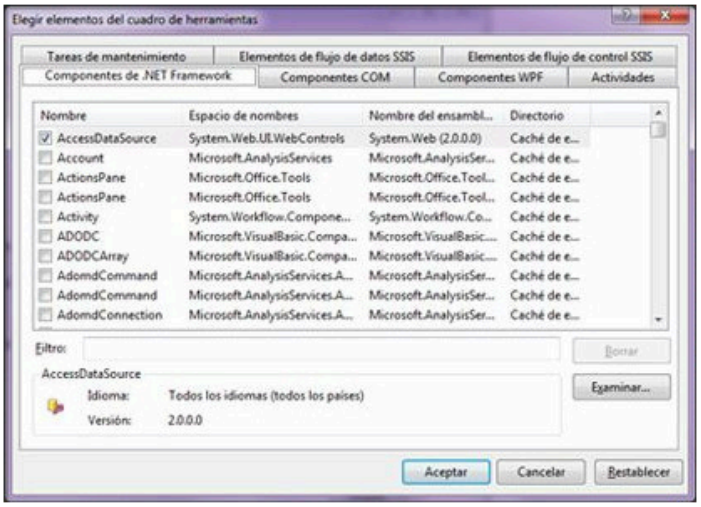
Tanto las propiedades como los métodos deben ser **definidos**[27] por el programador. Las propiedades dejan un pequeño margen de actuación, los métodos son en su totalidad definidos por el programador. Estos métodos forman parte del programa. Podría incluso hacerse una aplicación que no tuviese más código que el introducido en los métodos.

Para colocar un control en un formulario, basta con arrastrarlo desde el cuadro de herramientas hasta el mismo. Se pueden agregar otros controles al cuadro de herramientas presionando el botón derecho del mouse sobre él y seleccionando la opción "Elegir elementos". Aparecerá la siguiente pantalla con el listado de controles instalados en la computadora:

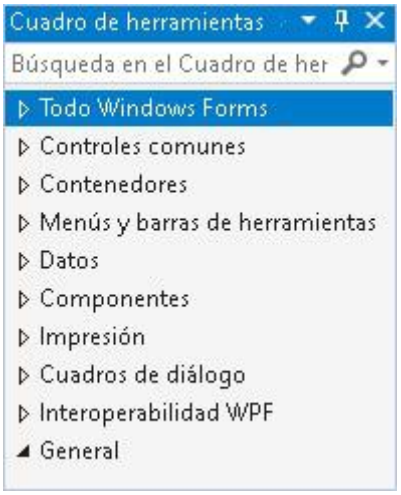
[27] Definidos: Se refiere a que deben ser programados por el usuario para que dicho método tenga funcionalidad.

Elegir elementos del cuadro de herramientas





Los controles *Windows* están agrupados en las siguientes categorías:



Ventanas o Formularios

Es el componente infaltable en cualquier interfaz GUI, es el área que ocupa un programa para desplegar todos sus componentes de interfaz con el usuario. Normalmente, suele ser rectangular y dispone de ciertos controles implícitos para controlar su apariencia, tamaño y ubicación.

En la imagen que sigue se visualizan tres ventanas, una de ellas está sobre las demás. Ésta se denomina ventana activa, ya que es la ventana en la que se está trabajando.

Note que los colores de la ventana activa son más oscuros que los colores de las ventanas que no están activas.

Ventanas sobre el escritorio de Windows



Propiedades de las Ventanas

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo frm (abreviatura del nombre general de las ventanas en inglés: Form) e **identifica unívocamente**[28] a la ventana en cuestión. Ejemplos de nombres: frmClientes, frmFacturas, frmControlSonido

- **Color del Fondo:** determina el color del fondo de la ventana. A veces se denomina área de trabajo de la ventana.

- **Título:** es el mensaje que aparecerá en la parte superior (barra de título). Generalmente tiene un tipo de letra, tamaño, alineación y color común a todas las ventanas del sistema.

- **Ancho:** determina el ancho de la ventana. (el espacio que ocupará sobre el escritorio).

[28] Propiedad Nombre: absolutamente todos los objetos GUI tienen un nombre que los identifica de manera unívoca en cada contexto o aplicación. De no tenerlo, no podríamos referirnos a ellos en el código. ¿Identificación unívoca en cada contexto? Quiere decir que no puede haber dos ventanas con el mismo nombre en una misma aplicación. Del mismo modo, no puede haber dos controles con el mismo nombre en una misma ventana, pero sí en ventanas diferentes.

- **Alto:** determina la altura de la ventana.
- **Estado (Activa/Inactiva):** cuando una ventana está activa, está disponible, se encuentra ubicada encima de las demás y el color de su barra de título es más fuerte que las otras. Cuando una ventana está inactiva, no está disponible, y el color de su barra de título es más tenue. Siempre es la ventana Activa la que recibe los comandos de teclado que el usuario teclee.
- **Tipo de Borde (Fijo / Redimensionable):** algunas ventanas tienen bordes fijos, es decir, que no se pueden redimensionar. Otras, en cambio, sí permiten que el usuario modifique su tamaño arrastrando y soltando sus bordes.
- **Ubicación (Coordenadas):** lugar donde estará situada la ventana con respecto a la pantalla.
- **Barra de desplazamiento (Vertical, Horizontal, Ambas o Ninguna):** son las barras horizontales y verticales que nos permiten desplazar el texto hacia arriba o abajo, hacia la derecha o la izquierda y así poder visualizarlo correctamente. Las ventanas pueden tener barras de desplazamiento o prescindir de ellas.
- **Ícono:** puede asignar un archivo icono (.ico) que aparecerá en la barra de título.
- **Botón maximizar:** puede activarse o desactivarse el botón en el formulario.
- **Botón minimizar:** puede activarse o desactivarse el botón en el formulario.

Encontrará todos los nombres equivalentes en Visual Studio en la siguiente página web.

Eventos de las Ventanas

- **Inicializar:** ocurre en el momento en que la ventana "se abre" por primera vez. No ocurre si estaba minimizada y se restaura.
- **Cerrar:** ocurre cuando la ventana se va a cerrar.
- **Foco:** ocurre cuando estando de fondo, la ventana se activa.

- **PerderFoco:** ocurre cuando la ventana deja de ser la ventana activa (porque alguna otra se ha vuelto la ventana activa).
- **Redimensionar:** ocurre cuando la ventana cambia de tamaño (a consecuencia de las acciones del usuario arrastrando sus bordes, maximizándola, minimizándola o restaurándola).
- **Pulsación de una tecla:** captura la pulsación del teclado que se genera en el objeto.

Encontrará todos los nombres equivalentes en *Visual Studio* en la siguiente página web.

Métodos de las Ventanas

- **Cerrar:** cierra la ventana.
- **Maximizar:** maximiza la ventana.
- **Minimizar:** minimiza la ventana.
- **Enfocar:** hace que la ventana se convierta en la ventana activa en el sistema.
- **Ocultar:** oculta el objeto de la pantalla.
- **Mostrar:** invoca o muestra el objeto en la pantalla.

Encontrará todos los nombres equivalentes en *Visual Studio* en la siguiente página web.

Ejercitación: busque todos los nombres equivalentes, dentro de Visual Studio, para cada una de estas propiedades, eventos y métodos, identificando su par en el lenguaje de programación. Realice esta ejercitación con todos los controles que se desarrollan a continuación.

Controles para mostrar e introducir texto

Los controles de Etiquetas (Label) y Cuadros de Texto, también llamadas Cajas de Texto, (textBox) se usan para mostrar o introducir texto. Utilice etiquetas cuando desee que la aplicación muestre texto en un formulario y utilice cajas de texto cuando desee

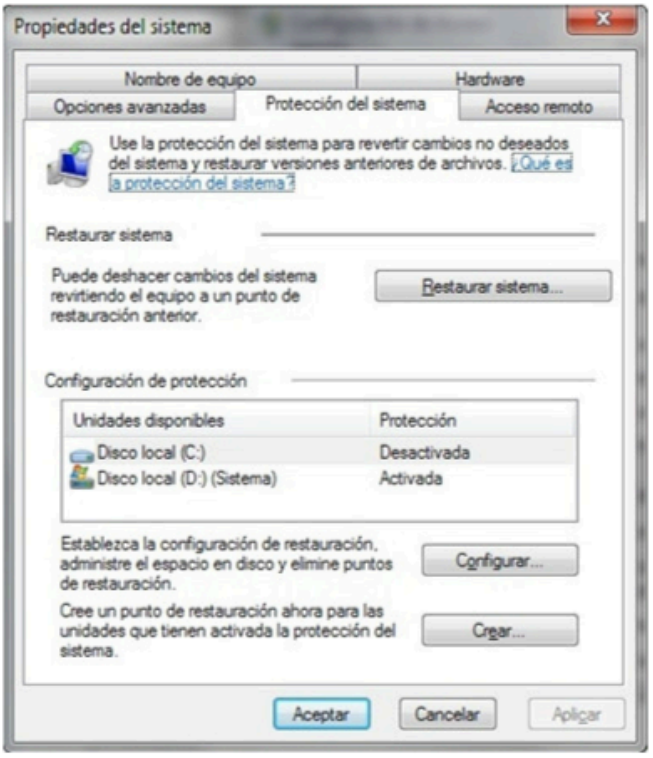
permitir al usuario ingresar o escribir texto. Las etiquetas contienen texto que solo se puede leer mientras que los cuadros de texto contienen texto que se puede modificar.

Para proporcionar esta caracterìstica	Use este control
Texto que el usuario puede ingresar o modificar; por ejemplo, un campo de entrada de pedidos o un cuadro de contraseña.	Caja de texto
Texto que solo se muestra; por ejemplo, para identificar un campo de un formulario o mostrar instrucciones al usuario.	Etiqueta

Las etiquetas y las cajas de texto se explican en las secciones siguientes:

Etiqueta (label)

Es una sucesión de caracteres que se utilizan para mostrar títulos o mensajes en pantalla. Estos datos son fijos, es decir, no se modifican.



Formulario de las propiedades generales de Windows.
Tanto los tìtulos como los resultados se muestran en etiquetas.

Propiedades de las etiquetas

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo lbl (por su acrónimo en inglés, "*label*"). Ejemplos de nombres: lblEstado, lblMemoriaVirtual, lblCategoríaIVA.
    - Sólo nos interesa darle nombre a las etiquetas sobre las que actuaremos desde el código del programa, por ejemplo, modificando su título. Las etiquetas que no sufrirán ningún tipo de modificación durante la ejecución del programa y con las que el usuario no interactuará de modo alguno no se tienen en cuenta para asignarles nombres particulares.
- **Color del Fondo:** determina el color del fondo de la etiqueta. Suele ser el mismo que el de la ventana.
- **Título:** es la leyenda que presenta.
- **Color, Tipo, Tamaño y Estilo de Letra:** características gráficas del texto del título.
- **Ancho, Alto, Ubicación:** las coordenadas de ubicación son relativas a la ventana y no al escritorio.

Encontrará todos los nombres equivalentes en *Visual Studio* en la siguiente página web.

Eventos de las etiquetas

No suele trabajarse con eventos sobre las etiquetas, son objetos pasivos.

A considerar:

- Estos objetos son capaces de reconocer eventos que ocasionalmente puede ser necesario utilizar, tales como *Click*, *Doble Click*, *MouseMove* y *otros*.
- Existe un objeto llamado ***LinkLabel*** que se comporta como un enlace en una página web, y le permite al usuario señalar una etiqueta y hacer *click* sobre ella produciendo un cambio en la interfaz de manera automática, con el solo hecho de señalarla, y desencadenando una acción como consecuencia del *click*.

Encontrará todos los nombres de eventos en *Visual Studio* en la siguiente página web.

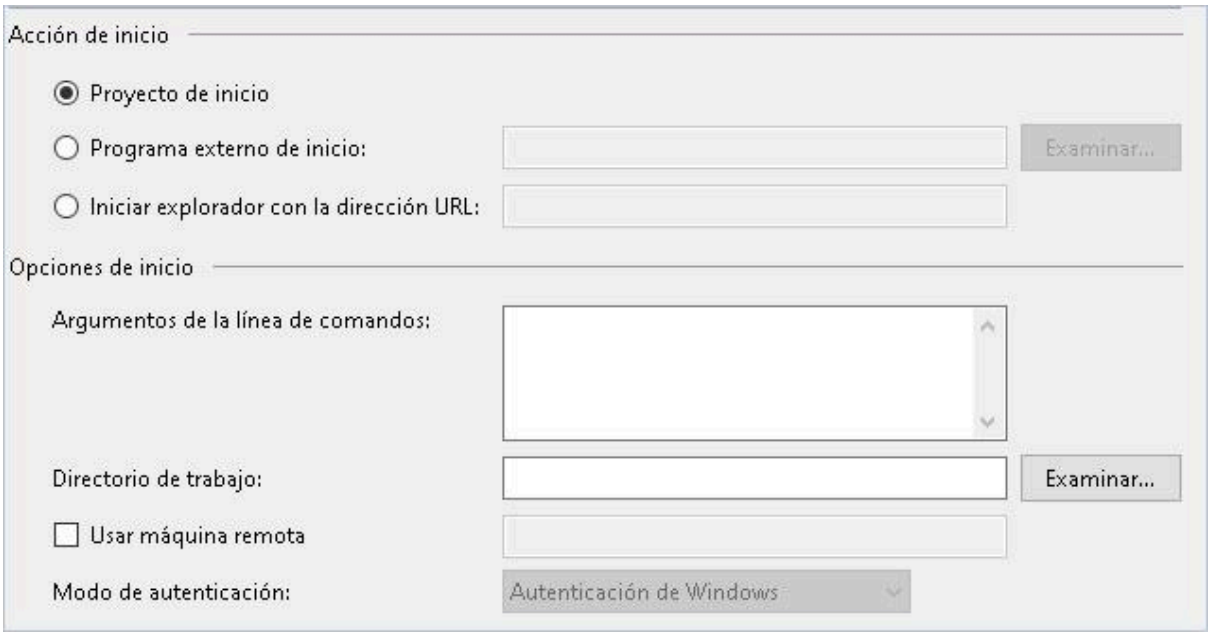
Métodos de las etiquetas

No se utilizan generalmente para ejecutar acciones, suelen ser objetos pasivos. No obstante, podemos encontrar métodos en el lenguaje de programación para hacer uso de estos controles.

Encontrará todos los nombres de métodos en *Visual Studio* en la siguiente página web.

Cajas de texto, también llamados Cuadros de texto (textBox)

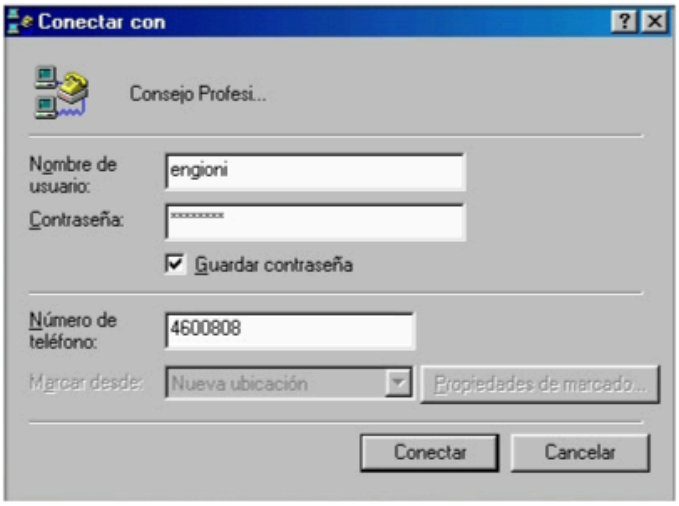
Es un control que se utiliza generalmente para que el usuario ingrese datos por teclado. Permite escribir y desplazarse sobre el texto, es decir, tiene funciones de "*scroll*" interno.



En este ejemplo pueden observarse algunos controles *textBox*, los dos primeros aparecen deshabilitados, el tercero es un *textBox* multilínea, es decir que permite escribir varias líneas de texto y el cuarto es un *textBox* de una sola línea, los dos últimos están habilitados para que el usuario pueda interactuar con ellos.

Propiedades de las Cajas de Texto

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo txt (por su acrónimo en inglés: Text Box). Ejemplos de nombres: txtPara, txtAsunto, txtNombreCliente.
- **Texto:** es el texto que está escrito en la caja de texto. A medida que el usuario modifica su contenido, esta propiedad va reflejando tal modificación.
- **Color del Fondo:** determina el color del fondo de la caja de texto. Suele ser blanco.
- **Habilitación:** Indica si el usuario podrá interactuar con el objeto o no. Los valores que puede asumir son:
    - **Habilitado:** indica que el objeto está disponible y el usuario puede interactuar con él.
    - **Inhabilitado:** indica que el objeto no está disponible y el usuario no puede interactuar con él. No puede modificar el texto que contiene. Sólo puede mirarlo. Cuando un objeto está Inhabilitado se muestra en un color atenuado (en gris) y no responde a ningún evento que sobre él se produzca.
- **Multilínea:** indica si la caja de texto tendrá varias líneas o no.
- **Contraseña:** indica que el texto que se ingrese tendrá formato de clave, es decir, no se visualiza en pantalla. (Ver próxima Figura)
- **Longitud máxima:** indica la cantidad de caracteres que podrán ser ingresados.
- **Resto de las características de aspecto, comunes a casi todos los controles:** Color, Tipo, Tamaño y Estilo de Letra; Ancho, Alto, Ubicación, etcétera.



Formulario con cajas de texto editables, una de ellas con contraseña.

Eventos de las Cajas de Texto

- **Cambio:** ocurre cuando el usuario modifica el contenido de la caja de texto.
- **Foco:** ocurre cuando la caja de texto recibe el foco dentro de la ventana activa. Con la tecla TAB el usuario puede ir moviéndose uno a uno por todos los controles activos de la ventana de trabajo.
- **PerderFoco:** ocurre cuando la caja de texto pierde el foco, es decir que el foco ha pasado a otro objeto.
- **Tecla pulsada: e**sto ocurre "antes" de que un nuevo carácter ingrese a la caja de texto, y permite anularlo según sea el mismo; es importante diferenciarlo del evento Change que tiene lugar cuando el texto "ya ha sido modificado".

Métodos de las Cajas de Texto

- **Enfocar:** hace que la caja de texto se convierta en el objeto enfocado dentro de la ventana activa.

Controles que ofrecen opciones a los usuarios

La mayoría de las aplicaciones necesitan presentar opciones a los usuarios, que van desde una simple opción de tipo sí o no hasta seleccionar de una lista que contiene

cientos de posibilidades. Son varios los controles GUI estándar que ayudan a presentar opciones. En la tabla siguiente se resumen estos controles y su uso adecuado.

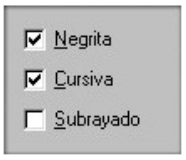
Para proporcionar esta característica	Use este control
Un conjunto pequeño de opciones entre las que el usuario puede elegir una o más.	Casillas de verificación.
Un conjunto pequeño de opciones entre las que el usuario solo puede y debe elegir una y solo una.	Botones de opción; use marcos si son necesarios grupos adicionales.
Una lista de opciones entre las que el usuario puede elegir solo una.	Cuadro de lista (monoselección) Lista Desplegable.
Una lista de opciones entre las que el usuario puede elegir más de una.	Cuadro de lista (multiselección).
Una lista de opciones en la que el usuario puede elegir un elemento, o escribir una opción (incluso no definida previamente) como si fuera un cuadro de texto.	Lista Desplegable (modalidad combo).

Las casillas de verificación, botones de opción, cuadros de lista y listas desplegables se explican en las secciones siguientes:

Casilla de verificación

Una casilla de verificación indica si una condición determinada está activada o desactivada. Se usan casillas de verificación en una aplicación para ofrecer a los usuarios una o varias opciones donde cada una de ellas puede establecerse como Activada (con tilde) o No (sin tilde).

Como las casillas de verificación funcionan independientemente una de otra, el usuario puede activar cualquier número de casillas de verificación al mismo tiempo, incluso ninguna. Por ejemplo, en la figura que sigue pueden estar activadas simultáneamente las casillas de **Negrita** y *Cursiva*.



El ejemplo de la figura usa una casilla de verificación para determinar si el texto se presentará en fuente negrita o no, otra para asignar fuente cursiva y una más para el subrayado. Si el usuario selecciona dos opciones el texto tomará ambas características. Puede suceder que el usuario decida no seleccionar ninguna. En este caso el texto quedará normal.

Propiedades de las Casillas de Verificación

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo chk (por su acrónimo en inglés: Check Box). Ejemplos de nombres: chkNegrita, chkCursiva, chkSubrayado
- **Título:** la etiqueta que se visualiza en la interfaz a la derecha de la casilla.
- **Valor:** las casillas de verificación, como se muestra en la figura siguiente, son triestables. Eso significa que la propiedad Valor puede tener tres estados posibles:
    - **Activo:** cuando ha sido seleccionada.
    - **Inactivo:** cuando no ha sido seleccionada.
    - **Indeterminado:** cuando no asume ninguno de los dos estados anteriores. La tilde y la casilla que lo contiene se ven en un color gris sólido. Habitualmente el usuario intercambia entre Activa e Inactiva con los *clicks* del mouse. Esta opción es mucho menos frecuente que las dos anteriores.

Normalmente la propiedad Valor es considerada dentro del código para condicionar la ejecución de alguna tarea.

- **Habilitación:** indica si el usuario podrá interactuar con el objeto o no. Los valores que puede asumir son:

- **Habilitado:** indica que el objeto está disponible y el usuario puede interactuar con él.
- **Inhabilitado:** indica que el objeto no está disponible y el usuario no puede interactuar con él. Sólo puede mirarlo. Cuando un objeto está Inhabilitado se muestra en un color atenuado (en gris) y no responde a ningún evento que sobre él se produzca.

- **El resto de las características de aspecto, comunes a casi todos los controles:** Color, Tipo, Tamaño y Estilo de Letra; Color del Fondo, Ancho, Alto, Ubicación, etcétera.



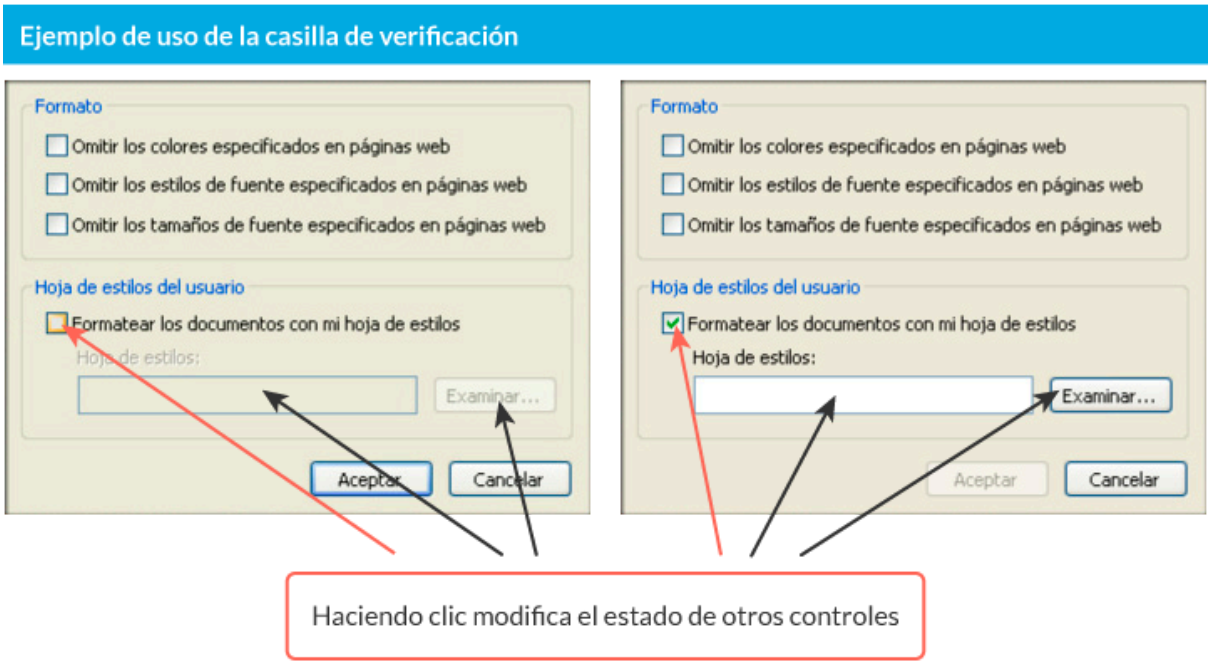
Habilitación y Valor (estado) de las casillas de verificación.

Eventos de las Casillas de Verificación

- **Click:** ocurre cuando el usuario hace *click* sobre la casilla. Automáticamente el objeto comprueba el estado de la casilla y lo modifica, alternando normalmente entre Activo e Inactivo. En algunas aplicaciones las casillas de verificación modifican el estado de otros controles. Por ejemplo, en la figura siguiente la selección de la casilla de verificación de "hoja de estilo" activa la caja de texto en la que el usuario especificará el nombre de la misma.
- **Foco, PerderFoco, etcétera:** los eventos comunes se mantienen en casi todos los controles GUI.

Métodos de las Casillas de Verificación

- **Enfocar:** hace que la casilla de verificación se convierta en el objeto enfocado dentro de la ventana activa.



Botón de opción

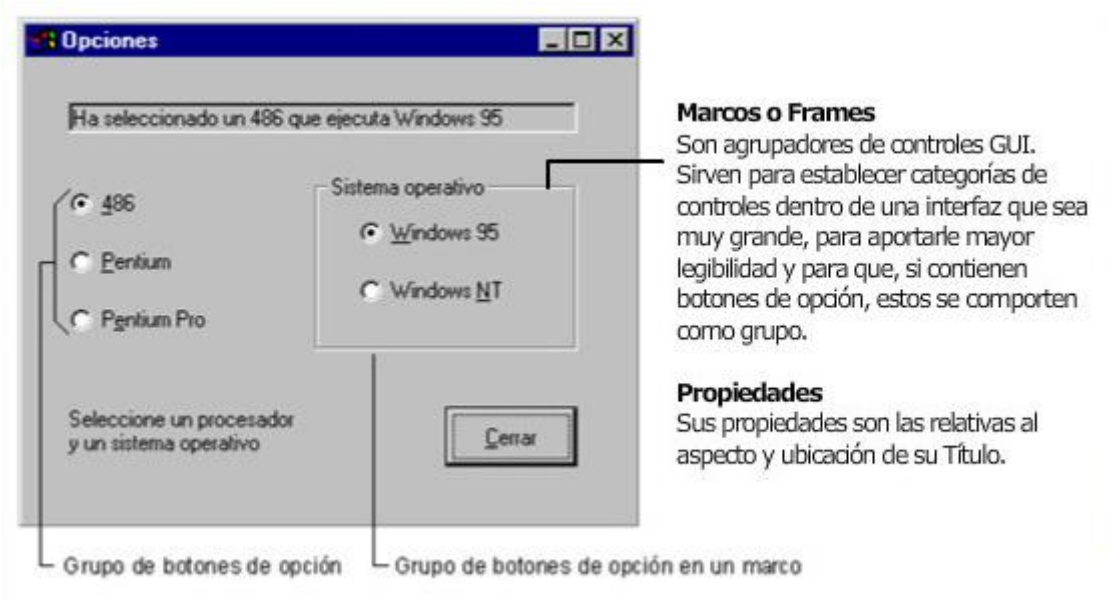
Los botones de opción presentan al usuario un conjunto de dos o más opciones. Pero, a diferencia de las casillas de verificación, los botones de opción deben funcionar siempre como parte de un grupo; al activar un botón de opción se desactivan inmediatamente todos los demás botones del grupo. Al definir un botón de opción se indica al usuario "Aquí tiene un conjunto de opciones entre las que puede elegir una y sólo una".

Por ejemplo, en el grupo de botones de opción que muestra a continuación, el usuario puede seleccionar uno de los tres botones de opción.



Fuente: Elaboración autor

Todos los botones de opción que se colocan directamente en un formulario constituyen un grupo. Si desea crear grupos adicionales de botones de opción, debe colocarlos dentro de marcos. Todos los botones de opción que hay dentro de un marco dado constituyen un grupo independiente. La imagen siguiente muestra un formulario con dos grupos de botones de opción.



Cuando dibuja botones de opción dentro de un marco, el usuario solo puede seleccionar un botón de opción del grupo.

Propiedades de los Botones de Opción

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo opt (por su acrónimo en inglés: *Option Button*). Ejemplos de nombres: opt486, optPentium, optWin95, optWinNT. Cada botón de opción de un grupo es un objeto en sí mismo y tratado independientemente dentro del programa.
- **Título:** la etiqueta que se visualiza en la interfaz a la derecha del botón.
- **Valor:** pueden tener dos valores posibles:
    - **Verdadero:** cuando fue seleccionado.
    - **Falso:** cuando no fue seleccionado



Normalmente la propiedad Valor es considerada dentro del código para condicionar la ejecución de alguna tarea.

- **Habilitación:** indica si el usuario podrá interactuar con el objeto o no. Los valores que puede asumir son:
  - **Habilitado:** indica que el objeto está disponible y el usuario puede interactuar con él.
  - **Inhabilitado:** indica que el objeto no está disponible y el usuario no puede interactuar con él. Sólo puede mirarlo. Cuando un objeto está Inhabilitado se muestra en un color atenuado (en gris) y no responde a ningún evento que sobre él se produzca.
- **El resto de las características de aspecto, comunes a casi todos los controles:** Color, Tipo, Tamaño y Estilo de Letra, Color del Fondo, Ancho, Alto, Ubicación, etcétera.

Eventos de los Botones de Opción

Los botones de opciones responden a los siguientes eventos:

- **Click:** ocurre cuando el usuario hace *click* sobre un botón de un grupo. Automáticamente el objeto pone en verdadero el valor del botón seleccionado por el usuario, y al mismo tiempo pone en Falso el valor de los demás botones del grupo. En algunas aplicaciones los botones de opción modifican el estado de otros controles. Al igual que se ejemplificará con las casillas de verificación, cuando un botón de opción se selecciona puede habilitarse algún otro control y cuando se deselecciona, inhibirse.
- **Foco, PerderFoco,** etcétera.

Métodos de los Botones de Opción

- **Enfocar:** hace que el botón de opción se convierta en el objeto enfocado dentro de la ventana activa.

Cuadros de Lista y Listas Desplegables

Los cuadros de lista y las listas desplegables presentan al usuario una lista de opciones. De forma predeterminada, las opciones se muestran verticalmente en una única columna, aunque también puede establecer múltiples columnas. Si el número de elementos supera a los que se pueden mostrar en la lista desplegable o el cuadro de lista, aparecen automáticamente barras de desplazamiento en el control. El usuario puede entonces desplazarse por la lista hacia arriba o hacia abajo, o de izquierda a derecha, según si la lista tiene una o varias columnas. Dependiendo de la configuración del cuadro de lista, el usuario podrá seleccionar allí una o varias de las opciones presentadas. Las figuras siguientes muestran un Cuadro de Lista de una única columna, con una y varias opciones seleccionadas.

Cuadro de Lista con una y varias opciones seleccionadas



Cuadros de lista de una única columna

Las Listas Desplegables en cambio se parecen bastante a las cajas de texto, sólo que disponen de una flechita a su derecha que permite desplegar una lista con las opciones disponibles. Este control sólo posibilita la selección un elemento (no varios). Las listas desplegables tienen una propiedad denominada Estilo, la que según su valor determinará el comportamiento de la lista. Si el estilo es "ListaDesplegable", el usuario podrá desplegar la lista (pulsando en la flechita a la derecha del cuadro de texto) y elegir una y solo una de las opciones disponibles. Si en cambio el estilo es

"CuadroCombinado", el control permite al usuario seleccionar una de las opciones disponibles, o bien escribir libremente un valor nuevo para la caja de texto. La figura siguiente muestra un cuadro combinado.

Cuadro combinado



Cuadro combinado

A diferencia de otros controles que contienen un único conjunto de valores posibles ({Verdadero/Falso}, {Activo/Inactivo/Indeterminado}, etc.), las cajas de lista y las listas desplegables contienen múltiples valores o una colección de valores. Tienen métodos integrados para agregar, quitar y recuperar valores de sus colecciones en **tiempo de ejecución**[29].

Los cuadros de lista y los cuadros combinados son una manera efectiva de presentar al usuario gran cantidad de opciones en un espacio limitado. Veamos a continuación estos controles más en detalle:

Cuadro de Lista

Es un cuadro que permite visualizar varios datos a la vez. Los elementos de la lista deben ser siempre pertenecientes a una misma categoría de objetos. Por ejemplo, en la figura siguiente se visualiza un cuadro de listas de donde se puede seleccionar las características de animación para un texto.

[29] Tiempo de Ejecución: Es el tiempo que transcurre para un programa mientras se encuentra ejecutándose en el ordenador. También se dice, "Mientras el programa está corriendo".



Los cuadros de listas se utilizan cuando se va a trabajar con una lista de opciones de las que el usuario pueda elegir una o varias, dependiendo de la configuración de la caja de lista (monoselección / multiselección). No se le permitirá al usuario especificar un valor no incluido en el cuadro de lista.

Propiedades de los Cuadros de Lista

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo lst (por su acrónimo en inglés: *List Box*). Ejemplos de nombres: lstModelo, lstEstilo, lstCiudad.

- **Columnas:** puede tener todos los elementos enlistados en una sola columna o en varias. Los posibles valores para esta propiedad son:
  - **MonoColumna:** los elementos se listan uno debajo del otro. Si son más de los que caben en la caja, aparece automáticamente una barra de desplazamiento vertical en el control para permitir acceder a la lista entera.
  - **MultiColumna:** la lista es más ancha y muestra los elementos en varias columnas. Si son pocos se ven en una columna. Si no caben en una, continúa en la siguiente, y así sucesivamente. Si la cantidad de columnas es mayor que el ancho de la caja de lista, aparecerá automáticamente una barra de desplazamiento horizontal en el control para permitir acceder a la lista completa.

- **Selección:** especifica si el usuario podrá seleccionar solo uno o varios elementos de la lista. Los posibles valores son:
  - **MonoSelección:** sólo permitirá seleccionar un elemento.
  - **MultiSelección:** permitirá la selección de múltiples elementos de la lista.

- **CantidadElementos:** propiedad de "solo lectura" en tiempo de ejecución, que indica la cantidad de elementos que la lista tiene cargados.

- **Elemento(N):** Valor (texto) del elemento ubicado en la posición indicada por N. Este subíndice N deberá estar comprendido entre 1 y el valor de la propiedad CantidadElementos.

- **Elemento(N).Seleccionado:** indica si el elemento ubicado en la posición N está seleccionado o no. Los posibles valores son Verdadero (seleccionado) y Falso (no seleccionado)

- **Ordenada:** los valores posibles son:
  - **Verdadero:** la lista presentará sus elementos siempre ordenados alfabéticamente.
  - **Falso:** la lista presentará sus elementos en el orden en que fueron cargados, sin forzar ningún tipo de ordenamiento interno.

- **Habilitación:** indica si el usuario podrá interactuar con el objeto o no. Los valores que puede asumir son:
  - **Habilitado:** indica que el objeto está disponible y el usuario puede interactuar con él.
  - **Inhabilitado:** indica que el objeto no está disponible y el usuario no puede interactuar con él. Solo puede mirarlo. Cuando un objeto está Inhabilitado se muestra en un color atenuado (en gris) y no responde a ningún evento que sobre él se produzca.

- **El resto de las características de aspecto, comunes a casi todos los controles:** Color, Tipo, Tamaño y Estilo de Letra; Color del Fondo, Ancho, Alto, Ubicación, etcétera.

Eventos

- **Click:** Ocurre cuando el usuario hace *click* sobre algún elemento de la lista. Si el elemento no se encontraba previamente seleccionado, se selecciona, y viceversa.
- **Foco, PerderFoco,** etcétera.

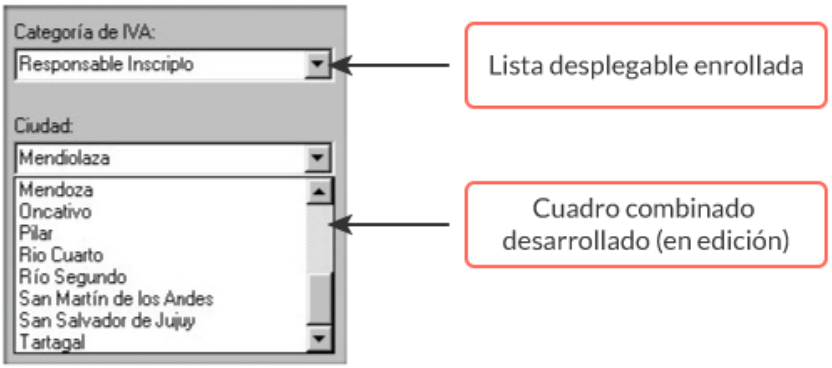
Métodos de los Cuadros de Lista

- **Enfocar:** Hace que el cuadro de lista se convierta en el objeto enfocado dentro de la ventana activa.
- **AgregarItem (Texto):** Agregará a la lista el elemento indicado como parámetro (Texto). El siguiente código de ejemplo agregará a la lista un ítem llamado "Agosto": *lstColores.AgregarItem ("Agosto")*
- **EliminarItem (N):** Eliminará el ítem ubicado en la posición N de la lista, corriendo todos los restantes un lugar hacia arriba.

Listas Desplegables y Cuadros Combinados

Como señalamos anteriormente, se trata de un mismo objeto, solo que su comportamiento depende del valor de la propiedad Estilo, que finalmente habilitará o no el ingreso de valores no enlistados en la caja de texto. Es la combinación de una caja de texto y una caja de lista. Generalmente, el cuadro combinado es el apropiado cuando hay una lista de opciones sugeridas (pero no obligatorias, pudiendo el usuario ingresar otra cosa diferente de las propuestas), mientras que el cuadro de lista es el apropiado cuando desea limitar la entrada a las opciones de la lista.

Un cuadro combinado contiene un campo de edición, de forma que en este campo se pueden introducir opciones que no figuran en la lista. Por ejemplo, observe en el siguiente cuadro combinado desplegado ("Ciudad") en el que se ingresó el nombre de una ciudad que no figura en el cuadro de lista. La ciudad de Mendoza no era una de las propuestas, pues de serlo, debería haber aparecido debajo de la ciudad de Mendiolaza, al estar la lista ordenada alfabéticamente.



En este ejemplo, la categoría de IVA del cliente debe seleccionarse en una Lista Desplegable. No sería correcto usar un Cuadro Combinado para este dato, pues el usuario no puede inventar nuevas categorías de IVA ni ingresar ningún otro valor que los predeterminados.

En cambio, el campo ciudad ofrece una lista con todas las ciudades conocidas por el sistema. No obstante, el usuario sí debe poder ingresar una ciudad nueva, que no figure previamente, pues es muy probable que haya alguna ciudad que la lista no contenga previamente y el usuario puede desear ingresarla de todas formas.

Propiedades de las Listas Desplegables

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo lst del mismo modo que las cajas de lista. (Otra posibilidad es el prefijo ddl, por su acrónimo en inglés: *Drop Down List*). Ejemplos de nombres: lstCategoríaIVA, lstCiudad, lstColor

- **Texto:** valor de la caja de texto. Contiene el texto correspondiente al elemento seleccionado o lo que haya tecleado el usuario (en caso de poder hacerlo).

- **Estilo:** indica si el objeto se comportará como una Lista Desplegable o como un Cuadro Combinado. Los valores posibles son ListaDesplegable o CuadroCombinado.

- **Habilitación:** indica si el usuario podrá interactuar con el objeto o no. Los valores que puede asumir son:

    - **Habilitado:** indica que el objeto está disponible y el usuario puede interactuar con él.
    - **Inhabilitado:** indica que el objeto no está disponible y el usuario no puede interactuar con él. Sólo puede mirarlo. Cuando un objeto está Inhabilitado se muestra en un color atenuado (en gris) y no responde a ningún evento que sobre él se produzca.

- **El resto de las características de aspecto, comunes a casi todos los controles:** Color, Tipo, Tamaño y Estilo de Letra; Color del Fondo, Ancho, Alto, Ubicación, etcétera.

Eventos de las Listas Desplegables

- **Click:** ocurre cuando el usuario selecciona un elemento de la lista. Automáticamente la propiedad Texto reflejará el valor del elemento seleccionado por el usuario.
- **Foco, PerderFoco, etcétera.**

Métodos de las Listas Desplegables

- **Enfocar:** hace que el cuadro de lista se convierta en el objeto enfocado dentro de la ventana activa.
- **AgregarItem (Texto):** agregará a la lista el elemento indicado como parámetro (Texto). El siguiente código de ejemplo agregará a la lista un ítem llamado "Azul": *lstColores.AgregarItem ("Azul")*
- **EliminarItem (N):** eliminará el ítem ubicado en la posición N de la lista, corriendo todos los restantes un lugar hacia arriba.

Controles que permiten al usuario ordenar comandos al sistema

Como parte del proceso de interacción entre el usuario y el sistema resulta necesario disponer de controles GUI que le permitan ordenar la ejecución de tareas, suspender la ejecución de otras que estén en desarrollo, dar por terminada la configuración de los demás controles en un cuadro de diálogo, etc. Esto controles permiten, en términos generales, que el usuario haga Click sobre ellos y que como reacción a dicha operación el sistema ejecute la tarea que corresponda.

Esta categoría de controles está compuesta por los siguientes elementos:

- Botones de Comando
- Barra de Menú
- Menús Desplegables

Analizaremos detenidamente cada uno de ellos.

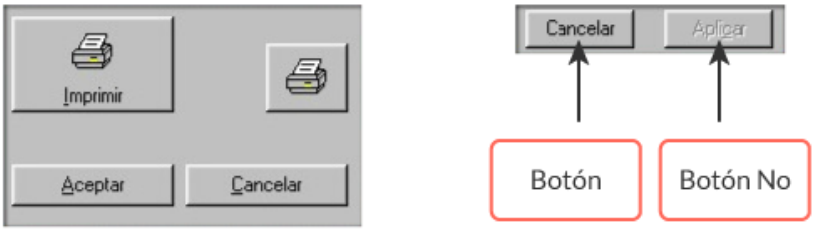
Botones de comando

La mayoría de las aplicaciones tienen botones de comando que permiten al usuario hacer click en ellos para realizar acciones. Cuando el usuario hace click en el botón, éste no solamente realiza una acción, sino que, además, ofrece al usuario el aspecto gráfico de hundirse en la pantalla y al soltarlo volver a su forma normal. Siempre que el usuario hace click en un botón se invoca el procedimiento de evento *click*. Para realizar cualquier acción que desee puede escribir código en el procedimiento de evento *click*.

Características principales

- Inician acciones en la interfaz
- Suelen ser rectangulares
- Deben tener un título que indique su función. Este normalmente puede componerse de un pequeño rótulo, un ícono, o ambos

Ejemplo botones de comando de diferentes tipos



Propiedades de los Botones de Comando

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo cmd (por su acrónimo en inglés: *Command Button*). Ejemplos de nombres: cmdAceptar, cmdCancelar, cmdConsultar, cmdBuscar
- **Título:** la leyenda que muestra en su superficie a modo de rótulo.
- **Ícono:** el icono que muestra en su superficie a modo de rótulo.
- **Tipo, tamaño, estilo, alienación y color de letra:** características del texto de la leyenda.
- **Habilitación:** indica si el usuario podrá interactuar con el objeto o no. Los valores que puede asumir son:
    - Habilitado: indica que el objeto está disponible y el usuario puede interactuar con él.
    - Inhabilitado: indica que el objeto no está disponible y el usuario no puede interactuar con él. Solo puede mirarlo. Cuando un objeto está Inhabilitado se muestra en un color atenuado (en gris) y no responde a ningún evento que sobre él se produzca.
- **El resto de las características de aspecto, comunes a casi todos los controles:** Color, Tipo, Tamaño y Estilo de Letra; Color del Fondo, Ancho, Alto, Ubicación, etcétera.

Eventos de los Botones de Comando

- **Click:** Ocurre cuando el usuario hace Click sobre el Botón.
- **Foco, PerderFoco, etcétera.**

Métodos de los Botones de Comando

- **Enfocar:** hace que el botón de comando se convierta en el objeto enfocado dentro de la ventana activa.

Barras de Menús / Menús Desplegables

Si desea que la aplicación proporcione un conjunto de comandos a los usuarios, los menús ofrecen una forma cómoda y coherente de agrupar comandos y una manera sencilla de que los usuarios tengan acceso a ellos. La barra de menú es un control que

permite agrupar una serie de tareas con un nombre que lo identifica. El usuario elegirá una de las opciones y a continuación aparecerán las acciones relacionadas con la opción elegida. En la figura siguiente se visualiza un menú desplegado.

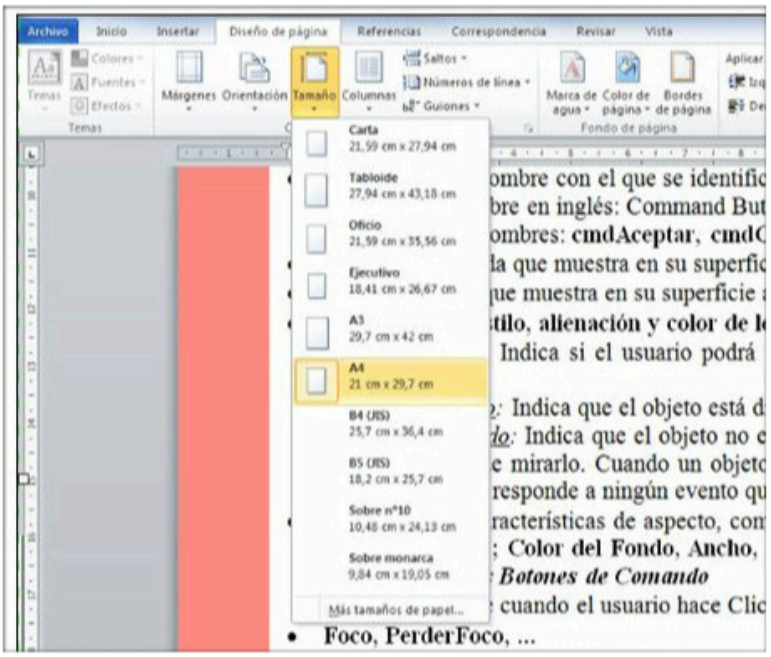


La barra de menús aparece en el formulario inmediatamente debajo de la barra de título y contiene uno o más títulos de menús. Cuando hace *click* en un título de menú (como Archivo, por ejemplo), se despliega una lista que contiene los elementos de menú.

Los elementos de menú pueden incluir:

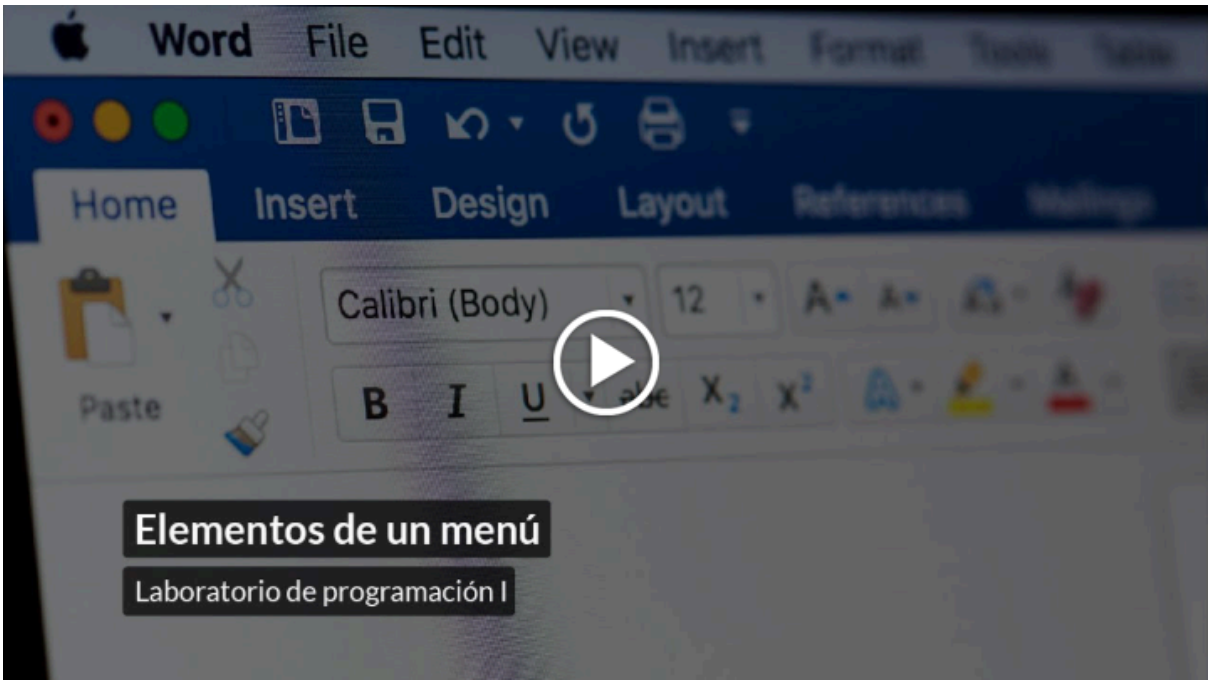
- **Comandos:** como Nuevo y Salir
- **Barras de separación**
- **Títulos de submenús.**

Cada elemento de menú que ve el usuario corresponde a un control de menú. Para que la aplicación sea más fácil de usar, debe agrupar los comandos de menú según su función.



Menú actual utilizando como "cinta" en office.
No obstante cumple la tarea de agrupar comandos para facilitar el uso a los usuarios.

Elementos del menú archivo



Link del video: https://www.youtube.com/watch?v=3fyFLZTJokU

Texto del video:

En esta imagen, por ejemplo, los comandos relacionados con archivos (Nuevo, Abrir y Guardar como...) se

encuentran todos en el menú Archivo, ya que son acciones que se realizan solamente para los archivos. Un ítem de menú puede acompañarse de un pequeño icono a su izquierda. Esto le brinda mayor legibilidad al menú. Esta característica es de utilidad cuando las mismas opciones del menú están disponibles en una barra de herramientas; entonces el icono que se le pone al menú es el mismo que al botón equivalente de la barra de herramientas. De ese modo, cuando el usuario aprenda las funciones del menú reconocerá sus formas equivalentes en la barra de herramientas, y viceversa. Los ítems de menú también pueden funcionar, como si fueran casillas de verificación, pero su comportamiento debe ser especificado en el código de la aplicación.

Acciones de las opciones de los menús

- Algunos elementos de menú ejecutan las tareas que tienen programadas de manera directa; por ejemplo, el elemento **Salir** del menú **Archivo** cierra el archivo y sale de la aplicación.
- Otros elementos de menú muestran un cuadro de diálogo (una ventana que requiere que el usuario proporcione información que la aplicación necesita para realizar la acción). Estos elementos de menú deben ir seguidos de puntos suspensivos (...). Por ejemplo, cuando elige **Guardar como**... en el menú **Archivo** aparece el cuadro de diálogo **Guardar como**.

Propiedades de los menús desplegables

Un control de menú es un objeto. Como otros objetos, tiene propiedades que puede usar para definir su apariencia y su comportamiento.

- **Nombre**: es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo mnu (por su nombre en inglés: Menú Item). Ejemplos de nombres: mnuAbrir, mnuCerrar, mnuEmisionFacturas
- **Título:** la etiqueta que se visualiza en la interfaz como nombre del menú.
- **Chequeado:** pueden tener dos valores posibles:
  - **Verdadero:** el ítem de menú se muestra con una tilde a su izquierda.
  - **Falso:** el ítem de menú se muestra normal (sin la tilde a su izquierda).

Normalmente esta propiedad es considerada dentro del código para condicionar la ejecución de alguna tarea.

- **Habilitación:** indica si el ítem de menú estará disponible o si se mostrará inhabilitado. Los valores que puede asumir son:
    - Habilitado: el elemento aparece disponible al usuario.
    - Inhabilitado: el ítem de menú se ve, pero griseado (inaccesible al usuario).

- **El resto de las características de aspecto, comunes a casi todos los controles:** Color, Tipo, Tamaño y Estilo de Letra; Color del Fondo, etc. no son particulares de cada ítem de menú sino comunes a todo el conjunto de menús de la aplicación.

Eventos de los Menús Desplegables

Los controles de menú contienen solo un evento, el evento *click*, que se invoca al seleccionar el control de menú con el mouse o mediante el teclado.

Métodos de los Menús Desplegables

Los controles de menú no implementan métodos que debamos considerar.

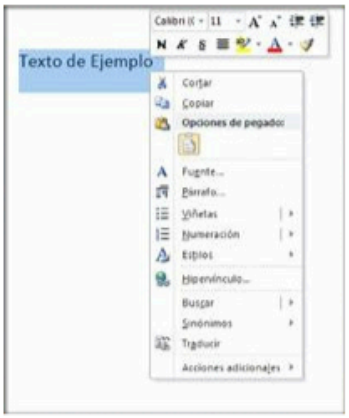
Menús emergentes

Un menú emergente es un menú flotante que aparece sobre un formulario y que es independiente de la barra de menús, como muestra la figura siguiente. Los elementos que aparecen en el menú emergente dependen de la ubicación del puntero cuando se presiona el botón secundario del mouse; por tanto, los menús emergentes se llaman también menús contextuales. Debe usar menús emergentes para proporcionar un método eficiente de tener acceso a comandos comunes que dependan del contexto o lugar donde esté ubicado el puntero del mouse.

Por ejemplo, si hace *click* con el botón secundario del mouse en un cuadro de texto, aparecerá un menú contextual, como muestran las siguientes imágenes.

Menú emergente o contextual

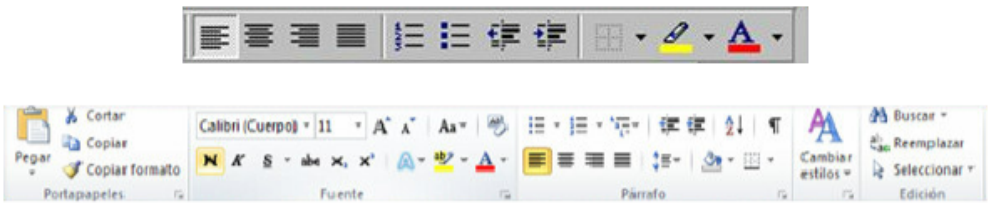




En tiempo de ejecución se puede mostrar como menú emergente cualquier menú que tenga al menos un elemento de menú.

Barras de herramientas

Puede mejorar aún más la interfaz de menús de su aplicación con barras de herramientas. Las barras de herramientas contienen botones específicos, que proporcionan un acceso rápido a los comandos que se usen con más frecuencia en una aplicación.



Barra de herramientas

Aunque los menús se diferencian de los botones de barra de herramientas por su apariencia, básicamente son, en esencia, el mismo tipo de objeto. Internamente son barras de comandos. Se caracterizan por tener botones que ejecutan comandos. Estos botones tienen siempre un icono que representa la función.

El criterio que se sigue para la distribución de comandos en la barra de herramientas de una aplicación y en su barra de menú es el siguiente: Normalmente la Barra de Menú contiene, jerárquicamente organizados, todos los comandos posibles para la aplicación; en tanto que la Barra de Herramientas solo ofrece aquellos de uso más frecuente, agrupados con separadores, y organizados de izquierda a derecha según su frecuencia de uso. Las Propiedades, Eventos y Métodos de las barras de herramientas son las mismas que se indicaron para los menús.

Otros controles fuera de categoría

Dentro del universo del GUI existen controles para gestionar casi cualquier cosa. Hemos visto hasta ahora solo los controles estándar, pero hay muchos otros que no hemos detallado. A continuación, mencionaremos los más relevantes:

Cuadro de Imagen

La utilización principal del control de cuadro de imagen es mostrar una imagen al usuario, desde un simple icono hasta toda una fotografía. La imagen que se muestra está determinada por la propiedad Imagen.

Propiedades de los cuadros de imagen

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo img (por su nombre en inglés: Image Box). Ejemplos de nombres: imgFotoPersonal, imgStatus, imgFondo.
- **Imagen:** indica el origen de la imagen a mostrar en el cuadro de imagen. Cambiar el valor de esta propiedad provocará que cambie la imagen exhibida en la interfaz.
- **Ancho, Alto, Ubicación,** etcétera.

Eventos de los cuadros de imagen

- **Click:** ocurre cuando el usuario hace *click* sobre el área de la imagen.
- **DobleClick:** ocurre cuando el usuario hace *doble click* sobre el área de la imagen.

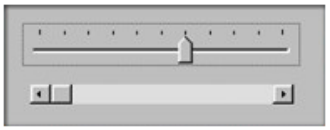
Métodos de los cuadros de imagen

- **Imprimir:** imprime el contenido del cuadro de imagen.
- **Borrar:** deja el cuadro de imagen en blanco.

Barra de desplazamiento y barra deslizante

Aunque las barras de desplazamiento suelen estar asociadas a cuadros de texto o a ventanas, algunas veces se usan como dispositivos de entrada de datos. Como estos controles pueden indicar la posición actual en una escala, se pueden usar individualmente los controles de barra de desplazamiento para controlar la entrada de datos en el programa; por ejemplo, para controlar el volumen de sonido o para ajustar los colores de una imagen.

Ejemplo de barra deslizante y barra de desplazamiento



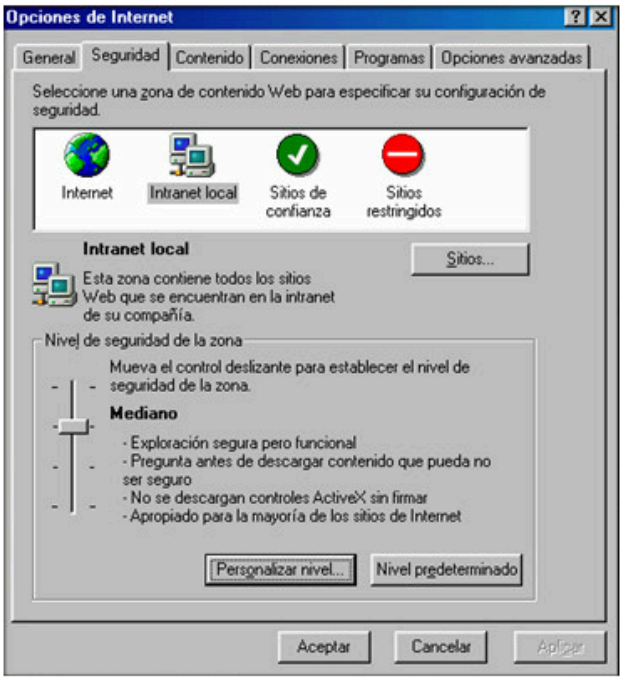
Los controles de barra de desplazamiento no son lo mismo que las barras de desplazamiento integradas que van adjuntas a los cuadros de texto, cuadros de lista y cuadros combinados. Las nuevas tendencias de diseño de interfaces GUI sugieren ahora usar como dispositivos de entrada controles deslizantes en lugar de barras de desplazamiento, como se ve en el ejemplo siguiente:

Control de volumen de Windows con barras deslizantes



El uso de los controles deslizantes no está limitado a las configuraciones de intensidad de volumen, brillo o color. Observe en la figura siguiente la utilización de controles deslizantes para la especificación de niveles de seguridad:

Propiedades de los Controles Deslizantes



Propiedades de los Controles Deslizantes

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo sld (por su nombre en inglés: Slider). Ejemplos de nombres: sldVolumen, sldSeguridad, sldIntensidad.
- **Ancho, alto, ubicación, colores,** etc.
- **Disponibilidad:** puede estar disponible o no.
- **Máximo y Mínimo:** los límites del intervalo de valores se devolverán.
- **Valor:** el valor actual dentro del intervalo.

Eventos de los Controles Deslizantes

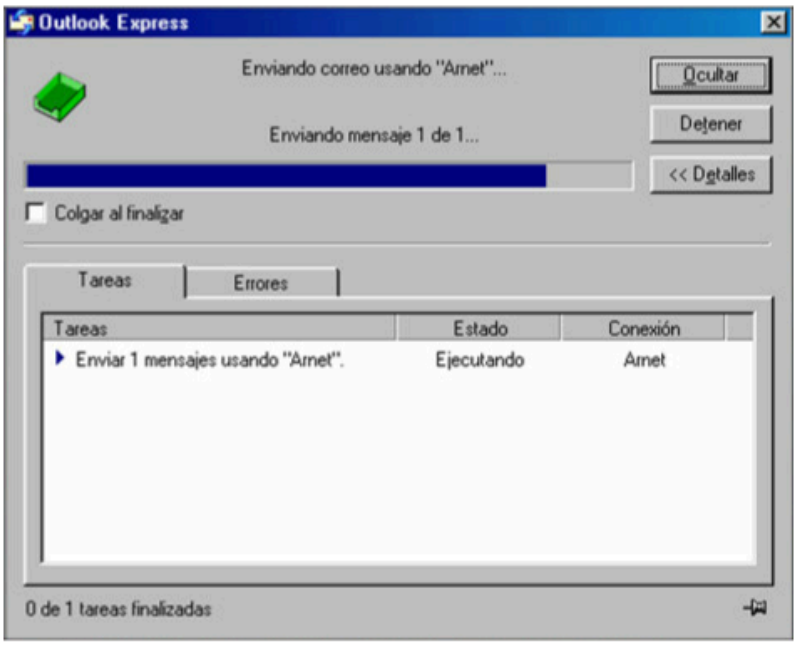
- **Cambio:** ocurre cuando el usuario hace *click* sobre el control y lo arrastra en algún sentido, o cuando usando el teclado lo mueve. En ambos casos, está "cambiando" el valor del control.
- **Foco, PerderFoco, etcétera.**

Métodos de los Controles Deslizantes

- **Enfocar:** hace que el botón de opción se convierta en el objeto enfocado dentro de la ventana activa.

Barra de Progreso

El control *barra de progreso* muestra la evolución de una operación prolongada llenando un rectángulo en bloques de izquierda a derecha.



La barra de progreso indica el porcentaje transcurrido y el restante

Este control tiene un intervalo y una posición actual. El intervalo representa la duración completa de la operación. La posición actual representa el progreso de la aplicación para completar la operación.

Propiedades de las Barras de Progreso

- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo prg (por su acrónimo en inglés: *Progress Bar*). Ejemplos de nombres: prgPorcentajeEnviado, prgOperacionesRealizadas
- **Máximo y Mínimo:** establecen los límites del intervalo.
- **Valor:** especifica la posición actual dentro del intervalo.
- **Cantidad de Segmentos:** determina el número y el tamaño de los segmentos que la llenan. Cuantos más segmentos se use, mayor será la precisión con que el control reflejará la evolución de la operación.

Las Barras de Progreso no responden a eventos ni ejecutan métodos que valga la pena citar.

Barra de Estado

El control **Barra de Estado** proporciona un área, normalmente ubicada en la parte inferior de una ventana, a través de la cual una aplicación puede mostrar varios tipos de datos de estado. Este control se divide en diferentes paneles, donde en cada uno se muestra cierta información de estado particular.



Cada panel puede contener texto o imagen. Generalmente se definen y configuran todos los elementos de los paneles durante el diseño, pero en tiempo de ejecución pueden modificarse paneles de la barra.

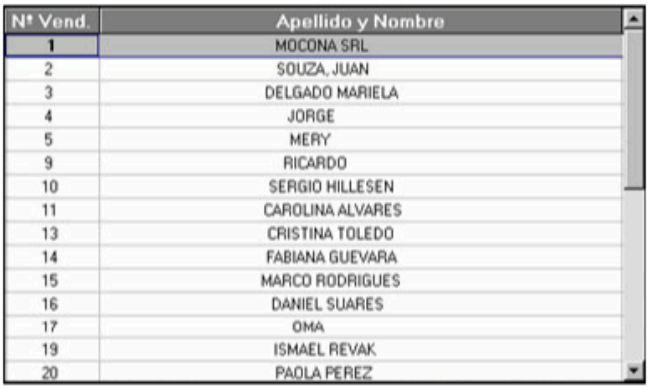
Propiedades de la Barra de Estado

- **Nombre** de cada panel de la barra de estado.
- **Alineación del texto y las imágenes.**
- **Estilo** para mostrar automáticamente datos habituales, como la fecha, la hora y los estados del teclado.

Normalmente, el control Barra de Estado muestra información sobre un objeto visible en el formulario, sus componentes o información de contexto relacionada con su funcionamiento. Este control, junto con otros controles como la Barra de Herramientas, proporciona las herramientas necesarias para crear una interfaz sencilla y rica en información.

Grilla de Datos

El control de cuadrícula es un objeto que presenta una colección de celdas dispuestas en filas y columnas. Las aplicaciones generalmente lo utilizan para mostrar los datos que se solicitan en las consultas.



Nº Vend.	Apellido y Nombre
1	MOCONA SRL
2	SOUZA, JUAN
3	DELGADO MARIELA
4	JORGE
5	MERY
9	RICARDO
10	SERGIO HILLESEN
11	CAROLINA ALVARES
13	CRISTINA TOLEDO
14	FABIANA GUEVARA
15	MARCO RODRIGUES
16	DANIEL SUARES
17	OMA
19	ISMAEL REVAK
20	PAOLA PEREZ

Se visualizan vendedores de una empresa en una grilla

Es un objeto contenedor que muestra datos en filas y columnas. Una Columna puede contener un Encabezado y controles de edición (casillas de verificación, listas desplegables, botón de comando, etcétera).

Propiedades de las Grillas

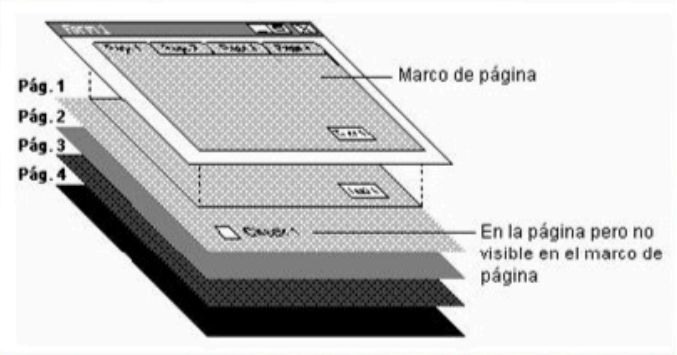
- **Nombre:** es el nombre con el que se identifica al objeto en el programa. Suele precederse del prefijo gri (por su acrónimo en inglés: *grid*). Ejemplos de nombres: **griVendedores, griResultados, griVentasAcumuladas**
- **Columna:** es el número de la columna donde está la celda activa de la grilla.
- **Fila:** indica el número de fila que contiene la celda activa.
- **MaxColumna y MaxFila:** contienen respectivamente el máximo número de columna y de fila disponible (con datos) en la grilla.
- **Nombre[X][Y]:** contiene el valor de la celda ubicada en la fila X y columna Y de la grilla.

Eventos de las Grillas

- **Cambio:** tiene lugar cuando se modifica el contenido de la celda activa.

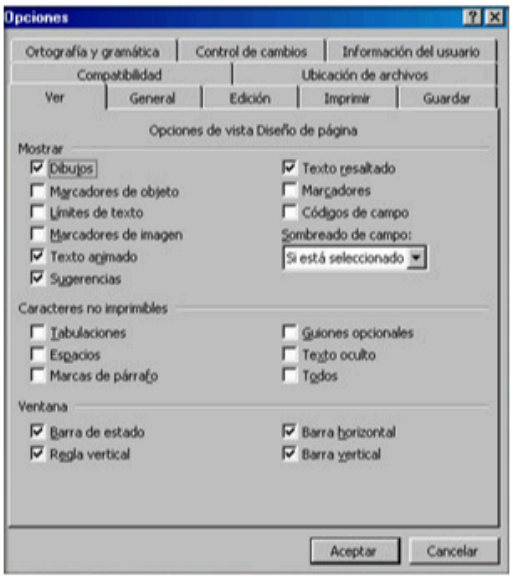
Fichas, Marco de Página o Solapas

Un control de Fichas es como los separadores de un cuaderno. Mediante el uso de este control se puede definir varias páginas para la misma área de una ventana o cuadro de diálogo en su aplicación.



Esquema de un Marco de Páginas

El control consiste en uno o más objetos Ficha de una colección de fichas. Cada ficha contiene controles o grupos de controles GUI. Fuera de las fichas pueden existir también botones de comando u otros controles que referencias elementos comunes a la ventana en general y no particulares de alguna ficha, con los que se puede interactuar sin depender de la ficha que se esté visualizando.



Fichas en un formulario. Cada ficha tiene controles.

En este ejemplo puede observarse una enorme cantidad de fichas en una misma ventana. Esto hace algo confusa la interfaz. Fuera del marco de páginas, se observan los botones de comando de "Aceptar" y "Cancelar". Estos no son propios de ninguna de las fichas, sino generales para la información gestionada en todas ellas.

Propiedades de los Marcos de Páginas

- **Nombre:** es el nombre con el que se identifica a cada Ficha al objeto en el programa. Suele precederse del prefijo tag (por su nombre en inglés: *tags*). Ejemplos de nombres: tagVer, tagGeneral, tagParámetros
- **Resto de las características de aspecto, comunes a casi todos los controles:** Color del Fondo, Ancho, Alto, Ubicación, etcétera.

Eventos de los Marcos de Páginas

- **Click:** responden al *click* sobre una lengüeta convirtiéndola en la Ficha Activa.

Estilos de interfaz

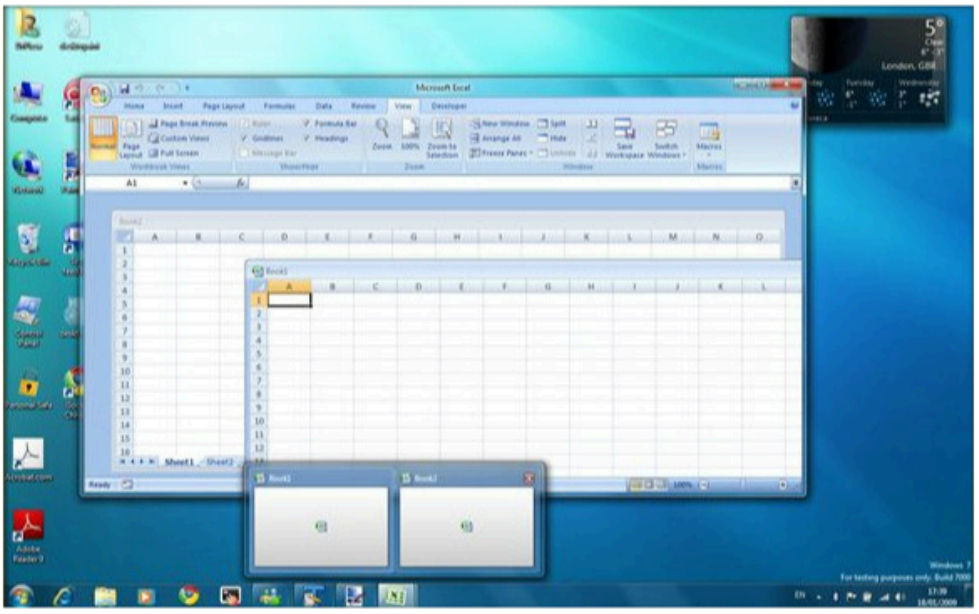
Si ha utilizado aplicaciones durante algún tiempo, se habrá dado cuenta que no todas las interfaces de usuario tienen la misma apariencia ni el mismo comportamiento. Hay dos estilos principales de interfaz de usuario: la **Interfaz de Documento Único** *(SDI: Simple Document Interface)* y la **Interfaz de Documentos Múltiples** *(MDI: Múltiple Document Interface)*.

Interfaz de documento único

Un ejemplo de la interfaz SDI es la aplicación *WordPad* incluida en *Microsoft Windows*. En *WordPad*, sólo se puede abrir un documento y se debe cerrar un documento para abrir otro.

Interfaz de documentos múltiples

Aplicaciones como *Microsoft Excel* y *Microsoft Word* son interfaces MDI; le permiten presentar varios documentos al mismo tiempo y cada documento se presenta en su propia ventana (llamada ventana "hija"), la cual se encuentra contenida dentro de la ventana de la aplicación (llamada ventana "padre"), en la que se encuentran disponibles la barra de menú y la de herramientas. Las aplicaciones MDI se pueden reconocer por la inclusión de un menú **Ventana** con **submenús** para cambiar entre las distintas ventanas o documentos.



Microsoft Excel, una aplicación con interfaz de documentos múltiples

Para determinar cuál es el mejor estilo de interfaz, tiene que basarse en el propósito de la aplicación. Por ejemplo:

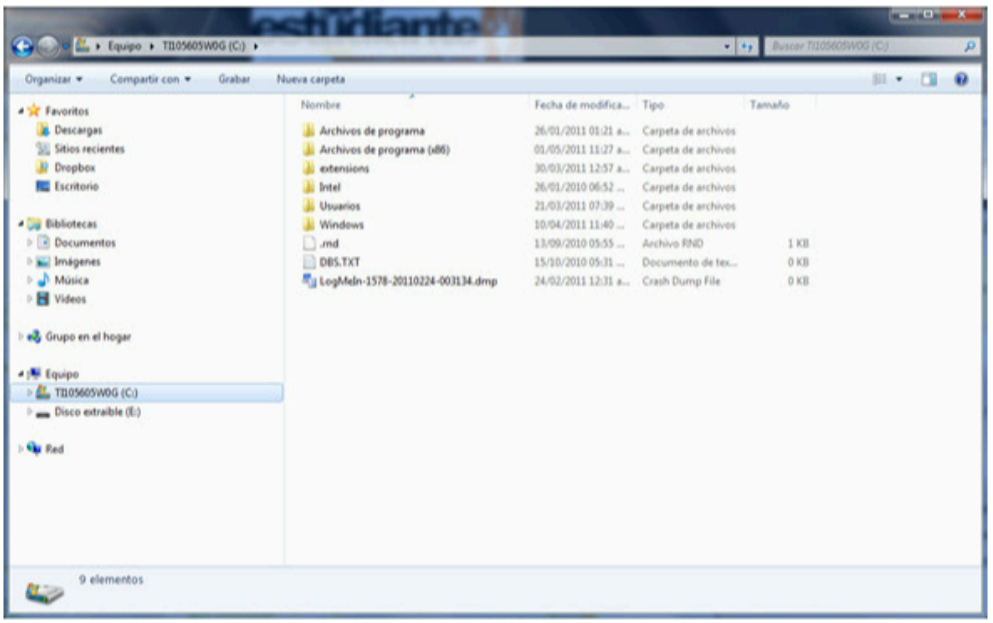
- Una aplicación para procesar reclamos de seguros podría sugerir el estilo MDI: puede que un empleado trabaje con más de un reclamo a la vez o puede que necesite comparar dos reclamos.
- Por otro lado, una aplicación de calendario es más adecuada para el estilo SDI: no parece que sea necesario tener más de un calendario abierto al mismo tiempo; en el caso poco probable de que lo fuera, podría abrir una segunda instancia de la aplicación SDI.

El estilo SDI es el más normal. Hay unas ciertas consideraciones y técnicas únicas para la creación de aplicaciones MDI.

Los formularios MDI presentan una ventana padre o principal, que determina el espacio de trabajo. Dentro de ella estarán las ventanas hijas o formularios secundarios. Cuando se minimiza una ventana hija, su icono aparece en la parte inferior, dentro de la ventana padre. Cuando se minimiza la ventana padre, desaparece con todo su contenido del escritorio, quedando su icono en la barra de tareas.

Interfaz al estilo explorador

Además de los dos estilos de interfaz más comunes, SDI y MDI, se está haciendo cada vez más popular un tercer estilo de interfaz: la interfaz al estilo explorador.



El Explorador de Windows. Interfaces estilo explorador

La interfaz estilo explorador es una única ventana que contiene dos o más paneles o regiones, que normalmente consisten en una vista de árbol o una vista jerárquica a la izquierda y un área de presentación a la derecha, como en el Explorador de Microsoft Windows. Este tipo de interfaz tiende por sí misma a la exploración o examen de grandes cantidades de documentos, imágenes o archivos.

Diseño pensando en el usuario

A menos que vaya a crear aplicaciones estrictamente para su propio uso, el valor de sus creaciones va a ser juzgado por otras personas. La interfaz de usuario de su aplicación produce el mayor impacto en la opinión del usuario; no importa el brillo técnico o la optimización del código; si el usuario cree que la aplicación es difícil de usar, no será bien recibida.

Como programador, sin duda está familiarizado con los aspectos técnicos de los equipos. Es bastante fácil olvidar que la mayoría de los usuarios no comprenden (y probablemente no les importa) la tecnología que hay detrás de una aplicación. Ven una aplicación como un medio para conseguir un fin; una manera de realizar una tarea, preferiblemente de una forma más fácil que sin la ayuda de un equipo.

Una interfaz de usuario bien diseñada aísla al usuario de los aspectos técnicos, facilitando la realización de la tarea para la que se ha diseñado.

Al diseñar la interfaz de usuario de su aplicación debe tener en cuenta a los usuarios.

- ¿Con qué facilidad puede un usuario descubrir las características de su aplicación sin necesidad de entrenamiento?
- ¿Cómo responde la aplicación cuando se producen errores?
- ¿Qué ayuda o asistencia al usuario va a proporcionar?
- ¿El diseño le parece estético al usuario?

En esta sección trataremos las respuestas a éstas y otras preguntas relativas al diseño orientado al usuario.

Fundamentos del diseño de la interfaz

No tiene que ser un artista para crear una gran interfaz de usuario; la mayoría de los principios del diseño de Interfaces de usuario son los mismos principios de diseño básicos que se aplican por igual a la pantalla de un equipo que a una hoja de papel o a un lienzo. Tienen que ver con los aspectos señalados a continuación. Es fundamental dibujar primero el formulario sobre papel, determinando los controles que vaya a necesitar, la importancia relativa de los diferentes elementos y las relaciones entre los controles.

Composición: la apariencia de una aplicación

La composición o distribución de un formulario no sólo afecta a su atractivo estético, sino que también tiene un tremendo impacto en la facilidad de uso de la aplicación. La

composición incluye factores tales como la colocación de los controles, la coherencia de los elementos, su facilidad de uso, el uso del espacio en blanco y la sencillez del diseño.



Colocación de los controles

En la mayoría de los diseños de interfaces, no todos los elementos son de igual importancia. Es necesario un diseño cuidadoso para asegurar que los elementos más importantes sean inmediatamente accesibles para el usuario. Los elementos importantes o utilizados con más frecuencia deben tener una posición prominente; los elementos menos importantes deben estar relegados a posiciones menos prominentes.

Como mencionáramos anteriormente, en la mayoría de los idiomas, se nos enseña a leer de izquierda a derecha y de arriba a abajo en una página. Lo mismo se aplica a la pantalla de un equipo: los ojos de la mayoría de los usuarios irán primero a la parte superior izquierda de la pantalla, por lo que el elemento más importante debe estar allí. Por ejemplo, si la información de un formulario se refiere a un cliente, el campo con el nombre del cliente debe mostrarse allí donde se vea primero. Los botones, como **Aceptar** o **Siguiente**, deben estar situados en la parte inferior derecha de la pantalla; normalmente el usuario no los usará hasta que haya terminado de trabajar con el formulario.

La agrupación de elementos y controles también es importante. Procure agrupar la información de forma lógica, de acuerdo con su función o su relación. Como las funciones están relacionadas, los botones para desplazarse por una base de datos deben estar agrupados visualmente en lugar de estar esparcidos por el formulario. Esto mismo puede aplicarse a la información; los campos de nombre y dirección se encuentran generalmente agrupados, ya que están estrechamente relacionados. En muchos casos, puede usar marcos para reforzar la relación entre controles, agrupando aquellos que pertenecen a una misma categoría.

Coherencia de los elementos de la interfaz

La coherencia es una virtud en el diseño de una interfaz de usuario. Una apariencia coherente aporta armonía a una aplicación: todo parece encajar perfectamente. La falta de coherencia de una interfaz puede provocar confusión y puede hacer parecer que la aplicación es caótica, desorganizada y barata, pudiendo llegar incluso a provocar al usuario dudas sobre la fiabilidad de la aplicación.

Para obtener coherencia visual, establezca una estrategia de diseño y convenciones de estilo antes de empezar el desarrollo. Los elementos de diseño tales como los tipos de controles, el estándar de tamaño y agrupación de controles, y las opciones de fuente deben establecerse previamente. Puede crear prototipos de los posibles diseños para ayudarle a tomar las decisiones de diseño. La gran variedad de controles disponibles tienta a usarlos todos. Evite esta tentación: elija el subconjunto de controles que mejor se adapte a su aplicación en particular. Mientras que los controles de cuadro de lista, cuadro combinado y grilla pueden usarse para presentar listas de información, es mejor adoptar un único estilo en la medida de lo posible.

Además, utilice los controles de forma apropiada; si bien un control de cuadro de texto puede definirse como de solo lectura y usarse para presentar texto, un control de etiqueta suele ser el más apropiado para este propósito. Mantenga la coherencia en el valor de las propiedades de los controles: si en un lugar utiliza un color de fondo blanco para el texto modificable, no utilice un fondo gris en otro sitio a menos que tenga una buena razón para ello. La coherencia entre los diferentes formularios de una aplicación es importante para su facilidad de uso. Si en un formulario utiliza un fondo gris y

efectos en 3D y un fondo blanco en otro, parecerá que los formularios no están relacionados entre sí. Adopte un estilo y manténgalo en toda la aplicación, incluso si ello implica volver a diseñar algunas características.

Facilidades: la forma sigue a la función

Las facilidades son pistas visuales de la función de un objeto. Aunque el término no sea muy familiar, a su alrededor hay muchos ejemplos de ellas. El manubrio de una bicicleta tiene hendiduras para colocar los dedos, una facilidad que hace obvio que ése es el sitio donde poner las manos. Los botones, las palancas y los conmutadores luminosos son todos facilidades; solo con mirarlos puede deducirse su propósito.

La interfaz de usuario también hace uso de estas facilidades. Por ejemplo, los efectos tridimensionales utilizados en los botones de comando hacen pensar que están para ser presionados. Si diseñara un botón de comando con bordes planos, perdería esta facilidad y el usuario no tendría claro que es un botón de comando. Hay casos en que los botones planos pueden ser apropiados, como en juegos o en aplicaciones multimedia; está bien mientras mantenga la coherencia en toda la aplicación.



Los cuadros de texto también aportan un grado de facilidad: los usuarios esperan que un cuadro con bordes y un fondo blanco contenga texto modificable. Aunque es posible

presentar un cuadro de texto sin bordes, eso lo haría parecer una etiqueta y para el usuario no sería obvio que es modificable.

Uso del espacio en blanco

El uso del espacio en blanco dentro de la interfaz de usuario puede contribuir a resaltar elementos y aumentar su facilidad de uso. Aunque el espacio en blanco no tiene que ser necesariamente blanco, hace referencia al uso del espacio vacío entre y alrededor de los controles de un formulario. Si hay demasiados controles en un formulario puede parecer una interfaz congestionada, dificultando la localización de un campo o control concreto. Hay que incorporar espacio en blanco al diseño para hacer énfasis en los elementos del diseño.

El espaciado coherente entre los controles y la alineación vertical y horizontal de los elementos hacen que el diseño sea también más sencillo. De igual forma que el texto de un periódico está dispuesto en columnas con el mismo espacio entre las líneas, una interfaz ordenada la hace más legible.

Simplificación

Quizás el principio más importante en el diseño de la interfaz sea la sencillez. Cuando se trata de aplicaciones, si la interfaz parece difícil, probablemente lo sea.

Pensando un poco se puede crear una interfaz que parezca (y sea) fácil de usar. Además, desde un punto de vista estético, siempre es preferible un diseño limpio y sencillo. Un error frecuente en el diseño de la interfaz es el de modelarla a partir de objetos del mundo real.

Imagine, por ejemplo, que se le pide que cree una aplicación para completar formularios de seguros. La reacción natural sería diseñar una interfaz que duplicara de forma exacta en la pantalla el formulario preimpreso en papel. Esto crea varios problemas: la forma y las dimensiones de un formulario preimpreso en papel son diferentes a las de una pantalla, la duplicación de un formulario preimpreso le limita al

uso de cuadros de texto y casillas de verificación, y el usuario no obtiene un beneficio real.



Uso del espacio en blanco

Es mucho mejor diseñar una nueva interfaz, proporcionando quizás un duplicado impreso (con presentación preliminar) del preimpreso original en papel. Agrupando los campos de forma lógica y utilizando una interfaz con fichas o varios formularios enlazados puede presentar toda la información sin tener que hacer desplazamientos de pantalla. También puede usar controles adicionales, como cuadros de lista con opciones predefinidas, lo que reduce la cantidad de datos que tiene que escribir el usuario.

También puede simplificar las aplicaciones si lleva las funciones utilizadas con poca frecuencia a sus propios formularios. Proporcionar acciones predeterminadas puede simplificar una aplicación; si nueve de cada diez usuarios seleccionan el texto en negrita, haga que el texto sea en negrita de forma predeterminada en vez de forzar al usuario a hacerlo cada vez (no olvide proporcionar una opción que invalide el valor predeterminado). Los asistentes también contribuyen a simplificar las tareas complejas o poco frecuentes. La mejor prueba de la sencillez de uso de una aplicación es observar la aplicación en uso. Si un usuario típico no puede realizar inmediatamente una tarea deseada sin ayuda, quizás debería volver a diseñarla.

Uso del color y de imágenes

El uso del color en una interfaz puede aportar atractivo visual, pero es fácil abusar de ello. Con los actuales monitores capaces de presentar millones de colores, se tiene la tentación de usarlos todos. El color, como todos los demás principios básicos de diseño, puede ser problemático si no se tiene en cuenta en el diseño inicial.

Las preferencias de color varían mucho; puede que los gustos de los usuarios no coincidan con los suyos. El color puede evocar emociones y si el destino de su diseño es internacional, ciertos colores pueden tener un significado cultural. Normalmente, lo mejor es ser conservador, y emplear colores más suaves y neutros.

Por supuesto, la elección de los colores también puede estar influida por los destinatarios de la aplicación, así como por el tono o el estado de ánimo que intente transmitir. Los rojos, verdes y amarillos brillantes pueden ser apropiados para aplicaciones infantiles, pero en raras ocasiones provocarían una impresión de responsabilidad fiscal en una aplicación bancaria.

Se pueden usar pequeñas cantidades de colores brillantes para poner énfasis y llamar la atención sobre un área importante. Como norma, debe limitar el número de colores en una aplicación y el esquema de colores debe ser coherente. Lo mejor es adoptar la paleta estándar de 16 colores si es posible; aunque los medios tonos pueden hacer que algunos colores desaparezcan cuando se presentan en un monitor de 16 colores.

Otra consideración en cuanto al uso del color es la ceguera a ciertos colores, o daltonismo. Mucha gente no es capaz de diferenciar entre distintas combinaciones de colores primarios como rojo y verde. Para alguien con esta limitación, el texto rojo sobre un fondo verde no sería visible.

Imágenes e iconos

El uso de imágenes e iconos también puede agregar interés visual a su aplicación, pero, de nuevo, es esencial un diseño cuidadoso. Las imágenes pueden transmitir información sin necesidad de incluir texto, pero las imágenes son percibidas de manera diferente por personas diferentes.

Las barras de herramientas con iconos que representen distintas funciones son un elemento útil de una interfaz, pero si el usuario no puede identificar la función representada por el icono, pueden ser contraproducentes. Al diseñar los iconos de una barra de herramientas, fíjese en otras aplicaciones y vea las normas estándar ya establecidas. Por ejemplo, muchas aplicaciones utilizan una hoja de papel con una esquina doblada para representar el icono de Nuevo archivo. Puede que haya una mejor metáfora para esta función, pero si se representa de forma diferente, podría provocar confusión en el usuario.

También es importante considerar el significado cultural de las imágenes. Muchos programas utilizan una imagen de un buzón de estilo rural con una bandera para representar las funciones de correo. Se trata de un icono principalmente norteamericano; los usuarios de otros países o culturas probablemente no lo reconozcan como buzón.

Cuando diseñe sus propios iconos e imágenes, procure que sean sencillos. Las imágenes complejas con muchos colores no se ven bien cuando se presentan como iconos de las barras de herramientas o cuando se presentan con resoluciones de pantalla superiores.

Elección de fuentes (Tipografías)

Las fuentes también son una parte importante de la interfaz de usuario, ya que con frecuencia comunican información importante al usuario. Tiene que elegir las fuentes que sean fácilmente legibles a resoluciones diferentes y en diferentes tipos de monitores. Lo mejor es adoptar fuentes sencillas. Script y otras fuentes decorativas generalmente quedan mejor impresas que en una pantalla y pueden ser difíciles de leer con tamaños en puntos pequeños.



A menos que piense distribuir fuentes junto con su aplicación, debe ceñirse a las fuentes estándar como **Arial**, **Times New Roman** o **System**. Si el sistema del usuario no incluye una fuente concreta, el sistema la sustituirá, dando como resultado una apariencia completamente distinta de la pretendida. Si el destino de su aplicación es internacional, tendrá que investigar las fuentes disponibles en los idiomas de destino. Además, deberá tener en cuenta la expansión del texto cuando diseñe para otros idiomas; los títulos y etiquetas pueden ocupar hasta un 50% más de espacio en otros idiomas.

De nuevo, es importante la coherencia del diseño a la hora de elegir las fuentes. En la mayor parte de los casos, no debe usar más de dos fuentes con dos o tres tamaños en puntos distintos dentro de una misma aplicación. Demasiadas fuentes pueden hacer que su sistema parezca una nota de pedido de rescate por un secuestro.

Diseño pensando en la facilidad de uso

La facilidad de uso de una aplicación la determina el usuario en última instancia. El diseño de la interfaz es un **proceso iterativo**[30]; en pocas ocasiones se consigue una interfaz perfecta a la primera. Si involucra a los usuarios en las primeras fases del proceso de diseño puede crear una interfaz mejor y más útil con menos esfuerzo.

Pensemos en nuestra situación profesional: seguramente propondremos al usuario un modelo de interfaz para que pueda emitir el reporte solicitado, aplicando en el diseño del mismo todo lo que ya sabemos sobre diseño de interfaces. No obstante, será una muy buena idea conversar con el usuario respecto a como se siente usando la pantalla

[30] Proceso Iterativo: Proceso que se repite una y otra vez. El diseño de interfaces se desarrolla en sucesivas instancias, probando el diseño, depurándolo y refinando sus características, hasta que sea verdaderamente funcional y fácil de usar.

que hayamos creado, a los efectos de ir afinando el diseño, pensando siempre en la facilidad de uso para el usuario.

¿Qué es una buena interfaz?

El mejor punto de partida al diseñar una interfaz de usuario es fijarse en las aplicaciones más vendidas; después de todo, puede que no sean las más vendidas a causa de sus pobres interfaces. Encontrará muchas cosas comunes, como las barras de herramientas, barras de estado, Información sobre herramientas, menús contextuales y cuadros de diálogo con fichas. También puede aprovechar su propia experiencia como usuario de programas. Piense en algunas de las aplicaciones que haya utilizado, en lo que es funcional y lo que no, y cómo lo solucionaría. Recuerde, sin embargo, que puede que sus gustos no coincidan con los de sus usuarios; tendrá que validar sus ideas con ellos.



Ejemplo de interfaz

También se habrá dado cuenta de que las mejores aplicaciones ofrecen opciones que se acomodan a las distintas preferencias de los usuarios. Por ejemplo, el Explorador de *Microsoft Windows* permite que los usuarios copien archivos con menús, comandos de teclado o mediante arrastrar y colocar. Al ofrecer opciones se amplía el atractivo de la aplicación; como mínimo debe hacer que todas las funciones sean accesibles desde el mouse y desde el teclado.

Interacción con los usuarios

En un mundo ideal, el software y el hardware funcionarían siempre bien y los usuarios nunca cometerían errores. La realidad nos dice que los errores son posibles y de hecho ocurren. Una parte del diseño de la interfaz de usuario tiene que ver con la decisión de cómo responderá la aplicación cuando haya algún problema.

Una respuesta común es presentar un cuadro de diálogo para pedir al usuario que decida cómo la aplicación tendría que tratar el problema. Una respuesta menos común (pero preferible) sería simplemente resolver el problema sin involucrar al usuario. Después de todo, al usuario le importa.

Crear cuadros de diálogo inteligentes

De vez en cuando se producen errores en las aplicaciones y es necesario tomar una decisión para resolver la situación. Esto ocurre normalmente como una bifurcación (condicional) dentro del código del programa. Si la decisión requiere la intervención del usuario, la cuestión se le presenta al usuario mediante un cuadro de diálogo. Los cuadros de diálogo forman parte de la interfaz de usuario, y como el resto de los elementos de la interfaz, su diseño es importante para la facilidad de uso de la aplicación.



Link del video: https://www.youtube.com/watch?v=BMEWR_1aB88

Texto del video:

Algunas veces parece como si los cuadros de diálogo hubieran sido diseñados por programadores que nunca han tenido una conversación inteligente con otro ser humano. Un mensaje como "Un sector del disco duro C está dañado o es inaccesible. Anular, Reintentar o Ignorar", no significa mucho para el usuario medio. Es como si un mozo de un bar le dijera "Nos hemos quedado sin sopa" ¿Anular, Reintentar, Ignorar? ¿Cómo respondería? Es importante formular las preguntas y las soluciones de manera que el usuario pueda entenderlas.

En el ejemplo anterior, un mensaje mejor podría ser "Hay un problema al guardar el archivo en la unidad C, ¿Quiere guardar el archivo en la unidad A? o ¿No desea guardar el archivo?

Cuando cree cuadros de diálogo para su aplicación, tenga en cuenta al usuario. ¿Transmite el mensaje información útil para el usuario? ¿Es inteligible? ¿Presentan opciones claras los botones de comando? ¿Son adecuadas las opciones para la situación dada? Recuerde que con un único cuadro de mensajes desalentador se da al usuario una mala impresión de la aplicación. Si va a diseñar sus propios formularios de diálogo personalizados, procure mantener un estilo estándar. Si se desvía mucho del diseño de los cuadros de mensajes estándar, puede que los usuarios no lo reconozcan como un cuadro de diálogo.

Tratar errores sin cuadros de diálogo

No siempre es necesario interrumpir al usuario cuando se produce un error. Algunas veces es preferible tratar el error desde el programa sin notificárselo al usuario o advertir al usuario de una manera que no le interrumpa su trabajo. Un buen ejemplo de esta técnica es la característica de **corrección automática** de *Microsoft Word*: si una palabra frecuente se escribe incorrectamente, Word la corrige automáticamente; si lo mismo ocurre con una palabra no tan común, queda subrayada en rojo para que el usuario la pueda corregir posteriormente.

Puede usar varias técnicas; usted decide cuál de ellas es la más apropiada para su aplicación. He aquí algunas sugerencias:

- Agregue una función Deshacer en el menú **Edición**. En lugar de interrumpir al usuario con un diálogo de confirmación en las eliminaciones, confíe en que ellos saben lo que hacen y proporcióneles una función Deshacer por si cambian de opinión más tarde.
- Presente un mensaje en una barra o icono de estado. Si el error no afecta a la tarea actual del usuario, no detenga la aplicación. Utilice una barra de estado o un icono de advertencia llamativo para advertir al usuario; podrá resolver el problema cuando esté preparado para ello.
- Corrija el problema. Algunas veces la solución de un error es obvia. Por ejemplo, si un disco está lleno cuando el usuario intenta guardar un archivo, busque espacio en las otras unidades del sistema. Si hay espacio disponible, guarde el archivo; envíe un mensaje a la barra de estado para hacer saber al usuario lo ocurrido.
- Guarde el mensaje para después. No todos los errores son críticos o demandan una atención inmediata; considere la posibilidad de registrar los errores en un archivo y presentarlos al usuario cuando salga de la aplicación o en otro momento más adecuado. Si el usuario comete un error tipográfico (por ejemplo, Córdoab en vez de Córdoba), regístrelo. Agregue un botón **Repasar entradas** y una función para presentar las discrepancias de modo que el usuario pueda corregirlas.

- No haga nada. Algunas veces el error no es lo bastante importante como para emitir ni siquiera una advertencia. Por ejemplo, el hecho de que una impresora en LPT1 no tenga papel no tiene mucha importancia hasta que se vaya a imprimir. Espere hasta que el mensaje adquiera importancia dentro de la tarea actual.

Diseñar un modelo de asistencia al usuario

No importa la calidad de la interfaz de usuario; habrá veces en que un usuario necesite asistencia. El modelo de asistencia al usuario de su aplicación incluye elementos como la Ayuda en pantalla y la documentación impresa; también puede contener elementos de asistencia al usuario como Información sobre herramientas, barras de estado, ayuda del tipo ¿Qué es esto? y asistentes. El modelo de asistencia al usuario debe diseñarse como cualquier otra parte de la aplicación: antes de empezar el desarrollo. El contenido del modelo variará dependiendo de la complejidad de la aplicación y de la audiencia prevista.

Ayuda y documentación

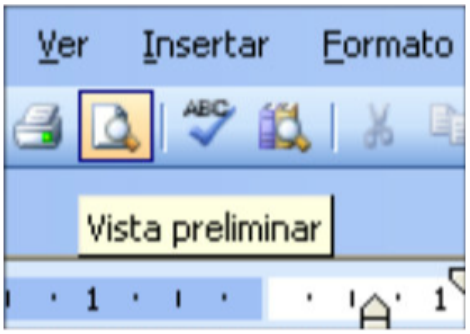
La Ayuda en pantalla es una parte importante de cualquier aplicación; normalmente es el primer lugar en el que el usuario busca cuando tiene alguna duda. Incluso una aplicación sencilla debe proporcionar Ayuda; no proporcionarla es como asumir que los usuarios nunca tendrán dudas o preguntas. Al diseñar el sistema de Ayuda, tenga en cuenta que su propósito principal es responder a preguntas. Intente pensar como el usuario cuando cree nombres de temas y entradas de índice; por ejemplo, "¿Cómo dar formato a una página?" en lugar de "Menú Edición, Formato de página" hará que el tema sea más fácil de encontrar. No olvide la ayuda contextual (dependiente del objeto que tiene foco, o de la pantalla activa); la mayoría de los usuarios encuentra desalentador presionar la tecla F1 para obtener Ayuda acerca de un campo o ventana concreta y ver que se les presenta, en cambio, el índice general de la ayuda.

Elementos de asistencia al usuario

Dentro de la interfaz de usuario, hay varias técnicas para proporcionar asistencia al usuario.

- **Información sobre herramientas (Tool Tips)**

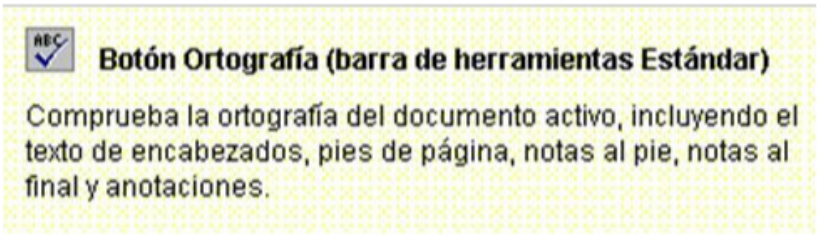
La información sobre herramientas (figura siguiente) es una excelente manera de presentar información a los usuarios mientras recorren la interfaz de usuario. Los Tool Tips son pequeñas etiquetas que se presentan cuando el puntero del mouse se mantiene durante un cierto período de tiempo sobre un control y que normalmente contienen una descripción de la función del control. Por lo general, se utilizan de forma conjunta con las barras de herramientas; Los Tool Tips funcionan bien casi en cualquier parte de la interfaz.



Información sobre la herramienta de la barra

Ayuda de tipo ¿Qué es esto?

La ayuda ¿Qué es esto? proporciona un vínculo con un tema de Ayuda emergente (vea la figura siguiente) cuando el usuario selecciona ayuda ¿Qué es esto? y hace *click* en el cursor ¿Qué es esto? de un control. La ayuda ¿Qué es esto? puede iniciarse desde un botón de una barra de herramientas, un elemento de menú o un botón de la barra de título de un cuadro de diálogo.



Una ventana emergente de ayuda ¿Qué es esto?

Presentación de estado

Una presentación del estado también se puede usar para proporcionar asistencia al usuario de manera muy parecida a Información sobre herramientas. La presentación de estado es una buena manera de ofrecer instrucciones o mensajes que no quepan en una Información sobre herramientas. Las barras de estado son útiles para presentar mensajes; también se puede usar un control de etiqueta como presentación de estado.

Asistentes

Un asistente es un elemento de asistencia al usuario que lo guía paso a paso a través de un procedimiento, trabajando con los datos reales de dicho usuario. Los asistentes se utilizan normalmente para proporcionar asistencia sobre tareas específicas. Ayudan al usuario a realizar una tarea que de otro modo requeriría un largo (y poco deseable) período de aprendizaje y también ofrecen, en ciertas ocasiones, información de experto a un usuario que tal vez no lo sea.

Hasta aquí, vemos los estándares de diseño de interfaces gráficas y los controles frecuentes que podemos encontrar en *Visual Studio*, cabe destacar que en varios IDE encontraremos los mismos controles con características similares. Para realizar los pedidos del usuario/cliente, debemos confeccionar una interfaz con los datos necesarios. Por ejemplo:

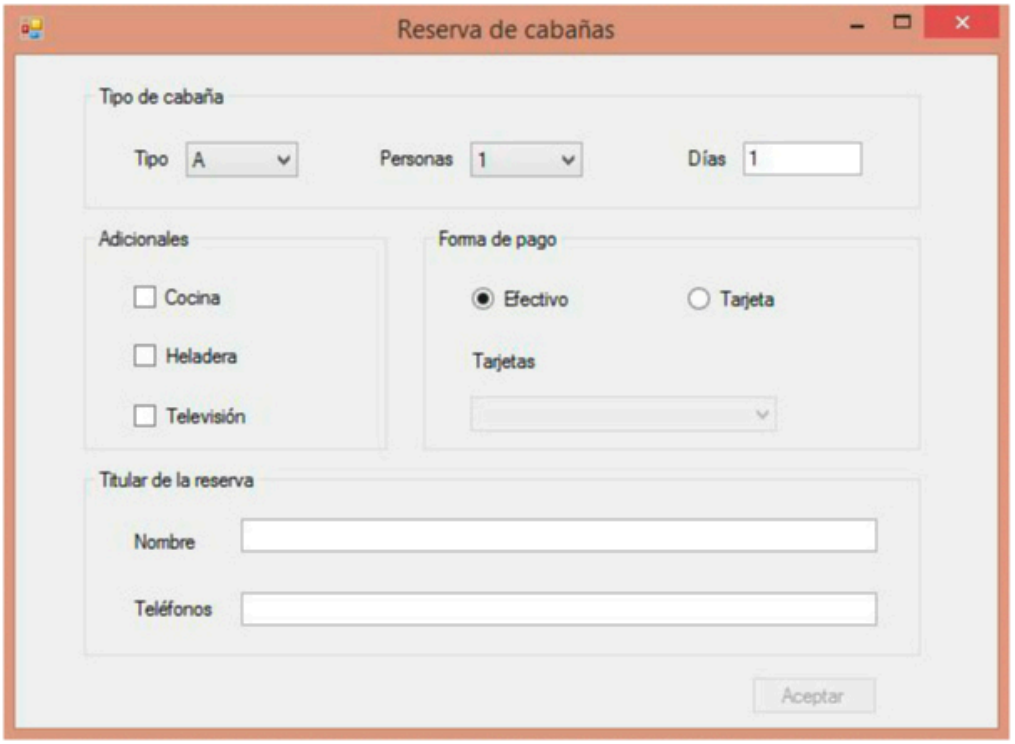
Título de la ventana "Reserva de cabañas"

Caja de grupo (**groupbox**) "Tipo de cabaña" donde contiene 3 etiquetas, 2 listas desplegables y 1 caja de texto.

Caja de grupo (**groupbox**) "Adicionales" donde contiene 3 casillas de verificación.

Caja de grupo (**groupbox**) "Forma de pago" donde contiene 2 botones de opción y 1 lista desplegable.

Caja de grupo (**groupbox**) "Titular de la reserva" donde contiene 2 cajas de texto.



Indique si es verdadero o falso

1) La propiedad MaxLength de un control de entrada de texto (textBox), se utiliza para definir la cantidad máxima de caracteres que se podrán introducir en él.

○ Verdadero

○ Falso

2) La diferencia fundamental entre los CheckBox y los RadioButton es su funcionalidad, en el caso del radioButton solo uno puede ser seleccionado.

○ Verdadero

○ Falso

3) Para agregar un ítem en un ComboBox o un ListBox, se utiliza Items.Add().

○ Verdadero

○ Falso

4) Los controles tienen solo propiedades y métodos.

○ Verdadero

○ Falso

5) Los controles básicos de C# .NET están compuestos por TextBox, Label, Button, entre otros.

○ Verdadero

○ Falso

6) La finalidad de la propiedad Enabled en la mayoría de los controles es habilitarlo o deshabilitarlo.

○ Verdadero

○ Falso

7) Un control Label (Etiqueta) permite modificar el texto que muestra en tiempo de ejecución, es decir cuando el programa se está ejecutando.

○ Verdadero

○ Falso

Respuestas correctas[31]

[31] 1)V; 2)F; 3)V; 4)F; 5)V; 6)V; 7)V.

SP2/H3: Identificadores, constantes, variables, tipos de datos y operadores

En la situación profesional se plantea la eventualidad de brindar información de precios por distintas categorías. Seguramente, en la resolución del problema, el programador de la aplicación deberá declarar variables para almacenar datos y realizar cálculos. Las variables son normalmente los elementos más usados en un programa, además disponemos de otros elementos como **constantes** y **funciones**. Todos ellos deben llevar un nombre apropiado que se define como "identificador". El uso correcto de Identificadores es, por lo tanto, el primer paso en la construcción de un programa.

Identificadores

Veamos algunos ejemplos de identificadores:

```
float Recargo; // 'Recargo' es el identificador de la variable declarada de tipo float
int Dias; // 'Dias' es el identificador de la variable declarada de tipo int
const string Titulo = "Listado Mensual"; // 'Titulo' es el identificador de la constante declarada de tipo string
```

Un identificador es el nombre empleado para identificar a cualquier elemento de un programa. Y debe tener las siguientes características:

- Comenzar con una letra o el guión bajo (_).
- No contener espacios, puntos y ciertos caracteres especiales.
- No puede ser una palabra clave reservada.
- No debe superar los veinticinco caracteres de longitud.

Con el proyecto que creamos anteriormente, podemos ver cómo funcionan las variables, y que algunas variables tienen propiedades. Para comprender mejor el código fuente que realizamos, vamos a ver en detalle qué tipos podemos crear en los siguientes párrafos.

Constantes

Las constantes son utilizadas para representar datos que no se modifican durante la ejecución del programa. Por ejemplo:

const int TOPE = 25;

Es muy útil en el código del programa emplear el nombre de la constante en lugar de su valor. Si el valor de una constante es utilizado en diferentes lugares del programa y debe ser modificado, el cambio se realiza en una sola parte del código. Veamos otro ejemplo:

const string AVISO = "Error en el ingreso de datos";

Si en lugar de utilizar en el programa la constante AVISO, el programador emplea el valor de la constante en distintas partes del programa, y este valor luego debe ser cambiado, lo cual resultará poco práctico.

El lenguaje posee muchas constantes definidas que pueden ser utilizadas sin ningún tipo de declaración.

Las constantes representan valores fijos utilizados en diferentes partes de un programa. Para declarar una constante, su sintaxis es la siguiente:

[public | private] const [Tipo] Nombre = Expresión;

- **public** es opcional e indica que la constante es pública y está disponible en todos los módulos.
- **private** es opcional e indica que la constante es privada y está disponible en el **módulo**[32] donde se declaró.
    - **Nombre:** es utilizado para identificar a la constante en el programa.
    - **Tipo:** es el tipo de dato de la constante.
    - **Expresión:** posee el valor que se le asigna a la constante.

[32] Módulo: Un módulo es un elemento del proyecto donde se pueden declarar variables, constantes, funciones, procedimientos, entre otros. Cuando la programación se realizaba en Visual Basic 6.0, estos módulos eran de gran utilidad, pero al utilizar Visual Basic .NET los módulos son reemplazados por clases.

Variables

Una variable se emplea en un programa para almacenar temporalmente un valor durante la ejecución del mismo. Una variable se identifica con un nombre, posee un tipo de dato y un valor. Los nombres asignados a variables deben respetar las siguientes reglas:

- El nombre debe comenzar con una letra o el guión bajo.
- El resto del nombre puede contener números.
- El nombre debe ser único dentro de su alcance.
- El nombre de la variable no puede ser una palabra reservada (*palabras que se utilizan en el lenguaje de programación como ser "for"*).

Para declarar una variable se utilizan las siguiente sintáxis:

Tipo Nombre;
Tipo Nombre = valor;

En el primer caso sólo se declara la variable, en el segundo caso se declara y se asigna un determinado valor.

Nombre es el nombre de la variable y Tipo, es el tipo de dato de dicha variable. A continuación se detallan los principales tipos de datos:

Tipo de dato	Descripción
bool	booleano, verdadero o falso
char	un único carácter
int	enteros
long	enteros de mayor rango
float	punto flotante
double	punto flotante de mayor precisión
decimal	valores de moneda
string	secuencia de caracteres

Ejemplo de declaraciones de variables:

```
int cantidad;
decimal precio;
float temperatura;
string nombre ;
```

Declaraciones y asignacion del valor inicial de las variables:

```
int cantidad = 0;
decimal precio = 115.50;
float temperatura = 27.3;
string nombre = "Juan Carlos";
```

Para los valores con decimales se usa el punto como separador decimal.

Cuando se declara una variable de tipo "*String*", en ella se pueden almacenar cadenas de longitud variable.

Existen buenas prácticas de nomenclatura recomendables, veamos algunas de ellas:

- Escribir variables en ***pascal case***: en lugar de declarar la variable **claveusuario**, se recomienda utilizar **ClaveUsuario**. Esta forma de nomenclatura da al programador una mejor visualización del nombre de la variable.
- Usar los nombres con la notacion **camel case**: similar al caso anterior, pero usando una minúscula para la primer palabra: **claveUsuario,** esta notacion también se conoce como ***lower camel case***.

Tipos de datos

Las constantes y variables pueden contener distintos tipos de datos o información. Se detallan a continuación los tipos de datos provistos por el lenguaje y el tamaño que ocupan en memoria.

Tipo de dato	Tamaño en memoria (bits)	Descripción
bool	8	valores True y False
char	16	un único carácter
int	32	enteros
long	64	enteros largos
float	32	con punto flotante
double	64	con punto flotante largo
decimal	128	con punto flotante de moneda
string	16 bits por cada carácter	secuencias de caracteres

Operadores aritméticos

Los operadores aritméticos son utilizados en un programa para realizar **cálculos matemáticos**; éstos se utilizan conjuntamente con variables y constantes para formar **expresiones** que retornan un valor, por ejemplo, multiplicar la cantidad pedida de un artículo por su precio y almacenar el importe resultante. A continuación, se detallan los operadores provistos por el lenguaje:

Operador	Operación
+	suma
-	resta
*	multiplicación
/	división
++	incremento unitario
--	decremento unitario
%	módulo: resto de la división

Ejemplo:

```
float Total = 0; // inicializamos las variables
float Importe = 0;
int acumulado = 0;
float Iva = 0.21;
Importe = 350; // asignación de un valor
Iva = Importe * 21 / 100; // se multiplica y divide
Total = Importe + Iva; // se suman dos variables
acumulado++; // se incrementa en una unidad el valor de la variable
```

Operadores de comparación

En el lenguaje los operadores de comparación son utilizados para realizar comparaciones entre expresiones. A continuación, se detallan los operadores provistos por el lenguaje:

Operador	Operación
==	igualdad
!=	desigualdad
<	menor que
>	mayor que
<=	menor o igual que
>=	mayor o igual que

Los operadores de comparación son utilizados en las condiciones de **estructuras de control** de un programa. Ejemplo:

```
int Valor;
int Tope;
...
Valor > 1000 // se compara la variable Valor con 1000 para saber si en mayor.
```

Valor != 0 // se compara la variable Valor para saber si es distinta de cero.

Valor == Tope // se comparan dos variables para saber si son iguales

Operadores lógicos

Los operadores lógicos permiten en el lenguaje conectar expresiones y determinar su veracidad o falsedad. Producen resultados de tipo verdadero o falso. La lista de operadores es la siguiente:

Operador	Operación
&&	conjunción lógica (AND)
||	disyunción lógica (OR)
!	negación (NOT)

Ejemplos:

Para dato1 = 25 y dato2 = 20

La expresión:

dato1 == 25 && dato2 == 37 dá como resultado false ya que, aunque dato1 es 25, la comparación de dato2 con 37 devuelve falso. Para obtener un resultado *True* ambas comparaciones deben ser verdaderas.

dato1 != 25 || dato2 >= 20 dá como resultado true, ya que aunque la primer comparacion es falsa, la segunda es verdadera.

Otros operadores

En el lenguaje el operador "+" se utiliza para concatenar expresiones de texto. Ejemplo:

string Nombre = "Pedro" + " " + "Pérez"; // resultado Nombre = "Pedro Pérez"

Otro operador utilizado en el lenguaje es el operador punto, que sirve para conectar objetos con propiedades y métodos. Recuperar y almacenar datos en variables creadas a partir de tipos de datos definidos por el usuario. Ejemplos:

```
txtDomicilio.Text = "San Martín 123";
Alumno.Edad = 20 ;
```

txtDomicilio es un control textBox y 'Text' es la propiedad que guarda el valor visualizado en el control

Alumno es un objeto que posee la propiedad 'Edad' de tipo numérica, guarda la edad del alumno

En el lenguaje, el operador "=" se emplea para asignar valores. Ejemplos:

```
int a; // a es una variable entera
a = 10; // se asigna el valor 10 a la variable 'a '
```

Se han mencionado los elementos básicos de un programa, algunos de ellos serán utilizados en la resolución de la situación profesional, por ejemplo, declarar constantes, variables de un determinado tipo, trabajar con operadores aritméticos en los cálculos, operadores relacionales para comparar datos y operadores lógicos para conectar expresiones para analizar.

Hasta aquí, la utilidad de las variables nos permite almacenar temporalmente información que nos puede ser útil para solucionar el planteo propuesto por nuestra situación profesional.

Indique verdadero o falso

1) El operador **"="** es utilizado solo para realizar operaciones de asignación.

- ○ Verdadero
- ○ Falso

2) El operador de comparación **"!="** se utiliza para verificar si el contenido de dos expresiones no son iguales.

- ○ Verdadero
- ○ Falso

3) El operador **"&&"** permite concatenar expresiones de cualquier tipo.

- ○ Verdadero
- ○ Falso

4) La expresión **(Valor % 4)** retorna el valor cero, siempre y cuando el contenido de la variable "Valor" no sea múltiplo de cuatro.

- ○ Verdadero
- ○ Falso

5) Si en una estructura de control **"If"** se analizan dos expresiones conectadas por el operador lógico **OR:**
"||" y una sola de las expresiones es falsa, se ejecuta el cuerpo de la estructura.

- ○ Verdadero
- ○ Falso

Indique la opción correcta

6) Indique si en el siguiente fragmento de código existe un error:

```
int A = 9;

string B = "0";

int C = A + B;
```

- ○ Si
- ○ No

Respuestas correctas[33]

[33] 1)Verdadero; 2)Verdadero; 3)Falso; 4)Falso; 5)Verdadero; 6)Si.

La herramienta anterior nos permitió aprender a declarar los elementos básicos de un programa, pero en la resolución de problemas se necesitan instrucciones para ejecutar un bloque de instrucciones o no, dependiendo de una condición: repetir la ejecución de un bloque de instrucciones hasta que se cumpla una condición. Estas instrucciones en los lenguajes de programación reciben el nombre de estructuras de control. En la situación profesional, el programa debe realizar un cálculo en función de la forma de pago, en este caso se debe utilizar una estructura de control alternativa.

Las estructuras de control se emplean en un programa para controlar el orden en que se ejecutan sus instrucciones. Las estructuras de control permiten tomar decisiones respecto de la ejecución de las instrucciones del programa. Pasemos a enumerar las distintas estructuras de control que posee el lenguaje.

Condicionales

• Estructura If

La estructura "if" permite ejecutar instrucciones en forma condicional. Su sintaxis es la siguiente:

C #: Estructura if

```
if (condicion)
{
  //bloque de sentencias que se ejecuta sólo si la condición es verdadera (True)
}
```

Lo podemos traducir como: si la condición es verdadera (*true*) ejecutar este bloque de sentencias, en caso contrario no ejecutar nada. La condición debe ser una expresión booleana, es decir que se pueda evaluar como verdadera o falsa, como, por ejemplo, (A > B) o (J < 10).

Ejemplo para programar:

C #: Ejemplo if

```
float Pago = 0;
float Interes = 0;
float Importe = 0;
float Saldo = 0;
// asignamos un importe
Importe = 1000;
// asignamos un pago
Pago = 875;
// usamos una condición para saber si el pago es menor al importe y asi aplicar un interés
if (Pago < Importe)
{
  // se aplica el 20% de interés sobre el Importe
  Interes = Importe * 20 / 100;
  Saldo = (Importe - Pago) + Interes;   // cálculo del saldo a pagar
  MessageBox.Show("Proximo pago: " + Saldo.ToString());
}
```

Ejercicio para el alumno: en base al ejemplo realizar el proyecto en *Windows Forms* (interfaz gráfica).

En este ejemplo, si el monto pagado es mayor al importe, se realiza el cálculo de intereses; pero si el monto pagado no es mayor a la fecha de vencimiento, no se ejecuta el cuerpo de la estructura de control "If", es decir, no se realiza el cálculo de intereses.

• Estructura if - else

La estructura de control "*if - else*" permite ejecutar un bloque de instrucciones si la condición es verdadera y otro bloque de instrucciones, si la condición es falsa. La sintaxis es la siguiente:

C #: Estructura if - else

```
if (condición)
{
  // bloque de sentencias que se ejecuta sólo si la condición es verdadera (true)
}else
{
  // bloque de sentencias que se ejecuta sólo si la condición es falsa (false)
}
```

A continuación se presenta un ejemplo para programar y complementar el ejemplo anterior:

C #: Ejemplo if - else

```
float Pago = 0;
float Interes = 0;
float Importe = 0;
float Saldo = 0;
// asignamos un importe
Importe = 1000;
// asignamos un pago
Pago = 1000;
// usamos una condición para saber si el pago es menor al importe y asi aplicar un
interés
if (Pago < Importe)
{
  // se aplica el 20% de interés sobre el Importe
  Interes = Importe * 20 /100;
  Saldo = (Importe - Pago) + Interes;   // cálculo del saldo a pagar
  MessageBox.Show("Proximo pago: " + Saldo.ToString());
}else
{
  MessageBox.Show("Su pago ha sido registrado, el monto es: " + Importe.ToString());
}
```

Ejercicio para el alumno: modifique este ejemplo para realizar el proyecto en *Windows Forms* (interfaz gráfica).

• Estructura switch

La estructura "*switch*" es muy útil para la legibilidad del código del programa cuando la expresión que se analiza puede tomar muchos valores; se evita de esta manera el uso de la estructura "*if - else*" con numerosos bloques anidados. "**Anidado**": se refiere a un bloque de código contenido en otro bloque de código, en este caso un bloque "if" dentro de otro bloque "if".

La sintaxis de la instrucción se detalla a continuación:

C #: Estructura switch

```
switch (expresion)
  {
     case valor1:
        // bloque de sentencias a ejecutar cuando 'expresion' concide con valor1
        break; // sale de la estructura switch
     case valor2:
        // bloque de sentencias a ejecutar cuando 'expresion' concide con valor2
        break; // sale de la estructura switch
     case valor3:
        // bloque de sentencias a ejecutar cuando 'expresion' concide con valor3
        break; // sale de la estructura switch
     default:
        // bloque de sentencias a ejecutar cuando 'expresion' no concide con
        // ningún valor anterior
        break; // sale de la estructura switch
  } // fin de switch
```

No siempre es posible emplear la estructura **switch**, ya que posee algunas restricciones:

1. La **expresion** debe ser de tipo int o string , si se necesita trabajar con otro tipo de dato, float o double, por ejemplo, se deberá usar la estructura if.
2. Los valores de cada **case**, deben ser constantes, no es posible emplear variables en los case,
3. No se pueden repetir los valores de los **case**.
4. El uso de la sentencia **break** es obligatorio, cada case debe finalizar con **break**, también si se usa **default** debe llevar su correspondiente break.

A diferencia de la estructura "if" que hace uso de una "*condición*", el "switch" usa una "*expresión*" que puede ser una variable o una operación que de por resultado un determinado valor, dicho valor se comparará con cada uno de los valores en los 'case' dentro del switch, de forma tal que sólo uno de esos bloques será ejecutado. Si el valor de la '*expresión*' no coincide con ninguno de los casos, se ejecuta el bloque de instrucciones correspondiente a la instrucción "d*efault*". Veamos un ejemplo para programar:

C #: Ejemplo switch

```
string nombre = "";
// ingresar un nombre
nombre = "Ana";
 switch(nombre)
 {
     case "Pedro":
         MessageBox.Show("Arquitecto");
         break;
     case "Ana":
         MessageBox.Show("Doctora");
         break;
     case "Carlos":
         MessageBox.Show("Estudiante);
         break;
     default:
         MessageBox.Show("Desconocido");
         break;
 }
```

Ejercicio para el alumno: modifique este ejemplo para realizar el proyecto en *Windows Forms* (interfaz gráfica).

En el ejemplo se analiza el contenido de la variable "Nombre"; en caso de que su valor sea igual a "Pedro", se muestra la cadena "Arquitecto"; en caso de valer "Ana", se muestra la cadena "Doctora"; en caso de valer "Carlos", se muestra la cadena "Estudiante" y si el valor de la variable "Nombre" no es igual a ninguno de los casos analizados, se muestra la cadena "Desconocido".

Se puede decir que la estructura de control "switch" es un "if" anidado más elegante desde el punto de vista de la claridad del código escrito. En el ejemplo anterior, utilizando la estructura de control "if - else", el código escrito por el programador debería haber sido el siguiente:

C #: Ejemplo if anidado

```
 if (Nombre == "Pedro")
{
   MessageBox.Show("Arquitecto");
}
else
{
   if (Nombre == "Ana")
   {
      MessageBox.Show("Doctora");
   }
   else
   {
      if (Nombre == "Carlos")
      {
         MessageBox.Show("Estudiante");
      }
      else
      {
         MessageBox.Show("Desconocido");
      }
   }
}
```

Se puede apreciar en el bloque de "*if else*" anidado, que es menos claro cuando se analizan muchos valores.

Ejercicio para el alumno: modifique este ejemplo para realizar el proyecto en *Windows Forms* (interfaz gráfica).

Ejercitación para el alumno: descargue el siguiente archivo con ejercicios para practicar las estructuras de control con C#.

REPETITIVAS

• Estructura "while" y "do while"

La estructura de control repetitiva "*while*" permite a un programa repetir la ejecución de un bloque de instrucciones, mientras que se cumpla una determinada condición. La sintaxis de la instrucción "*Mientras*" tiene dos variantes:

C #: Estructura while y do - while

```
while (condicion)
 {
    // bloque de instrucciones a ejecutar mientras la condición sea verdadera
 }
 do {
    // bloque de instrucciones a ejecutar
 } while(condicion);
 // si la condición es verdadera se volverá a ejecutar el mismo bloque de instrucciones
```

En el primer caso se analiza la *condicion*, si la misma es verdadera, se ejecuta el bloque de instrucciones delimitado por las llaves { } que definen el bloque de instrucciones, el proceso se repite nuevamente mientras la condición sea verdadera y finaliza solamente cuando la condición se vuelve falsa. Para el segundo caso se ejecuta el bloque de instrucciones y luego se analiza la condición, si la misma es verdadera, se repite el proceso mientras la condición sea verdadera. Note que el bloque do-while finaliza con punto y coma.

La diferencia entre una variante y otra es el orden en el que se ejecutan las instrucciones, en un caso se analiza la condición y luego se ejecuta el bloque de instrucciones y en el otro caso, se ejecuta el bloque de instrucciones y luego se analiza la condición.

Ejemplo:

C #: Ejemplo while

```
int Dato = 1;
while (Dato < 6)
{
   Dato++;
}
MessageBox.Show("Dato: " + Dato.ToString());
```

Aquí se muestra el valor final de la variable "Dato", el bucle analiza la condición "Dato < 6" y si la expresión es verdadera, se incrementa Dato en una unidad. El proceso se repite y el mismo termina cuando la variable "Dato" toma el valor 6; en este caso, la expresión es falsa (6 no es menor a 6).

Otro ejemplo para agregar al proyecto que pryRepetitivas:

C #: Ejemplo do - while

```
int Dato = 5;
do {
   Dato--;
} while (Dato > 0);
MessageBox.Show("Dato: " + Dato.ToString());
```

En este caso el bucle decrementa la variable "*Dato*" en una unidad, luego se analiza la condición (Dato es mayor a 0); como la condición es verdadera, se repite el proceso, que termina cuando la variable "*Dato*" es igual a cero, en ese momento la condición es falsa (0 no es mayor a 0).

Entonces qué valor mostrará el mensaje al final del ciclo *do-while* ?

• La estructura "for"

La estructura "while" se utiliza normalmente cuando el número de veces que se debe repetir un bloque de instrucciones no se conoce. Para el caso de conocer el número de veces que se debe repetir el bloque de instrucciones, existe la estructura "for".

La sintaxis de la estructura "for" es la siguiente:

C #: Estructura for

```
for (inicializacion; condición; incremento)
{
   // bloque de sentencias a ejecutar mientras la condición sea verdadera
}
```

- **inicializacion**: consiste en una o más instrucciones de asignación inicial a la variable que se usa para controlar la cantidad de repeticiones del ciclo. La inicialización se ejecuta una única vez, al comienzo del ciclo.
- **condición**: es una expresión que resulta en verdadero o falso (booleana) y que determinará el final de las repeticiones cuando su resultado sea falso. La condición se evalúa antes de comenzar a ejecutar el cuerpo del ciclo, si es falsa, el ciclo finaliza.

- **incremento**: variación que tiene la variable de control en cada ciclo de la repetitiva, en general es un incremento unitario, pero puede ser con otra variación, de dos en dos , por ejemplo, o también puede ser una variación en forma de decremento. El incremento se ejecuta siempre al final de cada iteración, justo antes de evaluar la condición.

Veamos algunos ejemplos.

Obtener la suma de todos los números enteros entre 1 y 10:

C #: Ejemplo for

```
  int n;
  int suma = 0;  // valor inicial de suma
  for (n=1; n <= 10; n++)  // se lee: para n desde 1 hasta 10, (mientras sea menor o
igual a 10), repetir
  {
     suma = suma + n;  // esta sentencia se ejecutará 10 veces, en cada ciclo, la
variable 'n' cambia su valor
              // de acuerdo al incremento en el ciclo for, en este caso se incrementa en
una unidad
  }
  MessageBox.Show("Suma: " + suma.ToString());
```

Como puede verse en el ejemplo anterior la variable suma va acumulando los valores de 'n', que comienza el ciclo con el valor 1 y se va incrementando en una unidad en cada iteración del ciclo for, como la condición, es mientras n <= 10, el último valor de 'n' que se acumula es 10, resultando en total 10 iteraciones y sumando entonces los valores 1, 2, 3, 4, 5, 6, 7, 8, 9 y 10.

Obtener la suma de todos los números impares entre 1 y 10:

C #: Ejemplo for con incremento en 2

```
  int n;
  int suma = 0;  // valor inicial de suma
  // observe que el incremento es de 2 en 2, y como 'n' se inicia en 1, se obtienen
valores impares
  for (n=1; n <= 10; n=n+2)  // n=1, 3, 5, 7 y 9
  {
     suma = suma + n;  // esta sentencia se ejecutará 5 veces
  }
  MessageBox.Show("Suma de impares: " + suma.ToString());
```

En este caso el incremento es de 2 en 2 y como el valor inicial de 'n' es 1, se obtienen siempre valores impares.

El bloque de instrucciones que se repite está delimitado por las llaves { }

Ejercicio para el alumno: modifique este ejemplo para realizar el proyecto en *Windows Forms* (interfaz gráfica).

Para concluir, se puede decir que, si en un programa se necesita ejecutar o no un bloque de instrucciones, se utilizan las estructuras de control alternativas (if, if else o switch). Mientras que para repetir la ejecución de un bloque de instrucciones, se utilizan las estructuras de control repetitivas (while, do while o for). La utilización de una u otra estructura estará dada por el algoritmo que plantea el programador para resolver un problema. En general, a medida que la complejidad del algoritmo crece se necesitan emplear varias de estas estructuras de control combinadas de distintas formas.

Estructuras de control alternativas

```
if (condición)
{
        // bloque de instrucciones
}


if (condición)
{
        // bloque de instrucciones
} else
{
        // bloque de instrucciones
}


switch (expresión)
{
        case valor1:
                // bloque de instrucciones
                break;
        case valor2:
                // bloque de instrucciones
                break;
        default:
                // bloque de instrucciones
                break;
}
```

Estructuras de control repetitivas

```
while (condición)
{
        // bloque de instrucciones
}

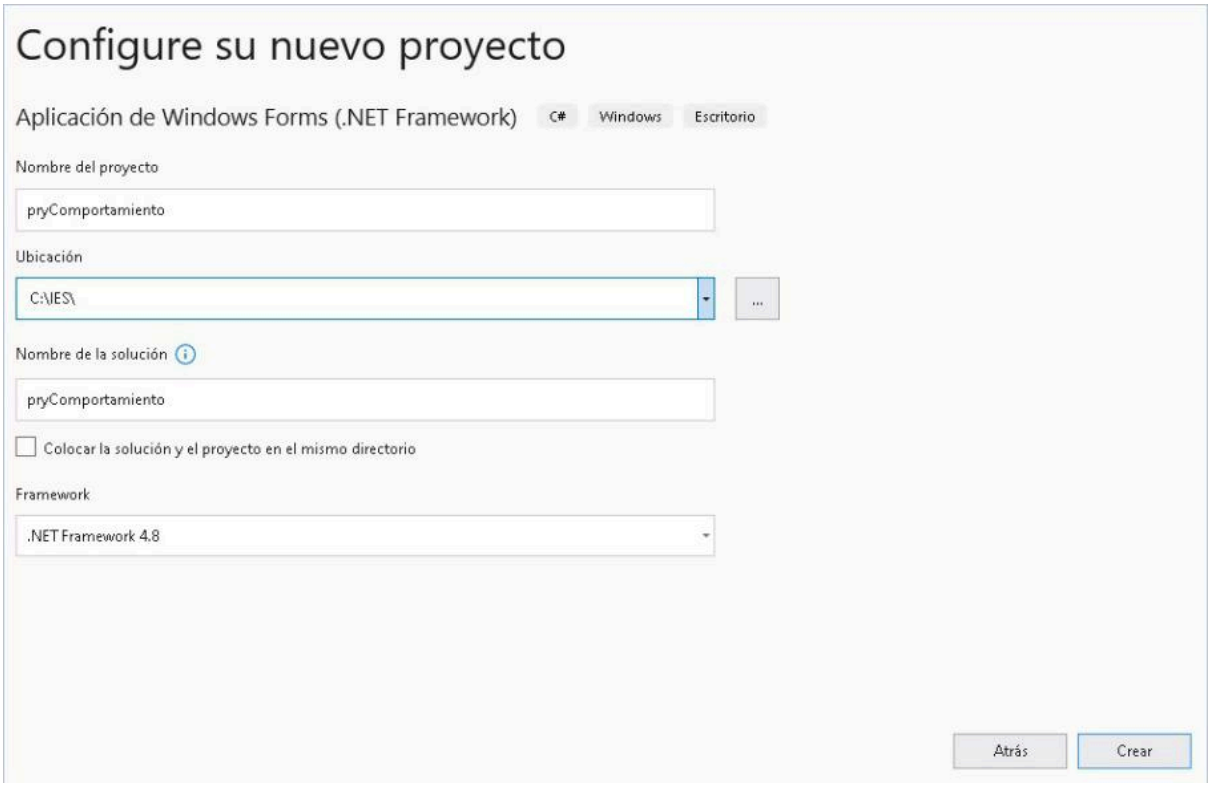
do {
        // bloque de instrucciones
} while (condición);


for ( inicialización, condición, incremento)
{
        // bloque de instrucciones
}
```

Estructuras de control

El comportamiento de la interfaz, es algo muy importante en una aplicación, ya que debemos guiar al usuario por medio de la interfaz gráfica, intentar que se realicen los menores errores de ingreso de datos y que sean de fácil utilización los controles. Veamos un ejemplo para programar:

1. Abra **Visual Studio.**
2. Cree un **Nuevo proyecto**. Seleccione la plantilla Aplicación de *Windows Forms* (.NET Framework) con lenguaje C#.
3. Nombre el proyecto pryComportamiento.



Fuente: Elaboración del autor

4. Nombre el proyecto pryComportamiento.
5. Hará la siguiente configuración:
    - a) En el **formulario**:

        i.NAME: frmComportamiento

        ii.Text: Comportamiento

        iii.Size: 320, 270

- b) Inserte dos etiquetas y configure:

  i.**Label1**

  1.Name: lblNombre

  2.Text: Nombre

  ii.**Label2**

  1.Name: lblApellido

  2.Text: Apellido

- c) Inserte dos cajas de texto

  i.**TextBox1**

  1.Name: txtNombre

  ii.**TextBox2**

  1.Name: txtApellido

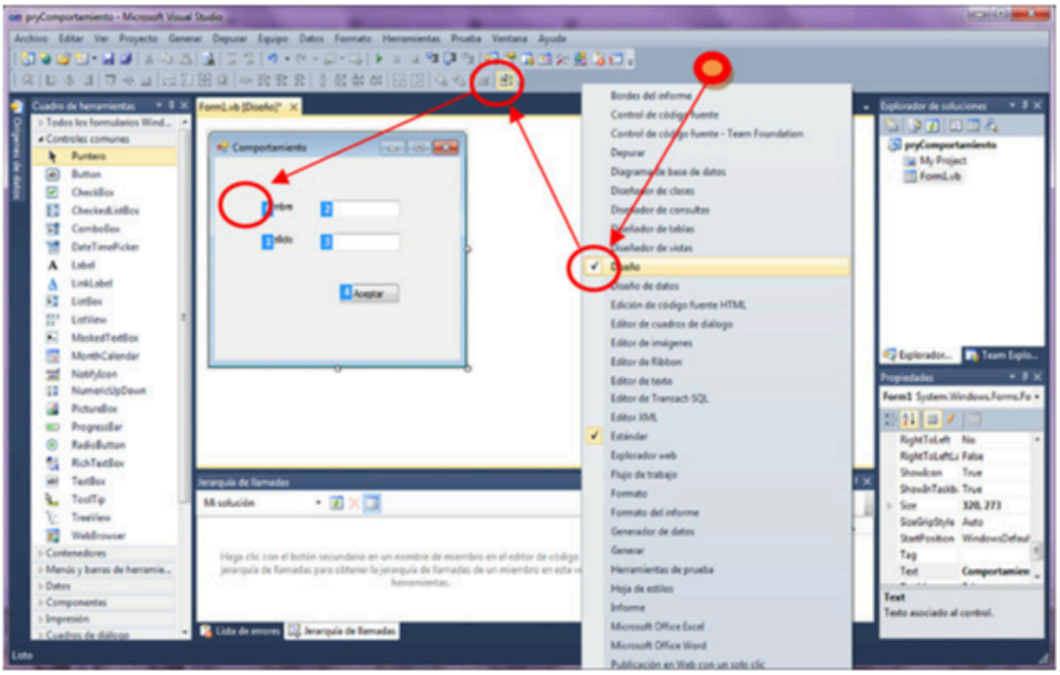
- d) Inserte un botón de comando

  i.**Button1**

  1.Name: btnAceptar

  2.Text: &Aceptar

6. Después de "dibujar" el formulario, veremos el comportamiento de TABULACIÓN, esto se lleva a cabo en el FOCO de cada componente en el formulario, presionando la Tecla TAB. Dentro del IDE Visual Studio hacemos clic en la barra de herramientas y elegimos la opción DISEÑO, aparecerá una barra de diseño, el último botón es Orden de Tabulación, como verá en el siguiente gráfico:

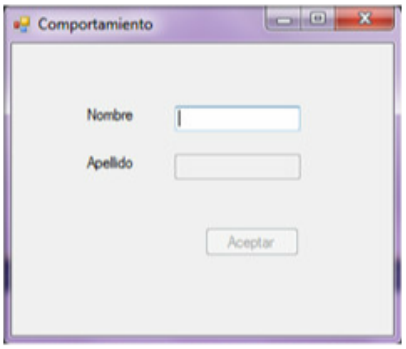


7. Observará el orden de tabulación. Haciendo click sobre los distintos componentes verá que cambia el orden por el cual va a tomar FOCO.

8. Comportamiento:

    - a) La secuencia de ingreso de datos en este formulario debería ser, al ingresar información en el **txtNombre** que se habilite **txtApellido**.
    - b) Al ingresar información en el **txtApellido** que se habilite **btnAceptar**.
    - c. Al hacer clic en el botón **btnAceptar** que aparezca una ventana de mensaje con el Nombre y Apellido. También que se "limpien" los controles de la pantalla.

9. En la interfaz seleccionar el control **txtApellido** y en la ventana de propiedades pondremos la propiedad ENABLED de txtApellido en FALSE para que inicialmente el control se muestre deshabilitado, repetir lo mismo con el control **btnAceptar**. Quedará de la siguiente manera:



Controles deshabilitados

10. Sobre **txtNombre** en diseño, haga doble click izquierdo, entre en el código, y directamente en el evento **txtNombre_TextChanged**. Dicho evento se dispara cuando se modifique el contenido de la caja de texto.
    - Dentro realizaremos el siguiente código:

C #: evento TextChanged

```
private void txtNombre_TextChanged(object sender, EventArgs e)
{
    if (txtNombre.Text != "")
    {
        txtApellido.Enabled = true;
    }else
    {
        txtApellido.Enabled = false;
    }
}
```

Sobre **txtApellido** en diseño, haga doble *click* izquierdo, entre en el código, y directamente en el evento txtApellido_TextChanged. Dentro realizará el siguiente código:

C #: Evento TextChanged en txtApellido

```
private void txtApellido_TextChanged(object sender, EventArgs e)
{
    if(txtApellido.Text != "")
    {
        btnAceptar.Enabled = true;
    }else
    {
        btnAceptar.Enabled = false;
```

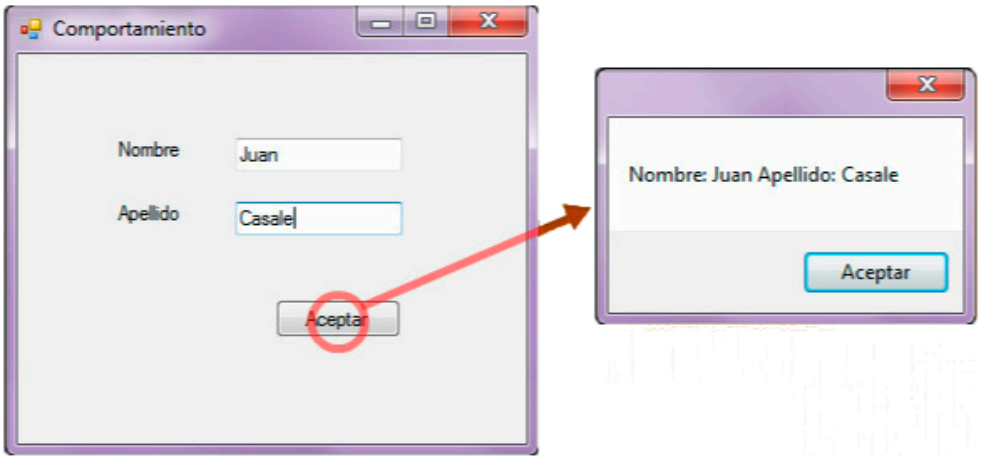
```
        }
    }
```

11. Sobre btnAceptar en diseño, haga doble *click* izquierdo, entre en el código, y directamente en el evento **btnAceptar_Click** realizaremos el siguiente código:

C #: Evento Click de btnAceptar

```
    private void btnAceptar_Click(object sender, EventArgs e)
    {
        MessageBox.Show("Nombre: " + txtNombre.Text + " Apellido: " +
txtApellido.Text);
        // limpiar controles
        txtNombre.Text = "";
        txtApellido.Text = "";
        // hacer foco en el nombre
        txtNombre.Focus();
    }
```

12. Cuando compile y ejecute el programa, después de escribir un nombre y apellido, haga clic en Aceptar debería aparecer:



Uso del espacio en blanco

Este proyecto muestra básicamente el comportamiento de una interfaz. En habilitación de controles y limpieza de los mismos.

Controles comunes

Los controles comunes son los que encontramos en el *Visual Studio* instalados por defecto, y generalmente los más utilizados por los desarrolladores; hemos visto desarrollado anteriormente en la Situación profesional una descripción de cada uno, ahora veremos la utilización de los mismos. Como ejemplo arrastraremos los controles y dejaremos sus nombres por defecto. Probaremos distintos tipos de propiedades en dichos controles, además deberemos utilizar estructuras lógicas y repetitivas para que lleven a cabo una determinada tarea. Crearemos un proyecto para programar:

1. Abra **Visual Studio.**
2. Cree un Nuevo proyecto. Seleccione Aplicación de Windows Form.
3. Nombre el proyecto pryControlesComunes.
4. Al Form1 asignaremos en la propiedad.
    - a. NAME: frmEjemplo
    - b. Text: **Prueba de Controles Comunes**
5. Arrastrare desde el cuadro de herramientas, los siguientes controles:
    - a. 3 controles CheckBox, en la propiedad Text cambie en cada uno de ellos:

        i. NAME: **chkRojo; chkAzul; chkAmarillo**

        ii. Text: **Rojo, Azul, Amarillo.**

    - b. 1 control Label, al cual asignar las siguientes propiedades:

        i. NAME: lblEjemplo

        ii. Text: **Texto de Ejemplo**

        iii.BorderStyle: Fixed3D

        iv.Font – Size: 16

    - c. COMPORTAMIENTO: cada vez que se seleccione un CheckBox, debe aplicar o quitar dicho color (rojo, azul o amarillo) a la etiqueta **lblEjemplo**.

- i. Cuando se seleccionen dos colores, deberá asignarse el color que forman a la etiqueta.

  ROJO-AZUL = VIOLETA

  ROJO-AMARILLO=NARANJA

  AZUL-AMARILLO=VERDE

6. El código a programar sería el siguiente:

C #: Codigo frmEjemplo

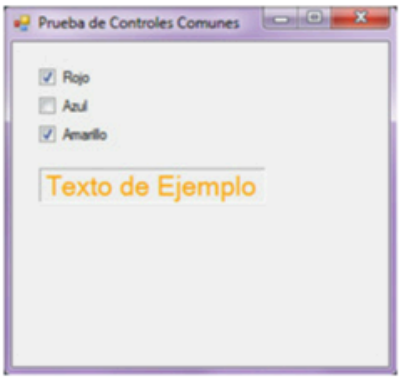
```
public partial class frmEjemplo: Form
{
        private void chkRojo_CheckedChanged(object sender, EventArgs e)
        {
            if (chkRojo.Checked == true) // Checked indica si el control está marcado o
no
            {
                lblEjemplo.ForeColor = Color.Red; // ForeColor determina el color del
texto
            }else
            {
                lblEjemplo.ForeColor = Color.Black; // 'Color' permite usar colores
            }
        }
        private void chkAzul_CheckedChanged(object sender, EventArgs e)
        {
            if (chkAzul.Checked == true)
            {
                lblEjemplo.ForeColor = Color.Blue;
            }else
            {
                lblEjemplo.ForeColor = Color.Black;
            }
        }
        private void chkAmarillo_CheckedChanged(object sender, EventArgs e)
        {
            if (chkAmarillo.Checked == true)
            {
                lblEjemplo.ForeColor = Color.Yellow;
            }else
            {
                lblEjemplo.ForeColor = Color.Black;
            }
        }
}
```

Compilar, apretando el botón Iniciar compilación o F5, y pruebe la aplicación. Hasta aquí podemos asignar un color a la etiqueta. Para poder asignar la combinación de colores, tendremos que utilizar una estructura alternativa condicional, como el IF anidado o el *Select Case*. Por ejemplo:

C #: Evento CheckedChanged de chkRojo

```
private void chkRojo_CheckedChanged(object sender, EventArgs e)
{
    if (chkRojo.Checked == true)
    {
        lblEjemplo.ForeColor = Color.Red;
        if (chkAzul.Checked == true)
        {
            lblEjemplo.ForeColor = Color.Violet; // Rojo + Azul
        }
        if (chkAmarillo.Checked == true)
        {
            lblEjemplo.ForeColor = Color.Orange; // Rojo + Amarillo
        }
    }else
    {
        lblEjemplo.ForeColor = Color.Black;
    }
}
```

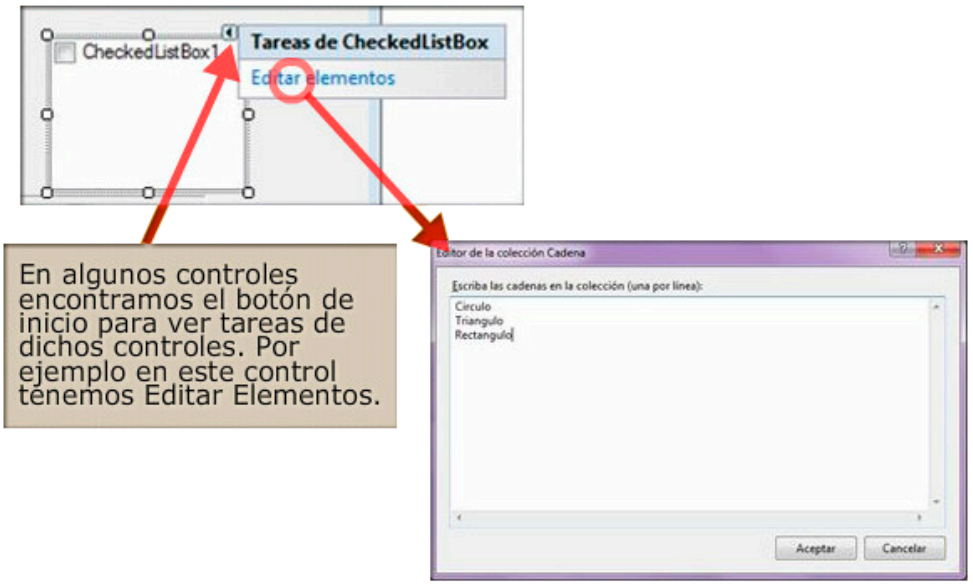
7. Repita la lógica en cada caja de verificación, con sus respectivos colores. Incluya el caso de tener los 3 *checkBox* marcados. Hasta aquí hemos utilizado controles *CheckBox* y *Label*, con algunas de sus propiedades.



Controles comunes, formulario

Para terminar apropiadamente este proyecto, utilizaremos más adelante funciones.

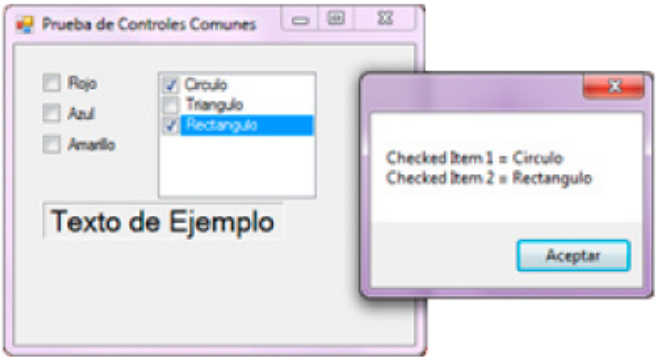
8. Agregaremos un *CheckedListBox*, para probar las mismas acciones que programamos anteriormente (dejamos el nombre del control por defecto). Configuraremos el control de la siguiente manera:



9. Para este ejemplo, realizaremos la siguiente codificación a modo de ejemplo:

C #: Evento SelectedIndexChanged de checkedListBox1

```
private void checkedListBox1_SelectedIndexChanged(object sender, EventArgs e)
      {
          if (checkedListBox1.CheckedItems.Count > 0)
          {
              int x;
              string s = "";
              for(x=0; x < checkedListBox1.CheckedItems.Count; x++)
              {
                  s = s + "Opción seleccionada " + (x + 1).ToString() + " "
                      + checkedListBox1.CheckedItems[x].ToString() + "\n";
              }
              MessageBox.Show(s);
          }
      }
```



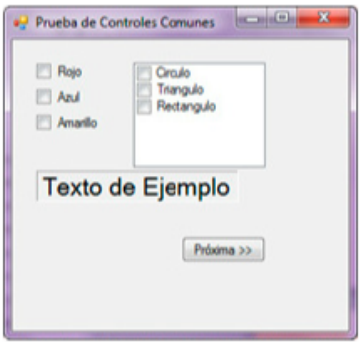
10. Agregar un botón, NAME: **btnProxima**, TEXT: **"Próxima>>".**

11. Agregamos un WindowsForm al proyecto, al mismo NAME: frmEjemplo2, Text: **"Prueba de Controles Comunes - 2".**

12. Hacemos doble clic en btnProxima para generar el evento Click y codificamos lo siguiente:

C #: Evento Click btnProxima

```
private void btnProxima_Click(object sender, EventArgs e)
{
    frmEjemplo2 frm = new frmEjemplo2();
    frm.Show();  // el método "Show" permite visualizar el formulario
}
```



13. Hacemos doble clic en Form2 / frmEjemplo2, luego al estar en tiempo de diseño hacemos doble clic sobre el formulario (que no debería tener ningún control insertado en el mismo).

C #: Evento Load de frmEjemplo2

```
      private void frmEjemplo2_Load(object sender, EventArgs e)
      {
       // 'OpenForms' nos permite acceder a los formularios abiertos
          Application.OpenForms[0].Hide();
          // el método 'Hide' sirve para ocultar el objeto, en este caso el primer
formulario (frmEjemplo)
      }
```

14. En el form2 / frmEjemplo2, agregaremos los siguientes controles y configuraciones:
    - a. 2 controles **TextBox**, en las propiedades cambiaremos en cada uno de ellos:

        i. NAME: **txtAgregar**; **txtMostrar**.

        ii. En txtAgregar cambiamos la propiedad **MAXLENGTH** en 12.

        iii.En txtMostrar habilitaremos la propiedad **MULTILINE** en **TRUE**. Extenderemos el control, dando forma de cuadrado al control.

    - b. 2 controles Button, a los cuales asignaremos las siguientes propiedades:

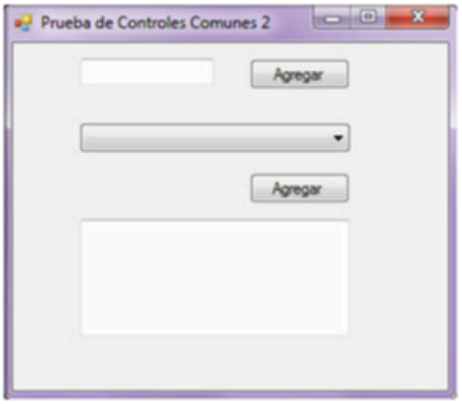
        i. NAME: b**tnAgregarCombo, btnAgregarCaja**

        ii. Text: Agregar

    - c. 1 control ComboBox

        i. Name: **cboLista**

        ii. DropDownStyle: DropDownList (*de esta forma no podrá escribir sobre el control*)



Controles comunes, segundo formulario

15. Comportamiento: txtMostrar, cboLista, btnAgregarCaja, y btnAgregarCombo estarán inhabilitados al entrar a la pantalla. Esta vez lo haremos por código y no por diseño, haciendo doble click sobre el formulario (sin seleccionar o posicionarnos sobre otros controles). Accederemos al evento *FORM LOAD* del formulario, esta sección se ejecuta antes de mostrar el mismo. Codificaremos lo siguiente:

C #: Evento Load de frmEjemplo2

```
private void FrmEjemplo2_Load(object sender, EventArgs e)
{
    txtMostrar.Enabled = false;
    cboLista.Enabled = false;
    btnAgregarCaja.Enabled = false;
    btnAgregarCombo.Enabled = false;
}
```

- a) Al ingresar texto en **txtAgregar** se debe habilitar **btnAgregarCombo**. Si no hay texto debe estar deshabilitado.
- b) Al hacer *click* en el botón debe habilitarse **cboLista** y agregar el contenido de la caja de texto **txtAgregarCombo** al combo box. Debe limpiarse **txtAgregarCombo** y hacer foco sobre el mismo.
- c) Cuando seleccione un elemento de cboLista, debe habilitarse **btnAgregarCaja**.
- d) Al hacer click en **btnAgregarCaja** se habilitará **txtMostrar** y se agregará el elemento seleccionado del combo box.

16. La codificación sería la siguiente:

Evento TextChanged() en txtAgregar: este evento se dispara cada vez que se modifica el contenido de la caja Agregar:

C #: Evento TextChanged de txtAgregar

```
private void txtAgregar_TextChanged(object sender, EventArgs e)
      {
          if (txtAgregar.Text != "")
          {
             // habilita al botón cuando hay algo escrito en txtAgregar
              btnAgregarCombo.Enabled = true;
          }else
          {
           // en caso contrario lo deshabilita
              btnAgregarCombo.Enabled = false;
          }
      }
```

Evento Click del boton AgregarCombo:

C #: Evento Click de btnAgregarCombo

```
      private void btnAgregarCombo_Click(object sender, EventArgs e)
      {
          cboLista.Enabled = true;
          cboLista.Items.Add(txtAgregar.Text);
          txtAgregar.Text = "";
      }
```

Evento SelectedIndexChanged() del combo Lista:

C #: Evento SelectedIndexChanged de cboLista

```
private void cboLista_SelectedIndexChanged(object sender, EventArgs e)
      {
          if (cboLista.SelectedIndex >= 0)
          {
              btnAgregarCaja.Enabled = true;
          }else
          {
              btnAgregarCaja.Enabled = false;
          }
      }
```

Evento Click del boton AgregarCaja:

C #: Evento Click de btnAgregarCaja

```
private void btnAgregarCaja_Click(object sender, EventArgs e)
{
    txtMostrar.Text = cboLista.SelectedItem.ToString();
}
```

17. Agregamos un formulario nuevo, y de nombre asignamos **frmEjemplo3**.

18. Agregamos un botón en el formulario **frmEjemplo2**, asignamos el nombre **btnProxima**, configuramos el mismo para que muestre al siguiente formulario.

C #: Evento Click de btnProxima

```
private void btnProxima_Click(object sender, EventArgs e)
{
    frmEjemplo3 frm = new frmEjemplo3();
    frm.Show();  // el método "Show" permite visualizar el formulario
}
```

19. En frmEjemplo3 agregaremos los siguientes controles:
    - a. 6 controles **Label**, en las propiedades cambiaremos en cada uno de ellos:
        - i. NAME: **lblFecha**.
        - ii. NAME: **lblDia**. TEXT: "DÍA:"
        - iii. NAME: **lblMes**. TEXT: "MES:"
        - iv. NAME: **lblAño**. TEXT: "AÑO:"
        - v. NAME: **lblInicio**. TEXT: "INICIO:"
        - vi. NAME: **lblFin**. TEXT: "FIN:"
    - b. 2 control **DateTimePicker**, dejaremos los nombres por defecto para este ejemplo.
        - i. DateTimePicker1
            1. Format: Short

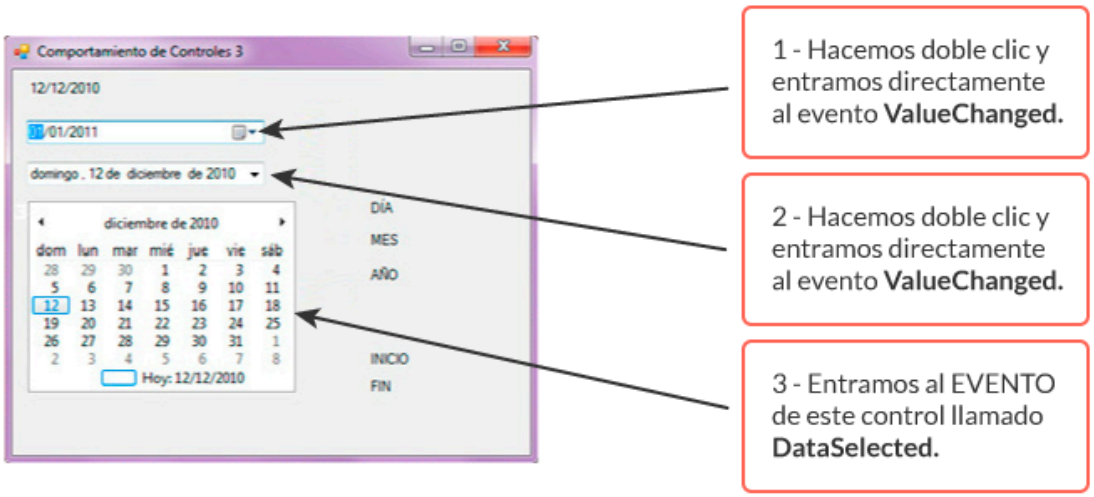
2.MaxDate: ´31/12/2021´

3.MinDate: ´01/01/2020´

ii.DateTimePicker2

1.Format: Long

- c. 1 control **MonthCalendar**. Dejamos el nombre por defecto.



Evento Load del formulario:

C #: Evento Load de frmEjemplo3

```
private void frmEjemplo3_Load(object sender, EventArgs e)
{
 // La clase DateTime provee métodos para trabajar con fechas
    lblFecha.Text = DateTime.Today.ToString();
}
```

Evento ValueChanged del primer y segundo DateTimePicker:

C #: Eventos ValueChnaged de los DateTimePicker

```
private void dateTimePicker1_ValueChanged(object sender, EventArgs e)
{
    lblDia.Text = "DIA: " + dateTimePicker1.Value.Day.ToString();
    lblMes.Text = "MES: " + dateTimePicker1.Value.Month.ToString();
    lblAnio.Text = "AÑO: " + dateTimePicker1.Value.Year.ToString();
}
private void dateTimePicker2_ValueChanged(object sender, EventArgs e)
{
    lblDia.Text = "DIA: " + dateTimePicker1.Value.Day.ToString();
    lblMes.Text = "MES: " + dateTimePicker1.Value.Month.ToString();
    lblAnio.Text = "AÑO: " + dateTimePicker1.Value.Year.ToString();
}
```

Evento DateChanged() del control monthCalendar1:

C #: Evento DateChanged de monthCalendar1

```
private void monthCalendar1_DateChanged(object sender, DateRangeEventArgs e)
{
    lblDia.Text = "DIA: " + e.Start.Day.ToString();
    lblMes.Text = "MES: " + e.Start.Month.ToString();
    lblAnio.Text = "AÑO: " + e.Start.Year.ToString();
    lblInicio.Text = "INICIO: " + e.Start.ToString();
    lblFin.Text = "FIN: " + e.End.ToString();
}
```

Con esto terminaríamos el proyecto de Controles Comunes.



Link del video: https://www.youtube.com/watch?v=2rqsCM457kw

Texto del video:

En esta situación profesional comenzamos a profundizar en los componentes de la interfaz gráfica, usando sus propiedades y métodos.

También trabajamos con los principales elementos del lenguaje C#, aprendimos el uso de constantes y variables, sus tipos de dato, operadores y las estructuras de control alternativas y repetitivas.

Conociendo todos estos elementos podremos desarrollar aplicaciones más elaboradas y complejas.

Indique verdadero o falso

1) La estructura de control "If" permite repetir un bloque de instrucciones.

- ○ Verdadero
- ○ Falso

2) La estructura de control "switch" puede evaluar una expresión con cualquier tipo de dato.

- ○ Verdadero
- ○ Falso

3) En la estructura de control "If else" el bloque de instrucciones de la sentencia "else" se ejecuta si la expresión analizada es falsa.

- ○ Verdadero
- ○ Falso

4) La estructura de control "while" primero ejecuta el bloque de instrucciones y luego analiza la condición.

- ○ Verdadero
- ○ Falso

5) La estructura de control "for" repite la ejecución del bloque de instrucciones hasta que la condición sea falsa.

- ○ Verdadero
- ○ Falso

Respuestas correctas[34]

[34] 1)Falso; 2)Falso; 3)Verdadero; 4)Falso; 5)Verdadero.

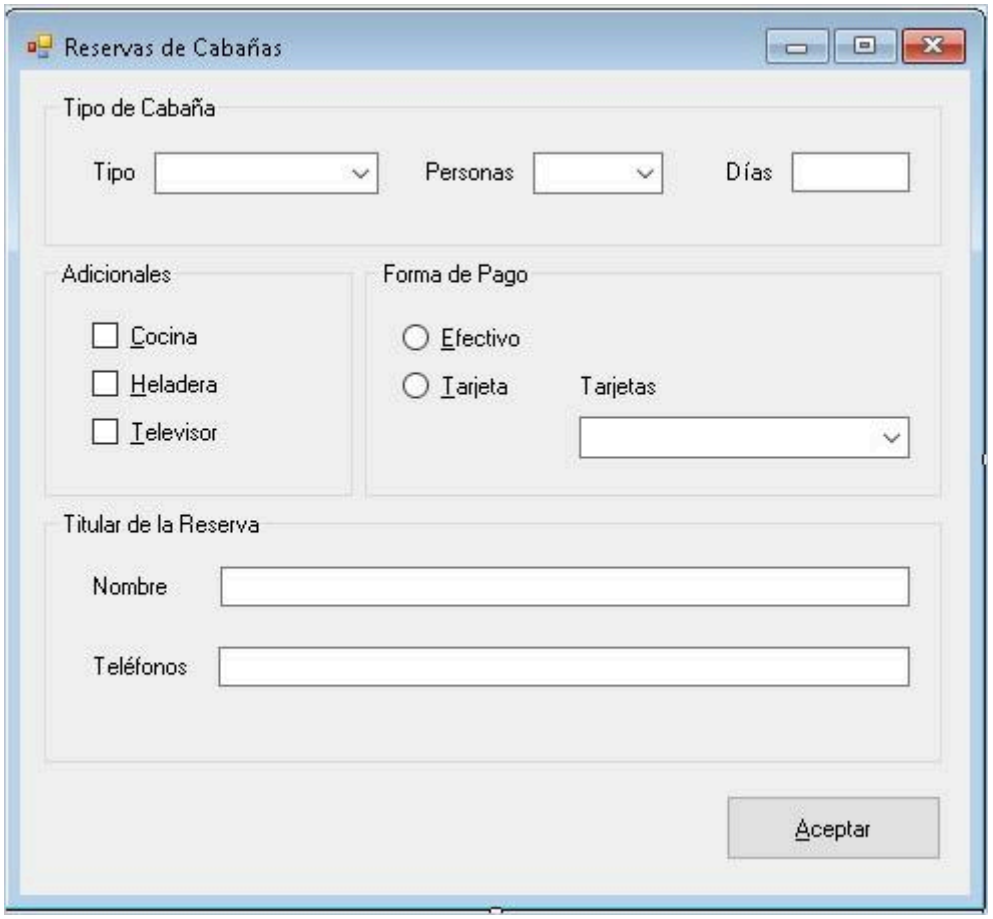
SP2/Ejercicio resuelto

Con todos los conocimientos adquiridos en el manejo de Visual Studio, y el estándar de generación de proyectos, podrá llevar a cabo la tarea requerida para resolver la situación profesional.

Pasos a seguir para completar la resolución de la situación planteada.

Crear el proyecto 'SP2' en Visual Studio, seleccionado el lenguaje C#, plataforma Windows y tipo de proyecto de Escritorio, usando el Framework de .Net

Realizar el diseño de la interfaz gráfica cómo se muestra a continuación:



'Tipo de Cabaña' es un groupBox que contiene los controles comboBox 'cmbTipo' y 'cmbPersonas' más el textBox 'txtDias'

'Adicionales' es otro groupBox con tres controles checkBox: 'chkCocina', 'chkHeladera' y 'chkTelevisor'

'Forma de Pago' es el tercer groupBox y contiene dos conrtroles radioButton: 'optEfectivo' y 'optTarjeta' y un control comboBox: 'cmbTarjeta'

El cuarto groupBox, 'Titular de la Reserva' contiene dos controles textBox: 'xtNombre' y 'txtTelefonos'

Finalmente se ubica un botón de comando: 'btnAceptar'.

El título del formulario será 'Reservas de Cabañas'

Una vez colocados todos los controles en la interfaz y ajustados sus tamaños y ubicaciones se asignarán los nombres indicados en el párrafo anterior.

Comenzaremos escribiendo las declaraciones de las constantes a usar para los cálculos del programa, esto se hará en la sección de declaraciones, al comienzo de la clase del formulario:

C #: Declaración de Constantes SP2

```
public partial class Form1 : Form
   {
       // constantes para los cálculos
       private const float TIPOA = 20;
       public const float TIPOB = 34;
       const float COCINA = 1;
       const float HELADERA = 1.5f;
       const float TELEVISOR = 2;
       const float PORPERSONA = 1;
```

Los valores de las constantes son los declarados en el enunciado de la situación profesional, los declaramos de tipo float para facilitar posibles cambios en sus valores y mantener los cálculos realizados sin cambios.

Seguidamente programaremos el evento 'Load' del formulario para establecer el estado inicial del mismo

C #: Evento Load SP2

```
private void Form1_Load(object sender, EventArgs e)
{
    // cargar los items en el primer control combobox
    cmbTipo.Items.Clear();
    cmbTipo.Items.Add("Tipo A");
    cmbTipo.Items.Add("Tipo B");
 // esta acción provoca el disparo del evento "SelectedIndexChanged"
    cmbTipo.SelectedIndex = 0;
    // se inicializa la cantidad de días en 1
    txtDias.Text = "1";
    // inicialzar los demás controles de la interfaz
    chkCocina.Checked = false;
    chkHeladera.Checked = false;
    chkTelevisor.Checked = false;
    // en los radiobuttons se asigna sólo el que debe quedar en true
    optEfectivo.Checked = true;
    txtNombre.Text = "";
    txtTelefonos.Text = "";
    // cargar los items del combo de tarjetas
    cmbTarjeta.Items.Clear();
    cmbTarjeta.Items.Add("Card Red");
    cmbTarjeta.Items.Add("Card Green");
    cmbTarjeta.Items.Add("Card Blue");
    // deshabiliar el botón "Aceptar"
    btnAceptar.Enabled = false;
}
```

Recordemos que el evento "Load" es el primero que se ejecuta antes de que el usuario pueda visualizar el formulario y es dónde normalmente podemos programar todas las acciones que establezcan el estado inicial de los componentes de la interfaz gráfica.

A continuación programamos el evento SelectedIndexChanged del comboBox 'cmbTipo' para que actualice la cantidad de personas según el tipo de cabaña que se seleccione:

C #: Evento SelectedIndexChanged cmbTipo en SP2

```
private void cmbTipo_SelectedIndexChanged(object sender, EventArgs e)
  {
      // cuando ocurra el evento se debe actualizar el
      // contenido del combo de personas
      int I = 0;
```

```
        // limpiar los items de personas
        cmbPersonas.Items.Clear();
        // si es tipo de cabaña A ->> cargar los items del 1 al 4
        if (cmbTipo.SelectedIndex == 0) // o cmbTipo.SelectedItem == "Tipo A"
        {
            for (I = 1; I <= 4; I++)
            {
                cmbPersonas.Items.Add(I);
            }
        }
        else { // si es tipo de cabaña B ->> cargar los items del 1 al 8
            for (I = 1; I <= 8; I++)
            {
                cmbPersonas.Items.Add(I);
            }
        }
        // establecer como preseleccionado el item 0 del combo
        cmbPersonas.SelectedIndex = 0;
    }
```

Nos interesa que el combobox que muestra la cantidad de personas se actualice cada vez que se modifique el tipo de cabaña seleccionado en el primer combobox, por eso debemos programar este evento cuyo código se encarga de mantener la cantidad de personas correcta para cada tipo de cabaña.

Continuamos con la programación de los eventos checkedChanged de los radioButtons que controlan la forma de pago, 'optEfectivo' y 'optTarjeta':

C #: Eventos CheckedChanged SP2

```
    private void optEfectivo_CheckedChanged(object sender, EventArgs e)
    {
        // se deshabilita el combo y no se muestra nada
        cmbTarjeta.Enabled = false;
        cmbTarjeta.SelectedIndex = -1;
    }
    private void optTarjeta_CheckedChanged(object sender, EventArgs e)
    {
        // se habilita el combo y se muestra el primer item
        cmbTarjeta.Enabled = true;
        cmbTarjeta.SelectedIndex = 0;
    }
```

A continuación hay que programar los eventos TextChanged de los controles textBox: 'txtDias', 'txtNombre' y 'txtTelefonos' para controlar si el botón 'Aceptar' debe habilitarse o no. Es importante destacar que es necesario programar el mismo código en los tres eventos para que la aplicación se comporte adecuadamente sin importar la secuencia de uso que haga el usuario de estos controles. Esto es así porque la condición para que el botón "Aceptar" se habilite está dada por los tres componentes textBox.

C #: Evento textChanged de txtDias SP2

```
private void txtDias_TextChanged(object sender, EventArgs e)
   {
       // condiciones para habilitar o no el botón "Aceptar"
       if (txtDias.Text != "" && txtDias.Text != "0" &&
          txtNombre.Text != "" && txtTelefonos.Text != "")
       {
           btnAceptar.Enabled = true;
       } else
       {
           btnAceptar.Enabled = false;
       }
   }
```

C #: Evento txtChanged txtNombre SP2

```
private void txtNombre_TextChanged(object sender, EventArgs e)
{
    // condiciones para habilitar o no el botón "Aceptar"
    if (txtDias.Text != "" && txtDias.Text != "0" &&
        txtNombre.Text != "" && txtTelefonos.Text != "")
    {
        btnAceptar.Enabled = true;
    }
    else
    {
        btnAceptar.Enabled = false;
    }
}
```

C #: Evento textChanged de txtTelefonos SP2

```
private void txtTelefonos_TextChanged(object sender, EventArgs e)
{
    // condiciones para habilitar o no el botón "Aceptar"
    if (txtDias.Text != "" && txtDias.Text != "0" &&
        txtNombre.Text != "" && txtTelefonos.Text != "")
    {
        btnAceptar.Enabled = true;
    }
    else
    {
        btnAceptar.Enabled = false;
    }
}
```

Finalmente programamos el evento Click del botón Aceptar:

En la primera parte de este evento declaramos las variables necesarias para los cálculos a realizar, obtenemos la cantidad de días a procesar y determinamos el precio base de acuerdo al tipo de cabaña seleccionado:

C #: Evento Click de btnAceptar parte1 SP2

```
private void btnAceptar_Click(object sender, EventArgs e)
{
    float PrecioBase;
    float Opcionales;
    float Recargo;
    int Dias;
    float Total;
    // obtener la cantidad de dìas ingresados
    Dias = int.Parse(txtDias.Text);
    // controlar el tipo de cabaña para determinar el precio base
    if (cmbTipo.SelectedIndex == 0)
    {
        PrecioBase = TIPOA;
    } else
    {
        PrecioBase = TIPOB;
    }
```

Seguidamente, y continuando en el mismo evento, controlamos los opcionales que se pueden haber seleccionado y calculamos el total teniendo en cuenta la cantidad de dias de permanencia ingresados:

C #: Evento Click de btnAceptar parte 2 SP2

```
// sumar al precio base el importe extra por persona (US$ 1)
PrecioBase = PrecioBase + (PORPERSONA * int.Parse(cmbPersonas.Text));
// controlar los adicionales por las opciones
Opcionales = 0;
if (chkCocina.Checked == true)
{
    Opcionales = Opcionales + COCINA;
}
if (chkHeladera.Checked == true)
{
    Opcionales = Opcionales + HELADERA;
}
if (chkTelevisor.Checked == true)
{
    Opcionales = Opcionales + TELEVISOR;
}
// determinar el total por la cantidad de dìas
Total = (PrecioBase + Opcionales) * Dias;
```

Finalmente controlamos la forma de pago seleccionada y aplicamos el recargo correspondiente si se seleccionó el pago con Tarjeta, se muestra el resultado final de la reserva y se restaura la interfaz al estado inicial:

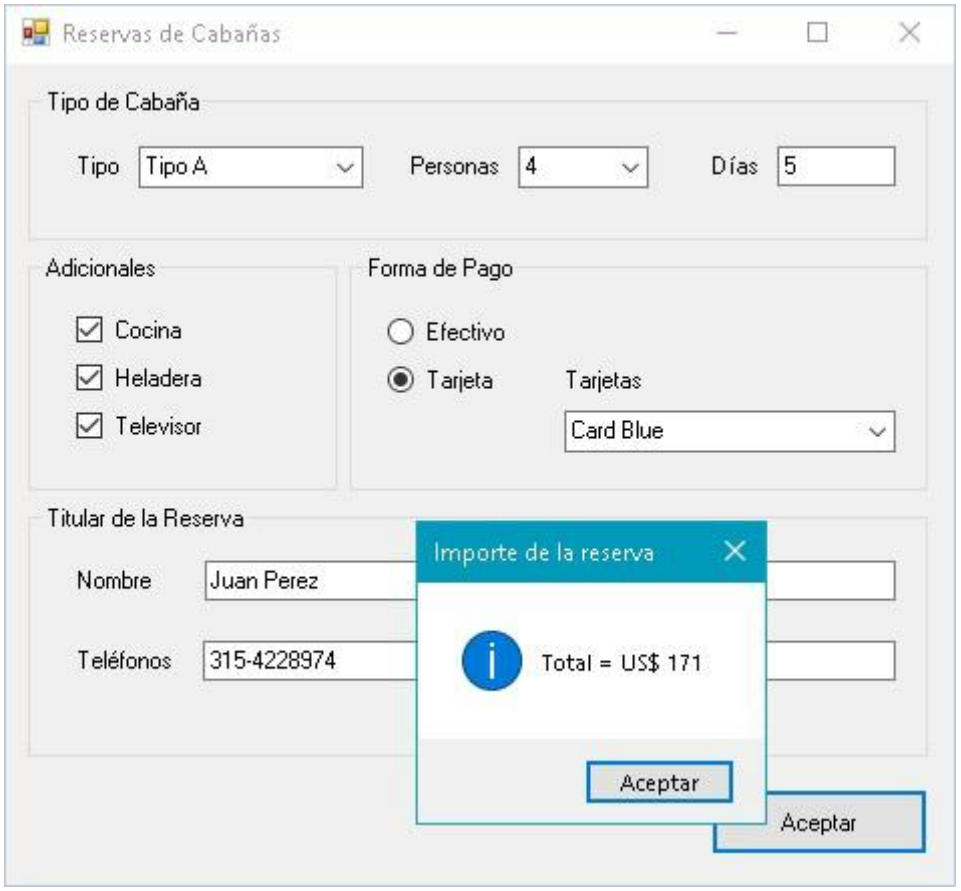
C #: Evento Click de btnAceptar parte3 SP2

```
// controlar la forma de pago
if (optTarjeta.Checked == true)
{
    if (cmbTarjeta.SelectedIndex == 0)
    {
        Recargo = Total * 10 / 100;
    } else {
        Recargo = Total * 20 / 100;
    }
    Total = Total + Recargo;
}
// mostrar el resultado
MessageBox.Show("Total = " + Total.ToString(), "Importe de la reserva",
```

```
            MessageBoxButtons.OK, MessageBoxIcon.Information);
        // inicialzar los controles de la interfaz
        cmbTipo.SelectedIndex = 0;
        txtDias.Text = "1";
        chkCocina.Checked = false;
        chkHeladera.Checked = false;
        chkTelevisor.Checked = false;
        // en los radiobuttons se asigna sólo el que debe quedar en true
        optEfectivo.Checked = true;
        txtNombre.Text = "";
        txtTelefonos.Text = "";
    }
```

Con eso se completa el ejercicio resuelto de la situación profesional 2.

Se muestra un ejemplo del resultado obtenido con cabaña tipo 'A', con 4 personas y 5 días de estadía, seleccionado todos los adicionales y pago con tarjeta Blue:



SP2/Ejercicio por resolver

Le solicitan en la organización Ferro que confeccione una aplicación para calcular los precios de los boletos de ferrocarril dependiendo de la distancia recorrida. El programa debe permitir al usuario ingresar la distancia hasta la localidad de destino y seleccionar la cantidad de días de estancia entre 1 y 30 días. El programa determina el precio de un boleto de ida y vuelta, calculando la distancia total a recorrer y sabiendo que, si el número de días de estancia es mayor o igual a 7 y la distancia mayor o igual a 100 kilómetros, el boleto tiene una reducción del 50%. El precio por kilómetro es de 5 pesos.

El resultado del cálculo se debe mostrar por pantalla en un cuadro de mensaje.

Resumen:

- Precio por kilómetro: 5 pesos
- Descuento: 50%
- Condición de descuento: distancia mayor o igual a 100 km. y estancia mayor o igual a 7 días.
- Días de estancia entre 1 y 30.

Debe confeccionar la interfaz gráfica y el código necesario para que la aplicación funcione.

Indique verdadero o falso

1) El Formulario es el soporte de objetos, podemos ubicar TextBox, Label, ComboBox, y otros controles comunes.

- ○ Verdadero
- ○ Falso

2) La estructura de control "switch" se puede construir con una expresión de tipo float.

- ○ Verdadero
- ○ Falso

3) Si en una estructura de control "If" se analizan dos expresiones conectadas por el operador lógico "OR" (||) y una sola de las expresiones es falsa, se ejecuta el cuerpo de la estructura.

- ○ Verdadero
- ○ Falso

4) Con la propiedad WindowsState podemos hacer que un formulario aparezca como minimizado si seleccionamos la opción minimizebox.

- ○ Verdadero
- ○ Falso

5) Con la estructura for podemos ejecutar un bloque de sentencias repetidas veces hasta que su condición sea verdadera.

- ○ Verdadero
- ○ Falso

Respuestas correctas[35]

[35] 1)Verdadero; 2)Falso; 3)Verdadero; 4)Verdadero; 5)Falso.

Situación profesional 3: Auto Test

La empresa *Auto Test* dedicada a la revisión técnica vehicular le solicita a usted en su carácter de pasante en el área informática el desarrollo de una aplicación para registrar y consultar los turnos que se otorgan a sus clientes. La aplicación deberá tener interfaz gráfica para la carga de datos y para su consulta.

Para la carga de datos se requiere que el usuario ingrese la información del vehículo a revisar que consiste en estos datos:

- **Dominio**: tipo alfanumérico, de 6 caracteres mínimo y 7 caracteres máximo.
- **Año de fabricación**: tipo numérico, entero.
- **Titular**: tipo alfanumérico, de 30 caracteres.

También deberá ingresar los datos del turno:

- **Número de turno**: tipo numérico, entero. 5 dígitos como máximo.

La aplicación debe permitir registrar hasta 50 turnos por vez.

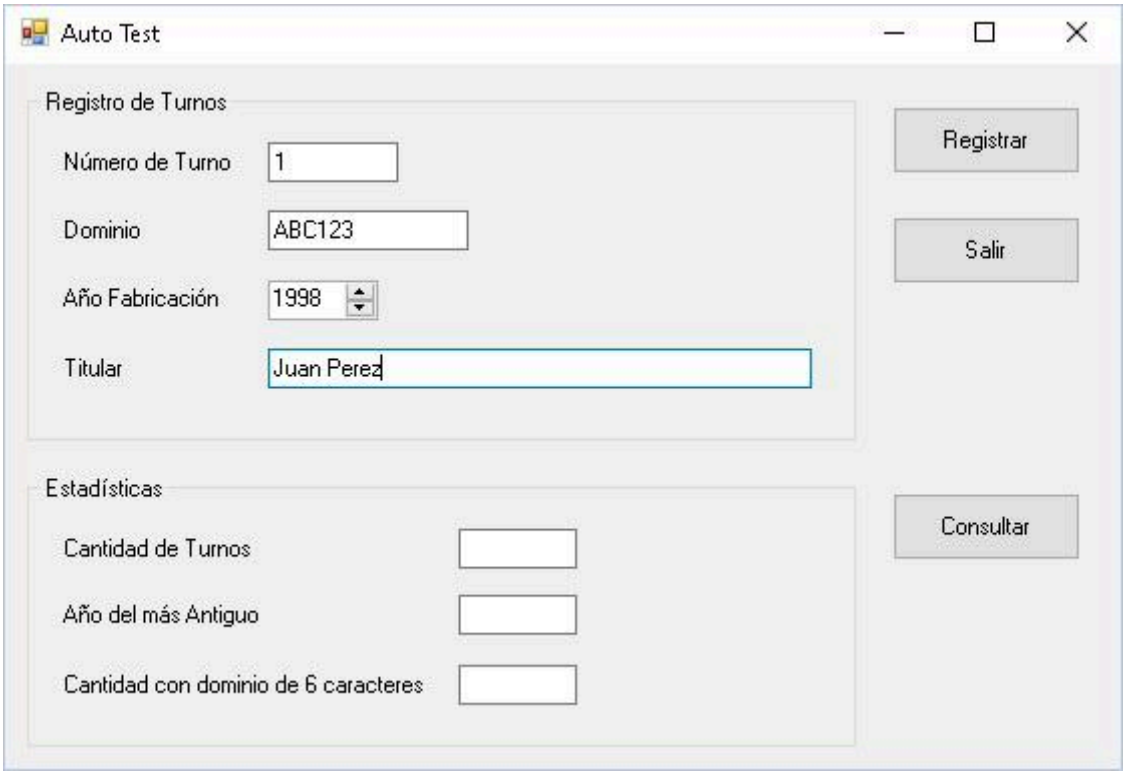
El registro de cada turno debe contar con validaciones que aseguren que los datos ingresados son correctos:

- El Dominio debe tener 6 o 7 caracteres, pudiendo ingresar solamente letras mayúsculas y los números del 0 al 9.
- El Año de fabricación debe estar comprendido entre 1950 y el año actual.
- Titular debe tener al menos dos caracteres.
- El Número de turno debe ser un número entero y no podrá repetirse. No es necesario que sean valores consecutivos.

Consulta de Turnos: debe mostrar datos estadísticos de la información ingresada:

- La cantidad total de turnos ingresados.
- El año de fabricación del vehículo más antiguo registrado.
- La cantidad de vehículos registrados con dominio de 6 caracteres.

A continuación, se muestra un modelo de la interfaz propuesta para la aplicación:


Auto Test
Registro de Turnos
Número de Turno
1
Dominio
ABC123
Año Fabricación
1998
Titular
Juan Perez
Registrar
Salir
Estadísticas
Cantidad de Turnos
Año del más Antiguo
Cantidad con dominio de 6 caracteres
Consultar

SP3/H1: Tipos de datos definidos por el usuario, tipos 'struct'

'Struct'

Hasta ahora hemos trabajado con los tipos base que posee el lenguaje C#, esto es **char, int, float, double**, etc. con los cuales podemos crear variables que nos permiten almacenar los datos que manejan nuestros problemas, sin embargo, aparecen situaciones en las cuales sería más eficiente contar con conjuntos de datos que respondan a necesidades específicas de cada problema a resolver.

Nos referimos a ciertos datos que guardan alguna relación lógica entre sí, por ejemplo, cuando necesitamos almacenar el nombre y el documento de una persona, normalmente usaríamos dos variables una para el nombre y otra para el documento, pero como ambos valores se refieren a la misma persona sería mejor contar con una única variable que almacene los dos valores al mismo tiempo.

Esto último es posible con los tipos 'struct' que C# nos permite definir. Se denominan estructuras y consisten en una declaración que el programador puede realizar para agrupar bajo un único nombre dos o más elementos que guardarán una relación lógica entre ellos.

Una estructura es, entonces, un nuevo tipo de dato declarado por el programador que se identificará con un nombre y contendrá en su declaración 'campos' o 'miembros' , cada uno de ellos con su respectivo nombre y tipo de dato.

Declaración de estructuras:

```
[private public] struct NOMBRE_ESTRUCTURA
{
public tipo_datos nombre_campo_1;
public tipo_datos nombre_campo_2;
...
public tipo_datos nombre_campo_N;
};
```

'**private**' y '**public**' determinan el ámbito de uso de la declaración, con 'private' la estructura se podrá usar solo donde fue declarada, con 'public' podrá emplearse en todos los demás componentes de la aplicación.

'**struct**' es la palabra reservada del lenguaje que nos permite hacer la declaración del nuevo tipo.

'**NOMBRE_ESTRUCTURA**' es el identificador asignado a la estructura que estamos creando.

Los c**ampos o miembros** de la estructura van en el cuerpo de la estructura, delimitados por las llaves { }, y cada campo debe especificar el tipo de dato y su nombre, se especifica 'public' para que los campos sean accesibles cuando se trabaje con las variables de la estructura.

Por ejemplo, y volviendo al ejemplo inicial de los datos de una persona, podemos declarar esta estructura:

```
public struct PERSONA

{

public string Nombre;

public long Documento;

};
```

Remarcamos que lo anterior es una declaración de tipo, no estamos creando ninguna variable para almacenar datos, la declaración le informa al compilador que a partir de ahora existe un nuevo tipo de dato llamado 'PERSONA' con el que sí podremos crear variables.

Declaración de variables tipo estructura

Una vez declarada la estructura ya podemos crear variables que se ajusten al nuevo tipo de dato, se sigue la regla general que aplicamos con los demás tipos de datos ya usados:

```
tipo_dato Nombre_Variable;
```

Entonces para crear una variable de la estructura de PERSONA escribimos:

```
PERSONA juan;
```

'juan' es una variable de tipo 'PERSONA' y por lo tanto puede almacenar un nombre y un documento en sus respectivos campos.

Acceso a miembros de la estructura

Para especificar a qué miembro o campo se quiere hacer referencia usamos el operador punto **(.)** de la siguiente forma

```
Nombre_Variable.Nombre_Campo = valor;
```

Para asignar valores a los campos de la variable 'juan' sería:

```
juan.Nombre = "Juan";

juan.Documento = 32123987;
```

Restricciones en la declaración de las estructuras

No está permitido asignar valores a los campos durante la declaración de la estructura, por ejemplo, esto genera un error de compilación:

```
struct PERSONA

{

public string Nombre = "";

public long Documento = 0;

};
```

Los valores se deben asignar sobre la variable que vamos a crear a partir de la declaración de la estructura.

Anidamiento de estructuras

Al declarar una estructura es posible que uno de sus campos sea a su vez una estructura, por ejemplo, **declaramos una estructura para poder almacenar datos de domicilios:**

```
struct DOMICILIO
{
public string Calle;
public int Numero;
public string CodigoPostal;
};
```

Declaramos una estructura para Clientes que contenga la de domicilio:

```
struct CLIENTE
{
public string RazonSocial;
public long Cuit;
public DOMICILIO direccion;
};
```

El campo 'direccion' es de tipo estructura 'DOMICILIO' y para accederlo en una variable de la estructura 'CLIENTE' continuamos aplicando el operador punto:

```
CLIENTE miCliente;
miCliente.RazonSocial = "Auto Test";
miCliente.Cuit = 302158964;
// ahora asignamos los 3 campos de la dirección:
```

```
miCliente.direccion.Calle = "Av Colon";

miCliente.direccion.Numero = 3150;

miCliente.direccion.CodigoPostal = "X5000";
```

El concepto de anidamiento puede repetirse en varios niveles, aunque no se recomienda emplear más de 3 o 4 niveles porque eleva la complejidad en el tratamiento de los datos.

Copiando variables de estructuras

Como las variables creadas con las estructuras son de tipo valor pueden ser usadas para asignar de forma completa o tomando sus campos en forma individual, por ejemplo, si tenemos dos variables de tipo PERSONA y queremos copiar los valores de una variable a la otra podemos operar así:

```
// creamos las dos variables

PERSONA persona1, persona2;

// asignamos valores a la primera variable

persona1.Nombre "Luis";

persona1.Documento = 31987123;

// ahora copiamos los datos de persona1 a persona2

persona2 = persona1; // esta es una copia completa
```

También se pueden copiar los campos de forma individual y obtenemos el mismo resultado:

```
persona2.Nombre = persona1.Nombre;

persona2.Documento = persona1.Documento;
```

Normalmente la primera forma es mucho más conveniente ya que requiere escribir menos código.

Si recordamos el enunciado de la situación profesional, vemos que hay datos que pueden asociarse para crear una estructura:

Para la carga de datos se requiere que el usuario ingrese la información del vehículo a revisar que consiste en estos datos:

- **Dominio: tipo alfanumérico, de 6 caracteres mínimo y 7 caracteres máximo.**
- **Año de fabricación: tipo numérico, entero**
- **Titular: tipo alfanumérico, de 30 caracteres.**

También deberá ingresar los datos del turno:

- **Número de turno: tipo numérico, entero.**

Como estamos hablando de datos de turnos, entonces podemos declarar una estructura 'turno' que contenga estos campos

```
public struct TURNO

{

public int NumeroTurno;

public string Dominio;

public int AnioFabricacion;

public string Titular;

};
```

Con esta estructura podemos crear las variables necesarias para resolver el almacenamiento de los datos de cada turno solicitado en la actual situación profesional.

SP3/Autoevaluación 1

Indique la opción correcta

1) La palabra reservada para declarar una estructura en C# es:

- ○ 'structure'
- ○ 'struct'
- ○ 'estructura'

2) Los elementos que componen la estructura son:

- ○ Siempre son todos del mismo tipo de dato
- ○ Pueden ser de distintos tipos de dato
- ○ Deben ser siempre de distintos tipos de dato

3) ¿Cuándo se presentan estructuras anidadas?

- ○ Cuando la declaración de una estructura contiene 3 o más miembros
- ○ Cuando la declaración de una estructura presenta todos los miembros del mismo tipo de dato
- ○ Cuando la declaración de una estructura contiene un campo que también es de tipo estructura
- ○ Cuando la creación de una variable de estructura es asignada a otra variable de estructura

Indique verdadero o falso

4) La declaración de una estructura crea un nuevo tipo de dato.

- ○ Verdadero
- ○ Falso

5) Al declarar una nueva estructura es posible asignar valores iniciales a sus campos miembro.

- ○ Verdadero
- ○ Falso

Respuestas correctas[36]

[36] 1)struct; 2)Pueden ser de distintos tipos de dato; 3)Cuando la declaración de una estructura contiene un campo que también es de tipo estructura; 4)Verdadero; 5)Falso.

SP3/H2: Arreglos

En general llamamos arreglos a un conjunto de elementos que ocupan posiciones de memoria consecutivas y son del mismo tipo de dato. Este conjunto se identifica con un único nombre y sus elementos se referencian con un número entero que hace las veces de índice para establecer la posición del elemento.

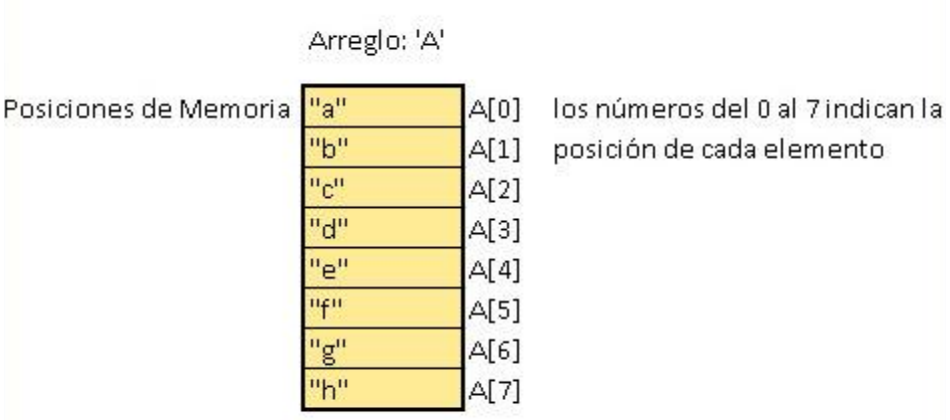
Podemos crear arreglos de cualquier tipo de dato, pueden ser de un tipo base, como char, int o float, por ejemplo, o tipos complejos como los struct vistos recientemente.

En distintos lenguajes de programación se aplica la denominación de **Arrays** para referirse a los arreglos.

Como podremos ver en la materia Programación Lógica, la teoría del uso de **estructuras de datos simple,** definidos como arrays (*vectores y matrices*), cuentan con características importantes que podemos considerar:

- Su almacenamiento es en la memoria del sistema, por tanto, la información es volátil.
- La estructura tiene dimensión (tamaño) establecida por el programador/desarrollador.
- Sus datos almacenados son homogéneos (mismo tipo de dato).
- Para recorrer la información de la estructura necesitamos algún indicador (índice).

Representación gráfica de los arreglos en memoria



Arreglos en C#

El lenguaje C# trata los arreglos como "tipos por referencia", a diferencia de los tipos primitivos, como 'int, 'float', que son tratados por valor. El concepto de "tipos por referencia" está asociado a las direcciones de memoria que ocupan los elementos referenciados, es decir que se conoce la dirección de memoria asignada en lugar de conocer el valor del elemento. Esto ocurre con todos objetos creados a partir de una determinada clase y los arreglos son tratados en ese sentido como si fueran objetos.

Debido a este tratamiento por referencia que tienen los arreglos en C# vamos a distinguir dos momentos en el uso de arreglos:

- **Declaración del arreglo.**
- **Creación del arreglo o nueva instancia del arreglo**

Declaración del arreglo

La **declaración** del arreglo le informa al compilador el tipo y el nombre del arreglo, con esta sintaxis:

```
tipo [] Nombre_Arreglo;
```

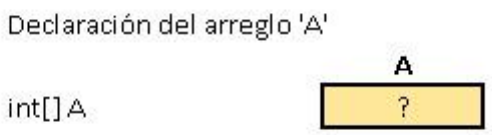
Donde 'tipo' es el tipo de dato que vamos a emplear, los corchetes vacíos '[]' indican que se trata de la declaración de un arreglo y no de una variable común y 'Nombre_Arreglo' es el identificador asociado al arreglo.

Esta declaración reserva espacio para alojar una dirección de memoria y no los datos que queremos guardar en el arreglo.

Ejemplo:

```
int[] Numeros; // se declara un arreglo de tipo entero con el nombre 'Numeros'
```

Después de la declaración tenemos un lugar de la memoria asignado al arreglo donde podemos guardar la dirección de memoria donde posteriormente se encontrarán los datos del arreglo. Pero aún no tenemos disponible el espacio para esos datos, por lo tanto, ese arreglo no se podrá usar hasta no completar la creación del mismo.



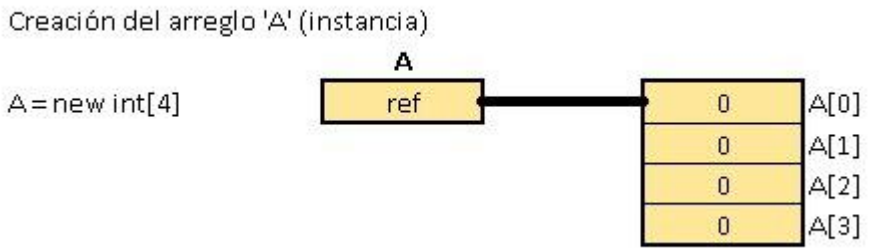
Creación del arreglo

La **creación** del arreglo es el segundo paso y sin él no será posible usar el arreglo, esta creación consiste en construir una instancia del arreglo, tal como ocurre con cualquier otro objeto en C#, la instancia se genera aplicando el operador 'new' de la siguiente forma:

```
arreglo = new tipo[cantidad de elementos];
```

Por ejemplo:

```
Numeros = new int[4];
```



La creación del arreglo reserva el espacio de memoria necesario para todos sus elementos y ese espacio de memoria queda referenciado por la variable declarada inicialmente, técnicamente la variable 'A' no almacena los datos del arreglo, sólo guarda una referencia al espacio de memoria que realmente contiene los datos de cada elemento.

El operador 'new' es el encargado de construir el objeto, en este caso el arreglo. La instancia del arreglo creada debe llevar la cantidad de elementos que contendrá el arreglo, eso es lo que finalmente permite al compilador reservar y asignar la cantidad de memoria suficiente para todos los elementos del arreglo, recordemos que debe ser un único bloque de memoria, con todas sus posiciones consecutivas.

Una vez creada la instancia del arreglo ya estamos en condiciones de usar el arreglo para asignar y leer valores en sus elementos, para ello debemos usar un número entero que hace las veces de índice para indicar la posición del arreglo que queremos acceder, por ejemplo:

```
Numeros[0] = 10; // asignamos el valor 10 al elemento del arreglo 'Numeros' que ocupa la posición cero.
```

Complementando lo visto sobre el uso de Arreglos, comentamos a título informativo que C# posee también una serie de clases que representan colecciones de objetos con un manejo especializado de sus elementos, en estas colecciones los elementos se manejan como objetos lo que los hace más genéricos y nos dan la posibilidad de adaptarlos a muchas situaciones de uso. A continuación se enumeran las clases que forman estas colecciones especializadas.

- **ArrayList**: permite entre otras cosas, eliminar, agregar e insertar elementos en el arreglo de manera más simple.
- **Queue**: implementa el mecanismo First-In, First-Out (FIFO[37]) que controla el ingreso y egreso de elementos del arreglo.
- **Stack**: implementa el mecanismo Last-In, First-Out (LIFO[38]) que controla el ingreso y egreso de elementos del arreglo.
- **Hashtable**: provee el mecanismo para mapear un índice entero con un valor lo que permite crear un diccionario de valores.
- **SortedList**: similar al Hashtable con la diferencia que sus elementos se mantienen siempre ordenados.

Se recomienda revisar la documentación oficial de *Microsoft* sobre estas colecciones de C# definidas en el espacio de nombres '*System.Collections*' y sus subespacios de nombres.

[37] FIFO: First In, First Out, su traducción en castellano significa primero en entrar, primero en salir.
[38] LIFO: Last In, First Out, su traducción al castellano sería último en entrar, primero en salir.

Indique verdadero o falso

1) Un Arreglo es un conjunto de elementos, del mismo tipo o de distintos tipos de dato, con un único nombre.

- ○ Verdadero
- ○ Falso

2) Los arreglos se pueden crear con los tipos base o también con los tipos struct.

- ○ Verdadero
- ○ Falso

3) Declarar y Crear un arreglo son acciones equivalentes.

- ○ Verdadero
- ○ Falso

Indique la opción correcta

4) Para acceder a los elementos de un arreglo se usa como índice:

- ○ Un valor alfanumérico
- ○ Un valor numérico entero
- ○ Un valor numérico de cualquier tipo
- ○ Cualquiera de las anteriores

5) El operador 'new' se usa para:

- ○ Declarar un arreglo
- ○ Crear un arreglo
- ○ Asignar valores a los elementos de un arreglo

Respuestas correctas[39]

[39] 1)Falso; 2)Verdadero; 3)Falso; 4)Un valor numérico entero; 5)Crear un arreglo.

SP3/H3: Arreglos de una dimensión

Como vimos en la herramienta anterior, un arreglo es un conjunto de elementos del mismo tipo y consecutivos en la memoria. Aparte del tipo de dato que asignaremos al arreglo debemos especificar la cantidad de dimensiones que tendrá el arreglo comenzando por los arreglos de una dimensión.

Dimensión del Arreglo

El término dimensión se usa para indicar la cantidad de índices que posee el arreglo para acceder a sus elementos individuales, eso determina la forma en que se organiza el contenido del arreglo. El caso más simple corresponde a los arreglos de una dimensión, conocidos también como 'vectores', luego tenemos los arreglos bidimensionales (de 2 dimensiones) y finalmente los arreglos multidimensionales, de 3 o más dimensiones.

En este apartado trataremos con los arreglos unidimensionales o vectores.

Arreglos unidimensionales (Vectores)

Como lo expresamos más arriba, los arreglos unidimensionales son los que poseen un único índice para acceder a sus elementos individuales y en C# cumplen con las siguientes propiedades:

- Sus elementos son todos del mismo tipo de dato.
- Deben tener un identificador válido como nombre.
- Sus elementos individuales se podrán acceder con un índice de tipo numérico entero.
- Deben tener un tamaño definido (cantidad de elementos que forman el arreglo).

Las actividades que podemos realizar con el arreglo son:

- Declarar el arreglo, con su tipo y nombre.
- Crear el arreglo, es decir su instancia.
- Hacer uso del arreglo:

- Asignar valores a sus elementos individuales.
- Recorrer el arreglo.
- Buscar un elemento en el arreglo.
- Modificar un elemento del arreglo

A continuación veremos cada uno de estos puntos usando C#

Declaración y Creación del Arreglo

Como ya lo vimos anteriormente la declaración del arreglo consiste en especificar el tipo de dato del arreglo y su nombre, colocando después del tipo de dato corchetes vacíos:

```
tipo [] Nombre_Arreglo;
```

Recordemos que el tipo de dato puede ser alguno de los tipos base, como char, int, float, etc. o un tipo struct o también objetos en general.

Ejemplos:

```
char[] Letras; // arreglo de tipo 'char', nos permitirá almacenar un caracter en cada una de sus posiciones

int[] Numeros; // arreglo de tipo 'int'

float[] Precios; // arreglo de tipo 'float'

PERSONA[] Clientes; // arreglo de tipo PERSONA, donde 'PERSONA' es una estructura
```

También vimos que luego de la declaración es necesario **crear** el arreglo haciendo uso del operador '**new**' para obtener una instancia y el espacio de memoria necesario para sus elementos. Para la creación debemos especificar la cantidad de elementos que tendrá el arreglo:

```
arreglo = new tipo[cantidad de elementos];
```

'arreglo' es el nombre del arreglo que previamente ya hemos declarado

'tipo' es el mismo tipo de dato que usamos en su declaración

'cantidad de elementos' es un valor de tipo entero (o una variable entera) que especifica el tamaño del arreglo

Ejemplos:

```
Letras = new char[30]; // crea el arreglo 'Letras' y le asigna espacio para 30 elementos

Numeros = new int[10]; // crea el arreglo 'Numeros' y le asigna 10 elementos

Precios = new float[5]; // crea el arreglo 'Precios' y le asigna 5 elementos

Clientes = new PERSONA[50] // crea el arreglo 'Clientes' y le asigna 50 elementos
```

Una vez completada la creación del arreglo ya estamos en condiciones de poder utilizar el arreglo en nuestro programa.

Importante: una vez creado el arreglo, su cantidad de elementos no podrá ser modificada, es decir no podremos ni agregar ni quitar elementos al arreglo ya que al crear su instancia la cantidad de memoria asignada al arreglo queda definida por la cantidad de elementos.

Hacer uso del arreglo

Asignación de valores

La asignación de valores a cada posicíon del arreglo se puede realizar en el mismo momento de la creación o posteriormente en forma individual a cada elemento.

Asignación en la creación del arreglo:

```
Letras = new char[30] { 'a', 'b', 'c', 'd', 'e', 'f', ... };
// 'a' es asignado a la posición 0 del arreglo y el resto de
los caracteres ocupará posiciones consecutivas.

Numeros = new int[10] { 10, 20, 30, 40, 50, ... 100 }; // es
obligación asignar todas las posiciones del arreglo
```

Los valores entre { } pueden ser constantes, como en los ejemplos anteriores, o variables o resultados en tiempo de ejecución, por ejemplo:

```
float p = 1.25f; // se usa 'f' para indicar que se trata de un
valor float

Precios = new float[5] { p, p*2, p*3, p*4, p*5 };
```

En el caso de los arreglos de estructuras los elementos de cada posición van también entre { }, por lo que la sintaxis resulta así:

```
struct MONITOR

{

public string Marca;

public float Pulgadas;

};

MONITOR[] a;

a = new MONITOR[4] { new MONITOR {Marca="LG", Pulgadas=22},

new MONITOR {Marca="HP", Pulgadas=19},

new MONITOR {Marca="JVC", Pulgadas=24},

new MONITOR {Marca="TCL", Pulgadas=23} };
```

En este último ejemplo el elemento de la posición 0 del arreglo 'a' recibe los valores "LG" en el campo 'Marca' y 22 en el campo 'Pulgadas', así los valores de todos los

campos que forman la estructura se colocan entre llaves { } y se debe usar new para crear cada uno de los elementos de estructura MONITOR.

Este tipo de asignación en el mismo momento de la creación del arreglo puede ser conveniente cuando la cantidad de elementos es reducida.

Asignación de valores individuales:

```
Letras[0] = 'a';

Letras[1] = 'b';

Letras[2] = 'c';
```

o también:

```
Numeros[0] = 10;

Numeros[1] = 20;

Numeros[2] = 30;

Numeros[3] = 40;
```

Lo que nos lleva a usar un índice para referenciar la posición deseada:

```
int valor = 10;

for(i=0; i <= 10; i++) // se usa el índice 'i' para acceder a
las posiciones del arreglo

{

Numeros[i] = valor; // el valor de 'i' va de 0 a 10

valor += 10;

}
```

Recorridos del Arreglo

Nos referimos con esto a las formas de acceder a todos los elementos del arreglo ya sea para asignar nuevos valores a sus elementos individuales o para obtener los valores de sus elementos. Los recorridos por lo general son **secuenciales**, usamos un índice que determina la posición del arreglo y que se irá incrementando de uno en uno hasta llegar al último elemento del arreglo. También podemos hacer recorridos inversos, es decir desde la última posición del arreglo hasta la posición cero que es el inicio del arreglo.

Veamos algunos ejemplos:

- 1- Recorrido directo del arreglo

```
int[] numeros;

numeros = new int[10]; // declaramos y creamos un arreglo de 10 elementos de tipo entero

// recorrer el arreglo para asignar valores a todas sus posiciones;

for(i=0; i<10; i++) // el indice i tomará los valores del 0 al 9

{

numeros[i] = i+1; // los valores asignados serán del 1 al 10

}
```

El ciclo **for** recorre todos los elementos del arreglo y equivale a ejecutar esto:

```
numeros[0] = 1;

numeros[1] = 2;

numeros[2] = 3;

...
```

```
numeros[9] = 10;
```

- 2- Recorrido inverso del arreglo

  Siguiendo con el ejemplo anterior haremos un recorrido inverso para obtener la suma de todos los elementos del arreglo

```
int suma = 0;

for(i=9; i >= 0; i--) // se parte de la última posición del arreglo

{

suma = suma + numeros[i]; // en 'suma' se van acumulado los valores de cada posicion del arreglo

}

Resulta suma = 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1
```

- 3- Recorridos parciales del arreglo, son los que no acceden a todas las posiciones del arreglo, por ejemplo:

  a- Obtener la suma de los primeros 5 elementos del arreglo:

```
int suma = 0;

for(i=0; i < 5; i++) // recorre sólo 5 elementos

{

suma = suma + numeros[i];

}

Resulta suma = numeros[0] + numeros[1] + numeros[2] + numeros[3] + numeros[4] = 1 + 2 + 3 + 4 + 5
```

  b- Sumar sólo los elemntos de las posiciones pares del arreglo

```
int suma = 0;
```

```
for(i=0; i < 10; i=i+2) // se incrementa el indice de 2
en 2, recorre sólo los elementos de posiciones pares

{

suma = suma + numeros[i];

}
```

Resulta suma = numeros[0] + numeros[2] + numeros[4] + numeros[6] + numeros[8] = 1 + 3 + 5 + 7 + 9

Buscar elementos en el Arreglo

Las búsquedas se basan por lo general en recorridos secuenciales del arreglo, según el caso puede ser necesario recorrer el arreglo en forma completa o parcialmente. Supongamos un arreglo 'numeros' de 100 elementos de tipo entero que ha sido cargado con valores aleatorios entre 1 y 1000 y necesitamos buscar la posición que ocupe el primer elemento con valor 38:

```
int posicion;

int i = 0;

// recorrer desde 0 hasta el fin del arreglo o hasta que se
encuentre el valor buscado

while( i < 100 && numeros[i] != 38 )

{

i++;

}

// al salir del ciclo hay que controlar si el valor buscado
fue encontrado o si por el contrario se llegó al final del
recorrido
```

```
if( i < 100 ) // si no se llegó al final del recorrido es
porque se encontró el valor buscado, en este caso 38

{

posicion = i; // guardamos en 'posicion' el valor del índice
'i' que corresponde a la posición donde está el valor 38

MessageBox.Show( "Encontrado en la posiciçon : " +
posicion.ToString() );

}else{

posicion = -1; // guardamos el 'posicion' un valor negativo
indicando que el valor 38 no fue encontrado en el arreglo

MessageBox.Show( "Valor no encontrado" );

}
```

Como vemos este tipo de búsqueda tiene dos posibles resultados: encontrar el valor buscado o no encontrarlo, como la comparación del valor está en la misma condición del ciclo repetitivo al producirse la igualdad en la comparación, la condición del 'while' se vuelve falsa y el ciclo finaliza, en ese caso no habremos alcanzado el fin del recorrido y el índice 'i' será menor a 100 que es la cantidad de elementos del arreglo. Por el contrario, si el valor 38 no estuviera cargado en el arreglo el índice 'i' alcanzará el valor 100 y el ciclo finalizará sin haber encontrado el valor 38, en ese caso 'i' contendrá el valor 100 indicando que el valor buscado no fue encontrado.

Otro tipo de búsqueda es la que necesita recorrer el arreglo completo para poder determinar el resultado del proceso, por ejemplo, buscar el mayor valor almacenado en el vector del ejemplo anterior, tenemos 100 elementos con valores que varían entre 1 y 1000 y queremos saber cuál es efectivamente el mayor valor que posee el arreglo, para este tipo de búsquedas la solución implica recorrer en forma completa todos los elementos del arreglo para llegar a la solución correcta.

```
int mayor = 0; // variable para guardar el mayor valor
encontrado, se inicializa en cero
```

```
int i = 0;

// recorrer desde 0 hasta el fin del arreglo

while( i < 100 )

{

if( numeros[i] > mayor) // comparar el valor del arreglo con
la variable de referencia

{

mayor = numeros[i]; // guardar en la variable 'mayor' el valor
del arreglo

}

i++; // avanzar a la siguiente posición

}

MessageBox.Show( "Mayor valor: " + mayor.ToString() );
```

Modificar valores del Arreglo

Nos referimos a cambiar los valores previamente guardados en el arreglo por otros nuevos, puede ser en una sola y determinada posición o en varias posiciones, dependerá del criterio adoptado para realizar las modificaciones, podría tratarse, por ejemplo, de una actualización de precios que afecte sólo a un grupo de elementos o a todo el conjunto.

Este proceso puede requerir recorrer todo el arreglo si no se conoce previamente la posición que ocupa el elemento que queremos modificar, por ejemplo, dado un arreglo de tipo float de 5 elementos denominado 'Precios', ya cargado con valores, y queremos modificar los precios mayores a 1000 sumando un 10% a cada uno. Esto requerirá recorrer el arreglo completo y en cada posición comparar el valor almacenado para determinar si es mayor a 1000, y en caso de que lo sea, modificar su valor sumando un 10%.

```
int i;

for( i=0; i < 5; i++ ) // se recorre el arreglo completo

{

if( Precios[i] > 1000 ) // si el valor es mayor a 1000

{

Precios[i] = Precios[i] *1.1; // modificar el valor agregando un 10%

}

}
```

También pueden producirse cambios en el contenido del arreglo si se aplica algún algoritmo para ordenar sus valores, agruparlos por rangos, invertirlos, etc. En todos los casos se aplicará algún tipo de recorrido para acceder a cada posición y alterar los valores previamente almacenados.

Propiedades de los Arreglos en C#

Propiedad Length

Esta propiedad devuelve la longitud del arreglo, o sea la cantidad de elementos que posee, por ejemplo:

int N = Precios.Length; // siendo 'Precios' el arreglo del ejemplo anterior, 'N' resulta con el valor 5

Indique la opción correcta

1) Para declarar un arreglo unidimensional 'datos', de tipo entero se debe usar este código:

- ○ int[] datos;
- ○ int datos[];
- ○ datos int[];
- ○ datos[] int;
- ○ Ninguna de las anteriores

2) En los arreglos unidimensionales el valor numérico colocado entre corchetes indica:

- ○ La posición del último elemento del arreglo
- ○ La cantidad total de elementos del arreglo
- ○ Ninguna de las anteriores

3) Para asignar el valor 10 a la primera posición de un arreglo 'N' de tipo entero previamente creado, se escribe:

- ○ N[1] = 10;
- ○ N(0) = 10;
- ○ N[0] = 10;
- ○ N(1) = 10;

Indique verdadero o falso

4) En C# es suficiente con declarar el arreglo para poder guardar valores en él, por ejemplo, int[] a;

- ○ Verdadero
- ○ Falso

5) Para crear el arreglo se debe emplear el operador 'new'.

- ○ Verdadero
- ○ Falso

6) Un arreglo puede crearse y recibir los valores para sus elementos, todo en un solo paso.

- ○ Verdadero
- ○ Falso

Respuestas correctas[40]

[40] 1)int[] datos; 2)La cantidad total de elementos del arreglo; 3)N[0] = 10; 4)Falso; 5)Verdadero; 6)Verdadero.

SP3/H4: Elementos del lenguaje para conversiones y validaciones de datos

Validaciones en el ingreso de datos

Cuando necesitamos controlar algún dato ingresado por el usuario en un control de tipo TextBox podemos adoptar, entre otras, dos alternativas bien diferenciadas, la primera de ellas es permitir que el usuario escriba lo que quiera en el control TextBox y luego con un proceso determinar si el valor ingresado será válido o no, la otra alternativa es limitar el ingreso a los caracteres que consideremos válidos según el caso. Por ejemplo, si esperamos que el usuario ingrese un valor numérico, sólo permitiremos que se tengan en cuenta las teclas numéricas y la tecla de borrado (backspace), esto último puede implementarse por medio de los eventos del teclado: KeyDown, KeyPress y KeyUp, estos tres eventos se ejecutan siempre, cada vez que el usuario presiona una tecla y el control TextBox tiene el foco.

A continuación veremos algunos ejemplos que nos permitirán realizar este tipo de filtrado de teclas para el ingreso de datos en los TextBox.

Estos eventos pueden ser programados en cualquier control TextBox que necesite controlar que teclas se tendrán en cuenta y cuáles deben ser descartadas para evitar que el usuario ingrese valores no deseados.

Hay una secuencia en la ejecución de los eventos que conviene tener en cuenta, el primero que se ejecuta es siempre el evento KeyDown, cuando se presiona una tecla, luego se ejecuta el evento KeyPress y finalmente el KeyUp cuando se libera la tecla.

Además KeyDown y KeyUp permiten detectar la presión de combinaciones de teclas, como CTRL + ALT o CTRL + FN, etc. Por lo general, para los casos más comunes se trabaja con el evento KeyPress que detecta una sola tecla.

Filtrado de teclas para permitir ingresar sólo valores numéricos

El evento **KeyPress** tiene este encabezado:

```
private void txtNumero_KeyPress(object sender,
KeyPressEventArgs e)

{

}
```

Analizando el encabezado vemos que uno de sus argumentos es el objeto 'e' de tipo KetPressEventArgs, este objeto posee 2 propiedades: '**KeyChar**', de tipo char y '**Handled**' de tipo boolean, la primera nos informa qué tecla fue presionada y la segunda propiedad nos permite borrar la tecla presionada para evitar que llegue a mostrarse en el TextBox. De ahí podemos deducir que si en 'KeyChar' detectamos una tecla no deseada la podremos borrar asignado el valor True a la propiedad 'Handled', veamos en un caso concreto cómo lo implementamos:

Supongamos el control TextBox de nombre 'txtNumero', y el evento KeyPress de ese TextBox con esta implementación:

```
private void txtNumero_KeyPress(object sender,
KeyPressEventArgs e)

{

// en condicion se coloca una expresión que compare e.KeyChar
con los valores que no queremos que ingresen

if ( e.KeyChar == 'X' ) // si es una tecla no deseada se debe
borrar, 'X' es la tecla que queremos borrar

{

e.Handled = true; // borrar la tecla ingresada

}

}
```

Este ejemplo evaluará como verdadera la condición cada vez que el usuario presione la tecla X mayúscula y procederá a borrar esa tecla al asignar True a la propiedad Handled.

Si queremos controlar que se ingresen sólo números deberemos comparar con los caracteres '0' al '9', resultará así:

```
private void txtNumero_KeyPress(object sender,
KeyPressEventArgs e)

{

// en condicion se coloca una expresión que compare e.KeyChar
con los valores que no queremos que ingresen

if ( e.KeyChar < '0' || e.KeyChar > '9' ) // se borrarán las
teclas fuera del rango '0' al '9' (menor a 0 o mayor a 9)

{

e.Handled = true; // borrar la tecla ingresada

}

}
```

A este ejemplo le falta modificar la condición del 'if' para que acepte la tecla Backspace y no la borre, Backspace tiene el valor 8 (numérico), entonces finalmente resultará:

```
private void txtNumero_KeyPress(object sender,
KeyPressEventArgs e)

{

// en condición se coloca una expresión que compare e.KeyChar
con los valores que no queremos que ingresen

if ( (e.KeyChar < '0' || e.KeyChar > '9') && e.Keychar != 8 )
// se borrarán las teclas fuera del rango '0' al '9' menos
Backspace
```

```
{
e.Handled = true; // borrar la tecla ingresada
}
}
```

La enumeración 'Keys'

C# cuenta con la enumeración[41] 'Keys' que contiene la definición de todas las teclas que podemos encontrar en un teclado estándar, puede encontrar toda la documentación de esta enumeración en este link.

Por ejemplo, si nos queremos referir a las teclas numéricas usaremos: Keys.D0, Keys.D1, Keys.D2, ... Keys.D9 donde 'D0' corresponde a la tecla '0' (cero) y así con las siguientes. La tecla de Backspace es Keys.Back.

Los elementos de la enumeración representan valores numéricos de tipo 'int' y para poder compararlos con la propiedad 'KeyChar' del evento se necesita tener un valor de tipo 'char' por lo que hay que aplicar el modificador o 'cast[42]' al elemento de la enumeración: (char)Keys.D0, por ejemplo.

Teniendo en cuenta esto último podemos volver a escribir el código del evento KeyPress usando los valores de la enumeración 'Keys', resulta:

```
private void txtNumero_KeyPress(object sender,
KeyPressEventArgs e)
{
// en condición se coloca una expresión que compare e.KeyChar
con los valores que no queremos que ingresen
if((e.KeyChar < (char)Keys.D0 || e.KeyChar > (char)Keys.D9) &&
e.KeyChar != (char)Keys.Back )
```

[41] Enumeración: conjunto de constantes que definen todos los valores posibles de una variable, ejemplo: los días de la semana.
[42] Cast: conversión explícita de tipo de dato. Es la forma en la que un valor es convertido de un tipo de dato en otro tipo de dato, además evita posibles errores en la compilación.

```
{
e.Handled = true; // borrar la tecla ingresada
}
}
```

Tal vez nos dé la impresión de tener mayor cantidad de código, pero los valores de la enumeración son más fáciles de identificar con sus nombres, sobre todo con las teclas que no son dígitos y letras, como el caso de Backspace.

La Estructura 'Char' (no confundir con el tipo de dato 'char')

Otro elemento del lenguaje C# que podemos usar para hacer comparaciones y conversiones de caracteres es el objeto 'Char' que posee una serie de métodos con los que podemos operar las teclas presionadas en los eventos KeyDown, KeyPress y KeyUp, veamos algunos de esos métodos.

Métodos de validación o comparación

- Char.IsDigit(char): indica si un carácter es clasificado como dígito o no
- Char.IsLetter(char): indica si un carácter es clasificado como una letra o no
- Char.IsLower(char): indica si un carácter es clasificado como una letra minúscula o no
- Char.IsUpper(char): indica si un carácter es clasificado como una letra mayúscula o no
- Char.IsWitheSpace(char): indica si un carácter es clasificado como un espacio en blanco o no

Estos métodos devuelven el valor 'true' si se cumple su condición y 'false' en caso contrario.

Métodos de conversión

- Char.ToLower(char): convierte el carácter en minúscula, si char no es una letra no ocurrirá ninguna conversión

- Char.ToUpper(char): convierte el carácter en mayúscula, si char no es una letra no ocurrirá ninguna conversión

Los métodos de validación de 'Char' pueden resultar más directos y simples de usar en los eventos de teclado, por ejemplo, volviendo al caso de controlar valores numéricos podríamos usar el método 'IsDigit' y la implementación quedaría así:

```
private void txtNumero_KeyPress(object sender,
KeyPressEventArgs e)

{

// en condición se coloca una expresión que compare e.KeyChar
con los valores que no queremos que ingresen

if( !(Char.IsDigit(e.KeyChar)) && e.KeyChar != (char)Keys.Back
) // preguntamos si NO es dígito por eso se niega (!) la
expresión

{

e.Handled = true; // borrar la tecla ingresada

}

}
```

Otro ejemplo: ingresar siempre las letras mayúsculas, sin importar como se presionen en el teclado (si se presionó una minúscula convertirla a su correspondiente mayúscula).

```
private void txtNombre_KeyPress(object sender,
KeyPressEventArgs e)

{

// usamos los métodos de la clase 'Char', (IsLower y ToUpper)

if(Char.IsLower (e.KeyChar)) // es una minúscula ?

{
```

```
e.KeyChar = Char.ToUpper(e.KeyChar); // convertir a mayúscula
}
}
```

Observe que en este último ejemplo el valor original de la propiedad KeyChar se modifica por uno nuevo, en este caso por la mayúscula de la letra presionada. De la misma forma se podría obtener cualquier otra conversión de caracteres, por ejemplo reemplazar un espacio en blanco por un guión (-) si no queremos que en el texto ingresado queden espacios en blanco.

Eventos KeyDown y KeyUp

Estos eventos tienen este tipo de encabezado:

```
private void txtNombre_KeyDown(object sender, KeyEventArgs e)
{
}
```

El argumento 'e' que recibe el evento es de tipo KeyEventArgs y posee varias propiedades:

- KeyCode // código de teclado de la tecla presionada, es un valor de la enumeración Keys
- Handled // permite borrar la tecla presionada asignado True
- Control // indica si se presionó la tecla CTRL
- Shift // indica si se presionó la tecla SHIFT
- Alt // indica si se presionó la tecla ALT

Como lo comentamos antes con estos eventos es posible detectar la presión de dos o más teclas en forma simultánea, incluyendo las teclas CTRL, ALT y SHIFT

Ejemplo: detectar la combinación de CTRL + B en un textBox y borrar su contenido si eso ocurre:

```
private void txtNumero_KeyDown(object sender, KeyEventArgs e)
```

```
{
if( && e.Control && e.KeyCode == Keys.B ) // se presionó CTRL
y la tecla es 'B' ?
{
txtNumero.Clear(); // borrar el contenido del TextBox
}
}
```

En la resolución de nuestra situación profesional deberemos recurrir a estos eventos para mejorar el control de los datos que ingrese el usuario en la interfaz gráfica de la aplicación.

Convertir texto en valores numéricos

Para poder procesar correctamente los valores ingresados por el usuario en los controles TextBox necesitamos convertir una expresión de tipo *string*, que es el valor guardado en la propiedad 'Text' de cada control, en un valor numérico, entero o float, en el caso de tratarse de valores con parte decimal. Para realizar esas conversiones podemos recurrir a los métodos '**Parse**' que poseen todos los tipos de datos de C#.

¿Pero qué es 'Parse'?, en inglés 'parsing' es un término que viene del latín 'pars' y significa parte. En informática 'parsing' es el proceso por el cuál se analiza una serie de símbolos de acuerdo a una serie de reglas de una gramática formal, ese análisis determina si la expresión analizada cumple con determinado formato o no. Llevado eso a los tipos de datos del lenguaje C# nos permite convertir, por ejemplo, una expresión de texto: "158" en un valor entero equivalente: 158

Así contamos con un método 'Parse' en cada tipo de dato:

int.Parse(string) // retorna el valor entero de la cadena de texto analizada

long.Parse(string) // retorna el valor entero largo de la cadena de texto analizada

float.Parse(string) // retorna el valor float de la cadena de texto analizada

double.Parse(string) // retorna el valor double de la cadena de texto analizada

bool.Parse(string) // retorna el valor booleano de la cadena de texto analizada

Forma de uso de los métodos Parse, ejemplos:

- Obtener un valor entero del texto ingresado en un TextBox de nombre 'txtNumero':

```
int numero = int.Parse( txtNumero.Text ); // si txtNumero contiene "5420", se asignará el valor 5420 a la variable 'numero'
```

- Obtener un valor float del texto ingresado en un TextBox de nombre 'txtPrecio':

```
float precio = float.Parse( txtPrecio.Text ); // si txtPrecio contiene "150.75", se asignará el valor 150.75 a la variable 'precio'
```

Otras Propiedades de los tipos de datos

Relacionado con las conversiones de tipos tenemos en cada tipo de dato otras propiedades que pueden resultar útiles para obtener valores de referencia que nos sirvan en procesos de validación, búsquedas, ordenación, etc. Nos referimos concretamente a las propiedades '**MinValue**' y '**MaxValue**' que nos entregan los valores mínimo y máximo respectivamente que soporta cada tipo de dato, esto puede ser variable dependiendo del compilador del lenguaje y del sistema operativo, por ejemplo, veremos diferencias en sistemas de 32 y de 64 bits por la diferente capacidad de direccionamiento de memoria.

Ejemplos:

En 32 bits

```
int min = int.MinValue; // asigna el valor -2.147.483.648 en un sistema operativo de 32 bits
```

```
int max = int.MaxValue; // asigna el valor 2.147.483.647 en un
sistema operativo de 32 bits
```

En 64 bits

```
int min = int.MinValue; // asigna el valor
-9.223.372.036.854.775.808 en un sistema operativo de 64 bits

int max = int.MaxValue; // asigna el valor
9.223.372.036.854.775.807 en un sistema operativo de 64 bits
```

De esta forma sabemos que cualquier variable de tipo 'int' podrá almacenar valores comprendidos en el rango de MinValue y MaxValue.



Link del video: https://www.youtube.com/watch?v=UPJBxQe19gw

Texto del video:

En la situación profesional 3 vimos el uso de tipos definidos por el usuario y el concepto de Arreglo, en este caso trabajamos con arreglos de una dimensión para almacenar y procesar gran cantidad de datos de manera más sencilla.

Además trabajamos con distintas formas de convertir valores y conocimos el uso de los eventos del teclado para realizar filtrado y validaciones en el ingreso de datos por parte del usuario.

SP3/Autoevaluación 4

Indique verdadero o falso

1) El orden de ejecución de los eventos KeyPress, KeyDown y KeyUp puede ser alterada por el programador.

○ Verdadero

○ Falso

2) La enumeración 'Keys' contiene la definición de todas las teclas que posee un teclado estándar.

○ Verdadero

○ Falso

3) Para convertir un carácter en dígito se puede emplear el método Char.ToDigit()

○ Verdadero

○ Falso

Indique las opciones correctas (Hay más de una respuesta)

4) El evento KeyPress permite:

☐ Borrar el ingreso de teclas no deseadas
☐ Convertir la tecla ingresada por otra
☐ Detectar combinaciones de teclas
☐ Todas las anteriores

Indique la opción correcta

5) Los eventos KeyPress, KeyDown y KeyUp se ejecutan siempre en este orden:

○ 1-KeyPress, 2-KeyDown, 3-KeyUp

○ 1-KeyUp, 2-KeyDown, 3-KeyPress

○ 1-KeyDown, 2-KeyPress, 3-KeyUp

○ 1-KeyDown, 2-KeyUp, 3-KeyPress

○ 1-KeyUp, 2-KeyPress, 3-KeyDown

Respuestas correctas[43]

[43] 1)Falso; 2)Verdadero; 3)Falso; 4)Borrar el ingreso de teclas no deseadas / Convertir la tecla ingresada por otra; 5)1-KeyDown, 2-KeyPress, 3-KeyUp.

SP3/Ejercicio resuelto

Recordemos la situación planteada inicialmente:

Desarrollar una aplicación con interfaz gráfica para registrar y consultar los turnos para las revisiones técnicas de vehículos.

Para la carga de datos se requiere que el usuario ingrese la información del vehículo a revisar que consiste en estos datos:

- Dominio: tipo alfanumérico, de 6 caracteres mínimo y 7 caracteres máximo.
- Año de fabricación: tipo numérico, entero.
- Titular: tipo alfanumérico, de 30 caracteres máximo.

También deberá ingresar los datos del turno:

- Número de turno: tipo numérico, entero, 5 dígitos como máximo.

La aplicación debe permitir registrar hasta 50 turnos por vez.

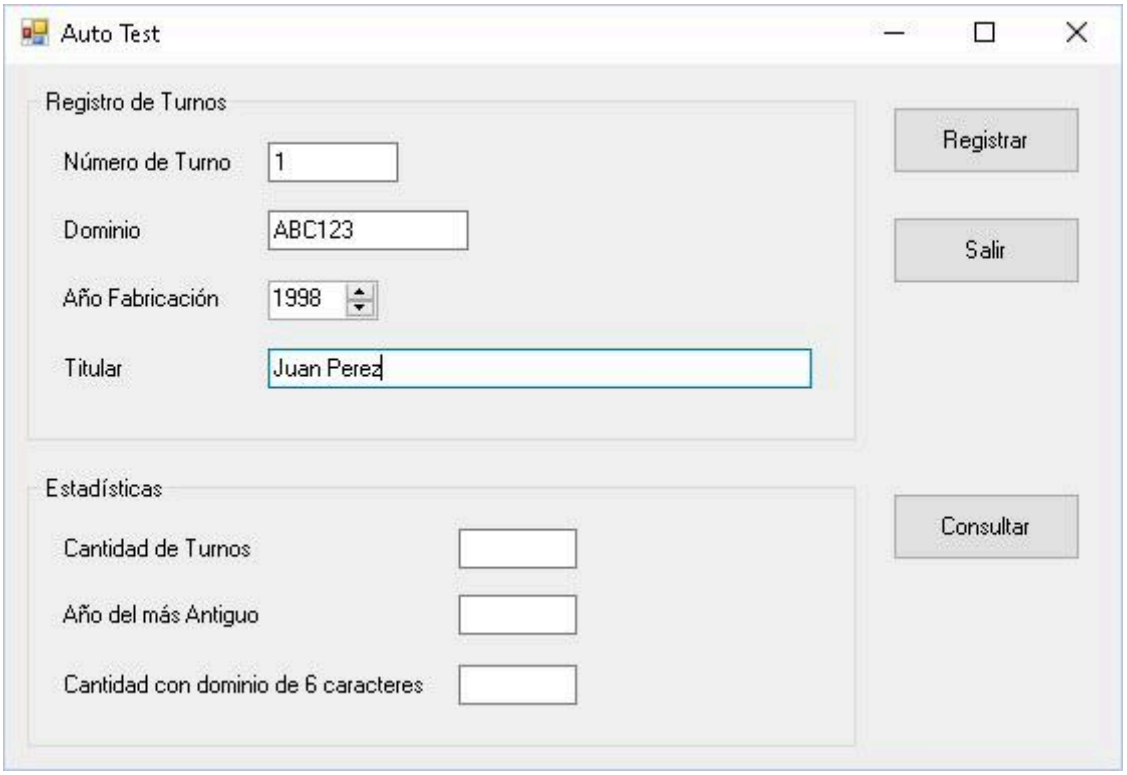
El registro de cada turno debe contar con validaciones que aseguren que los datos ingresados son correctos:

- El Dominio debe tener 6 o 7 caracteres, pudiendo ingresar solamente letras mayúsculas y los números del 0 al 9.
- El Año de fabricación debe estar comprendido entre 1950 y el año actual.
- Titular debe tener al menos dos caracteres.
- El Número de turno debe ser un número entero y no podrá repetirse. No es necesario que sean valores consecutivos.

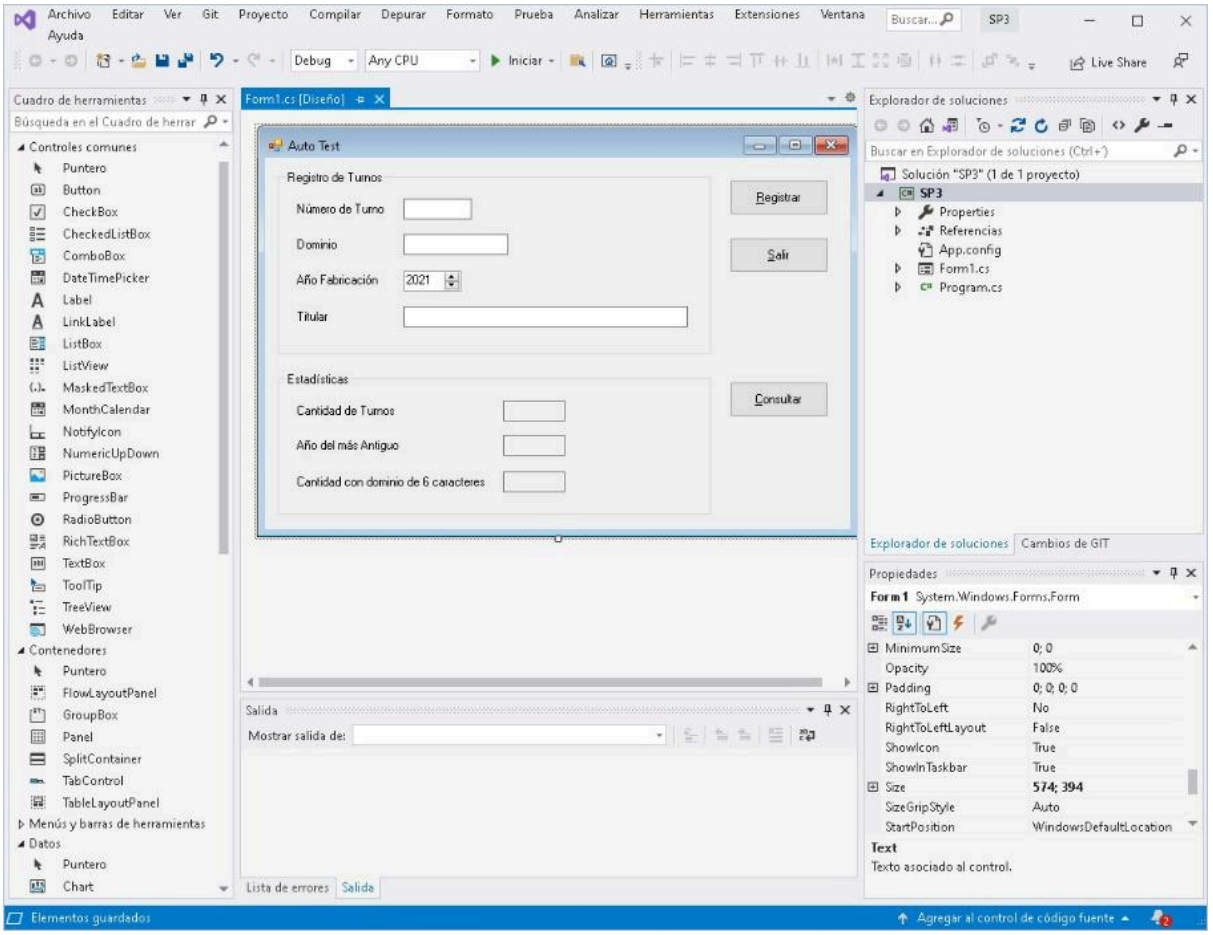
Consulta de Turnos: debe mostrar datos estadísticos de la información ingresada:

- La cantidad total de turnos ingresados.
- El año de fabricación del vehículo más antiguo registrado.
- La cantidad de vehículos registrados con dominio de 6 caracteres.

La interfaz gráfica podría tener este diseño:



Comenzamos creando, como lo hacemos habitualmente, un nuevo proyecto con Visual Studio .Net con leguaje C#, para la plataforma Windows y de tipo Aplicación de Escritorio, a continuación diseñamos el formulario de la aplicación con todos los componentes necesarios:.



Los nombres que usaremos para identificar los componentes (las etiquetas no se listan), comenzando de arriba a la izquierda, son:

grpRegistro (GroupBox), que contiene estos otros componentes:

- txtNroTurno (textBox)
- txtDominio (textBox)
- numAnioFabricacion (numericUpDown)
- txtTitular (textBox)

Para la parte de consultas tenemos el grpConsulta (GroupBox) que a su vez contiene a:

- txtCantidadTurnos (textBox)
- txtAnioMasAntiguo (textBox)
- txtDominio6Caracteres (textBox)

Y luego a la derecha del formulario los botones de comando:

btnRegistrar (Button)

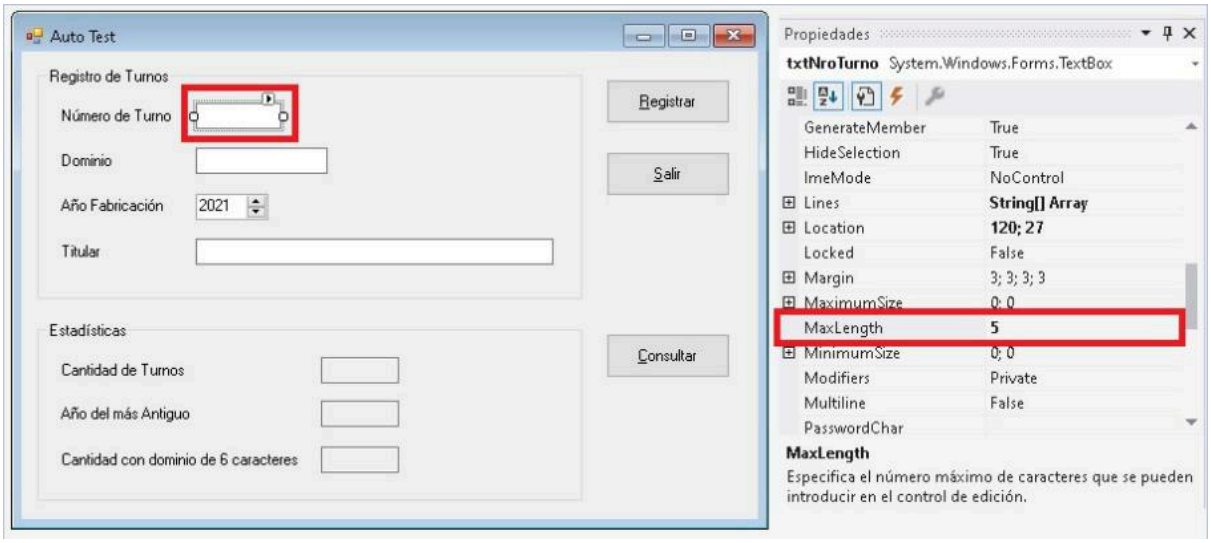
btnSalir (Button)

btnConsultar (Button)

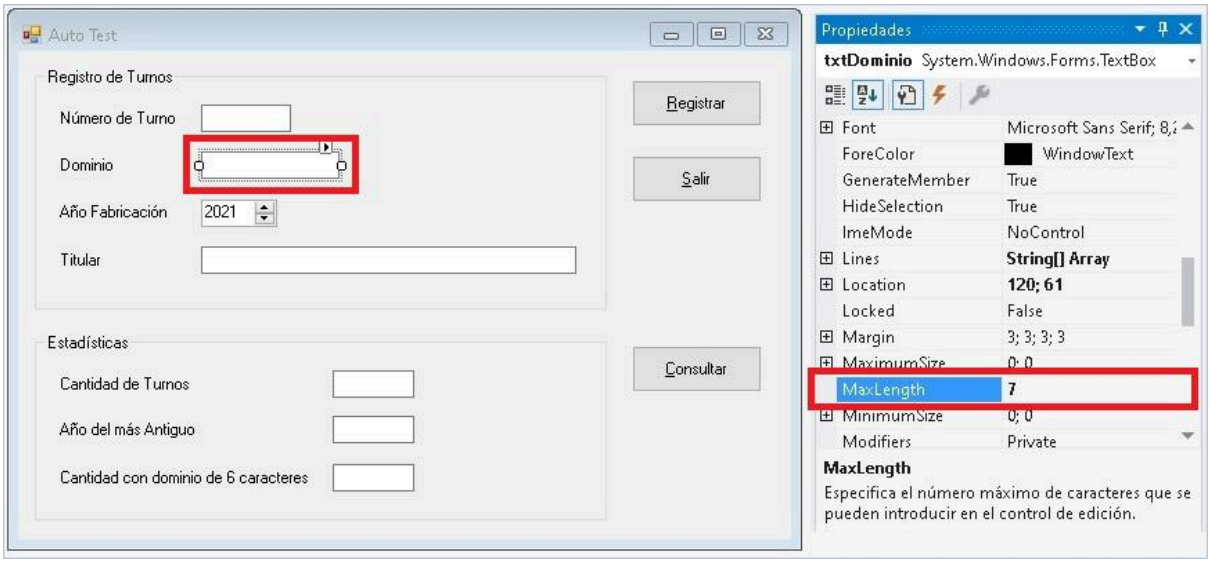
Una vez asignados los nombres a todos los elementos que necesitamos acceder desde el código que vamos a desarrollar, debemos modificar algunas propiedades para que el funcionamiento se ajuste a los requerimientos.

Ajustamos la propiedad 'Text' del formulario con el valor "Auto Test".

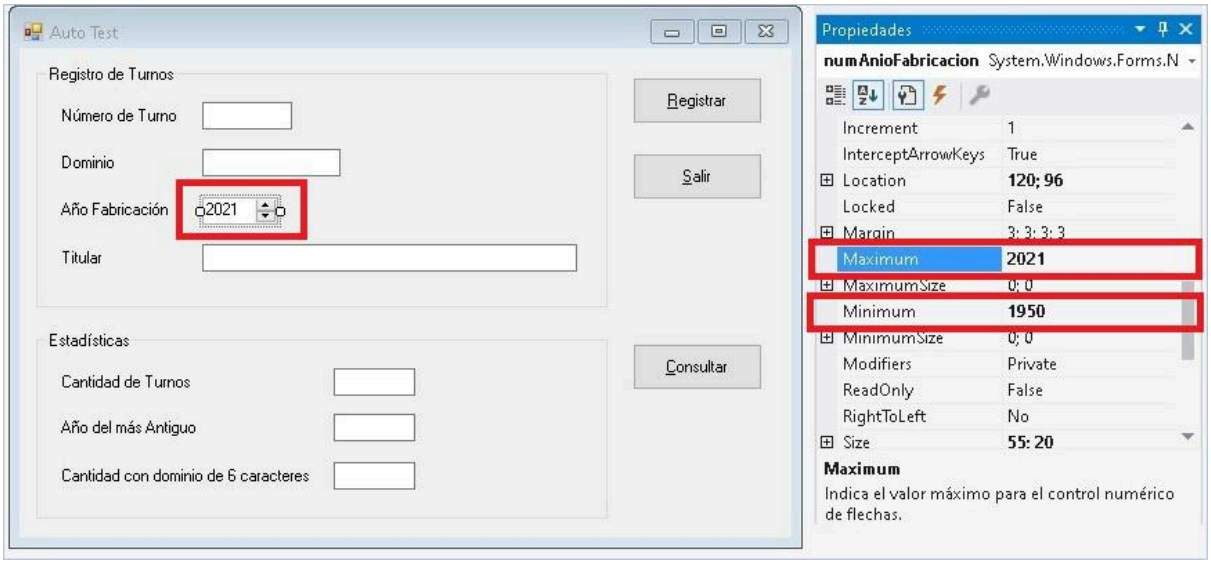
Del control txtNroTurno hay que limitar la cantidad de caracteres a ingresar para que no se supere la cantidad de 5, se usará la propiedad 'MaxLength' y el valor a asignar es 5.



Algo similar ocurre con txtDominio, se debe establecer MaxLength en 7

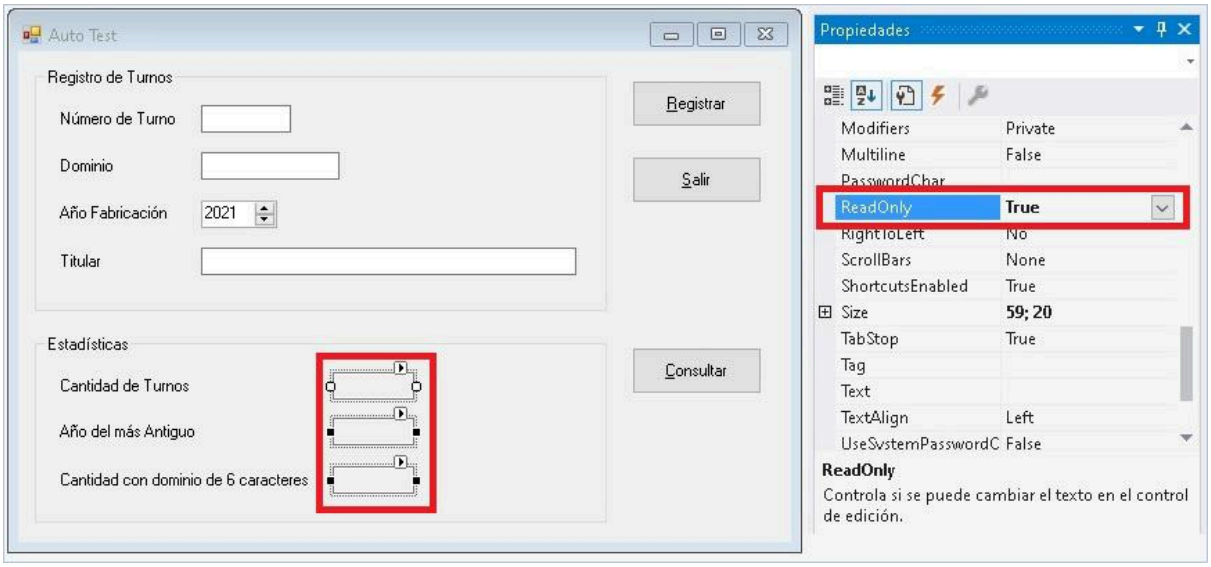


Para el control numericUpDown hay que asignar el valor mínimo y máximo de los valores que puede adoptar el control:



Para txtTitular también hay que modificar la propiedad MaxLength, pero en este caso el valor debe ser 30.

En la parte de la consulta y para evitar que los resultados mostrados por el programa puedan ser alterados por el usuario colocaremos en los 3 controles textBox la propiedad 'ReadOnly' (solo lectura) en verdadero, de esa forma el contenido de los textBox se podrá visualizar pero no se podrá modificar



Observe que se puede trabajar con los 3 componentes seleccionados al mismo tiempo para hacer la asignación en un solo paso.

Completada esta etapa de diseño y configuración de propiedades comenzaremos con la implementación de los eventos y procedimientos necesarios.

En primer lugar escribiremos la declaración de la estructura que debemos usar para almacenar la información de los turnos, la definición del arreglo unidimensional de turnos y algunas constantes y variables auxiliares, como ya lo hicimos anteriormente, este código se coloca en la sección de declaraciones del formulario:

C #: Declaraciones en SP3 Resuelta

```
public partial class Form1 : Form
   {
       // declaración de la estructura para los turnos
       public struct TURNO
       {
           public int NumeroTurno;
           public string Dominio;
           public int AnioFabricacion;
           public string Titular;
       };
       // constante para la cantidad total de elementos del arreglo
       const int MAX = 50;
       // declaración del arreglo unidimensional de 50 elementos
       public TURNO[] turnos;
       // variable para controlar la cantidad de elementos cargados
       public int Cantidad = 0;
```

Vemos la declaración de la estructura TURNO, el arreglo 'turnos' y la constante 'MAX' que usaremos para establecer la cantidad de elementos del arreglo, también se declara la variable 'Cantidad' que usaremos para contar y controlar la cantidad de turnos que se van cargando en el arreglo.

En segundo lugar trabajaremos, con el evento 'Load' del formulario para establecer el estado inicial de la aplicación, el código en este evento resulta así:

C #: Evento Load SP3 Resuelto

```
private void Form1_Load(object sender, EventArgs e)
{
    // creación del arreglo
    turnos = new TURNO[MAX];
    // inicializar la variable que controla la cantidad de elementos cargados
    Cantidad = 0;
    // establecer el estado inicial de todos los componentes de la interfaz
    inicializarInterfaz();
}
```

Se crea el arreglo, usando la constante 'MAX' para establecer la cantidad de elementos, asignamos cero a la variable 'Cantidad' y luego se ejecuta el procedimiento 'inicializarInterfaz':

C #: inicializarInterfaz SP3 Resuelta

```
// método para establecer el estado inicial del formulario
private void inicializarInterfaz()
{
    limpiarControles(); // estado inicial de los controles de carga de datos
    // estado inicial de los controles de consultas
    txtCantidadTurnos.Clear();
    txtAnioMasAntiguo.Clear();
    txtDominio6Caracteres.Clear();
}
```

En este método se ejecuta 'limpiarControles' que se encarga de los controles usados para la carga de datos, seguidamente se limpian los controles usados para las consultas.

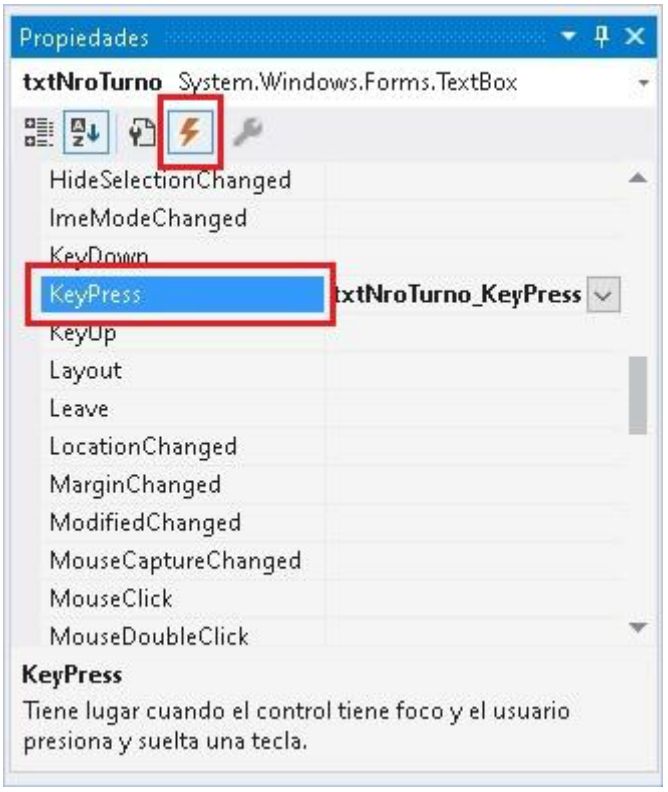
C #: limpiarControles SP3 Resuelta

```
private void limpiarControles()
{
    // valores para el estado inicial de los controles de carga
    txtNroTurno.Clear();
    txtDominio.Clear();
    numAnioFabricacion.Value = 2021;
    txtTitular.Clear();
}
```

Como veremos más adelante, 'limpiarControles' se volverá a usar en otra parte del código, por eso se implementó como un proceso separado de 'inicializarInterfaz'

Resumiendo, al ejecutarse la aplicación, primero se produce el evento 'Load' del formulario, de ahí se invoca al método 'inicializarInterfaz' y éste a su vez llama al método 'limpiarControles', al finalizar su ejecución tendremos el arreglo y la interfaz listos para comenzar a cargar datos.

Seguidamente comenzaremos a programar los eventos 'KeyPress' de los controles TextBox que requieran implementar algún filtro para las teclas que no se deben aceptar como válidas. Comenzaremos con el control 'txtNroTurno'. Como el evento 'KeyPress' no es el evento predefinido de los TextBox hay que agregarlo explícitamente desde la ventana de propiedades:



En primer lugar y con el control TextBox seleccionado, hacemos click en el ícono de los eventos (el rayo de color amarillo), veremos la lista de todos los eventos que soporta el control seleccionado, en esa lista buscamos el evento deseado, en este caso 'KeyPress', y seguidamente hacemos doble click sobre su nombre, automáticamente se generará el encabezado del evento en la ventana de código:

C #: Encabezado KeyPress

```
private void txtNroTurno_KeyPress(object sender, KeyPressEventArgs e)
  {
  }
```

Ahora ya podemos proceder a completar el código del evento, en este caso debemos filtrar las teclas y permitir solamente que se ingresen dígitos y que se pueda borrar usando la tecla Backspace, nos quedará esta implementación:

C #: Evento KeyPress de txtNroturno

```
// Evento KeyPress: debe permitir ingresar solamente números (y borrar)
  private void txtNroTurno_KeyPress(object sender, KeyPressEventArgs e)
  {
```

```
        // sin NO es un dígito y NO es Backspace (para borrar)
        if (!(Char.IsDigit(e.KeyChar)) && (e.KeyChar != (char)Keys.Back))
        {
             e.Handled = true;  // borrar la tecla ingresada
        }
    }
```

Procedemos de forma similar con el control txtDominio, agregamos el evento 'KeyPress', y su código es:

C #: Evento KeyPress txtDominio

```
    // Evento KeyPress: convierte minúsculas en mayúsculas y no permite
    // ingresar más que letras y números
    private void txtDominio_KeyPress(object sender, KeyPressEventArgs e)
    {
        // usamos los métodos de la clase 'Char', (IsLower y ToUpper)
        if(Char.IsLower (e.KeyChar)) // es una minúscula ?
        {
            e.KeyChar = Char.ToUpper(e.KeyChar); // convertir a mayúscula
        }
        // no es ni letra ni número y es distinto de Backspace ?
        if (!Char.IsLetterOrDigit (e.KeyChar) && e.KeyChar != (char)Keys.Back)
        {
            e.Handled = true; // borrar la tecla
        }
    }
```

En este evento las letras minúsculas serán convertidas a sus correspondientes mayúsculas y si se presiona una tecla que no sea una letra o un número, dicha tecla será borrada, excepto la tecla BackSpace que se debe aceptar para permitir modificar el valor ingresado en caso de error.

Primero se hace uso de los métodos 'IsLower()' para determinar si la tecla corresponde a una letra minúscula y del método 'ToUpper() para convertir a mayúscula la letra ingresada. Más abajo se usa el método 'IsLeeterOrDigit()' para determinar si la tecla presionada es una letra o un número, y si no lo fuera y es diferente de la tecla BackSpace será borrada y su ingreso anulado. Note que para el valor de la tecla BackSpace se usa la enumeración 'Keys'. Con esa implementación el contenido de

txtDominio queda asegurado a letras mayúsculas y números, cualquier otro carácter será eliminado.

El próximo paso consiste en desarrollar una función que realice una validación completa de los datos del turno que se pretende ingresar, es decir que controle si los datos están completos, si la longitud del dominio es de al menos 6 caracteres y si el número del turno no existe ya en el arreglo, esta función nos facilitará realizar la carga del turno devolviendo verdadero si todo es correcto y falso en caso contrario. Resulta:

C #: Funcion ValidarDatos

```
    private bool validarDatos()
      {
          bool resultado = false; // valor a devolver si no se cumplen todas las
condiciones
          // validar la existencia de los datos a ingresar
          if (txtNroTurno.Text != "" && txtDominio.Text != "" && txtTitular.Text != "")
          {
              if (txtDominio.Text.Length >= 6) // validar el contenido del Dominio
              {
                  // validar que NO exista el número de turno a cargar
                  if(!buscarTurno(int.Parse(txtNroTurno.Text)))
                  {
                      resultado = true; // si todo está bien devuelve verdadero
                  }
                  else {
                      MessageBox.Show("El Número de Turno ingresado ya existe",
                          "ERROR", MessageBoxButtons.OK, MessageBoxIcon.Error);
                  }
              }
              else {
                  MessageBox.Show("El Dominio debe tener 6 o 7 caracteres",
                      "ERROR", MessageBoxButtons.OK, MessageBoxIcon.Error);
              }
          }
          else {
              MessageBox.Show("Debe completar los datos faltantes", "ERROR",
                  MessageBoxButtons.OK, MessageBoxIcon.Error);
          }
          return resultado;
      }
```

En 'validarDatos' se controlan 3 condiciones, la primera verifica que txtNroTurno, txtDominio y txtTitular tengan datos, ninguna puede estar vacía, la segunda condición controla que txtDominio tenga al menos 6 caracteres y la tercera condición usa la función 'buscarTurno' para determinar que el número de turno que se pretende ingresar no existe ya en el arreglo de turnos. Para cada condición que no se cumpla se mostrará un mensaje de error particular.

Veamos entonces el desarrollo de la función 'buscarTurno'

C #: Funcion buscarTurno

```
// función para buscar un número de turno en el arreglo de turnos
// recibe por parámetro el número a buscar
// devuelve verdadero si el número existe en el arreglo y falso en caso contrario
private bool buscarTurno(int numero)
{
    bool existe = false; // valor inicial a devolver
    int pos = 0; // primera posición del arreglo
    // recorrer el arreglo hasta la posición que tenga datos cargados
    while(pos < Cantidad)
    {
        // comparar el número buscado con el número en el arreglo
        if(numero == turnos[pos].NumeroTurno)
        {
            existe = true; // indica que el número fue encontrado
            break; // sale del ciclo while
        }
        pos++; // próxima posición del arreglo
    }
    return existe;  // devuelve verdadero o falso
}
```

Como se observa, la función hace un recorrido secuencial de los elementos del arreglo 'turnos' usando como índice la variable 'pos' que se inicializa en cero y se incrementa hasta la cantidad de elementos cargados en el arreglo. En cada posición del arreglo compara el número de turno buscado con el número de turno guardado en el arreglo, si lo encuentra asigna verdadero a la variable 'existe' y sale del ciclo repetitivo (con 'break'). Si el índice alcanza la última posición cargada, saldrá del ciclo y devolverá falso indicando que el número de turno buscado no se encuentra en el arreglo.

Ahora ya tenemos todo preparado para poder implementar la carga de los turnos, lo haremos entonces desde el evento 'Click' del botón 'Registrar':

C #: Evento Click btnRegistrar

```
private void btnRegistrar_Click(object sender, EventArgs e)
{
    // validar los datos antes de ingresarlos al arreglo
    if(validarDatos())
    {
        // agregar los datos al arreglo en la posición indicada por
        // la variable 'Cantidad'
        turnos[Cantidad].NumeroTurno = int.Parse(txtNroTurno.Text);
        turnos[Cantidad].Dominio = txtDominio.Text;
        turnos[Cantidad].AnioFabricacion =
                    int.Parse(numAnioFabricacion.Value.ToString());
        turnos[Cantidad].Titular = txtTitular.Text;
        // incrementar la cantidad de elementos cargados
        Cantidad++;
        // verificar si queda espacio en el arreglo
        if(Cantidad == MAX)
        {
            MessageBox.Show("Se completó la capacidad de carga de datos",
                "ATENCIÓN", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
            btnRegistrar.Enabled = false; // deshabilitar el botón Registrar
        }
        limpiarControles(); // restablecer el estado inicial del formulario
    }
}
```

En este evento haremos uso de la función 'validarDatos', si obtenemos verdadero procedemos a asignar en el arreglo los valores de los campos definidos en la estructura 'TURNO', comenzando por 'NumeroTurno', luego 'Dominio', 'AnioFabricacion' y finalmente 'Titular', todos los valores se obtienen de la misma interfaz, teniendo la precaución de convertir los que sean numéricos con el método 'Parse' del tipo 'int', como ocurre con los valores de 'NumeroTurno' y 'AnioFabricacion'. Para la posición a usar del arreglo se usa la variable 'Cantidad' que se va incrementando después que se carga un nuevo turno.

Cuando la variable 'Cantidad' alcance el valor máximo que soporta el arreglo se mostrará un mensaje indicando que la carga de turnos está completa y el botón 'Registrar' quedará deshabilitado permanentemente.

Finalmente se llama al método 'limpiarControles' que ya vimos y usamos al comienzo del desarrollo, para restablecer el estado de la interfaz y dejarla lista para un nuevo registro.

Así se completa todo el código necesario para la carga de turnos, nos resta resolver las consultas estadísticas que se deben mostrar en la parte inferior del formulario.

Botón 'Consultar': necesitamos trabajar en el evento 'Click' del botón, como son tres los valores a obtener se puede implementar un proceso para cada valor o realizar las tres consultas en un único proceso, nosotros optamos por un único proceso que obtenga y muestre los tres resultados solicitados:

C #: Evento Click btnConsultar-parte1

```
// botón Consultar
  private void btnConsultar_Click(object sender, EventArgs e)
  {
      // primer consulta: cantidad de turnos registrados
      // se obtiene directamente de la variable 'Cantidad'
      txtCantidadTurnos.Text = Cantidad.ToString();
```

La cantidad de turnos está almacenada en la variable 'Cantidad', por lo que simplemente mostramos su valor.

C #: Evento Click btnConsultar-parte2

```
      // segunda consulta: se debe recorrer el arreglo y determinar
        // el menor valor del campo 'AnioFabricacion'
        int menor = int.MaxValue; // mayor valor posible
        int pos;
        // recorrer el arreglo hasta la posición con datos
        for(pos=0; pos<Cantidad; pos++)
        {
            // comparar el valor del elemento en el arreglo
            if(turnos[pos].AnioFabricacion < menor)
            {
                menor = turnos[pos].AnioFabricacion; // guarda el menor valor
```

```
            }
        }
        // mostrar el resultado
        txtAnioMasAntiguo.Text = menor.ToString();
```

Para la segunda consulta se recorre el arreglo y se compara el año de fabricación de cada elemento con la variable 'menor' usada para almacenar el menor valor a medida que se recorre el arreglo, al finalizar el recorrido se mostrará su valor. Observar que el valor inicial de la variable 'menor' se obtiene de la propiedad 'MaxValue' del tipo 'int', esa propiedad nos devuelve el valor máximo que se puede guardar en una variable de tipo 'int', esto nos asegura que cualquier valor del año de fabricación en el arreglo será menor a él.

C #: Evento Click btnConsultar-parte3

```
        // tercera consulta: cantidad de vehículos con dominio de 6 caracteres
        int contador = 0; // contador en cero
        // recorrer el arreglo hasta la posición con datos
        for (pos = 0; pos < Cantidad; pos++)
        {
            // controlar si la longitud del dominio es 6
            if (turnos[pos].Dominio.Length == 6)
            {
                contador++; // incrementar el contador
            }
        }
        // mostrar el resultado
        txtDominio6Caracteres.Text = contador.ToString();
```

En la tercera consulta también es necesario recorrer todas las posiciones del arreglo que fueron cargadas, pero en este caso tenemos un 'contador', que es una variable inicializada en cero y que se irá incrementando cada vez que en el arreglo el campo 'Dominio' tenga 6 caracteres, lo que se resuelve usando la propiedad 'Length' del tipo 'string'.

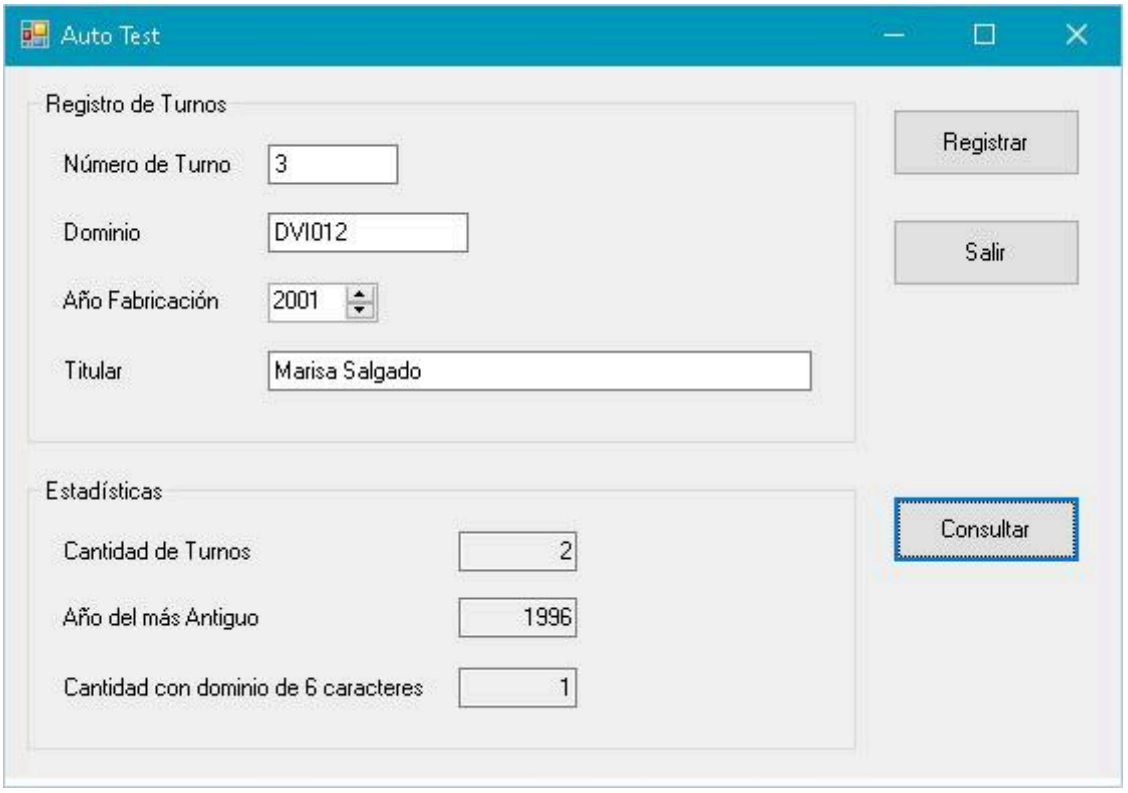
Para completar la resolución de la aplicación implementaremos el evento 'Click' del botón 'Salir'

C #: Evento Click btnSalir

```
// botón Salir
  private void btnSalir_Click(object sender, EventArgs e)
  {
      this.Close(); // cierra el formulario
  }
```

Acá simplemente ejecutamos el método 'Close' del formulario para cerrar la aplicación. Y así finalizamos la implementación de toda la funcionalidad solicitada en la situación profesional.

Vemos acá un ejemplo de la aplicación funcionado



SP3/Ejercicio por resolver

El administrador de una empresa de venta de repuestos para autos le ha solicitado a usted, en calidad de pasante, que desarrolle un programa para ingresar repuestos y, luego, poder consultar el detalle de los repuestos ingresados.

La interfaz gráfica debe tener en su parte superior los controles necesarios para ingresar los datos de los repuestos, que consisten en:

- La marca del repuesto, una letra que representa la fábrica del repuesto (P) Peugeot, (F) Fiat o (R) Renault.
- El origen del repuesto, una letra que puede ser (N) Nacional o (I) Importado.
- Número del repuesto, valor numérico de 6 dígitos como máximo.
- Descripción del repuesto, texto de hasta 50 caracteres como máximo.
- Precio del repuesto, de tipo float.

Se deberá definir la estructura con los campos adecuados para contener los datos enumerados anteriormente y un arreglo de 100 elementos para cargar los repuestos.

Debe controlar que los datos ingresados de cada repuesto sean correctos antes de ser almacenados en el arreglo, de lo contrario debe informar el error. No permita ingresar dos repuestos con el mismo número. Informe también en este caso el error. Al completar la carga de los 100 elementos en el arreglo mostrar un mensaje y no permitir más ingresos.

En la parte inferior de la interfaz gráfica se realizará la consulta de los datos ingresados, para ello se dispondrá un control comboBox para elegir la marca (P, F, R) y dos botones de opción para elegir el origen del repuesto (Nacional, Importado), se deberá permitir consultar por marca y origen, listando todos aquellos repuestos cuya marca y origen coincida con los valores seleccionados. Mostrar el número, la descripción y el precio de los repuestos encontrados. Para listar los datos de los repuestos se puede emplear un control TextBox (con la propiedad Multiline en verdadero), o un ListBox.

Indique la opción correcta

1) Dada la siguiente declaración de una estructura

```
public struct TELEFONO
{
   public int codigoArea;
   public int Numero;
};
```

- ○ Es incorrecta porque 'struct' debería ser 'structure'
- ○ Es incorrecta porque 'TELEFONO' no existe
- ○ Es correcta
- ○ Es incorrecta porque el tipo 'int' no es adecuado para números de 9 dígitos
- ○ Es incorrecta porque los campos deberían ser 'private'

2) En los arreglos unidimensionales el valor numérico colocado entre corchetes indica:

- ○ La posición del último elemento del arreglo
- ○ La cantidad total de elementos del arreglo
- ○ Ninguna de las anteriores

Indique verdadero o falso

3) Para acceder a los elementos de un arreglo se usa como Índice un valor entero

- ○ Verdadero
- ○ Falso

4) El operador 'new' se usa para declarar un arreglo.

- ○ Verdadero
- ○ Falso

5) Para convertir una letra mayúscula en su correspondiente minúscula se puede emplear el método Char.ToLower()

- ○ Verdadero
- ○ Falso

Respuestas correctas[44]

[44] 1)Es correcta; 2)La cantidad total de elementos del arreglo; 3)Verdadero; 4)Falso; 5)Verdadero.

Situación profesional 4: Arreglos multidimensionales - Triatlón

El director de la Asociación de Triatlón de la provincia le ha solicitado a usted, en carácter de pasante del área de informática, que desarrolle una aplicación con interfaz gráfica para registrar la participación de los 6 finalistas de la prueba de triatlón que se realiza todos los años.

La aplicación debe permitir ingresar el nombre y la nacionalidad de los 6 finalistas de la prueba, a continuación, se debe poder cargar el puesto obtenido por cada participante en las pruebas de natación, ciclismo y carrera, que son las tres pruebas que componen el triatlón. Se ingresarán los números del 1 al 6 para cada participante y prueba. Una vez completada la carga de datos la aplicación deberá, primero validar que los datos ingresados sean correctos, seguidamente deberá procesar las posiciones de cada participante y determinar los ganadores de la competencia, considerando el primero, segundo y tercer puesto. Para esto último se asignan los siguientes puntajes para cada posición de las pruebas:

Primer puesto: 1000 puntos

Segundo puesto: 920 puntos

Tercer puesto: 830 puntos

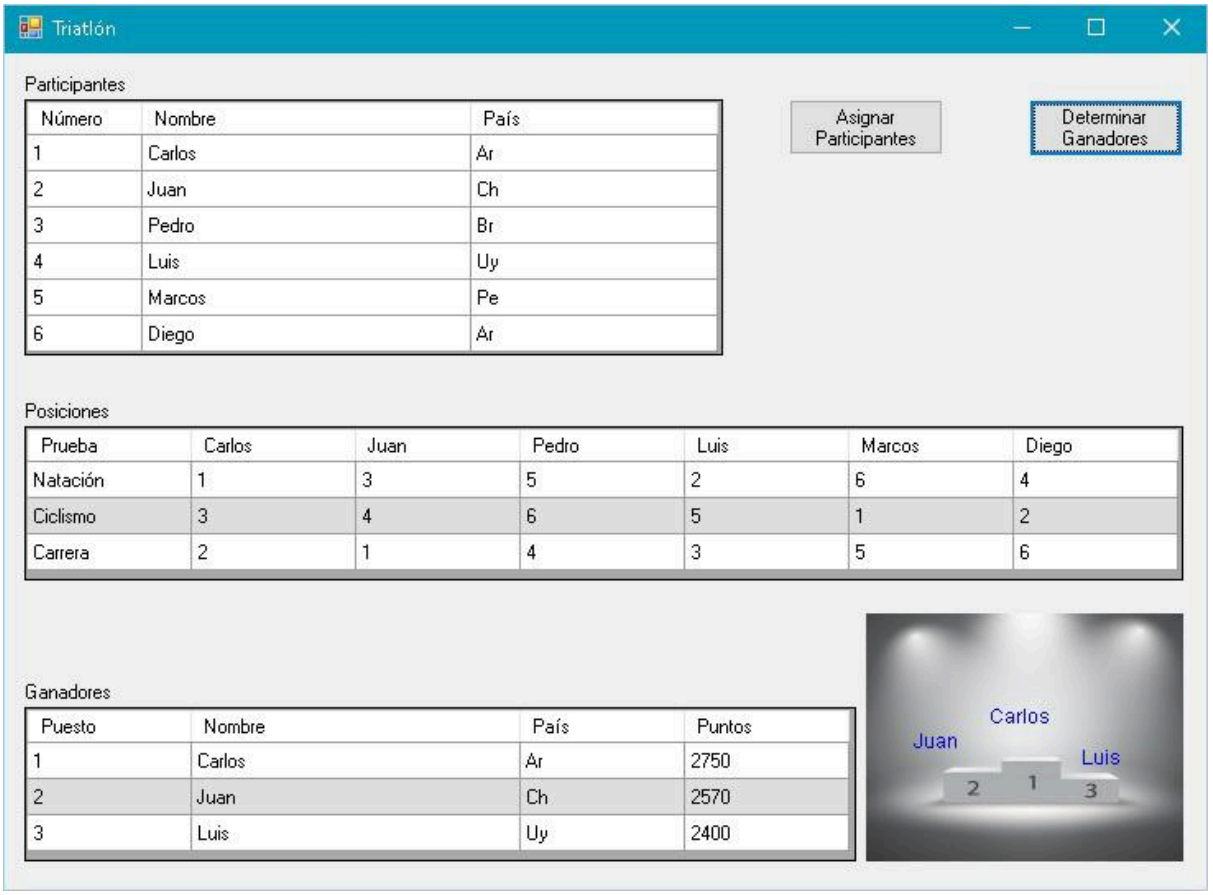
Cuarto puesto: 740 puntos

Quinto puesto: 650 puntos

Sexto puesto: 560 puntos

Los datos de los 3 ganadores del triatlón se mostrarán en una grilla, donde se debe visualizar el puesto obtenido, el nombre, nacionalidad, y puntaje total de cada participante.

La interfaz propuesta para la aplicación es la siguiente:



En la primera grilla, el usuario de la aplicación cargará los datos de los 6 participantes, nombre y nacionalidad. Las celdas de la grilla permiten ingresar los datos directamente sobre ellas, una vez completada con los 6 participantes se usará el botón "Asignar Participantes" para configurar el encabezado de la segunda grilla colocando en ella los nombres de los participantes.

En la segunda grilla el usuario ingresará las posiciones que ocupó cada participante en cada una de las 3 pruebas del triatlón. Las posiciones son los números del 1 al 6. Una vez que se completen las posiciones obtenidas por todos los participantes se podrá usar el botón "Procesar Ganadores" para hacer una validación de los datos cargados en la grilla, validación que consiste en verificar que en cada prueba estén las 6 posiciones, sin repetidos ni faltantes y seguidamente calcular el puntaje total de cada participante para finalmente mostrar en la tercera grilla los datos de los 3 ganadores. Adicionalmente se mostrará una imagen del podio con el agregado de los nombres de los ganadores.

Si la validación encuentra algún error se mostrará un mensaje informativo y el cálculo de los ganadores no se realizará hasta que los datos no sean corregidos.

Arreglos

Arreglos multidimensionales

Como ya vimos en la situación profesional 3, los arreglos unidimensionales son los que poseen un único índice para acceder a sus elementos individuales, revisaremos ahora los multidimensionales que son aquellos que poseen más de un índice y en C# también cumplen con las siguientes propiedades:

- Sus elementos son todos del mismo tipo de dato.
- Deben tener un identificador válido como nombre.
- Sus elementos individuales se podrán acceder con índices de tipo numérico entero.
- Deben tener un tamaño definido (cantidad de elementos que forman el arreglo).

Las actividades que podemos realizar con el arreglo multidimensional son:

- Declarar el arreglo, con su tipo y nombre.
- Crear el arreglo, es decir su instancia.
- Hacer uso del arreglo:
    - Asignar valores a sus elementos individuales.
    - Recorrer el arreglo.
    - Buscar un elemento en el arreglo.
    - Modificar un elemento del arreglo

A continuación, veremos cada uno de estos puntos usando C#

Declaración y creación del arreglo

Con los arreglos multidimensionales debemos tener en cuenta la cantidad de índices que usarán, el caso más común es el de 2 índices, creando así las llamadas matrices de datos. En ese caso el primer índice representa las filas de la matriz y el segundo índice representa las columnas. Para cada par de valores (fila, columna) tendremos un dato almacenado en el arreglo.

La declaración del arreglo consiste en especificar el tipo de dato del arreglo y su nombre, colocando después del tipo de dato corchetes y la coma como separador de los índices que tendrá el arreglo:

```
tipo [,] Nombre_Arreglo; // arreglo de 2 dimensiones

tipo [,,] Nombre_Arreglo; // arreglo de 3 dimensiones
```

Recordemos que el tipo de dato puede ser alguno de los tipos base, como char, int, float, etc. o un tipo struct o también objetos en general.

Ejemplos:

```
char[,] Letras; // arreglo de tipo 'char', nos permitirá almacenar un caracter en cada una de sus posiciones

int[,] Números; // arreglo de tipo 'int'

float[,] Precios; // arreglo de tipo 'float'

PERSONA[,] Clientes; // arreglo de tipo PERSONA, donde 'PERSONA' es una estructura
```

También vimos que luego de la declaración es necesario **crear** el arreglo haciendo uso del operador '**new**' para obtener una instancia y el espacio de memoria necesario para sus elementos. Para la creación debemos especificar la cantidad de elementos que tendrá el arreglo, en el caso de 2 dimensiones:

```
arreglo = new tipo[cantidad de filas, cantidad de columnas];
```

'arreglo' es el nombre del arreglo que previamente ya hemos declarado

'tipo' es el mismo tipo de dato que usamos en su declaración

'cantidad de filas' es un valor de tipo entero (o una variable entera) que especifica el tamaño de la primera dimensión

'cantidad de columnas' es un valor de tipo entero (o una variable entera) que especifica el tamaño de la segunda dimensión.

Ejemplos:

```
Letras = nuevo char[2, 30]; // crea el arreglo 'Letras' y le asigna espacio para 2 filas de 30 elementos cada una

Números = nuevo int[5, 10]; // crea el arreglo 'Numeros' y le asigna 5 filas de 10 elementos cada una

Precios = new float[5, 5]; // crea el arreglo 'Precios' y le asigna 5 filas de 5 elementos cada una

Clientes = new PERSONA[10, 50] // crea el arreglo 'Clientes' y le asigna 10 filas de 50 elementos cada una
```

En el caso de arreglos de 3 dimensiones:

```
arreglo = new tipo[cantidad de elementos 1, cantidad de elementos 2, cantidad de elementos 3];
```

Como en los casos anteriores hace falta especificar la cantidad de elementos que tendrá el arreglo en cada dimensión.

Eso se repite de la misma forma para arreglos de n-dimensiones.

Completada la creación del arreglo ya podemos utilizar el arreglo en nuestro programa.

Recalcamos nuevamente que una vez creado el arreglo, su cantidad de elementos no podrá ser modificada, es decir no podremos ni agregar ni quitar elementos al arreglo ya

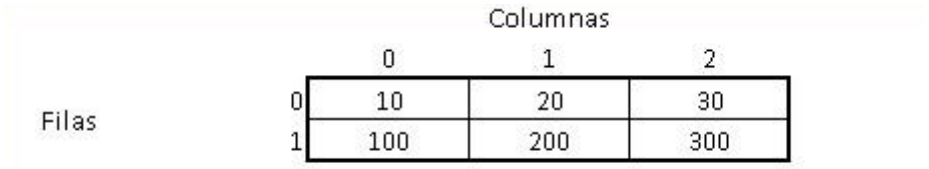
que al crear su instancia la cantidad de memoria asignada al arreglo queda definida por la cantidad de elementos.

Valores iniciales en un arreglo multidimensional

Es posible realizar una asignación de valores en el mismo momento de crear el arreglo:

```
int[,] mat = nuevo int[2,3] { {10, 20, 30}, {100, 200, 300} };
```

El resultado lo podemos graficar así:



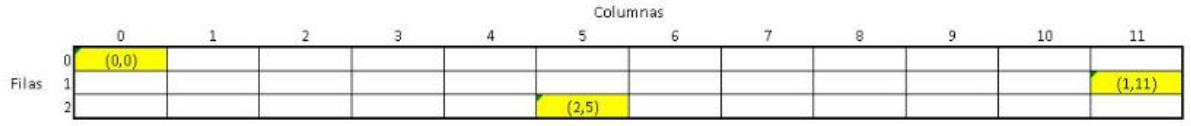
El primer grupo de valores {10,20,30} corresponde a la primera fila del arreglo, el segundo grupo {100,200,300} corresponde a la segunda fila del arreglo.

Índices en un arreglo bidimensional (matriz)

Como dijimos anteriormente, los arreglos más usados son los de 2 dimensiones, conocidos como bidimensionales o matrices, veremos seguidamente un ejemplo para explicar mejor la forma de usar sus índices y su significado. Supondremos que tenemos 3 empleados y queremos registrar el sueldo que cobran durante 12 meses, podríamos crear un arreglo bidimensional con 3 filas, una para cada empleado y 12 columnas, una para cada mes.

```
float[,] sueldos = new float[3,12];
```

La representación de este arreglo lo podríamos graficar así:



Como podemos ver las filas se numeran de 0 a 2 y las columnas de 0 a 11, entonces para acceder a una posición del arreglo debemos especificar ahora 2 valores, uno para la fila y otro para la columna, a diferencia de los arreglos unidimensionales que usan un único índice.

De esta forma la expresión sueldos[0,0] contendrá el valor del sueldo del primer vendedor en el primer mes trabajado:

sueldos[1, 11] tendrá el sueldo del segundo vendedor en el último mes trabajado (mes 12).

sueldos[2, 5] tendrá el sueldo del tercer vendedor en el sexto mes.

Límites de un arreglo de 2 dimensiones

Podemos conocer los valores máximos de los índices en cada una de las dimensiones de un arreglo multidimensional y la cantidad total de elementos que contiene con el método GetUpperBound() y la propiedad Length.

Por ejemplo:

Creamos un arreglo de 2 dimensiones de 2 filas y 5 columnas:

```
int[,] arr = nuevo int[2,5];
```

El método "GetUpperBound()" nos permite obtener el mayor índice en cada una de las dimensiones del arreglo, solo debemos especificar como parámetro el número de la dimensión que deseamos conocer, el número de dimensión comienza en cero para la primera dimensión del arreglo, por ejemplo:

```
int max1 = arr.GetUpperBound(0); // asignamos a la variable
"max1" el mayor índice de la primera dimensión del arreglo
"arr", en este caso se obtendrá el valor 1, ya que en las
filas tenemos los índices 0 y 1 (2 filas).
```

De forma similar:

```
int max2 = arr.GetUpperBound(1); // asignamos a la variable "max2" el mayor índice de la segunda dimensión del arreglo "arr", en este caso se obtendrá el valor 4, ya que en las columnas tenemos los índices 0, 1, 2, 3 y 4 (5 columnas).
```

Con la propiedad "Length" podemos obtener la cantidad total de elementos que posee un arreglo:

```
int cantidad = arr.Length; // en este caso la variable "cantidad" recibirá el valor 10 (2 filas por 5 columnas)
```

Recorrido de un arreglo de 2 dimensiones

Como ya vimos con los arreglos de una dimensión, para recorrer sus elementos se utiliza una estructura de control repetitiva, un índice y se necesita conocer el límite superior del arreglo. Ahora con un arreglo de 2 dimensiones vamos a necesitar **un índice para cada dimensión y conocer el límite superior en cada una de las dimensiones**, también **necesitaremos emplear 2 estructuras de control repetitivas, pero anidadas**, es decir una repetitiva incluida dentro del recorrido de otra repetitiva.

Veamos un ejemplo aplicado al arreglo del ejemplo anterior:

Inicialización del arreglo bidimensional: vamos a implementar un recorrido del arreglo para asignar a todos sus elementos un valor inicial, en este caso 1 (uno).

C#: Inicializando el arreglo

```
int indF; // indice para las filas
int indC; // indice para las columnas
// creación del arreglo de 2 filas y 5 columnas
int[,] arr = new int[2, 5];
indF = 0; // primer ciclo: recorre las filas
while (indF <= arr.GetUpperBound(0))
{
    indC = 0; // segundo ciclo: recorre las columnas
    while (indC <= arr.GetUpperBound(1))
    {
        // se asigna un valor a cada elemento del arreglo
```

```
            arr[indF, indC] = 1; // valor inicial
            indC++; // incremento del indice de columna
        }
        indF++; // incremento del indice de fila
    }
```

En este ejemplo se usaron ciclos repetitivos implementados con la estructura "while" pero también se pueden implementar con la estructura "for" de forma similar.

El primer ciclo recorre las filas, desde la fila 0 hasta la fila 1 y para cada fila, con el segundo ciclo, se recorren las columnas, desde la columna 0 hasta la columna 4, de esa forma todos los elementos del arreglo son asignados con el valor 1.

Este proceso también puede resolverse recorriendo primero las columnas y luego las filas, es decir, invertimos el orden en que se recorren los elementos del arreglo. En ambos casos el resultado final será el mismo.

Recorrido de lectura del arreglo bidimensional:

Cuando se trabaja con arreglos de tipo numérico, es muy frecuente tener que hacer cálculos con los elementos del arreglo, veamos los dos casos más frecuentes: procesar los datos por filas y por columnas.

- 1- Procesar los datos del arreglo recorriendo sus filas, por ejemplo, calcular la suma de todos los elementos de cada fila.

C #: Suma por fila

```
  int sumaFila; // resultado de la suma de cada fila
   int indF; // indice para las filas
   int indC; // indice para las columnas
   for(indF=0; indF <= arr.GetUpperBound(0); indF++)
   {
       sumaFila = 0; // suma inicial en cero
       for(indC=0; indC <= arr.GetUpperBound(1); indC++)
       {
           sumaFila += arr[indF, indC];
       }
       MessageBox.Show("Suma de fila " + indF.ToString() + " = " + sumaFila.ToString
());
   }
```

- 2- Procesar los datos del arreglo recorriendo sus columnas, por ejemplo, calcular la suma de todos los elementos de cada columna.

C #: Suma por Columna

```
 int sumaColumna; // resultado de la suma de cada columna
  int indF; // indice para las filas
  int indC; // indice para las columnas
  for (indC = 0; indC <= arr.GetUpperBound(1); indC++)
  {
      sumaColumna = 0; // suma inicial en cero
      for (indF = 0; indF <= arr.GetUpperBound(0); indF++)
      {
          sumaColumna += arr[indF, indC];
      }
      MessageBox.Show("Suma de columna " + indC.ToString() + " = " +
sumaColumna.ToString());
  }
```

Como se puede ver comparando los dos ejemplos anteriores, la lógica es la misma, pero en el segundo caso tenemos invertido el orden en que se recorren filas y columnas, en el primer caso buscamos sumar todos los elementos de cada fila, es decir que mientras el índice de la fila permanece fijo hacemos variar el índice de las columnas y así accedemos a todos los elementos que componen esa fila, el proceso se repetirá con todas las filas del arreglo.

En el segundo caso buscamos sumar todos los elementos de cada columna, entonces el ciclo externo controla la variación de la columna y el ciclo interno controla el índice de la fila, obteniendo así la suma de cada columna.

Existe otro caso que se puede resolver da forma mucho más sencilla y es cuando necesitamos obtener un valor de todos los elementos del arreglo, sin distinguir filas o columnas, por ejemplo, la suma del arreglo completo:

C#: Ciclo Foreach

```
 int total = 0;
  foreach (int elemento in arr)
  {
      total += elemento;
  }
  MessageBox.Show("Suma total = " + total.ToString());
```

Podemos hacer uso del ciclo "foreach" indicado para recorrer colecciones completas de elementos, necesitamos una variable que sea del mismo tipo que los elementos que componen el arreglo, en este caso "int" con la variable "elemento". El ciclo "foreach" se encarga de tomar uno por uno todos los elementos del arreglo y asignarlos a la variable "elemento", de forma tal que dentro del ciclo podemos usar la variable para leer el valor que contiene, en este caso por tratarse de un arreglo de dos dimensiones el ciclo "foreach" recorrerá el arreglo completo como si lo estuviéramos haciendo con dos ciclos anidados. Con "foreach" no podemos saber que posición ocupa cada elemento en el momento de ser procesado, para esos casos se debe implementar el recorrido con los ciclos "while" o "for".

Otra limitación importante del ciclo "foreach" está dada por colocar el arreglo en modo solo lectura, es decir que no es posible realizar modificaciones de sus elementos, si eso fuera necesario hay que recurrir a los recorridos con "while" o "for".

Precauciones en el uso de arreglos

Uno de los errores más comunes que aparecen en el tratamiento de arreglos es intentar acceder a una posición del arreglo que no existe, ya sea fila o columna, o ambos. Esto se da cuando la variable usada como índice para el recorrido de alguna de las dimensiones del arreglo excede el valor máximo permitido, obtenemos en esos casos una excepción de tipo "IndexOutOfRangeException" (Excepción índice fuera de rango) que nos advierte que el índice usado está fuera de rango.

Por ese motivo se recomienda usar los métodos "GetUpperBound()" para obtener el valor máximo que puede tomar el índice en cada dimensión del arreglo y controlarlo para no superarlo.

También es común obtener un error "ArgumentNullException" (Excepción argumento nulo) cuando intentamos usar un elemento del arreglo que no ha recibido un valor inicial, sobre todo con los de tipo "string". Para evitar esto se recomienda realizar la inicialización del arreglo colocando algún valor inicial en todos sus elementos.

Muchos de los conceptos desarrollados se deberán usar en la resolución de la situación profesional actual.

Indique verdadero o falso

1) En C# los arreglos multidimensionales sólo pueden ser de 2 o 3 dimensiones;

- ○ Verdadero
- ○ Falso

2) Un arreglo multidimensional no puede recibir los valores para sus elementos en el momento de su creación, las asignaciones de valores deben hacerse posteriormente.

- ○ Verdadero
- ○ Falso

3) En un arreglo bidimensional los elementos que contiene pueden ser de 2 tipos de dato diferentes

- ○ Verdadero
- ○ Falso

Indique la opción correcta

4) Para declarar un arreglo bidimensional 'datos', de tipo entero se debe usar este código:

- ○ int[2] datos;
- ○ int datos[,];
- ○ int[,] datos;
- ○ datos[,] int;
- ○ Ninguna de las anteriores

5) Para asignar el valor 100 a la posición de la fila 2, columna 5 de un arreglo bidimensional 'A' de tipo entero previamente creado, se escribe:

- A[1,4] = 100;
- A[2,5] = 100;
- A[5,2] = 100;
- A(2,5) = 100;

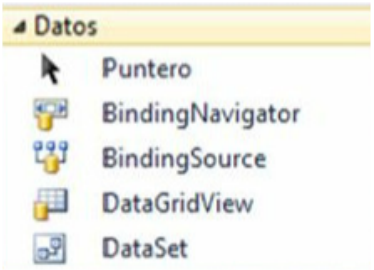
Respuestas correctas[45]

[45] 1)Falso; 2)Falso; 3)Falso; 4)int[,] datos; 5)A[2,5] = 100;.

Control Grilla (DataGridView)

Este componente presenta una gran diversidad de usos y sus propiedades y métodos nos permiten configurarla de diferentes maneras por lo que resulta un componente muy usado para presentar datos en formato tabla y también para permitir el ingreso, edición y eliminación de datos, trabajando en estos últimos casos en forma interactiva con el usuario de la aplicación.

El control DataGridView se encuentra en el cuadro de Herramientas de Visual Studio, en la pestaña Datos.



Pestaña Datos

El DataGridView se compone de filas (Rows) y columnas (Columns), como una tabla, las columnas las podemos configurar en modo diseño desde la ventana de propiedades de Visual Studio o desde la ventana de diseño de la propia grilla. Desde esa ventana podemos agregar columnas, asignar un texto de encabezado (header), regular el ancho de la columna y varias propiedades más. Todo esto también es posible configurar desde el mismo código de la aplicación y obtener así un comportamiento dinámico de la grilla según las necesidades, por ejemplo, se podrían ocultar columnas, agregar o quitar, incluso cambiar completamente una configuración por otra.

Las filas de la grilla en cambio, sólo es posible agregarlas o quitarlas por código, en tiempo de ejecución.

Otra funcionalidad muy interesante que presenta este control nos permite "enlazar" una fuente externa de datos, por ejemplo, una tabla de una base de datos, y visualizar directamente los datos en la grilla sin necesidad de programar nada más.

Configurar la grilla

La configuración de las columnas es normalmente la primera tarea que debemos realizar con la grilla, es un proceso bastante intuitivo. pero lo veremos a continuación con un ejemplo.

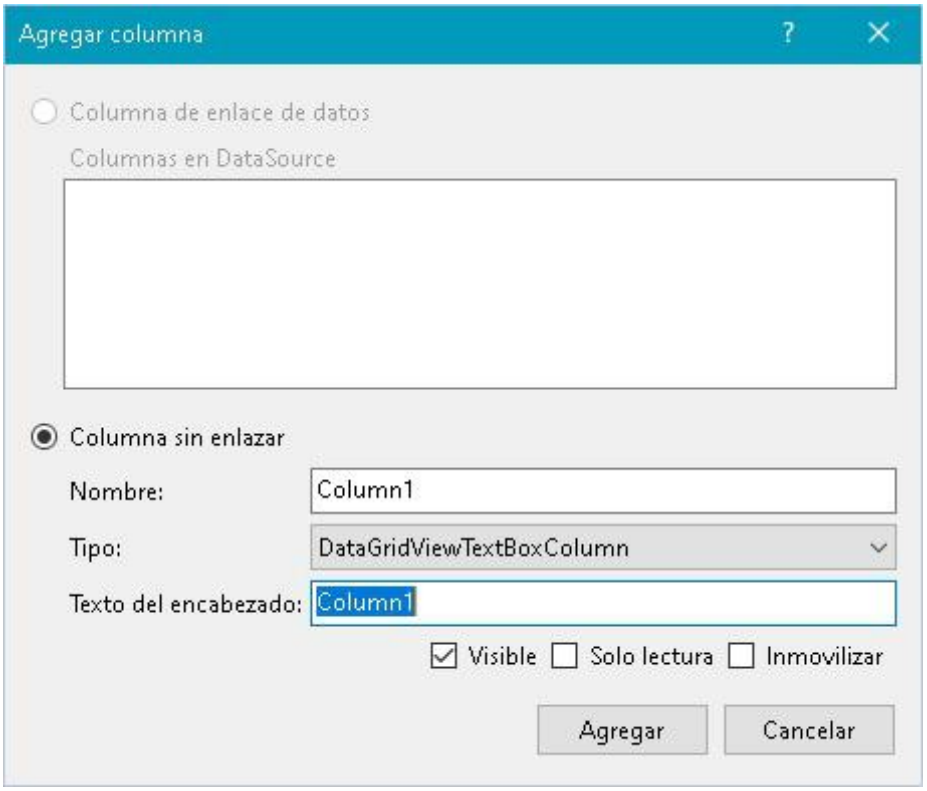
Ubicamos el control DataGridView en el formulario y hacemos click en el botón que se muestra en la parte superior derecha de la grilla, se desplegará este menú de opciones del que elegiremos la opción "Agregar columna..."



Tenemos ahora la ventana para agregar la información básica de la nueva columna, usaremos "Columna sin enlazar" que es el tipo por defecto, en el campo "Nombre" podemos asignar un nombre al objeto columna que estamos creando, este nombre debe seguir las reglas de los identificadores del lenguaje, luego podemos elegir el tipo de columna, siendo el tipo "TextBoxColumn" el que normalmente se usa. A continuación, colocamos el "Texto del encabezado" que es el texto que se verá en la columna de la grilla, como es un dato de tipo string y no un nombre de objeto podemos colocar cualquier texto.

Más abajo disponemos de tres propiedades que podemos cambiar: "Visible", "Solo lectura" e "Inmovilizar".

Finalmente, con el botón "Agregar" la nueva columna será agregada a la grilla.

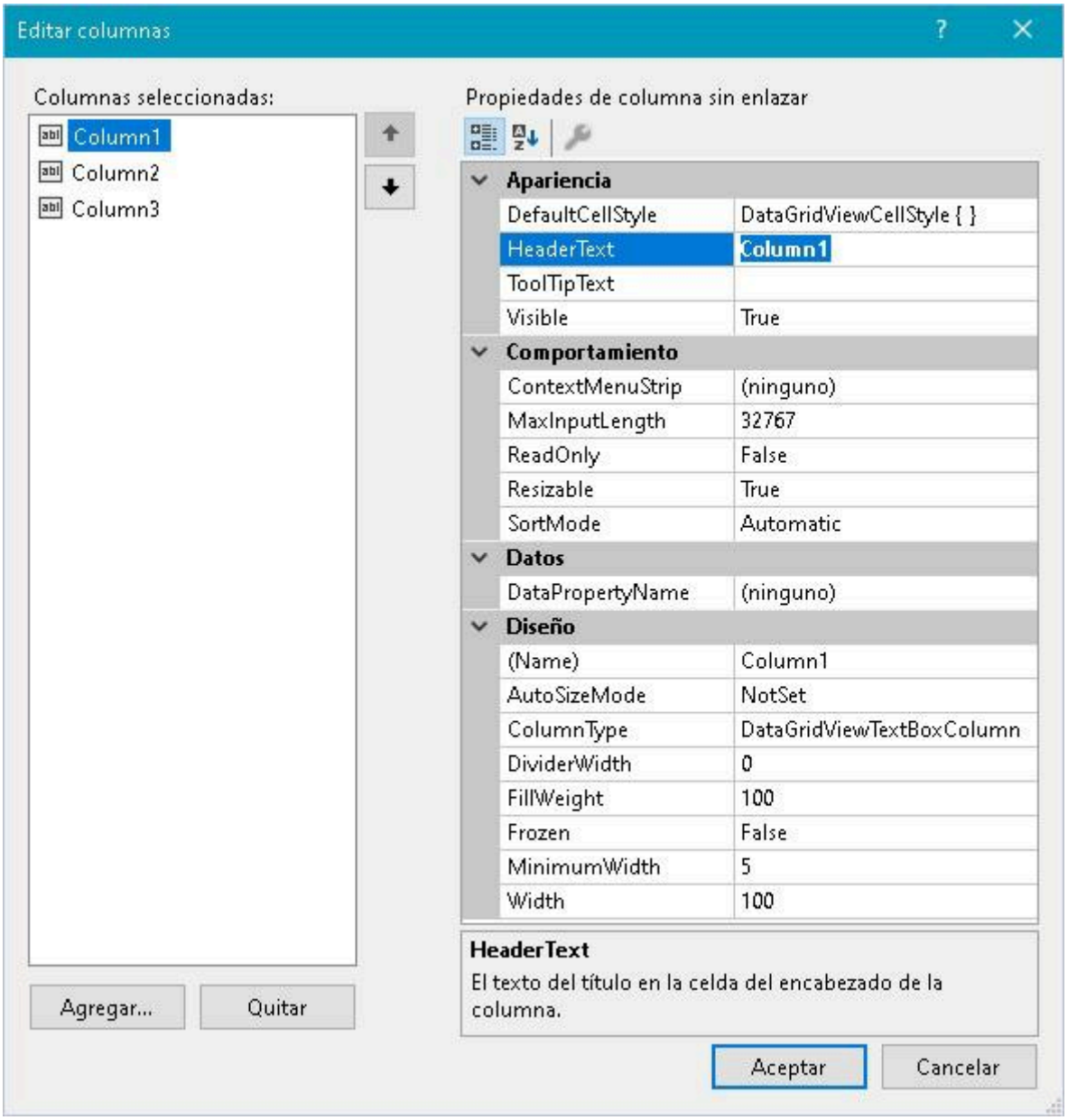


Por lo general, en una grilla que vamos a usar solo para mostrar datos que genera la aplicación, asignaremos el texto del encabezado y dejaremos los demás campos con sus valores iniciales.

Esto lo podemos repetir con todas las columnas que necesitemos configurar en nuestra grilla.

Editar columnas existentes

Si posteriormente necesitamos modificar algunas propiedades de las columnas podemos seleccionar en el menú desplegable la opción "Editar columnas..." y tendremos esta ventana:



Seleccionado una columna en la lista de la izquierda tenemos disponibles todas las propiedades que podemos modificar de la columna, "HeaderText" es la propiedad que guarda el texto del encabezado asignado a la columna, "Width" es la propiedad que establece el ancho de la columna y que a veces conviene modificar para que su contenido pueda verse en forma completa.

A la derecha de la lista de las columnas se ubican dos botones con una flecha hacia arriba y hacia abajo, esos botones mueven la columna seleccionada en el sentido de la flecha para reordenar su ubicación en la grilla. La columna en la primera posición de esta lista aparecerá a la izquierda en la grilla.

Funcionalidad interactiva con la grilla

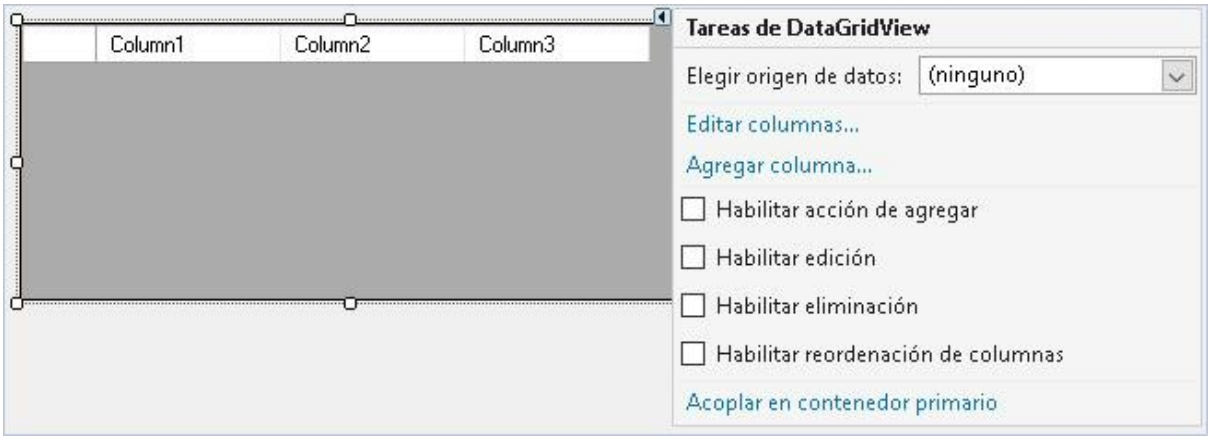
En el menú desplegable de la grilla hay cuatro opciones que se pueden habilitar o deshabilitar a voluntad, estas opciones regulan el comportamiento de la grilla completa cuando la aplicación está en ejecución y el usuario puede interactuar con el control.

"Habilitar acción de agregar", si está activada, permitirá al usuario agregar datos en una nueva fila de la grilla, en ese caso la fila para agregar datos aparece al final de todas y queda identificada con un **"*"** asterisco en la izquierda de la fila.

"Habilitar edición", si está activa, posibilita al usuario modificar valores en cualquier celda de la grilla.

"Habilitar eliminación", si está activa, permite al usuario seleccionar una fila y removerla de la grilla, se usa en ese caso el botón suprimir del teclado.

En los casos donde la grilla cumple con la funcionalidad de visualizar datos solamente estas tres propiedades deben desactivarse para evitar que el usuario realice cambios en los datos.



Manipular la grilla desde el código

Comentamos anteriormente que las configuraciones de las columnas y las propiedades de la grilla se pueden realizar desde el mismo código de la aplicación, veremos algunos ejemplos:

- **Asignar el texto de encabezado de una columna:**

La grilla tiene una colección de columnas (propiedad Columns) que podemos acceder con un índice, que tiene base cero, además cada columna dispone de la propiedad "HeaderText" que almacena el encabezado de la columna, para modificarlo podemos hacer:

```
datagridview1.Columns[0]. HeaderText = "texto de encabezado";
```

De todas las columnas estamos referenciando a la primera, la de la posición 0: Columns[0] y de esa columna en particular a la propiedad **HeaderText** para modificar su contenido.

- **Establecer una columna en modo solo lectura (ReadOnly)**

  Si nuestra grilla está activada para permitir la edición de los valores que muestra y queremos mantener una columna con los valores originales para que el usuario no pueda modificarlos debemos establecer la propiedad "ReadOnly" (sólo lectura) en verdadero para esa columna:

```
datagridview1.Columns[0]. ReadOnly = true; // coloca la columna 0 en modo solo lectura
```

- **Agregar filas vacías a la grilla**

  Para agregar una fila nueva a la grilla usaremos la colección "Rows" (filas) del control y el método "Add" (agregar) de la siguiente forma:

```
datagridview1.Rows.Add(); // agrega al final de la grilla una nueva fila vacía.
```

  También se puede agregar una nueva fila y asignar valores en cada celda que forma la fila, deberíamos tener tantos valores como columnas tenga la grilla. En general sería:

```
datagridview1.Rows.Add( valor1, valor2, ..., valorN );
```

  En este caso todos los valores asignados, desde valor1 hasta valorN, deben ser de tipo string.

La fila agregada se ubicará al final de todas las filas ya existentes.

- **Asignar valores a las celdas de una columna**

  Supongamos que tenemos 3 filas en una grilla y queremos asignar ciertos valores, por ejemplo, nombres de personas, a las celdas que forman parte de la columna 0 (cero). Usaremos en este caso la coleccion "Cells" que posee cada "Row" de la grilla, Cada "Cell" representa una celda de la grilla. La celda almacena su valor en la propiedad "Value".

```
datagridview1.Rows[0].Cells[0].Value = "Juan Pablo";

datagridview1.Rows[1].Cells[0].Value = "Ana María";

datagridview1.Rows[2].Cells[0].Value = "Luis";
```

  El índice de Rows indica sobre qué fila estamos actuando, el índice de Cells indica qué columna de esa fila accedemos y la propiedad Value es la que recibe el valor que será almacenado en esa celda particular. Resumiendo, estamos asignando valores a la columna 0 de la grilla en sus 3 primeras filas.

  Importante: para que esto no genere errores durante la ejecución del programa deben existir las filas 0, 1 y 2 en la grilla, en caso contrario estaríamos tratando de acceder a elementos que no existen.

  En general podemos extender este código para asignar un valor a cualquier celda de la grilla, solo debemos conocer en qué fila y en qué columna se encuentra la celda a modificar.

```
datagridview1.Rows[ Fila].Cells[Columna].Value =
nuevo_valor;
```

  Fila: número entero que indica la fila

  Columna: número entero que indica la columna

  nuevo_valor: expresión o variable de tipo string con el valor que se asignará a la celda

- **Recorrer una fila de la grilla**

  Si necesitamos recorrer todas las celdas de una determinada fila podemos emplear un ciclo repetitivo con una variable que tome las posiciones de cada celda y tenga en cuenta la cantidad de columnas que posee la grilla.

  Ejemplo:

  Debemos sumar todos los valores que posee una grilla, en su tercera fila, sería la fila número 2 ya que comenzamos a contar desde la posición cero. Asumimos que la grilla posee 5 columnas.

```
int suma = 0;

int columna;

string valor;

// ciclo for para recorrer las columnas de la grilla

for(columna=0; columna < datagridview1. ColumnCount;
columna++)

{

valor =
datagridview1.Rows[2].Cells[columna].Value.ToString ();

suma += int.Parse(valor); // se acumulan en 'suma' los
valores de las celdas

}
```

  Usamos la propiedad '**ColumnCount**' de la grilla para determinar la cantidad de columnas que posee la grilla, en nuestro ejemplo dijimos que la grilla tiene 5 columnas entonces ese será el valor que tendrá la propiedad 'ColumnCount'.

  En el interior del ciclo repetitivo tomamos el valor de la celda que está ubicada en la fila número 2 (tercera fila) y la columna indicada por la variable 'columna', que inicia con valor cero y se irá incrementando en cada iteración del ciclo hasta

que alcance el valor 5, momento en que saldrá del ciclo repetitivo. Cada valor obtenido es de tipo string y para poder sumarlo necesitamos convertirlo a tipo entero, para ello usamos el método '**Parse**' de la clase 'int', tal como lo vimos en el ejercicio resuelto de la SP3.

- **Recorrer una columna de la grilla**

  De forma similar al recorrido de una fila podemos implementar el recorrido de una columna, sólo tenemos que adaptar el índice usado. Para determinar la cantidad de filas que posee la grilla usamos la propiedad '**RowCount**'.

```
int suma = 0;

int fila;

string valor;

// el ciclo for recorre las filas de la grilla

for (fila=0; fila < datagridview1. RowCount; fila++)

{

// toma los valores de la columna 0 en cada fila

valor =
datagridview1.Rows[fila].Cells[0].Value.ToString();

suma += int.Parse(valor);

}
```

Dada la similitud de la grilla con un arreglo de dos dimensiones, cualquier proceso de recorrido, ya sea de filas o columnas o de la grilla completa puede fácilmente ser replicado en la grilla controlando correctamente los valores de los índices que permiten acceder a las filas y columnas.

Muchos de los temas desarrollados en esta herramienta, deberás utilizarlas en la resolución de la actual situación profesional.

Indique verdadero o falso

1) El control DataGridView se basa en filas y columnas para mostrar información.

○ Verdadero

○ Falso

2) El control DataGridView se usa para visualizar datos, pero no permite ingresar nuevos datos.

○ Verdadero

○ Falso

3) En un control DataGridView las filas (Rows) se pueden agregar tanto en tiempo de diseño como en tiempo de ejecución.

○ Verdadero

○ Falso

4) En un control DataGridView las columnas (Columns) se pueden configurar durante el diseño para configurar su encabezado y la cantidad de columnas de la grilla.

○ Verdadero

○ Falso

Indique las opciones correctas

5) El control DataGridView se puede configurar si el usuario puede interactuar con los datos de la grilla seleccionando algunas opciones. Marque las opciones que se pueden configurar para lograr esa interacción.

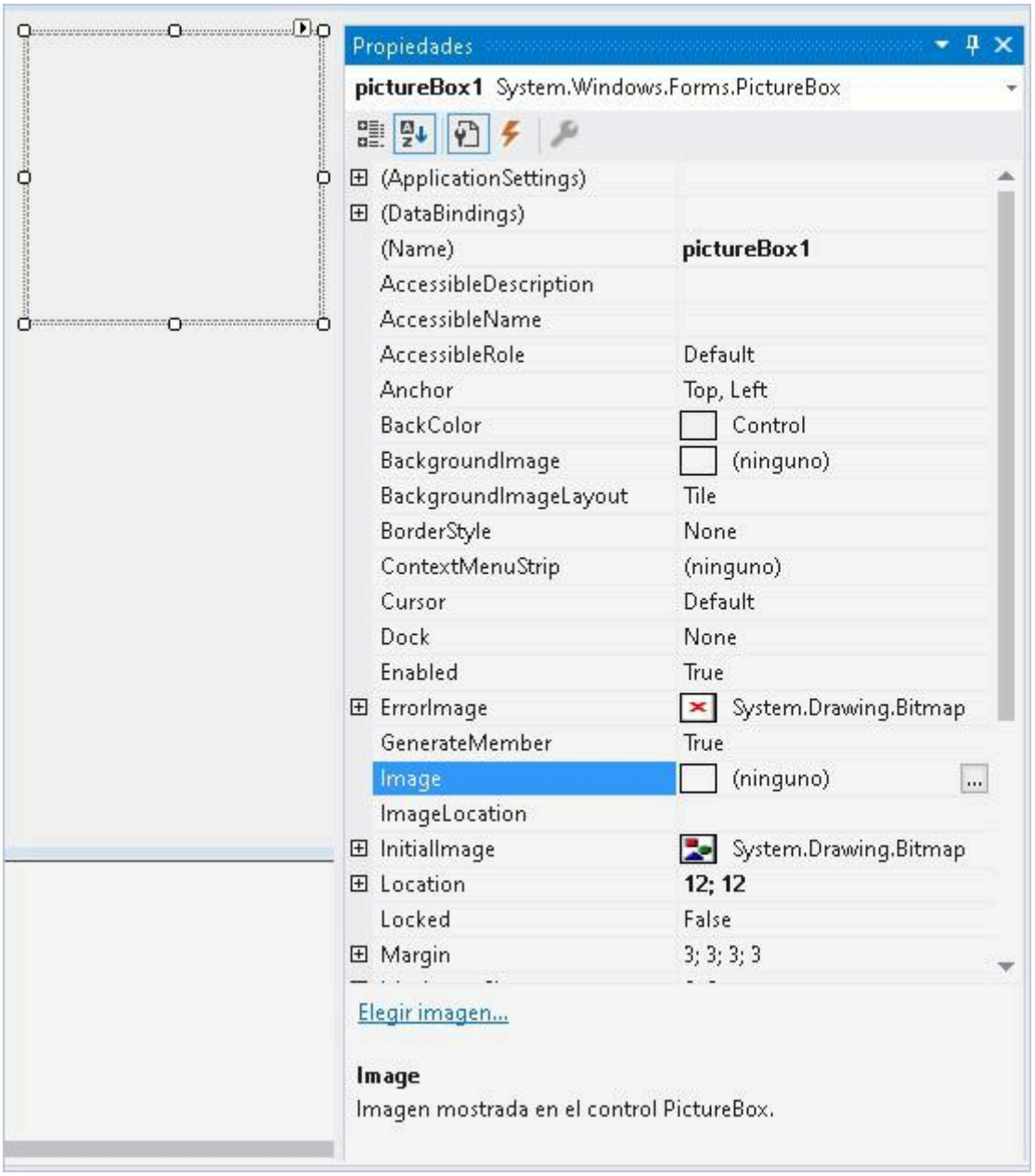
☐ Habilitar acción de agregar columnas
☐ Habilitar reordenación de filas
☐ Habilitar acción de agregar
☐ Habilitar eliminación
☐ Habilitar edición
☐ Habilitar reordenación de columnas

Respuestas correctas[46]

[46] 1)Verdadero; 2)Falso; 3)Falso; 4)Verdadero; 5)Habilitar acción de agregar/Habilitar eliminación/abilitar edición

Control PictureBox

El control PictureBox se usa para mostrar imágenes cargadas desde archivos y para generar gráficos y texto simples, como figuras geométricas, líneas, curvas, etc.



Veremos a continuación dos formas de cargar una imagen desde un archivo, el formato puede ser .jpg, .bmp, .png, etc.

- **Cargar un archivo de imagen por código**

Necesitamos tener el archivo de la imagen ubicado en el disco de la computadora donde se ejecuta la aplicación, supongamos que el archivo se llama "dibujo.jpg" y está ubicado en "C:/archivos/imagenes"

Con estás simples sentencias logramos cargar y visualizar la imagen en el control picturebox:

```
picturebox1.ImageLocation = "C://archivos/imagenes/dibujo.jpg";

picturebox1.Load();

picturebox1.SizeMode = PictureBoxSizeMode.StretchImage;
```

"**ImageLocation**" es la propiedad que almacena la ubicación y nombre del archivo

"**Load**()" es el método que realiza la lectura del archivo y transfiere la imagen al control picturebox.

"**SizeMode**" es la propiedad que determina de qué manera se ajusta el tamaño real de la imagen al tamaño del picturebox, por lo general el valor "StretchImage" es el que mejor resultado da, las otras opciones son:

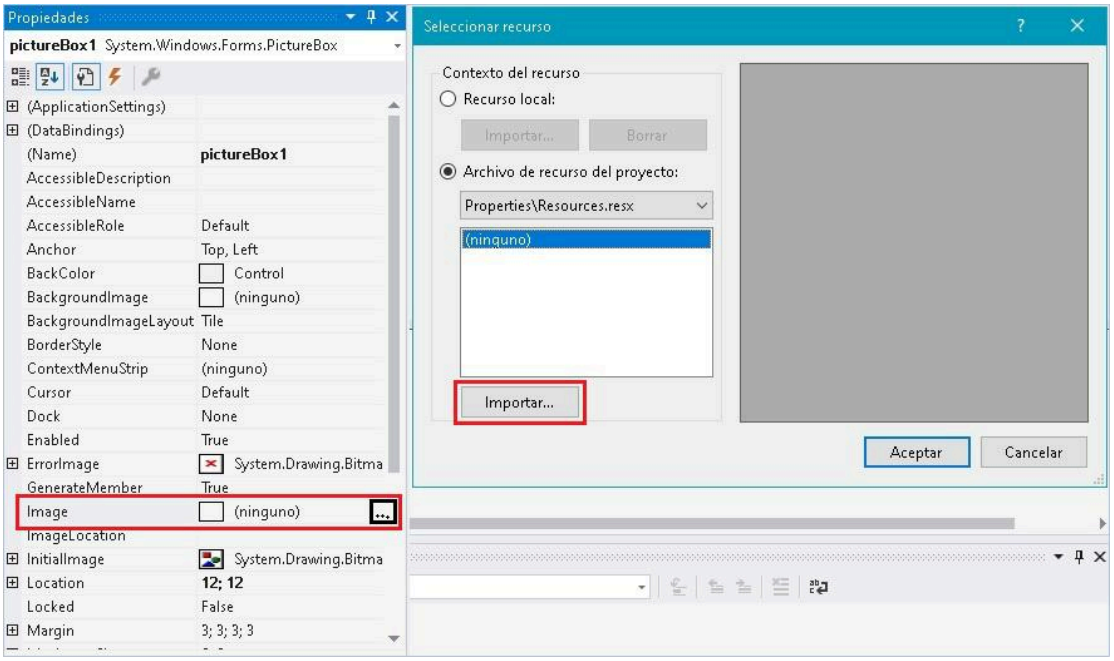
"Normal", "Autosize", "Zoom" y "CenterImage".

Esta forma de cargar la imagen nos obliga a tener el archivo disponible en la computadora donde se desee ejecutar la aplicación, además deberá tener la misma ubicación que se asigna en la propiedad "ImageLocation", en caso contrario el método "Load()" no podrá encontrar y cargar el archivo.

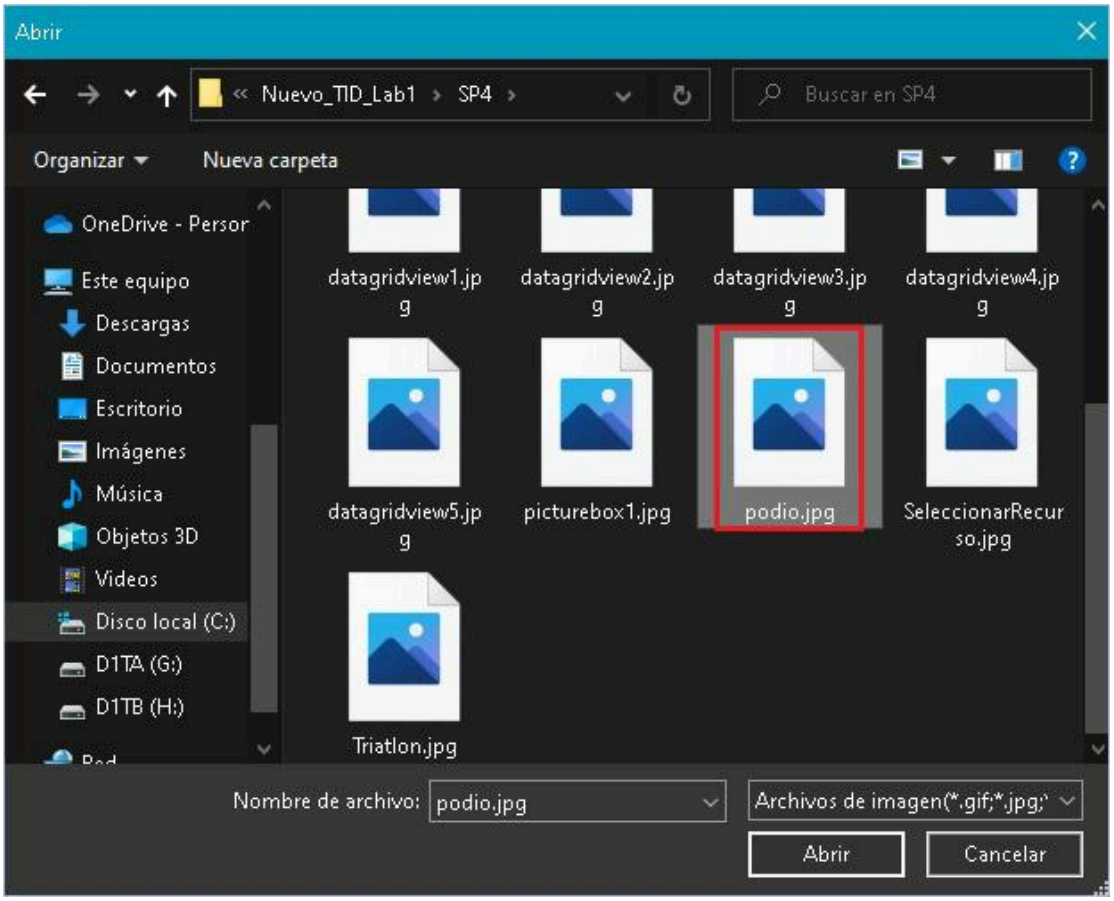
- **Cargar un archivo asignado en tiempo de diseño un recurso nuevo al proyecto**

  Esta alternativa consiste en incorporar el archivo que queremos mostrar en el picturebox al proyecto de Visual Studio en la forma de un nuevo recurso, así el archivo formará parte de nuestro ejecutable y ya no dependerá de tenerlo en la computadora que ejecute la aplicación.

  Para esto seleccionamos el control picturebox y buscamos en la ventana de propiedades la propiedad "Image":

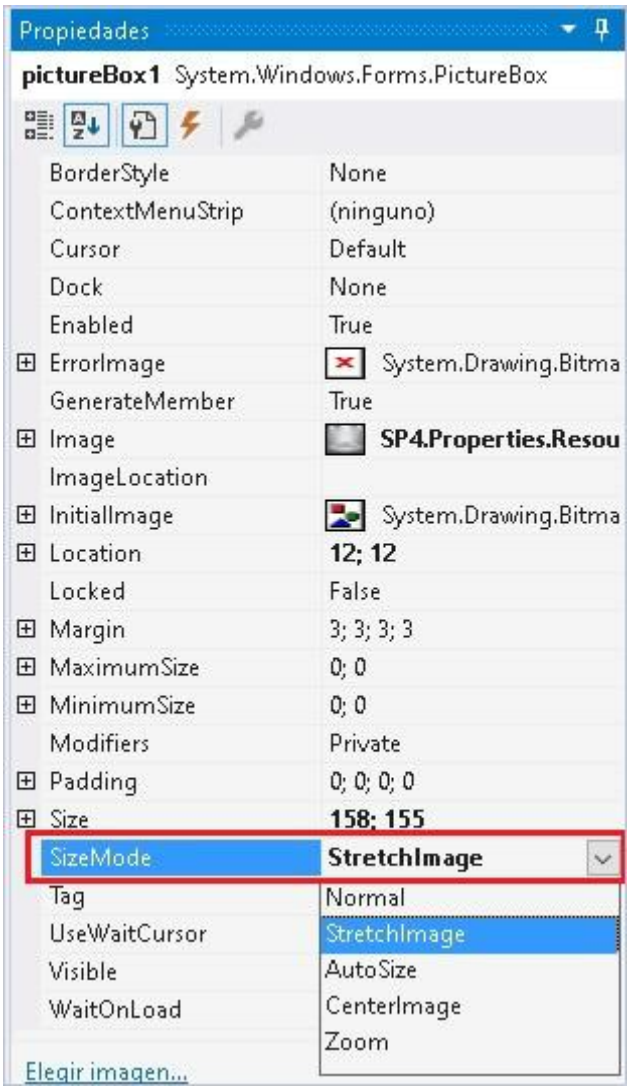


Hacemos click en el botón que figura a la derecha de la propiedad "Image" (el de los 3 puntos) y se abrirá la ventana para seleccionar un recurso, en esta ventana elegimos el botón "Importar..." para buscar nuestro archivo de imagen.



Una vez seleccionado el archivo deseado, tendremos una pre visualización del mismo en la ventana de seleccionar recurso, posiblemente la imagen no se vea completa pero eso lo podremos ajustar en los pasos siguientes. Hacemos click en el botón "Aceptar" y se cierra la ventana.

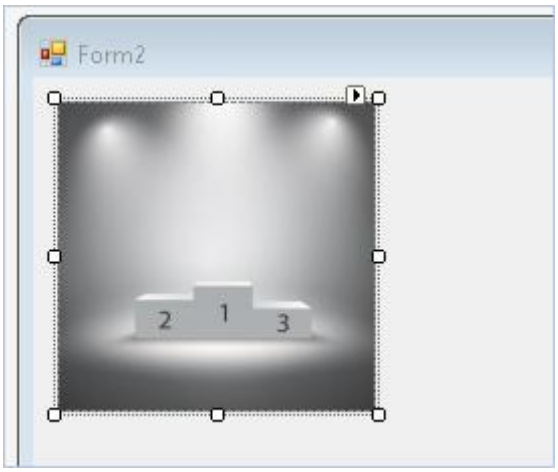
A continuación, para ajustar la visualización de la imagen en el control picturebox buscamos en la ventana de propiedades, siempre con el control picturebox como elemento seleccionado en el formulario que lo contiene, la propiedad "SizeMode":



Y de las opciones que se muestran elegimos "StretchImage", opción que trata de ajustar la imagen completa al tamaño del picturebox, en general esa opción es la que mejor resultado da, pero si no fuera así disponemos de otras opciones de

ajuste que se pueden probar, dependerá del tamaño y formato (alto y ancho) de la imagen original y de las proporciones que tenga con respecto al picturebox.

Finalmente obtenemos la imagen cargada en el control picturebox:



La ventaja de este proceso es que la imagen queda incorporada dentro del código ejecutable de nuestra aplicación y ya no necesitamos tener el archivo original disponible.

Podemos agregar varios archivos como recursos y asociarlos a diferentes controles picturebox o al mismo en distintos momentos, en ese caso usamos la propiedad "Image" para indicar el nombre del recurso que deseamos cargar en el control.

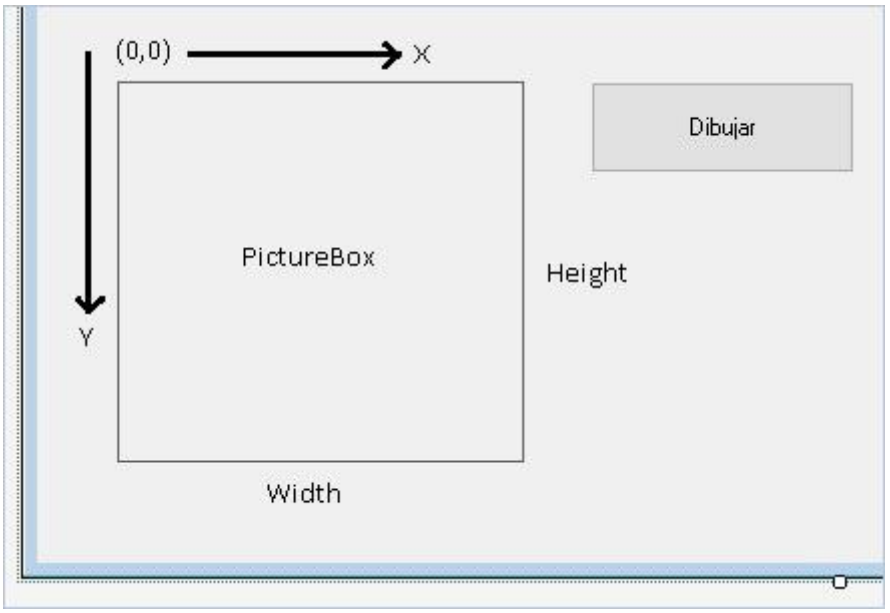
Cómo realizar dibujos simples en el control PictureBox

Otro posible uso del control picturebox es el de realizar dibujos de figuras simples, como líneas, rectángulos, círculos, polígonos y texto, pudiendo elegir colores, tipos de trazo, tipos de letra, patrones de relleno y varias características más.

Veremos algunos ejemplos de cómo realizar esto en un control PictureBox.

Nos ayudaremos del objeto "Graphics" desarrollado para la creación de gráficos en pantalla y también para generar impresiones. El objeto "Graphics" posee los métodos para dibujar líneas, rectángulos, círculos, y texto entre otros más, todos estos métodos se basan en el sistema de coordenadas X, Y, con unidades en puntos o pixels y con el origen de coordenadas en la esquina superior izquierda del picturebox, es decir la

coordenada (0,0) se encuentra arriba y a la izquierda. Los valore de X crecen en sentido horizontal hacia la derecha y los valores de Y crecen en sentido vertical hacia abajo.



Además, para obtener los valores máximos de las coordenadas, disponemos de las propiedades "Width" (ancho) y "Heigth" (alto) con las cuáles podremos controlar la ubicación de los dibujos sobre el picturebox.

Creación del objeto Graphics

Como lo comentamos en el párrafo anterior necesitamos un objeto de tipo Graphics para ejecutar los métodos de dibujo, y además necesitamos que ese objeto esté asociado al control picturebox que servirá como contenedor de los dibujos, la creación de objeto Graphics se realiza en este caso con el método "CreateGraphics()" del picturebox:

```
Graphics graf = picturebox1.CreateGraphics();
```

A partir de ahora "graf", el objeto que hemos creado, será capaz de ejecutar los métodos de dibujo que se verán reflejados en el contenido del control picturebox1.

Dibujar líneas

Para dibujar una línea necesitamos especificar el tipo y color del trazo (objeto Pen), las coordenadas del punto inicial y las coordenadas del punto final, se usa el método "**DrawLine**()" con los siguientes parámetros:

```
DrawLine( Pen, int x1, int y1, int x2, int y2);
```

Ejemplos:

- **A**- Dibujar una línea horizontal de color negro, desde la coordenada (0,0), que sería la esquina superior izquierda del picturebox, hasta la coordenada (100,0)

  ```
  graf.DrawLine( Pens.Black, 0, 0, 100, 0);
  ```

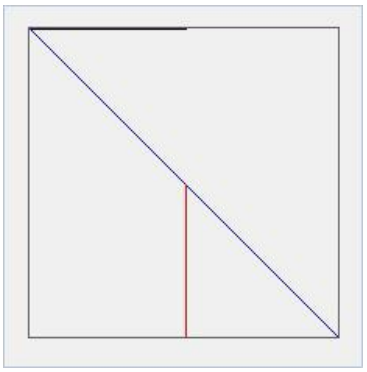
- **B**- Dibujar una línea de color azul, en diagonal desde la esquina superior izquierda (0,0) hasta la esquina inferior derecha. Los valores de la coordenada inferior derecha dependen del tamaño del picturebox, es decir, de su ancho y alto, valores que podemos obtener de las propiedades Width y Heigth:

  ```
  graf.DrawLine( Pens.Blue, 0, 0, picturebox1.Width,
  picturebox1.Heigth);
  ```

- **C**- Dibujar una línea vertical de color rojo desde el centro del picturebox hasta el borde inferior, acá también podemos usar la propiedad Width y Height para calcular el centro:

  ```
  graf.DrawLine( Pens.Red, picturebox1.Width/2,
  picturebox1.Heigth/2, picturebox1.Width/2,
  picturebox1.Width);
  ```

  PictureBox con las 3 líneas dibujadas (A, B, y C):

  El picturebox del ejemplo tiene dimensiones de 200 x 200 puntos.

Dibujar Texto

Para dibujar un texto usaremos el método "DrawString()" y necesitamos, además del texto a dibujar, un objeto "Font" que será el tipo de letra, con sus propiedades, un objeto "Brush" que sería el pincel para definir el color y relleno de las letras y las coordenadas x, y a partir de las cuáles se dibuja el texto.

```
DrawString( "Texto", Font, Brush, x, y);
```

Ejemplos

- A- Dibujar el texto "Programación .Net" en la coordenada (10,10) del picturebox, usar fuente Arial de 12 puntos y pincel de color azul.

```
graf.DrawString("Programación .Net", new Font("Arial",
12), Brushes.Blue, 10, 10);
```

El objeto "Font" se crea en el mismo momento de ejecutar el método DrawString o puede crearse antes y asignar a un nombre de objeto, al crear el objeto "Font" especificamos el nombre de la fuente y el alto en puntos. El pincel se toma de la enumeración "Brushes" que ya tiene muchos valores predefinidos.

La coordenada especificada, en este caso (10, 10) es el punto que queda en la esquina superior izquierda del rectángulo que contiene al texto dibujado: el rectángulo se muestra sólo para aclarar desde donde se comienza a dibujar el texto en referencia a los valores x, y de las coordenadas indicadas.



- B- Dibujar el texto "Programación .Net" con fuente "Courier" de tamaño 10 y de color rojo, el texto debe quedar centrado horizontal y verticalmente en el picturebox:

  Para lograr centrar el texto debemos agregar una especificación de formato en el texto, lo cual se puede hacer con un objeto "StringFormat" (formato de texto) de la siguiente manera, primero creamos el formato y asignamos sus propiedades para logar un texto centrado y luego dibujamos el texto agregando el formato previamente creado:

```
StringFormat formato = new StringFormat(); // objeto para definir el formato del texto

formato.Alignment = StringAlignment.Center; // alineado centrado en horizontal

formato.LineAlignment = StringAlignment.Center; // alineado centrado en vertical

graf.DrawString("Programación .Net", // se dibuja el texto con el formato creado

new Font("Courier", 10),

Brushes.Red,

picturebox1.Width /2, picturebox1.Height /2,

formato);
```



Otros métodos del objeto Graphics

El objeto Graphics posee varios métodos más para dibujar figuras y rellenos, por ejemplo:

```
DrawRectangle(), DrawArc(), DrawEllipse(), DrawLines(),
DrawPie(), DrawPolygon(), FillRectangle(), FillEllipse(),
FillPie(), FillPolygon().
```

Recomendamos investigar algunos de estos métodos con los cuáles podrá enriquecer el desarrollo de interfaces gráficas en sus aplicaciones de .Net.

Clase Graphics

Estas herramientas deberán ser utilizadas en la resolución de la situación profesional.



Link del video: https://www.youtube.com/watch?v=X13N4QpnJ9k

Texto del video:

En esta situación profesional profundizamos el uso de arreglos, trabajamos con arreglos de 2 dimensiones, algo fundamental para gran variedad de procesos de datos que se presentan en muchos desarrollos.

También aprendimos cómo usar el control de Grilla para mostrar información de forma más eficiente para el usuario final y estudiamos una introducción al uso del control PictureBox para manejar archivos gráficos de diferentes formatos y agregar en ellos elementos básicos como texto, líneas, recuadros, etc.

Indique verdadero o falso

1) El control PictureBox permite mostrar imágenes de archivos y también crear dibujos de figuras simples.

○ Verdadero

○ Falso

2) Para poder cargar la imagen desde un archivo, éste debe estar ubicado siempre en el mismo lugar que ocupa el ejecutable de la aplicación.

○ Verdadero

○ Falso

3) El picturebox puede mostrar una imagen tomada desde un recurso agregado al proyecto.

○ Verdadero

○ Falso

4) Para realizar dibujos sobre el control picturebox el sistema de coordenadas x, y tiene su origen en la esquina inferior izquierda del control.

○ Verdadero

○ Falso

Indique la opción correcta

5) La propiedad que almacena el nombre del archivo que se mostrará en el picturebox es:

○ ImageName

○ ImageLocation

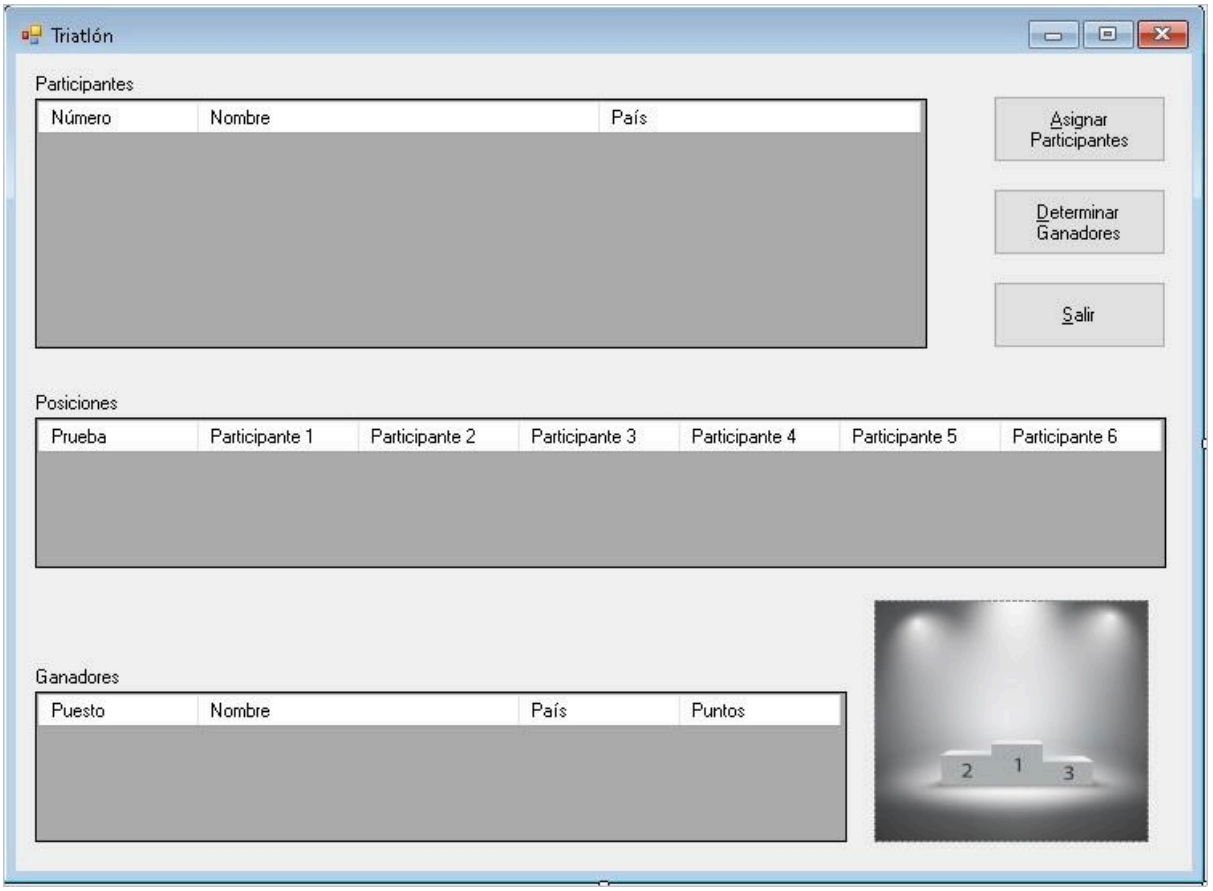
○ ImageSource

○ Ninguna de las anteriores

Respuestas correctas[47]

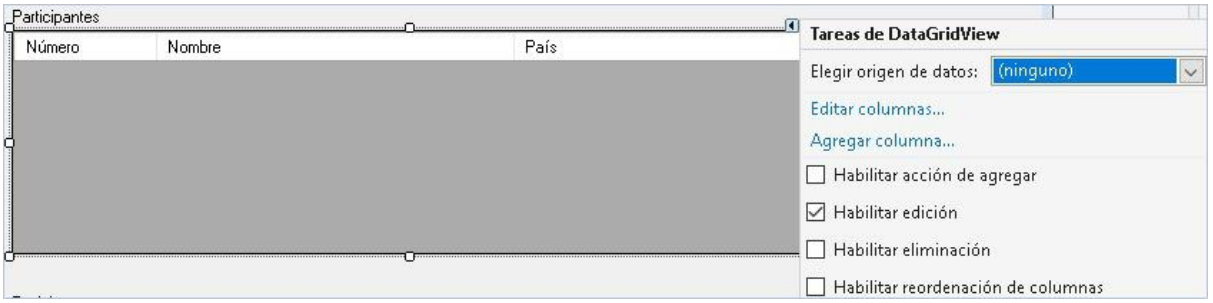
[47] 1)Verdadero; 2)Falso; 3)Verdadero; 4)Falso; 5)ImageLocation.

Para resolver la situación profesional tendremos que crear un nuevo proyecto en Visual Studio y trabajar en el diseño del formulario de la aplicación.



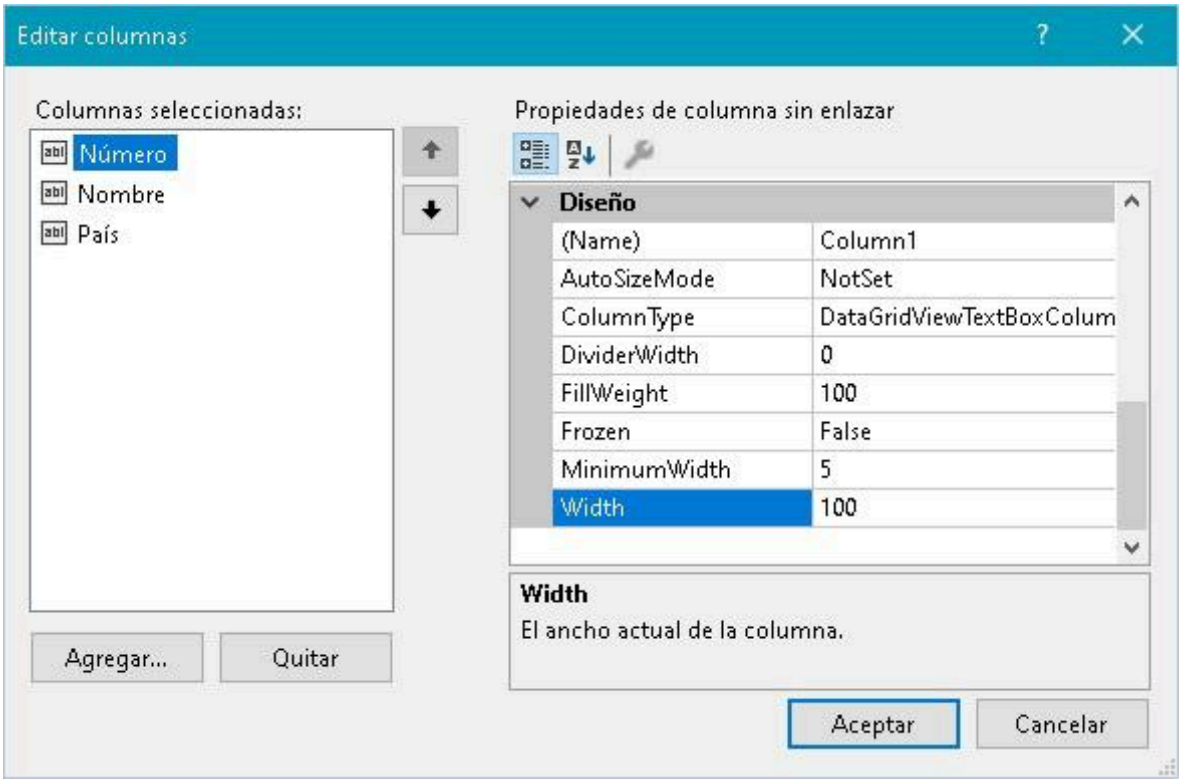
Colocaremos tres controles de tipo DataGridView y les daremos los nombres "dgvParticipantes", "dgvPosiciones" y "dgvGanadores" siguiendo el orden que se ve en el diseño del formulario. También agregamos tres controles "Button" (btnAsignarParticipantes, btnDeterminarGanadores y btnSalir) y un control PictureBox (picGanadores).

Las tres grillas se deben configurar con la cantidad de columnas y texto de cabecera que se muestra en el diseño, para la grilla dgvParticipantes tenemos seleccionada sólo la opción de "Habilitar edición" ya que los datos de los participantes se cargarán directamente sobre las celdas de la grilla:

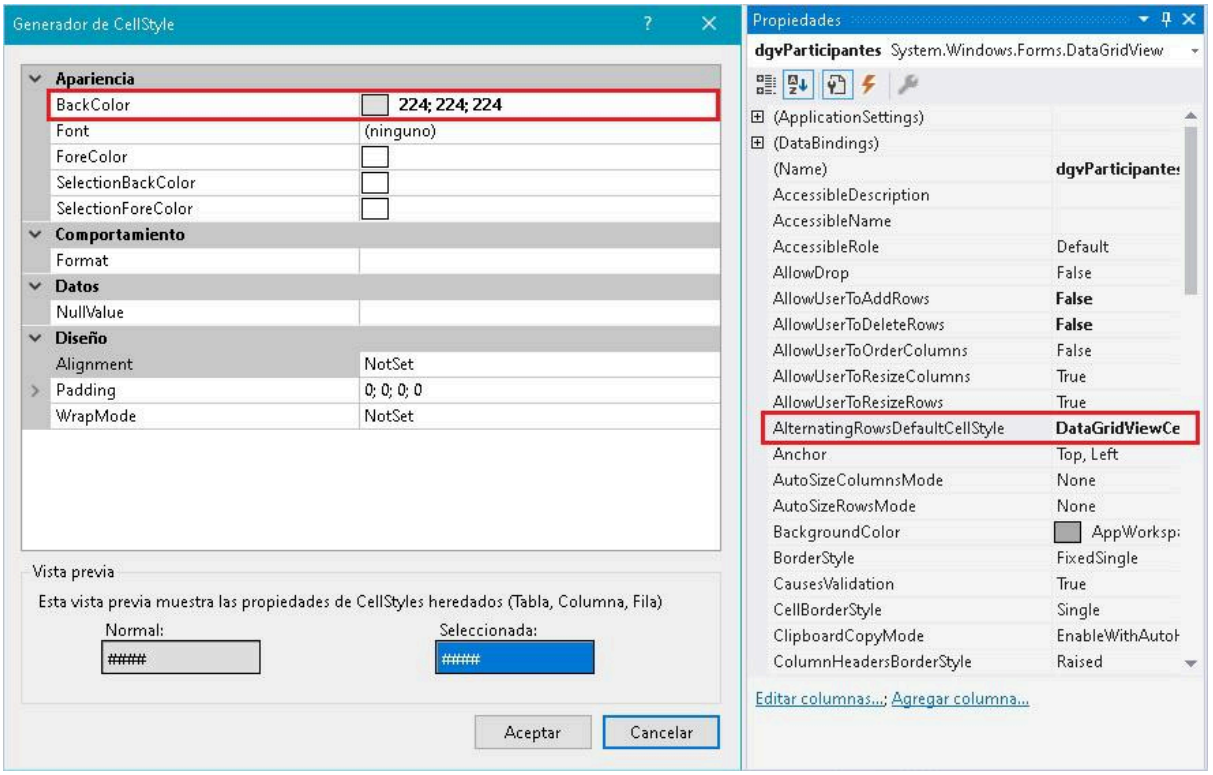


La grilla de posiciones (dgvPosiciones) también tendrá la misma configuración para que el usuario de la aplicación pueda escribir sobre las celdas de la grilla, en cambio la grilla dgvGanadores tendrá todas las opciones de edición deshabilitadas ya que en el caso de los ganadores el usuario no debe poder modificar los datos mostrados por la aplicación.

En cada grilla se deberán editar las columnas para darles un ancho adecuado (ver propiedad Width), por ejemplo, para la grilla de los participantes tenemos las columnas Número de ancho 100, Nombre de 250 y País de 200.



Otra propiedad que podemos modificar es 'AlternatingRowsDefaultCellsStyle' que nos permite asignar un color de fondo distinto al color normal de la grilla, que por defecto es blanco, y así tener colores alternados ente una fila y la siguiente:



En este caso se asigna un tono de gris para el color de fondo en las filas alternadas.

Con respecto al control PictureBox (picGanadores) se debe configurar con un archivo agregado como recurso del proyecto, por ejemplo, con esta imagen (descarga el archivo del podio en formato zip). Revisar el procedimiento indicado en la Herramienta 3 de esta SP.

Una vez configurados todos los componentes de la interfaz presentes en el formulario comenzaremos a desarrollar el código de la aplicación para obtener la funcionalidad solicitada.

La secuencia de uso de la aplicación está prevista para:

- Permitir la carga de los datos de cada participante del Triatlón, nombre y nacionalidad de cada uno de los seis finalistas en la primera grilla del formulario.
- Trasladar los nombres de los participantes ingresados a los encabezados de columna de la segunda grilla y configurar sus filas para cada prueba, esto se realiza con el botón "Asignar Participantes", luego permitir que el usuario cargue los datos de las posiciones ocupadas en cada una de las tres pruebas.
- Con el botón "Determinar Ganadores" la aplicación debe validar los datos ingresados en la grilla de posiciones y si los mismos son correctos determinará los tres ganadores con los mejores puntajes obtenidos en las tres pruebas del

Triatlón, los datos de los ganadores se mostrarán en la tercera grilla del formulario y además sus nombres se agregarán al picturebox del podio. Si la validación de datos detecta algún error, la aplicación debe mostrar un mensaje de error y no se calcularán los ganadores.

- El botón "Salir" cerrará el formulario y finalizará la aplicación.

Para almacenar las posiciones de los participantes en cada prueba usaremos un arreglo bidimensional de tipo entero y para los datos de los ganadores un arreglo unidimensional basado en una estructura.

Iniciamos entonces el código desde el evento Load() del formulario:

C #: Evento Load SP4

```
private void Form1_Load(object sender, EventArgs e)
  {
      // inicializar formulario
      int i;
      // agregar 6 filas a la grilla de participantes
      for (i = 1; i <= 6; i++)
      {
          // asignar en cada fila sólo el número de participante (i)
          dgvParticipantes.Rows.Add(i.ToString());
      }
      // configurar la primera columna para que sea de sólo lectura
      dgvParticipantes.Columns[0].ReadOnly = true;
      // deshabilitar la grilla de posiciones
      dgvPosiciones.Enabled = false;
  }
```

En el evento Load se carga la grilla de participantes con 6 filas que contendrán inicialmente los números de participante del 1 al 6 en la primera columna de la grilla, quedando el resto de las celdas en blanco para que el usuario complete los datos de los nombres y nacionalidades de cada participante. Para evitar que los números de participante se puedan modificar se asigna la propiedad "ReadOnly" (sólo lectura) a la columna cero de la grilla. Finalmente, la grilla de Posiciones se deshabilita para que no pueda ser usada hasta que todos los participantes estén asignados.

Botón "Asignar Participantes":

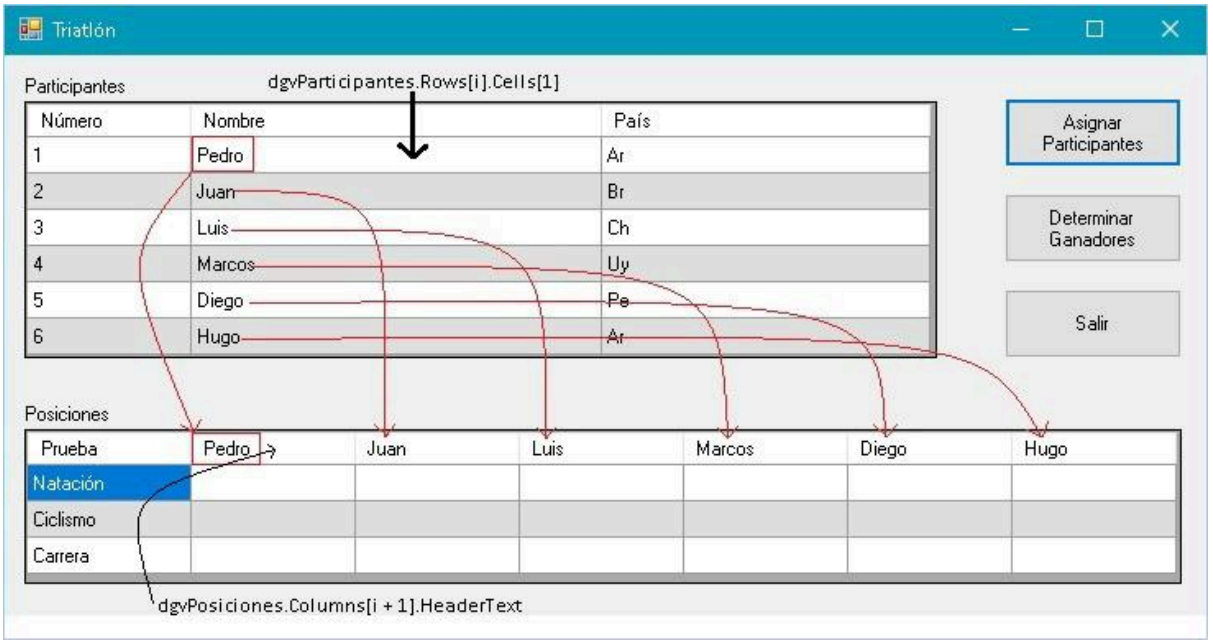
C #: Asignar Participantes

```
    private void btnAsignarParticipantes_Click(object sender, EventArgs e)
      {   int i;
          dgvPosiciones.Rows.Clear();
          // verificar que estén cargados todos los participantes
          for (i = 0; i < 6; i++)
          {
              if (dgvParticipantes.Rows[i].Cells[1].Value == null ||
                  dgvParticipantes.Rows[i].Cells[2].Value == null)
              {
                  MessageBox.Show("Debe completar los datos de todos los
participantes",
                      "Error", MessageBoxButtons.OK);
                  return;
              }
          }
          // configurar las columnas de la grilla
          for (i = 0; i < 6; i++)
          {
              dgvPosiciones.Columns[i + 1].HeaderText =
                      dgvParticipantes.Rows[i].Cells[1].Value.ToString();
          }
          // configurar las filas con las 3 pruebas del Triatlón
          for (i = 0; i < 3; i++)
          {
              dgvPosiciones.Rows.Add();
          }
          dgvPosiciones.Rows[0].Cells[0].Value = "Natación";
          dgvPosiciones.Rows[1].Cells[0].Value = "Ciclismo";
          dgvPosiciones.Rows[2].Cells[0].Value = "Carrera";
          dgvPosiciones.Columns[0].ReadOnly = true;
          dgvParticipantes.ClearSelection();
          dgvPosiciones.Enabled = true;
      }
```

En el evento Click del botón "Asignar Participantes" primero se controla que la columna 1, la que contiene los nombres de los participantes no tenga valores nulos, si eso ocurre se muestra el mensaje de error y no se prosigue con la asignación de los participantes.

Si los datos son correctos, se configuran las columnas de la grilla de Posiciones asignado en sus encabezados, por medio de la propiedad "HeaderText", los nombres de

los participantes que se toman de la primera grilla, las asignaciones comienzan en la columna 1 porque la 0 tendrá los nombres de las pruebas que componen el triatlón.



Seguidamente se agregan 3 filas vacías a la grilla de Posiciones y finalmente se asignan los nombres de las pruebas en la columna 0 de las 3 filas agregadas.

De forma similar a lo realizado con la grilla de Participantes, acá también se deja la columna 0 en modo sólo lectura poniendo "true" en la propiedad "ReadOnly" de la columna cero.

El método "ClearSelection" se encarga de borrar la marca de la última celda editada en la grilla de Participantes, es sólo para que visualmente resulte más limpia la interfaz gráfica.

Por último, se habilita la grilla de Posiciones para que el usuario pueda ingresar los datos de cada prueba.

Continuamos el desarrollo del código ahora con el evento Click del botón "Determinar Ganadores", las acciones que deben considerarse en este evento son:

- Realizar una validación de los datos cargados en la grilla de posiciones.
- Copiar los datos de las posiciones en cada prueba a un arreglo bidimensional para poder hacer los cálculos.
- Calcular los puntos totales obtenidos por cada uno de los 6 participantes.

- Determinar los 3 mejores puntajes.
- Mostrar los datos de los 3 ganadores en la tercera grilla.
- Escribir los nombres de los 3 ganadores en el picturebox del podio.

Como son varias acciones consecutivas las iremos desarrollando una por una y en la medida de lo posible implementaremos funciones para resolver algunas de ellas y que no quede un código muy extenso con todo el proceso en un único bloque.

En primer lugar, agregaremos la definición de una estructura de datos que nos facilite mantener la información que necesitamos de cada participante.

C #: Estructura de Datos

```
public partial class Form1 : Form
   {
       public struct DATOS
       {
           public int participante;
           public int puntos;
       }
```

Escribimos entonces la estructura "DATOS" compuesta por 2 campos: "participante" y "puntos", el primer campo guardará el número de participante y el segundo campo guardará el total de puntos alcanzado en las 3 pruebas del triatlón.

Esta estructura se declara al comienzo del código del formulario con alcance público.

Evento Click:

C #: Determinar Ganadores

```
private void btnDeterminarGanadores_Click(object sender, EventArgs e)
{
   int fila;
   int col;
   int i;
   string valor;
   string nombre;
   string nacionalidad;
   // controlar los datos ingresados en la grilla de posiciones
   if(!ValidarPosiciones())
   {
```

```
        MessageBox.Show("Los datos son incorrectos, revise las posiciones asignadas",
            "Error", MessageBoxButtons.OK);
        return;
    }
    dgvPosiciones.ClearSelection();
    int[,] puestos = new int[3, 6]; // arreglo bidimensional
    // copiar el contenido de la grilla al arreglo bidimensional
    for (fila = 0; fila < 3; fila++) // se recorren las filas de la grilla
    {
        for (col = 0; col < 6; col++) // se recorren las columnas de la grilla
        {
            // obtener el valor de cada celda
            valor = dgvPosiciones.Rows[fila].Cells[col + 1].Value.ToString();
            puestos[fila, col] = int.Parse(valor); // convertirlo a entero
        }
    }
```

En esta primera parte se invoca a la función "ValidarPosiciones()" para controlar los datos que se ingresaron en la grilla de posiciones, si hay algún error se muestra un cuadro de mensaje y finaliza el evento. Si los datos son correctos la ejecución continúa creando el arreglo bidimensional "puestos" y copiando en él los datos de las posiciones cargadas en la grilla.

La funcion "ValidarPosiciones()" se explica más adelante.

El proceso continúa así:

C #: Arreglo de DATOS

```
// inicializar el arreglo para computar los puntos de cada participante
 DATOS[] datos = new DATOS[6];
 for(i=0; i<6; i++)
 {
     datos[i].participante = i + 1;  // número de participante
     datos[i].puntos = 0; // cantidad de puntos del participante
 }
 // recorrer el arreglo bidimensional de los puestos y sumar los puntos
 for (col = 0; col < 6; col++) // las columnas son los participantes
 {
     // las filas son las posiciones obtenidas por cada participante
     for (fila = 0; fila < 3; fila++)
     {
         datos[col].puntos += ObtenerPuntos(puestos[fila, col]);
```

```
    }
}
```

Procedemos a crear un arreglo de 6 elementos usando la estructura "DATOS" definida previamente, en éste arreglo tendremos inicialmente los números de participantes (del 1 al 6) y el campo "puntos" inicializado en cero. Luego procederemos a obtener los puntos que el participante obtiene de acuerdo a la posición alcanzada en cada una de las 3 pruebas del triatlón que quedarán almacenadas en ese campo "puntos". Para el cálculo de los puntos que corresponden a cada puesto usaremos la función "ObtenerPuntos()" que veremos más adelante.

Al finalizar el recorrido del arreglo "puestos" tendremos los puntos totales de cada participante en "datos".

Como dijimos en el paso anterior, el arreglo "datos" tiene los puntos totales de cada uno de los 6 participantes del triatlón, debemos obtener los 3 mejores puntajes para determinar el primer puesto, segundo y tercero, para hacer eso se pueden aplicar diferentes soluciones, por ejemplo, podemos buscar el mayor puntaje de los 6 valores obtenidos y tendríamos el primer puesto, luego repetimos el proceso pero descartando de la búsqueda el mayor valor, con lo que obtendríamos el segundo lugar y así lo mismo para encontrar el tercer puesto.

C #: Buscar Ganadores

```
// arreglo de tipo DATOS para los ganadores
DATOS[] ganadores = new DATOS[3];
// obtener el primer puesto
ganadores[0] = BuscarMayorPuntaje(datos, ganadores);
// obtener el segundo puesto
ganadores[1] = BuscarMayorPuntaje(datos, ganadores);
// obtener el tercer puesto
ganadores[2] = BuscarMayorPuntaje(datos, ganadores);
```

Empleamos entonces un arreglo de 3 elementos para guardar los valores de 'participante' y 'puntos' de los 3 ganadores, el arreglo se llama 'ganadores' y en él asignaremos los valores que nos devuelve la función 'BuscarMayorPuntaje()', la función

recibe 2 parámetros: el arreglo 'datos' con los puntos totales de los 6 participantes y el mismo arreglo 'ganadores' que nos servirá de referencia para encontrar el primer puesto, segundo o tercero. ¿Cómo se determina ese valor tope ?, la idea es la siguiente: el mayor puntaje posible es 3000 ya que la primera posición en cada prueba otorga 1000 puntos y tenemos 3 pruebas (natación, ciclismo y carrera), entonces a la función le pedimos que busque en el arreglo 'datos' el mayor valor, pero que a su vez el participante que tiene ese puntaje no esté ya cargado en el arreglo de 'ganadores', en éste primer caso 'ganadores' está vacío, encontrará por lo tanto, el mayor puntaje de todos.

Supongamos que ese mayor valor sea del participante 3 con 2920 puntos, que corresponde a 2 pruebas ganadas y un segundo puesto en la prueba restante, para encontrar el segundo ganador invocamos a la función 'BuscarMayorPuntaje' con 'datos' y con el arreglo de 'ganadores' ahora cargado con los datos del participante 3, de esa forma la función obtendrá el mayor valor del arreglo pero que sea menor o igual a 2920 y que no corresponda al participante 3, así descartará de la búsqueda el primer puesto y obtendrá el segundo mayor valor.

Consideremos este ejemplo para explicar mejor el proceso:

Participante	Puntos
1	2490
2	1950
3	2920
4	1770
5	2750
6	2220

Cómo se puede observar el primer lugar le corresponde al participante 3, que obtuvo 2920 puntos, esos datos son los que retorna la primera invocación a la función 'BuscarMayorPuntaje(datos, ganadores)', valores que son asignados al elemento ganadores[0], para la segunda invocación tendremos los datos del participante 3 ya cargados en 'ganadores':

'BuscarMayorPuntaje(datos, ganadores)', los valores que retornará esta invocación son los del participante 5, con 2750 puntos, esos valores quedarán en 'ganadores[1]' y corresponden al segundo ganador.

Para obtener el tercer lugar volvemos a invocar a la función 'BuscarMayorPuntaje' pero ahora los parámetros serán:

'BuscarMayorPuntaje(datos, ganadores)' con 'ganadores' ahora cargado con los participantes 3 y 5, con lo que volverá a buscar en el arreglo 'datos' el mayor valor pero que a su vez no corresponda a los participantes 3 ni 5, el valor devuelto será el del participante 1 con 2490 puntos que se guarda en 'ganadores[2]' resultando ser el tercer ganador.

Con esto ya contamos con los datos de los 3 ganadores del triatlón, tenemos los participantes y los puntos obtenidos en el arreglo 'ganadores', nos resta cargar la tercera grilla, 'dgvGanadores', y luego agregar sus nombres en el picturebox del podio. Veamos entonces cómo cargar la grilla de ganadores:

C #: Grilla de Ganadores

```
// cargar la grilla de ganadores
 dgvGanadores.Rows.Clear();
 for (fila = 0; fila < 3; fila++)
 {
     dgvGanadores.Rows.Add(); // se agrega una nueva fila
     dgvGanadores.Rows[fila].Cells[0].Value = fila + 1; // posición final
     // obtener la fila que ocupa el participante en la grilla 1
     int f = ganadores[fila].participante - 1;
     // obtener el nombre del participante
     nombre = dgvParticipantes.Rows[f].Cells[1].Value.ToString();
     dgvGanadores.Rows[fila].Cells[1].Value = nombre;
     // obtener la nacionalidad del participante
     nacionalidad = dgvParticipantes.Rows[f].Cells[2].Value.ToString();
     dgvGanadores.Rows[fila].Cells[2].Value = nacionalidad;
     // puntos obtenidos por el participante
     dgvGanadores.Rows[fila].Cells[3].Value = ganadores[fila].puntos.ToString();
 }
 dgvGanadores.ClearSelection();
```

Hacemos un ciclo repetitivo con la variable 'fila' que usaremos como índice para acceder a los arreglos 'ganadores' y 'datos', además nos servirá para indicar sobre qué fila de la grilla estaremos asignado los valores.

El ciclo 'for' itera desde 0 hasta 3 porque son 3 filas las que debemos agregar a la grilla de ganadores, el primer paso es agregar una nueva fila vacía: 'dgvGanadores.Rows.Add()', luego asignamos los 4 valores que contiene cada fila.

En la columna 0 de la grilla se muestra la posición final de participante ganador, 1, 2 o 3

En la columna 1 el nombre del participante que se obtiene de la primera grilla, donde se cargaron los datos de los 6 participantes, para acceder a ese valor primero debemos saber en qué fila de la grilla de participantes está ubicado el participante que nos interesa, y ese número de fila lo podemos obtener con el número del participante menos uno. El resultado queda en la variable 'f' con la que finalmente accedemos a la grilla de participantes.

En la columna 2 la nacionalidad del participante, valor que también recuperamos de la grilla de participantes.

En la columna 3 los puntos obtenidos por el participante.

Finalmente con la función 'EscribirGanadores()' agregamos los nombres a la imagen del podio que muestra el picturebox:

C #: Escribir Ganadores

```
    // agregar al picturebox los nombres de los ganadores
    EscribirGanadores();
}
```

Y así completamos el código del evento Click en el botón 'DeterminarGanadores'.

Detallamos a continuación las funciones complementarias implementadas:

- **Validación de datos**

  Este proceso lo implementamos con una función que retorne un valor booleano, verdadero si los datos ingresados en la grilla de posiciones son correctos y falso

en caso contrario. Los datos son correctos si los valores ingresados para cada prueba (cada fila de la grilla de Posiciones) están comprendidos entre 1 y 6 y además no hay ningún valor repetido.

C #: funcio´n ValidarPosiciones

```
private bool ValidarPosiciones()
{
   int f;
   int c;
   int i;
   int valor;
   bool resultado = true;
   for(f=0; f < dgvPosiciones.RowCount; f++) // recorre las filas de la grilla
   {
       for(c=1; c < dgvPosiciones.ColumnCount; c++) // recorre las columnas de la grilla
       {
           valor = int.Parse(dgvPosiciones.Rows[f].Cells[c].Value.ToString());
           if(valor < 1 || valor > 6) // deben ser valores entre 1 y 6
           {
               resultado = false;
               break;
           }
           // verificar que el valor de esa celda no esté repetido
           // en las siguientes posiciones de la fila
           for(i=c+1; i < dgvPosiciones.ColumnCount; i++)
           {
               if (valor == int.Parse(dgvPosiciones.Rows[f].Cells[i].Value.ToString()))
               {
                   resultado = false;
                   break;
               }
           }
       }
   }
   return resultado;
}
```

Para realizar la validación hay que recorrer la grilla fila por fila y de cada fila recorrer las columnas para ir tomando cada uno de los valores ingresados por el usuario. El valor de la celda se compara con 1 y 6 para verificar que no esté fuera de ese rango y luego se compara con el resto de valores sobre la misma fila para verificar que no esté repetido. Si alguna de estas condiciones no se

cumple la función retornará el valor falso, en caso contrario al completar el recorrido de las tres filas de la grilla y todo está correcto el valor retornado será verdadero.

- **Obtener Puntos**

  La función 'ObtenerPuntos()' nos permite asignar el puntaje total a cada participante de acuerdo a las posiciones obtenidas en cada una de las 3 pruebas del triatlón.

C #: Obtener Puntos

```
private int ObtenerPuntos(int puesto)
{
    // arreglo con los puntos por el puesto en cada prueba:
    // puesto 1: 1000 puntos
    // puesto 2: 920 puntos
    // puesto 3: 830 puntos
    // puesto 4: 740 puntos
    // puesto 5: 650 puntos
    // puesto 6: 560 puntos
    int[] puntos = new int[6] {1000, 920, 830, 740, 650, 560 };
    // se devuele el valor del elemento puesto-1
    return puntos[puesto-1];
}
```

La función recibe por parámetro el puesto obtenido por el participante en una prueba, y devuelve la cantidad de puntos que le corresponden de acuerdo a los valores definidos en el arreglo 'puntos'.

- **Obtener los 3 Ganadores**

  La función 'BuscarMayorPuntaje()' recibe por parámetro el arreglo 'datos' (d) ya cargado con los puntos totales de los 6 participantes y el arreglo 'ganadores' (g) inicialmente vacío. Podemos resumirlo como: buscar el mayor puntaje pero que no esté cargado en el arreglo de 'ganadores'.

C #: Buscar Mayor Puntaje

```
private DATOS BuscarMayorPuntaje(DATOS[] d, DATOS[] g)
  {
```

```
        DATOS resultado;
        int i;
        int f;
        int ganadorPrevio;
        resultado.participante = 0;
        resultado.puntos = 0;
        for(i=0; i<6; i++)
        {
            if(d[i].puntos > resultado.puntos)
            {
                ganadorPrevio = 0;
                for(f=0; f < g.Length; f++ ) // recorre 'g' (ganadores)
                {
                    if(d[i].participante == g[f].participante )
                    {
                        ganadorPrevio = 1;
                    }
                }
                if (ganadorPrevio == 0)
                {
                    resultado.puntos = d[i].puntos;
                    resultado.participante = d[i].participante;
                }
            }
        }
        return resultado;
    }
```

Este proceso recorre el arreglo 'datos' y va comparando el valor del campo 'puntos' para ir guardando en la variable 'resultado' el mayor valor que va encontrando, además considera que el valor del participante que compara no esté ya cargado en el arreglo de 'ganadores', eso permite encontrar el mayor puntaje de todo el arreglo, el segundo mayor y el tercer mayor invocando a la función con distinto contenido en el arreglo 'ganadores'.

Escribir los ganadores en el PictureBox

La función 'EscribirGanadores()' toma los nombres de los 3 participantes de la grilla de ganadores y los dibuja sobre el picturebox 'picGanadores', se usa el objeto Graphics y se aplica el formato centrado a cada nombre para facilitar su ubicación en las coordenadas x, y del picturebox. Se usa letra "Arial" de 9 puntos y color azul.

C #: escribir Ganadores

```
private void EscribirGanadores()
{
    int ancho = picGanadores.Width;
    int alto = picGanadores.Height;
    StringFormat formato = new StringFormat();
    formato.Alignment = StringAlignment.Center;
    formato.LineAlignment = StringAlignment.Center;
    // agregar los nombre de los ganadores al gráfico del podio
    Graphics gra = picGanadores.CreateGraphics();
    string nombre = dgvGanadores.Rows[0].Cells[1].Value.ToString();
    gra.DrawString(nombre, new Font("Arial", 9), Brushes.Blue,
                                (ancho / 2), alto/4, formato);
    nombre = dgvGanadores.Rows[1].Cells[1].Value.ToString();
    gra.DrawString(nombre, new Font("Arial", 9), Brushes.Blue,
                                (ancho / 4), alto/2.75f, formato);
    nombre = dgvGanadores.Rows[2].Cells[1].Value.ToString();
    gra.DrawString(nombre, new Font("Arial", 9), Brushes.Blue,
                                (ancho / 1.33f), alto/2, formato);
}
```

Nos ayudamos de las propiedades 'Width' y 'Heigth' para obtener el ancho y alto del picturebox y así calcular las posiciones donde se centrarán los nombres de los ganadores, por ejemplo, para el primer nombre, las coordenadas resultantes corresponden a la mitad del ancho y a un cuarto del alto, tomado desde el borde superior, y de forma similar para los otros dos nombres. La notación '2.75f' y '1.33f', indican, con el agregado de la letra 'f' que se trata de valores de tipo float, esto es necesario para que la operación de división se realice correctamente.

Cómo se vio en la herramienta 3 de ésta SP, usamos 'StringFormat' para crear un formato centrado, tanto en horizontal como en vertical, para aplicar a cada nombre que se dibuja sobre el control picturebox.

Con esta última función completamos la implementación del evento Click en el botón 'DeterminarGanadores', cómo resumen podemos mencionar que hemos trabajado con un arreglo bidimensional ('puestos'), que almacena las posiciones de los participantes en cada prueba del triatlón y dos arreglos unidimensionales ('datos' y 'ganadores') para obtener el puntaje total de cada participante y los 3 ganadores del triatlón.

Recomendamos analizar detenidamente el uso de los índices empleados en los recorridos y asignaciones de los distintos elementos en cada uno de los arreglos, el correcto manejo de esos índices nos permitirá resolver cualquier requerimiento en el uso de arreglos.

También destacamos la similitud en la lógica empleada para acceder al arreglo bidimensional con la aplicada para acceder a las celdas de las grillas, ya que éstas al estar organizadas en filas y columnas resultan equivalentes a los arreglos bidimensionales.

- **Botón Salir**

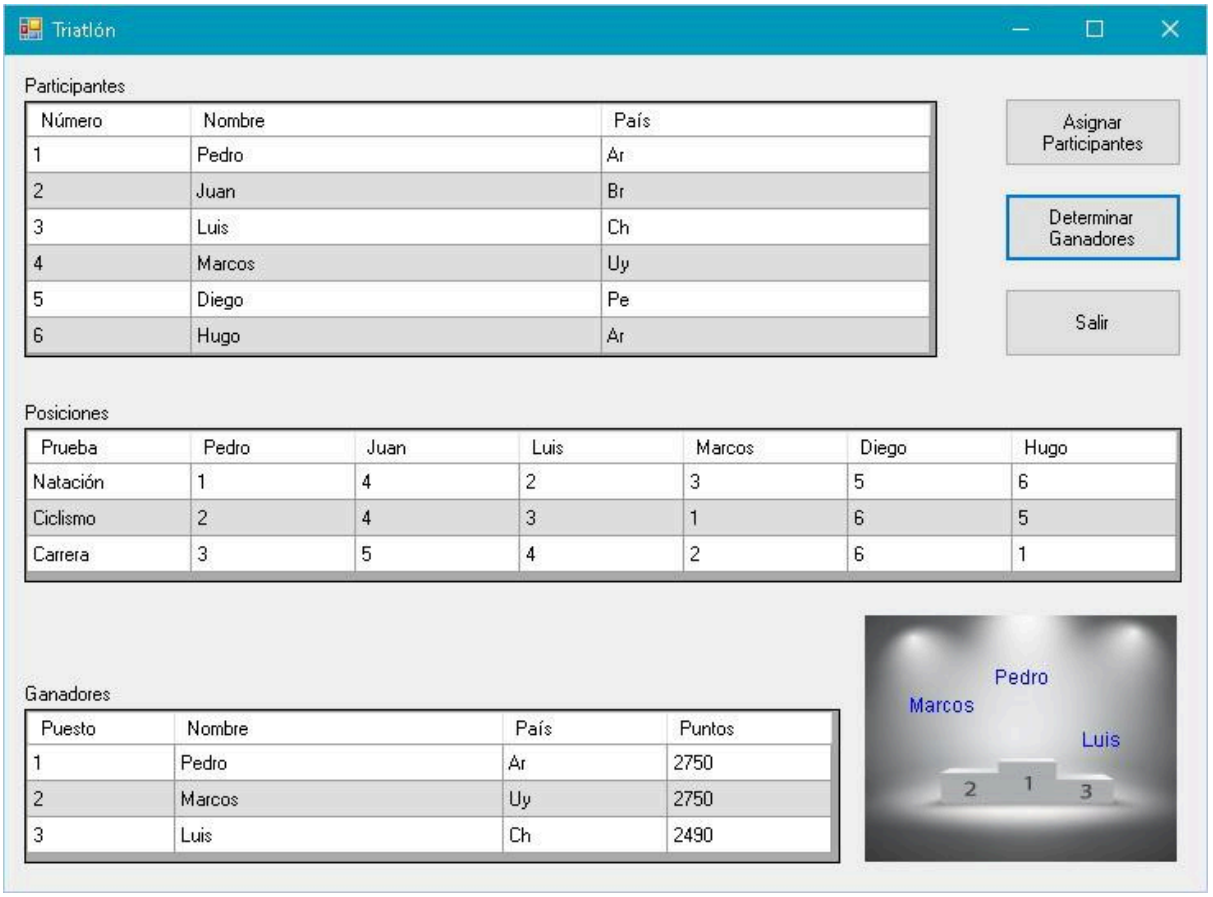
Nos resta implementar el evento Click del botón 'Salir':

C #: Boton Salir

```
private void btnSalir_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Y así completamos la resolución de la situación planteada.

Ejemplo del resultado obtenido al ejecutar la aplicación:



Las posiciones asignadas en este ejemplo determinan que Pedro y Marcos tengan la misma cantidad de puntos empatando el primer lugar, en estos casos se toma por defecto el número de participante para determinar el ganador: Pedro es el participante 1 y Marcos el participante 4, resulta Pedro primero y Marcos segundo. Esto resulta naturalmente así ya que los arreglos se recorren desde la primera posición hasta la última.

Bar La Milanga

El encargado de un negocio gastronómico le ha solicitado a usted en calidad de pasante, desarrolle un programa que permita ingresar las ventas diarias de cada mozo que atiende en el negocio para luego poder realizar cálculos que generen distinto tipos de información.

Las ventas se discriminan en 4 categorías: Comidas, Bebidas sin alcohol, Bebidas con alcohol y Postres.

Los mozos son 5: Julio, Esteban, Javier, Gonzalo y Alberto.

La interfaz debe contar con una grilla configurada con 5 filas, una para cada mozo y 4 columnas para las categorías de venta. El usuario deberá cargar los importes finales de cada mozo en cada una de las categorías directamente sobre las celdas de la grilla.

Una vez cargada la totalidad de la grilla y con un botón denominado 'Validar Datos' se deberá verificar que todas las celdas tengan un valor numérico, admitiendo como válido el valor cero. Si se detecta algún error se debe mostrar un mensaje de error y se deshabilitarán los botones de las consultas.

Los botones de las consultas se ubican en la parte inferior del formulario y son dos, el primero denominado "Mozo del día" deberá determinar qué mozo tiene el mayor importe de venta total, considerando todas las categorías y mostrará su nombre e importe en controles de tipo 'TextBox' o 'Label' ubicados a la derecha del botón.

El segundo botón de consulta "Totales" deberá calcular el importe total vendido por todos los mozos y los totales por categoría, son 5 valores que se mostrarán también a la derecha del botón. Como en el caso anterior, pueden colocarse controles 'TextBox' o 'Label' para mostrar los resultados.

El formulario tendrá también un botón para cerrar la aplicación.

Usar un arreglo bidimensional de tipo float para almacenar los importes ingresados en la grilla.

Indique verdadero o falso

1) float[,] precios = new float[5, 10]; declara un arreglo bidimensional de 10 filas y 5 columnas

- ○ Verdadero
- ○ Falso

2) En un arreglo de C# cada elemento puede ser a su vez un arreglo.

- ○ Verdadero
- ○ Falso

3) La creación de un arreglo en C# puede incluir la inicialización de sus valores.

- ○ Verdadero
- ○ Falso

4) En un control DataGridView la opción "Habilitar acción de agregar" permite que el usuario agregue en tiempo de ejecución filas o columnas a la grilla.

- ○ Verdadero
- ○ Falso

5) Una de las mejores formas de mostrar información es por medio de un control DataGridView conocido también como Grilla.

- ○ Verdadero
- ○ Falso

6) Para borrar las filas existentes en un control DataGridView debemos llamar al método Rows.Clear().

- ○ Verdadero
- ○ Falso

7) El control PictureBox se usa solamente para mostrar imágenes cargadas desde archivos.

- ○ Verdadero
- ○ Falso

8) El sistema de coordenadas X, Y en el control picturebox tiene su origen en la esquina superior izquierda del control.

- ○ Verdadero
- ○ Falso

Indique la opción correcta

9) El método para leer una imagen y mostrarla en el control picturebox es:

- ○ LoadImage()
- ○ Load()
- ○ AddImage()
- ○ CreateImage()
- ○ Insert()
- ○ Ninguno de los anteriores

Respuestas correctas[48]

[48] 1)Falso; 2)Verdadero; 3)Verdadero; 4)Falso; 5)Verdadero; 6)Verdadero; 7)Falso; 8)Verdadero; 9)Load().

Cierre



Link del video: https://www.youtube.com/watch?v=HvAiWgR9hag

Los avances vertiginosos de la tecnología en el mundo nos condicionan día a día, por ello es muy importante conocer qué herramientas se utilizan para el desarrollo de software en la resolución de problemas de una organización.

Este texto fue desarrollado para exponer de manera sencilla los conceptos introductorios de la programación en lenguaje C# sobre la plataforma .NET, teniendo en cuenta la complejidad que requiere explicar el funcionamiento de un lenguaje de programación orientado a objetos con interfaces gráficas.

Esperamos que sea sumamente útil teniendo en cuenta que hemos elegido una carrera en donde todo el contexto está rodeado de tecnología, software y hardware.

Deseamos de todo corazón, que puedan desarrollarse profesionalmente y utilicen los conceptos aprendidos en este texto de estudio para fortalecer su desarrollo, sabiendo que además de los contenidos expuestos aquí, pueden contar con los docentes de la materia para resolver cualquier situación que la realidad les presente.

Para finalizar, esperamos que disfruten del texto, y que sirva para aumentar sus conocimientos tanto para el ámbito profesional como personal.

El Autor

Bibliografía

Bibliografía Básica:

Bongionavanni, E. y Obegón, D. (2010). *Programación Lógica*. Texto de Interactivo Digital . Córdoba. Argentina. Editorial IES Siglo 21.

Rubble. (1998). Análisis y Diseño de Sistemas Orientado a Objetos con GUI. México. Prentice Hall.

Bibliografía Ampliatoria:

Wirth, N. (1989). *Algoritmos + Estructuras de Datos = Programas*. México. Ed. del Castillo S.A.

Joyanes Aguilar, L. (1988). *Fundamentos de Programación-Algoritmos y Estructuras de Datos*. España. McGraw-Hill/Interamericana de España S. A. U.

Bishop, J. (1989). *Abstracción de Datos*. España. Paraninfo.

Farrell, J. (2001). *Introducción a la Programación Lógica y Diseño*. Barcelona, España. Editorial Thomson Internacional.

Garcia Merayo, F. *Control de Errores en la Algoritmia*. España. Paraninfo. 1991.

Material Soporte de Información:

Farnetti, A. y Obregon, D. *Programación Lógica*. Texto Multimedial. (2002). Córdoba, Argentina. Editorial IES Siglo 21.

MSDN - Documentacion de C#